

O'REILLY®

Learning eBPF

Programming the Linux Kernel for Enhanced
Observability, Networking, and Security



Compliments of

ISOVALENT

Liz Rice

O'REILLY®

Learning eBPF

Programming the Linux Kernel for Enhanced
Observability, Networking, and Security



Compliments of

ISOVALENT

Liz Rice

Table of Contents

1. [Learning eBPF](#)
2. [学习 eBPF](#)
3. [Preface](#)
4. [前言](#)
5. [Who This Book Is For](#)
6. [这本书适合谁](#)
7. [What This Book Covers](#)
8. [这本书包括哪些内容。](#)
9. [Prerequisite Knowledge](#)
10. [先决知识](#)
11. [Example Code and Exercises](#)
12. [示例代码和练习](#)
13. [Is eBPF Only for Linux?](#)
14. [eBPF只适用于Linux吗?](#)
15. [Conventions Used in This Book](#)
16. [本书中使用的约定](#)
17. [Using Code Examples](#)
18. [使用代码示例](#)
19. [O'Reilly Online Learning](#)
20. [奥莱利在线学习](#)
21. [How to Contact Us](#)
22. [如何联系我们](#)
23. [Acknowledgments](#)
24. [致谢](#)

25. Chapter 1. What Is eBPF, and Why Is It Important?
26. 第一章：什么是eBPF，它为何重要？
27. eBPF's Roots: The Berkeley Packet Filter
28. eBPF的起源：伯克利数据包过滤器
29. From BPF to eBPF
30. 从BPF到eBPF
31. The Evolution of eBPF to Production Systems
32. eBPF到生产系统的演进
33. Naming Is Hard
34. 选择一个合适的名字很难
35. The Linux Kernel
36. Linux内核
37. Adding New Functionality to the Kernel
38. 向内核添加新功能
39. Kernel Modules
40. 内核模块
41. Dynamic Loading of eBPF Programs
42. eBPF程序的动态加载
43. High Performance of eBPF Programs
44. eBPF程序的高性能
45. eBPF in Cloud Native Environments
46. 在云原生环境中的eBPF
47. Summary
48. 摘要
49. Chapter 2. eBPF's "Hello World"
50. 第二章 eBPF的“Hello World”
51. BCC's "Hello World"

- 52. 嗨，世界！
- 53. Running “Hello World”
- 54. 运行“你好，世界”
- 55. BPF Maps
- 56. BPF地图
 - 1. Hash Table Map
 - 2. 哈希表映射
 - 3. Perf and Ring Buffer Maps
 - 4. 性能和环形缓冲区映射
- 57. Ring Buffers
- 58. 环形缓冲器
 - 1. Function Calls
 - 2. 函数调用
 - 3. Tail Calls
 - 4. 尾调用
- 59. Summary
- 60. 摘要
- 61. Exercises
- 62. 练习
- 63. Chapter 3. Anatomy of an eBPF Program
- 64. 第三章：eBPF程序的解剖 eBPF (Extended Berkeley Packet Filter，扩展伯克利数据包过滤器)程序的解剖，是理解eBPF的基础。在本章中，我们将深入了解eBPF程序的结构和组成部分。eBPF程序是用C语言编写的，并通过特定的编译器转换为可以在Linux内核中运行的字节码。这些eBPF程序通过网络套接字与内核通信，以实现网络流量的处理和分析。一个eBPF程序由多个指令组成。每个指令都有对应的操作码和参数。这些指令按照顺序依次

执行，实现特定的功能。eBPF程序还包括一组注册的内存映射，用于存储程序执行期间的变量和数据。这些内存映射可以被eBPF程序中的指令访问和修改。eBPF程序还可以调用一组特定的内核函数，以实现与内核的交互。这些内核函数提供了访问系统调用、网络数据包和其他内核资源的能力。除了具体的指令和内存映射外，eBPF程序还可以定义一组过滤条件和钩子点。这些过滤条件和钩子点决定了eBPF程序在何时和何地执行。通过理解eBPF程序的结构和组成部分，我们可以更好地利用eBPF技术来解决网络 and 系统性能调优等问题。在后续章节中，我们将深入探讨如何编写和使用eBPF程序来实现各种功能。

65. The eBPF Virtual Machine

66. eBPF虚拟机

1. eBPF Registers
2. eBPF寄存器
3. eBPF Instructions
4. eBPF指令

67. eBPF “Hello World” for a Network Interface

68. eBPF 网络接口的“Hello World”

69. Compiling an eBPF Object File

70. 编译一个eBPF对象文件

71. Inspecting an eBPF Object File

72. 检查eBPF对象文件

73. Loading the Program into the Kernel

74. 将程序加载到内核中

75. Inspecting the Loaded Program

76. 查看加载的程序

1. The BPF Program Tag

- 2. [BPF程序标签](#)
- 3. [The Translated Bytecode](#)
- 4. 请帮我翻译成简体中文：“The Translated Bytecode”。请回答仅包含翻译后的内容，不包含原文内容。
- 5. [The JIT-Compiled Machine Code](#)
- 6. [JIT 编译的机器码](#)
- 77. [Attaching to an Event](#)
- 78. [附加到事件](#)
- 79. [Global Variables](#)
- 80. [全局变量](#)
- 81. [Detaching the Program](#)
- 82. [分离程序](#)
- 83. [Unloading the Program](#)
- 84. [卸载程序](#)
- 85. [BPF to BPF Calls](#)
- 86. [BPF到BPF调用](#)
- 87. [Summary](#)
- 88. [总结](#)
- 89. [Exercises](#)
- 90. [练习](#)
- 91. [Chapter 4. The bpf\(\) System Call](#)
- 92. [第四章 bpf\(\) 系统调用。](#)
- 93. [Loading BTF Data](#)
- 94. [加载BTF数据](#)
- 95. [Creating Maps](#)
- 96. [制作地图](#)
- 97. [Loading a Program](#)

- 98. 装载程序
- 99. [Modifying a Map from User Space](#)
- 00. 从用户空间修改地图
- 01. [BPF Program and Map References](#)
- 02. [BPF程序和Map参考](#)
 - 1. [Pinning](#)
 - 2. 固定
 - 3. [BPF Links](#)
 - 4. [BPF链接](#)
- 03. [Additional Syscalls Involved in eBPF](#)
- 04. [附加的系统调用参与 eBPF](#)
 - 1. [Initializing the Perf Buffer](#)
 - 2. 初始化性能缓冲
 - 3. [Attaching to Kprobe Events](#)
 - 4. 附加到Kprobe事件
 - 5. [Setting Up and Reading Perf Events](#)
 - 6. 建立和阅读 Perf 事件
- 05. [Ring Buffers](#)
- 06. 环形缓冲区
- 07. [Reading Information from a Map](#)
- 08. 从地图中阅读信息
 - 1. [Finding a Map](#)
 - 2. 找地图
 - 3. [Reading Map Elements](#)
 - 4. 阅读地图要素
- 09. [Summary](#)
- 10. 摘要

- 11. Exercises
- 12. 练习
- 13. Chapter 5. CO-RE, BTF, and Libbpf
- 14. 第五章。CO-RE, BTF和Libbpf。
- 15. BCC's Approach to Portability
- 16. BCC对可移植性的方法
- 17. CO-RE Overview
- 18. CO-RE 概述
- 19. BPF Type Format
- 20. BPF类型格式
 - 1. BTF Use Cases
 - 2. BTF 使用案例
 - 3. Listing BTF Information with bpftool
 - 4. 使用bpftool列出BTF信息
 - 5. BTF Types
 - 6. BTF类型
 - 7. Maps with BTF Information
 - 8. 地图与BTF信息
 - 9. BTF Data for Functions and Function Prototypes
 - 10. 函数和函数原型的BTF数据
 - 11. Inspecting BTF Data for Maps and Programs
 - 12. 检查BTF数据的地图和程序
- 21. Generating a Kernel Header File
- 22. 生成内核头文件
- 23. CO-RE eBPF Programs
- 24. CO-RE eBPF程序
 - 1. Header Files

- 2. 头文件
 - 1. Kernel header information
 - 2. 内核头部信息
 - 3. Headers from libbpf
 - 4. 来自libbpf的标题
 - 5. Application-specific headers
 - 6. 应用特定标头
- 3. Defining Maps
- 4. 定义地图
- 5. eBPF Program Sections
- 6. eBPF程序部分
- 7. Memory Access with CO-RE
- 8. 使用CO-RE进行内存访问
- 9. License Definition
- 10. 定义许可
- 25. Compiling eBPF Programs for CO-RE
- 26. 为CO-RE编译eBPF程序
 - 1. Debug Information
 - 2. 调试信息
 - 3. Optimization
 - 4. 优化
 - 5. Target Architecture
 - 6. 目标架构
 - 7. Makefile
 - 8. BTF Information in the Object File
 - 9. 目标文件中的BTF信息
- 27. BPF Relocations

- 28. [BPF 搬迁](#)
- 29. [CO-RE User Space Code](#)
- 30. [使用者空间代码](#)
- 31. [The Libbpf Library for User Space](#)
- 32. [用户空间的Libbpf库](#)
 - 1. [BPF Skeletons](#)
 - 2. [骨架BPF](#)
 - 1. [Loading programs and maps into the kernel](#)
 - 2. [将程序和地图加载到内核中](#)
 - 3. [Accessing existing maps](#)
 - 4. [访问现有的地图](#)
 - 5. [Attaching to events](#)
 - 6. [连接到事件 \(Liánjiē dào shìjiàn\)](#)
 - 7. [Managing an event buffer](#)
 - 8. [管理事件缓冲区](#)
 - 3. [Libbpf Code Examples](#)
 - 4. [Libbpf 代码示例](#)
- 33. [Summary](#)
- 34. [摘要](#)
- 35. [Exercises](#)
- 36. [练习](#)
- 37. [Chapter 6. The eBPF Verifier](#)
- 38. [第六章. eBPF验证器](#)
- 39. [The Verification Process](#)
- 40. [验证过程](#)
- 41. [The Verifier Log](#)
- 42. [核实者日志](#)

- 43. Visualizing Control Flow
- 44. 可视化控制流程
- 45. Validating Helper Functions
- 46. 验证助手函数
- 47. Helper Function Arguments
- 48. 辅助函数参数,
- 49. Checking the License
- 50. 查看许可证
- 51. Checking Memory Access
- 52. 检查内存访问
- 53. Checking Pointers Before Dereferencing Them
- 54. 在解引用指针之前检查它们
- 55. Accessing Context
- 56. 获取上下文
- 57. Running to Completion
- 58. 运行到完成
- 59. Loops
- 60. 循环 (Xún huán)
- 61. Checking the Return Code
- 62. 检查返回代码
- 63. Invalid Instructions
- 64. 无效指令
- 65. Unreachable Instructions
- 66. 到达不了的指示
- 67. Summary
- 68. 摘要
- 69. Exercises

- 70. [练习](#)
- 71. [Chapter 7. eBPF Program and Attachment Types](#)
- 72. [第七章：eBPF程序和附件类型](#)
- 73. [Program Context Arguments](#)
- 74. [程序上下文参数](#)
- 75. [Helper Functions and Return Codes](#)
- 76. [助手函数和返回码](#)
- 77. [Kfuncs](#)
- 78. [功能函数](#)
- 79. [Tracing](#)
- 80. [追踪](#)
 - 1. [Kprobes and Kretprobes](#)
 - 2. [Kprobes 和 Kretprobes](#)
 - 1. [Attaching kprobes to syscall entry points](#)
 - 2. [将`Attaching kprobes to syscall entry points`翻译为简体中文: 将kprobes附加到系统调用入口点](#)
 - 3. [Attaching kprobes to other kernel functions](#)
 - 4. [连接kprobes到其他内核函数](#)
 - 3. [Fentry/Fexit](#)
 - 4. [进入/退出](#)
 - 5. [Tracepoints](#)
 - 6. [追踪点](#)
 - 7. [BTF-Enabled Tracepoints](#)
 - 8. [BTF-启用跟踪点](#)
 - 9. [User Space Attachments](#)
 - 10. [用户空间附件](#)
 - 11. [LSM](#)

12. 主观地说, `LSM` 在简体中文中没有固定的意思。

81. Networking

82. 网络化

1. Sockets

2. 套接字

3. Traffic Control

4. 交通管制

5. XDP

6. XDP translates to "XDP" in simplified Chinese. There is no translation available for this term.

7. Flow Dissector

8. 流分析器

9. Lightweight Tunnels

10. 轻量级隧道

11. Cgroups

12. Cgroups -> 控制组

13. Infrared Controllers

14. 红外控制器

83. BPF Attachment Types

84. BPF附件类型

85. Summary

86. 摘要

87. Exercises

88. 练习

89. Chapter 8. eBPF for Networking

90. 第8章。eBPF用于网络

91. Packet Drops

92. 数据包丢失

1. XDP Program Return Codes
2. XDP程序返回码
3. XDP Packet Parsing
4. XDP数据包解析

93. Load Balancing and Forwarding

94. 负载均衡和转发

95. XDP Offloading

96. XDP 卸载

97. Traffic Control (TC)

98. 运输管制 (TC)

99. Packet Encryption and Decryption

100. 数据包加密与解密

1. User Space SSL Libraries
2. 用户空间SSL库

101. eBPF and Kubernetes Networking

102. eBPF和Kubernetes网络

1. Avoiding iptables
2. 避免使用 iptables
3. Coordinated Network Programs
4. 协调网络计划
5. Network Policy Enforcement
6. 网络策略执行
7. Encrypted Connections
8. 加密连接

103. Summary

104. 摘要

- !05. [Exercises and Further Reading](#)
- !06. [练习和进一步阅读](#)
- !07. [Chapter 9. eBPF for Security](#)
- !08. [第9章 安全性的eBPF](#)
- !09. [Security Observability Requires Policy and Context](#)
- !10. [安全可观察性需要策略和上下文](#)
- !11. [Using System Calls for Security Events](#)
- !12. [使用系统调用进行安全事件](#)
 - 1. [Seccomp](#)
 - 2. [系统调用过滤器](#)
 - 3. [Generating Seccomp Profiles](#)
 - 4. [生成 Seccomp 话题](#)
 - 5. [Syscall-Tracking Security Tools](#)
 - 6. [系统调用跟踪安全工具](#)
- !13. [BPF LSM](#)
- !14. [BPF LSM will be translated to: BPF LSM 将被翻译为:](#)
- !15. [Cilium Tetragon](#)
- !16. [四方眼鱼鳍](#)
 - 1. [Attaching to Internal Kernel Functions](#)
 - 2. [内核函数的附加](#)
 - 3. [Preventative Security](#)
 - 4. [防范性安全 \(Fángfàn Xìng Ānquán\)](#)
- !17. [Network Security](#)
- !18. [网络安全](#)
- !19. [Summary](#)
- !20. [总结](#)
- !21. [Chapter 10. eBPF Programming](#)

- !22. [第十章。eBPF编程](#)
- !23. [Bpftrace](#)
- !24. [Bpftrace: 抽丝剥茧](#)
- !25. [Language Choices for eBPF in the Kernel](#)
- !26. [内核中eBPF的语言选择](#)
- !27. [BCC Python/Lua/C++](#)
- !28. [请帮我将`BCC Python/Lua/C++`翻译成简体中文。](#)
- !29. [C and Libbpf](#)
- !30. [C和Libbpf](#)
 - 1. [Go](#)
 - 2. [去](#)
 - 3. [Gobpf](#)
 - 4. [Gobpf \(高级 Berkeley Packet Filter\) 的翻译是高级伯克利包过滤器。](#)
 - 5. [Ebpf-go](#)
 - 6. [Ebpf-go的翻译为简体中文为"白名单引导程序-go"。](#)
 - 7. [Libbpfgo](#)
 - 8. [Libbpfgo 的翻译是 "Libbpfgo"](#)
- !31. [Rust](#)
- !32. [锈](#)
 - 1. [Libbpf-rs](#)
 - 2. [Libbpf-rs --> Libbpf-rs](#)
 - 3. [Redbpf](#)
 - 4. [红色安全平台 \(Hóngsè ānquán píngtái\)](#)
 - 5. [Aya](#)
 - 6. [阿雅](#)
 - 7. [Rust-bcc](#)

8. Rust-bcc => Rust-bcc

!33. [Testing BPF Programs](#)

!34. [测试BPF程序](#)

!35. [Multiple eBPF Programs](#)

!36. [多个 eBPF 程序](#)

!37. [Summary](#)

!38. [摘要](#)

!39. [Exercises](#)

!40. [练习](#)

!41. [Chapter 11. The Future Evolution of eBPF](#)

!42. [第11章。eBPF的未来演进](#)

!43. [The eBPF Foundation](#)

!44. [eBPF基金会](#)

!45. [eBPF for Windows](#)

!46. [eBPF为Windows](#)

!47. [Linux eBPF Evolution](#)

!48. [Linux eBPF进化](#)

!49. [eBPF Is a Platform, Not a Feature](#)

!50. [eBPF不是一个功能，而是一个平台](#)

!51. [Conclusion](#)

!52. [结论](#)

!53. [About the Author](#)

!54. [关于作者](#)

!55. [Colophon](#)

!56. [版权页](#)

Learning eBPF

学习 eBPF

Preface

前言

In the cloud native community and beyond, eBPF has become one of the hottest technical topics of recent years. A new generation of [powerful tools and projects](#) in networking, security, observability, and more have been built (and more continue to be created) using eBPF as a platform, offering better performance and accuracy compared to their predecessors. eBPF-related conferences such as the [eBPF Summit](#) and [Cloud Native eBPF Day](#) have attracted thousands of attendees and viewers, and at the time of this writing, the [eBPF Slack](#) community has more than 14,000 members.

在云原生社区及其他领域，eBPF已成为近年来最热门的技术话题之一。利用eBPF作为平台，已经建立了一代新的强大工具和项目，涉及网络、安全、可观测性等多个领域，其性能和准确性较之前的产品有了显著提升，而且还在不断开发中。与eBPF相关的会议，如eBPF峰会和云原生eBPF日，吸引了数千名与会者和观众，而截至撰写本文时，eBPF的Slack社区已拥有超过14,000名成员。

Why is eBPF being selected as the underlying technology for so many infrastructure tools? How does it deliver the promised improvements to performance? How is eBPF useful in such disparate technical fields, which range from performance tracing to network traffic encryption?

为什么eBPF被选择作为许多基础施工工具的基础技术？它如何实现所承诺的性能改进？eBPF在如此不同的技术领域，从性能追踪到网络流量加密，是如何有用的？

This book aims to answer these questions by giving the reader an understanding of how eBPF works, as well as providing an introduction to writing eBPF code.

本书旨在通过让读者了解eBPF的工作原理，以及提供编写eBPF代码的入门介绍，来解答这些问题。

Who This Book Is For

这本书适合谁

This book is for developers, system administrators, operators, and students who are curious about eBPF and want to know more about how it works. It will provide a foundation for those who want to explore writing eBPF programs themselves. Since eBPF provides a great platform for a whole new generation of instrumentation and tooling, there will likely be gainful employment for eBPF developers for some years to come.

这本书是为开发者、系统管理员、运维人员和对eBPF感兴趣并想进一步了解其工作原理的学生而编写的。它将为那些希望自己探索编写eBPF程序的人提供基础。由于eBPF为全新一代工具和仪器提供了良好平台，因此未来几年内eBPF开发者可能会有丰厚的就业机会。

But you don't necessarily need to be planning to write eBPF code yourself for this book to be useful to you. If you work in operations, security, or any other role that involves software infrastructure, you're likely to come across eBPF-based tooling, now or over the next few years. If you understand something about the internals of these tools, you'll be in a better position to use them effectively. For example, if you know how events can trigger eBPF programs, you'll have a better mental model for exactly what an eBPF-based tool is really measuring when it shows you performance metrics. If you're an application developer, you might also come into contact with

some of these eBPF-based tools—for example, if you are performance tuning an application, you might use a tool like [Parca](#) to generate flame graphs showing which functions are taking the most time. If you are evaluating security tools, this book will help you understand where eBPF shines and how to avoid using it in a naïve way that is less effective against attacks.

但您并不一定需要计划自己编写eBPF代码，这本书对您仍然很有用。如果您从事运维、安全或其他与软件基础设施相关的工作，您可能会在现在或未来几年里遇到基于eBPF的工具。如果您能理解这些工具的内部原理，您将能更有效地使用它们。例如，如果您了解事件如何触发eBPF程序，那么当它以性能指标显示给您时，您会对eBPF工具实际测量了什么有更清晰的理解。如果您是应用程序开发人员，您可能也会接触到一些基于eBPF的工具，例如，如果您正在对应用程序进行性能调优，您可以使用类似Parca的工具生成展示哪些函数占用最多时间的火焰图。如果您正在评估安全工具，本书将帮助您了解eBPF的优势，以及如何避免以幼稚的方式使用它以提高对抗攻击的效果。

Even if you're not using eBPF tools today, I hope this book will give you interesting insights into areas of Linux that you might not have considered before. Most developers take the kernel for granted, as they use programming languages with convenient higher-level abstractions that allow them to focus on the work of application development—which is plenty hard enough! They use tools like debuggers and performance analyzers to help them do their job effectively. Knowing the internals of how a debugger or performance tool works might be interesting, but it's not essential. Yet, for many of us, it's fun and fulfilling to go down the rabbit hole to find out more.¹

In the same way, most people will use eBPF tools without having to worry about how they are built. [Arthur C. Clarke wrote](#) that “any sufficiently advanced technology is indistinguishable from magic,” but personally, I like to dig in and find out how the magic trick works. You might be like me and feel compelled to explore eBPF programming to get a better feel for what is possible with this technology. If so, I think you’ll enjoy this book.

即使您今天不使用eBPF工具，我希望这本书能为您带来有趣的见解，让您了解以前可能没有考虑过的Linux领域。大多数开发人员很容易忽视内核，因为他们使用具有便捷高级抽象的编程语言，使他们能够专注于应用程序开发工作 - 这已经足够困难了！他们使用像调试器和性能分析器这样的工具来帮助他们有效地完成工作。了解调试器或性能工具的内部工作原理可能很有趣，但并非必要。然而，对我们许多人来说，深入了解更多是有趣而充实的。同样地，大多数人将使用eBPF工具而无需担心其构建方式。亚瑟·C·克拉克写道：“任何足够先进的技术都无法与魔法区分”，但就个人而言，我喜欢深入挖掘并找出魔术的工作原理。您可能和我一样，会被迫探索eBPF编程，以更好地了解这项技术的潜力。如果是这样，我想您会喜欢这本书。

What This Book Covers

这本书包括哪些内容。

eBPF continues to evolve at quite a rapid pace, which makes it rather difficult to write a comprehensive reference that doesn't constantly need updating. However, there are some fundamentals and basic principles that are unlikely to change significantly, and that's what this book discusses.

eBPF在快速发展中不断演变，这使得编写一本不需要不断更新的全面参考书变得相当困难。然而，有一些基础和基本原则是不太可能发生重大变化的，这就是本书讨论的内容。

[Chapter 1](#) sets the scene by describing why eBPF is so powerful as a technology and explaining how the ability to run custom programs in the operating system kernel enables so many exciting capabilities.

第一章 通过描述为什么eBPF作为一种技术强大，并解释在操作系统内核中运行自定义程序的能力如何实现了许多令人激动的功能，为故事情节定下了基调。

Things become more concrete in [Chapter 2](#), where you'll see some "Hello World" examples that introduce you to the concepts of eBPF programs and maps.

第二章中的事情变得更加具体，在其中你将看到一些“Hello World”示例，介绍了eBPF程序和映射的概念。

[Chapter 3](#) dives into more detail about eBPF programs and how they run in the kernel, and [Chapter 4](#) explores the interface between user space applications and eBPF programs.

第3章深入讨论了eBPF程序的更多细节，以及它们在内核中的运行方式；第4章探索了用户空间应用程序与eBPF程序之间的接口。

One of the big challenges of eBPF in recent years has been the question of compatibility across kernel versions. [Chapter 5](#) looks at the “compile once, run everywhere” (CO-RE) approach that solves this problem.

近年来，eBPF面临的一个重大挑战是在不同内核版本间的兼容性问题。第5章介绍了解决这个问题的“编译一次，随处运行”（CO-RE）方法。

The verification process is perhaps the most important characteristic that distinguishes eBPF from kernel modules. I'll introduce you to the eBPF verifier in [Chapter 6](#).

验证过程或许是区分eBPF与内核模块最重要的特征。我将会在第6章向您介绍eBPF验证器。

In [Chapter 7](#) you'll get an introduction to the many different types of eBPF programs and their attachment points. Many of those attachment points are within the networking stack, and [Chapter 8](#) explores the application of eBPF for networking features in more detail. [Chapter 9](#) looks at how eBPF is being used to build security tools.

在第7章中，您将了解到许多不同类型的eBPF程序及其附加点的介绍。其中许多附加点位于网络栈内部，在第8章中将更详细地探讨eBPF在网络功能中的应用。第9章讨论了eBPF如何被用于构建安全工具。

If you want to write a user space application that interacts with eBPF programs, there are many libraries and frameworks available to help. [Chapter 10](#) gives an overview of the options for various programming languages.

如果您想编写一个与eBPF程序交互的用户空间应用程序，有许多库和框架可供使用。第10章概述了各种编程语言的选择。

Finally, in [Chapter 11](#) I'll gaze into my crystal ball and tell you about some future developments that are likely to unfold in the eBPF world.

最后，在第11章中，我将凝视着我的水晶球，告诉你一些可能在eBPF世界中展开的未来发展。

Prerequisite Knowledge

先决知识

This book assumes you are comfortable with basic shell commands on Linux and with the idea of using a compiler to turn source code into an executable program. There are some simple example extracts from Makefiles, on the assumption that you have at least a minimal understanding of how `make` uses these files.

本书假设您对Linux上的基本shell命令以及使用编译器将源代码转换为可执行程序的概念已经非常熟悉。书中引用了一些简单的Makefile示例，假设您至少具备对make工具如何使用这些文件的最基本理解。

There are lots of code examples in Python, C, and Go. You won't need in-depth knowledge of those languages to get something out of these examples, but you'll get the most out of the book if you are generally happy to read some code. I'm also assuming you are familiar with the idea of *pointers*, which identify a memory location.

Python、C和Go中有很多的代码示例。你不需要深入了解这些语言就能够从这些示例中获得一些东西，但是如果你乐意阅读一些代码的话，你将能够从本书中获得最多的收益。我还假设你对指针的概念很熟悉，它们用来标识内存位置。

Example Code and Exercises

示例代码和练习

There are lots of code examples in this book. If you want to try them out for yourself, you'll find an accompanying GitHub repository and instructions for installing and running the code at <https://github.com/lizrice/learning-ebpf>.

这本书中有很多代码示例。如果你想亲自尝试它们，你可以在<https://github.com/lizrice/learning-ebpf>找到相关的GitHub存储库，并获取安装和运行代码的说明。

I have also included exercises at the end of most of the chapters to help you explore eBPF programming by extending the examples or

writing your own programs.

我还在大多数章节的结尾处包含了练习，帮助你通过扩展示例或编写自己的程序来探索eBPF编程。

Because eBPF is continually evolving, the features available to you depend on the kernel version you're running. Many of the restrictions that apply to earlier versions have been lifted or relaxed in later versions. The [lovisor project](#) has a useful overview of the [kernel versions in which different BPF features were added](#), and in this book I have attempted to note when the particular capabilities I'm describing were added. The examples were tested using version 5.15 of the kernel, and at the time of this writing some of the popular Linux distributions don't yet support such a recent kernel version. If you're reading this book shortly after it was published, you might find that some of the features won't work on the Linux kernel your organization uses in production.

因为eBPF在持续演进，所以可用的功能取决于您正在运行的内核版本。许多在早期版本中适用的限制已在后续版本中解除或放宽。lovisor项目提供了内核版本不同BPF功能添加的有用概述，本书中我已尝试记录特定能力的添加时间。这些示例是在内核5.15版本上进行测试的，写作时一些热门Linux发行版尚不支持如此新的内核版本。如果您在本书出版后不久开始阅读，可能会发现一些功能无法在您的生产环境中使用的Linux内核上运行。

Is eBPF Only for Linux?

eBPF只适用于Linux吗？

eBPF was originally developed for Linux. There's no particular reason why the same approach couldn't be used in other operating systems too—indeed, Microsoft has been developing an [eBPF implementation for Windows](#). I discuss that briefly in [Chapter 11](#), but throughout the rest of the book I focus on the Linux implementation, and all the examples will be from Linux.

eBPF最初是为Linux开发的。在其他操作系统中使用相同的方法并没有特殊的理由——实际上，微软一直在为Windows开发eBPF实现。我在第11章中简要讨论了这一点，但在本书的其余部分中，我将专注于Linux实现，并且所有示例都来自Linux。

Conventions Used in This Book

本书中使用的约定

The following typographical conventions are used in this book:

本书使用以下排版约定：

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

指示新术语，网址，电子邮件地址，文件名和文件扩展名。

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

用于程序清单，以及在段落中用来指代程序元素，例如变量或函数名、数据库、数据类型、环境变量、语句和关键字。

Constant width bold

Shows commands or other text that should be typed literally by the user.

显示用户应该逐字输入的命令或其他文本。

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

显示应用用户提供的值或上下文确定的值的文本。

Tip

This element signifies a tip or suggestion.

这个元素表示一个提示或建议。

Note

This element signifies a general note.

这个元素表示一般的注释。

Warning

This element indicates a warning or caution.

这个元素表示警告或注意。

Using Code Examples

使用代码示例

Supplemental material (code examples, exercises, etc.) is available for download at <https://github.com/lizrice/learning-ebpf>.

补充材料（代码示例，练习等）可在以下链接下载：

<https://github.com/lizrice/learning-ebpf>。

If you have a technical question or a problem using the code examples, please send email to bookquestions@oreilly.com.

如果您有技术问题或在使用代码示例时遇到问题，请发送电子邮件至 bookquestions@oreilly.com。

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of

example code from this book into your product's documentation does require permission.

这本书的目的是帮助你完成工作。一般情况下，如果书中提供了示例代码，你可以在自己的程序和文档中使用它。除非你复制了代码的大部分内容，否则无需联系我们索取许可。例如，编写一个使用了本书中多个代码片段的程序不需要许可。销售或分发《O'Reilly》图书中的示例代码需要许可。如果你引用了本书中的例子代码来回答问题，也无需许可。但是，如果你要将本书中大量的例子代码整合到你产品的文档中，需要许可。

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Learning eBPF* by Liz Rice (O'Reilly). Copyright 2023 Vertical Shift Ltd., 978-1-098-13512-6.”

我们赞赏，但通常不要求署名。署名通常包括标题、作者、出版社和ISBN。例如：“《学习eBPF》Liz Rice（O'Reilly出版社）2023年版权归Vertical Shift Ltd.所有，978-1-098-13512-6。”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

如果你觉得你使用的代码示例超出了公平使用范围或以上给予的许可，请随时联系我们，发邮件至permissions@oreilly.com。

O'Reilly Online Learning

奥莱利在线学习

Note

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

40多年来，O'Reilly媒体为公司的成功提供技术和商业培训、知识和见解。

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

我们独特的专家和创新者网络通过书籍、文章和我们的在线学习平台分享他们的知识和专长。O'Reilly的在线学习平台为您提供按需访问现场培训课程、深入学习路径、互动编码环境以及来自O'Reilly和其他200多家出版商的大量文本和视频资源。更多信息，请访问 <https://oreilly.com>。

How to Contact Us

如何联系我们

Please address comments and questions concerning this book to the publisher:

请向出版商提出有关本书的评论和问题。

- O'Reilly Media, Inc.
- 奥莱利媒体有限公司
- 1005 Gravenstein Highway North
- 1005 Gravenstein Highway North - 南格拉文斯坦高速公路北段1005号
- Sebastopol, CA 95472
- 塞巴斯托波尔, 加利福尼亚州95472
- 800-998-9938 (in the United States or Canada)
- 800-998-9938 (美国或加拿大)
- 707-829-0515 (international or local)
- 707-829-0515 (国际或本地)
- 707-829-0104 (fax)
- 707-829-0104 (传真)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at

<https://oreil.ly/learning-eBPF>.

我们为这本书设有一个网页，上面列出了勘误、示例和所有额外信息。您可以在<https://oreil.ly/learning-eBPF> 访问该网页。

Email bookquestions@oreilly.com to comment or ask technical questions about this book.

发送电子邮件至bookquestions@oreilly.com 来对本书进行评论或提出技术问题。

For news and information about our books and courses, visit <https://oreilly.com>.

有关我们的书籍和课程的新闻和信息，请访问<https://oreilly.com>。

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>.

在领英上找到我们: <https://linkedin.com/company/oreilly-media>。

Follow us on Twitter: <https://twitter.com/oreillymedia>.

关注我们的Twitter: <https://twitter.com/oreillymedia>。

Watch us on YouTube: <https://youtube.com/oreillymedia>.

在YouTube上观看我们: <https://youtube.com/oreillymedia>。

Acknowledgments

致谢

I would like to thank the many people who have contributed enormously to the writing of this book:

我想要感谢许多为这本书做出巨大贡献的人们。

- My technical reviewers—Timo Beckers, Jess Males, Quentin Monnet, Kevin Sheldrake, and Celeste Stinger—provided detailed, actionable feedback and great ideas for improving the examples, for which I'm very grateful.

我的技术评审员Tim Beckers, Jess Males, Quentin Monnet, Kevin Sheldrake和Celeste Stinger提供了详细而可操作的反馈意见和改进示例的好主意, 对此我非常感激。

- I'm standing on the shoulders of the giants who built, popularized, and continue to maintain eBPF, including Daniel Borkmann, Thomas Graf, Brendan Gregg, Andrii Nakryiko, Alexei Starovoitov, and countless others who have contributed not just code but also conference talks and blog posts to the community.

我站在那些建立、推广和维护eBPF的巨匠们的肩膀上, 他们包括Daniel Borkmann、Thomas Graf、Brendan Gregg、Andrii Nakryiko、Alexei Starovoitov和无数其他为社区贡献了不仅仅代码, 还有会议演讲和博客文章的人们。

- Thank you to my talented and lovely colleagues at Isovalent, many of whom are eBPF and kernel specialists, from whom I continue to learn so much.

感谢我在Isovalent的才华横溢又可爱的同事们, 其中许多人都是eBPF和内核专家, 从他们身上我继续学到了很多。

- Thanks also to the team at O'Reilly, especially my editor, Rita Fernando, who gave me endless support during the writing process, as well as the planning that helped keep the book on schedule; and John Devins, for encouraging me to write the book in the first place.

还要感谢O'Reilly团队, 特别是我的编辑Rita Fernando, 在写作过程中给予我无尽的支持, 还有为了帮助保持书稿进度所做的计划;

还有John Devins，鼓励我首次写这本书。

- Phil Pearl not only gave helpful feedback on the content, but he also made sure I ate and took breaks. I am forever grateful for his support and encouragement.

菲尔·珍珠不仅仅就内容提供了有用的反馈，还确保我吃饭和休息。我将永远感激他的支持和鼓励。

I also want to thank all the wonderful people who, over the years, have taken the time to make encouraging comments about my work, whether it's in person at an event or on social media. It's incredibly inspiring to know that something I have written or recorded has helped someone else get to grips with a technical concept or has given them the desire to build or write something themselves. Thank you!

我还要感谢多年来那些美妙的人们，他们花时间给我的作品留下鼓励性的评论，无论是在活动现场还是在社交媒体上。知道我所写的或录制的内容帮助到其他人理解技术概念，或者激发他们有意愿去构建或写作，这真的非常鼓舞人心。谢谢！

¹ At the dotGo Paris conference in 2017, [I gave a talk that showed how a debugger works.](#)

在2017年的dotGo巴黎会议上，我做了一个演讲，展示了调试器的工作原理。

Chapter 1. What Is eBPF, and Why Is It Important?

第一章：什么是eBPF，它为何重要？

eBPF is a revolutionary kernel technology that allows developers to write custom code that can be loaded into the kernel dynamically, changing the way the kernel behaves. (Don't worry if you're not confident about what the kernel is—we'll come to that shortly in this chapter.)

eBPF是一种革命性的内核技术，允许开发人员编写自定义代码，并能够动态加载到内核中，从而改变内核的行为方式。（如果你对内核不太了解也不用担心，在本章节中我们将很快介绍。）

This enables a new generation of highly performant networking, observability, and security tools. And as you'll see, if you want to instrument an app with these eBPF-based tools, you don't need to modify or reconfigure the app in any way, thanks to eBPF's vantage point within the kernel.

这使得一系列新一代高性能的网络、可观察性和安全工具成为可能。正如您将看到的，如果您想要使用这些基于eBPF的工具来监测一个应用程序，您无需以任何方式修改或重新配置该应用程序，这要归功于eBPF在内核中的视角。

Just a few of the things you can do with eBPF include:

使用eBPF可以做的一些事情包括：

- Performance tracing of pretty much any aspect of a system

几乎系统的任何方面的性能追踪

- High-performance networking, with built-in visibility

高性能网络，具备内置可见性

- Detecting and (optionally) preventing malicious activity

检测并（可选）防止恶意活动

Let's take a brief journey through eBPF's history, starting with the Berkeley Packet Filter.

让我们简要回顾一下eBPF的历史，从伯克利数据包过滤器开始。

eBPF's Roots: The Berkeley Packet Filter

eBPF的起源：伯克利数据包过滤器

What we call “eBPF” today has its roots in the BSD Packet Filter, first described in 1993 in a paper¹ written by Lawrence Berkeley National Laboratory's Steven McCanne and Van Jacobson. This paper discusses a pseudomachine that can run *filters*, which are programs written to determine whether to accept or reject a network packet. These programs were written in the BPF instruction set, a general-

purpose set of 32-bit instructions that closely resembles assembly language. Here's an example taken directly from that paper:

我们今天所称的“eBPF”起源于1993年由加州伯克利国家实验室的Steven McCanne和Van Jacobson撰写的一篇论文中描述的BSD数据包过滤器。该论文讨论了一种伪机器，可以运行过滤器，这些过滤器是编写的程序，用于确定是否接受或拒绝网络数据包。这些程序是用BPF指令集编写的，该指令集是一组与汇编语言非常相似的通用32位指令。以下是直接摘自该论文的示例：

```
ldh      [12]
jeq      #ETHERTYPE_IP, L1, L2
L1:      ret      #TRUE
L2:      ret      #0
```

This tiny piece of code filters out packets that aren't Internet Protocol (IP) packets. The input to this filter is an Ethernet packet, and the first instruction (`ldh`) loads a 2-byte value starting at byte 12 in this packet. In the next instruction (`jeq`) that value is compared with the value that represents an IP packet. If it matches, execution jumps to the instruction labeled `L1`, and the packet is accepted by returning a nonzero value (identified here as `#TRUE`). If it doesn't match, the packet is not an IP packet and is rejected by returning `0`.

这段小代码用于过滤非Internet协议（IP）数据包。该过滤器的输入是以太网数据包。第一条指令（`ldh`）会加载该数据包中从第12个字节开始的2个字节的值。在下一条指令（`jeq`）中，该值将与表示IP数据包的值进行比较。如果匹配成功，程序将跳转到标记为L1的指令处，并通过返回一个非零值（在这里被标识为`#TRUE`）来接受该数据包。如果不匹配，则该数据包不是IP数据包，并通过返回0来拒绝它。

You can imagine (or, indeed, refer to the paper to find examples of) more complex filter programs that make decisions based on other aspects of the packet. Importantly, the author of the filter can write their own custom programs to be executed within the kernel, and this is the heart of what eBPF enables.

你可以设想（或者可以参考文献中的示例）更复杂的过滤器程序，这些程序可以根据数据包的其他方面进行决策。重要的是，过滤器的作者可以编写自己的自定义程序来在内核中执行，这正是eBPF的核心所在。

BPF came to stand for “Berkeley Packet Filter,” and it was first introduced to Linux in 1997, in kernel version 2.1.75,² where it was used in the tcpdump utility as an efficient way to capture the packets to be traced out.

BPF起初代表“伯克利封包过滤器”，于1997年首次引入Linux的内核版本2.1.75中。它被用于tcpdump工具中，作为一种高效捕获封包以进行追踪的方式。

Fast-forward to 2012, when seccomp-bpf was introduced in version 3.5 of the kernel. This enabled the use of BPF programs to make decisions about whether to allow or deny user space applications from making system calls. We’ll explore this in more detail in [Chapter 10](#). This was the first step in evolving BPF from the narrow scope of packet filtering to the general-purpose platform it is today. From this point on, the words *packet filter* in the name started to make less sense!

快进到2012年，在内核版本3.5中引入了seccomp-bpf。这使得可以使用BPF程序来决定是否允许或拒绝用户空间应用程序进行系统调用。我们将在第10章中对此进行更详细的探讨。这是将BPF从狭义的数据包过滤发展为如今通用平台的第一步。从此以后，名称中的"数据包过滤器"这个词变得越来越没有意义了！

From BPF to eBPF

从BPF到eBPF

BPF evolved to what we call “extended BPF” or “eBPF” starting in kernel version 3.18 in 2014. This involved several significant changes:

BPF发展成我们所称的“扩展BPF”或“eBPF”，从2014年的内核版本3.18开始。这涉及了几个重大变化：

- The BPF instruction set was completely overhauled to be more efficient on 64-bit machines, and the interpreter was entirely rewritten.

BPF指令集被完全改进，以在64位机器上更高效运行，并且解释器被完全重写。

- eBPF *maps* were introduced, which are data structures that can be accessed by BPF programs and by user space applications, allowing information to be shared between them. You’ll learn about maps in [Chapter 2](#).

引入了eBPF映射，这是一种可以被BPF程序和用户空间应用程序访问的数据结构，允许它们之间共享信息。您将在第二章了解有关映射的内容。

- The `bpf()` system call was added so that user space programs can interact with eBPF programs in the kernel. You'll read about this system call in [Chapter 4](#).

`bpf()`系统调用被添加以使用户空间程序可以与内核中的eBPF程序进行交互。您将在第4章中了解该系统调用。

- Several BPF helper functions were added. You'll see a few examples in [Chapter 2](#) and some more details in [Chapter 6](#).

添加了几个BPF助手函数。您将在第2章中看到一些示例，并在第6章中了解更多细节。

- The eBPF verifier was added to ensure that eBPF programs are safe to run. This is discussed in [Chapter 6](#).

eBPF验证器的添加是为了确保eBPF程序的安全运行。这在第六章中进行了讨论。

This put the basis for eBPF in place, but development did not slow down! Since then, eBPF has evolved significantly.

这为eBPF奠定了基础，但开发并没有放慢！从那时起，eBPF有了显著发展。

The Evolution of eBPF to Production Systems

eBPF到生产系统的演进

A feature called *kprobes* (kernel probes) had existed in the Linux kernel since 2005, allowing for traps to be set on almost any instruction in the kernel code. Developers could write kernel modules that attached functions to kprobes for debugging or performance measurement purposes.³

自2005年起，Linux内核就存在着一项名为kprobes（内核探针）的特性，它允许在内核代码中设置陷阱来捕获几乎任何指令。开发人员可以编写内核模块，将函数附加到kprobes上，用于调试或性能测量目的。

The ability to attach eBPF programs to kprobes was added in 2015, and this was the starting point for a revolution in the way tracing is done across Linux systems. At the same time, hooks started to be added within the kernel's networking stack, allowing eBPF programs to take care of more aspects of networking functionality. We'll see more of this in [Chapter 8](#).

在2015年，我们增加了将eBPF程序附加到kprobes的能力，这是在Linux系统中进行跟踪的方式发生革命的起点。同时，内核的网络堆栈中开始添加钩子，允许eBPF程序处理网络功能的更多方面。在第8章中，我们会看到更多相关内容。

By 2016, eBPF-based tools were being used in production systems. [Brendan Gregg's](#) work on tracing at Netflix became widely known in infrastructure and operations circles, as did [his statement](#) that eBPF “brings superpowers to Linux.” In the same year, the Cilium project

was announced, being the first networking project to use eBPF to replace the entire datapath in container environments.

截至2016年，基于eBPF的工具已经被应用于生产系统。Brendan Gregg在Netflix上关于追踪的工作在基础设施和运营领域广为人知，他曾说过eBPF“为Linux带来了超级能力”。同年，Cilium项目宣布启动，成为第一个在容器环境中使用eBPF替代整个数据路径的网络项目。

The following year Facebook (now Meta) made [Katran](#) an open source project. Katran, a layer 4 load balancer, met Facebook's need for a [highly scalable and fast solution](#). Every single packet to [Facebook.com](#) since 2017 has passed through eBPF/XDP.⁴ For me personally, this was the year that ignited my excitement about the possibilities enabled by this technology, after seeing [Thomas Graf's talk](#) about eBPF and the [Cilium project](#) at DockerCon in Austin, Texas.

次年，Facebook（现在是Meta）将Katran开源项目化。Katran是一个4层负载均衡器，满足了Facebook对高度可扩展和快速解决方案的需求。自2017年以来，所有发送到Facebook.com的数据包都经过了eBPF/XDP。对我个人而言，这一年引发了我对这项技术所带来可能性的兴奋，因为我在德克萨斯州奥斯汀的DockerCon上听到了Thomas Graf关于eBPF和Cilium项目的演讲。

In 2018, eBPF became a separate subsystem within the Linux kernel, with [Daniel Borkmann](#) from Isovalent and [Alexei Starovoitov](#) from Meta as its maintainers (they were later joined by [Andrii Nakryiko](#), also from Meta). The same year saw the introduction of

BPF Type Format (BTF), which makes eBPF programs much more portable. We'll explore this in [Chapter 5](#).

2018年，eBPF成为Linux内核中的一个独立子系统，由Isovalent的Daniel Borkmann和Meta的Alexei Starovoitov担任维护者（后来还加入了Meta的Andrii Nakryiko）。同年还推出了BPF类型格式（BTF），使得eBPF程序更具可移植性。我们将在第5章中探讨这一点。

The year 2020 saw the introduction of LSM BPF, allowing eBPF programs to be attached to the Linux Security Module (LSM) kernel interface. This indicated that a third major use case for eBPF had been identified: it became clear that eBPF is a great platform for security tooling, in addition to networking and observability.

在2020年，引入了LSM BPF。这使得eBPF程序可以附加到Linux安全模块(LSM)内核接口上。这表明已经确定了第三个主要的eBPF使用案例：eBPF不仅在网络和观测性方面是一个很好的平台，还可以作为安全工具的载体。

Over the years, eBPF's capabilities have grown substantially, thanks to the work of more than 300 kernel developers and many contributors to the associated user space tools (such as `bpftool`, which we'll meet in [Chapter 3](#)), compilers, and programming language libraries. Programs were once limited to 4,096 instructions, but that limit has grown to 1 million verified instructions⁵ and has effectively been rendered irrelevant by support for tail calls and function calls (which you'll see in [Chapters 2 and 3](#)).

多年来，由于300多名内核开发人员以及许多相关用户空间工具（例如我们将在第3章中遇到的bpftool）、编译器和编程语言库的贡献，eBPF的功能大大增强。程序曾经被限制为4,096条指令，但这个限制已经增长到了100万条经过验证的指令，并且通过对尾调用和函数调用的支持（在第2章和第3章中将会看到），这个限制实际上已经变得无关紧要。

Note

For deeper insight into the history of eBPF, who better to refer to than the maintainers who have been working on it from the beginning?

为了对eBPF的历史有更深入的了解，有谁比最初开始工作的维护者更适合去参考呢？

Alexei Starovoitov gave a fascinating presentation about the [history of BPF](#) from its roots in software-defined networking (SDN). In this talk, he discusses the strategies used to get the early eBPF patches accepted into the kernel and reveals that the official birthday of eBPF is September 26, 2014, which marked the acceptance of the first set of patches covering the verifier, BPF system call, and maps.

Alexei Starovoitov关于BPF的历史发表了一场令人着迷的演讲，从软件定义网络（SDN）的根源开始。在此演讲中，他讨论了早期eBPF补丁被内核接受的策略，并透露eBPF的正式生日是2014年9月26日，标志着第一批涵盖验证器、BPF系统调用和映射的补丁被接受的时刻。

Daniel Borkmann has also discussed the history of BPF and its evolution to support networking and tracing functionality. I highly recommend his talk “[eBPF and Kubernetes: Little Helper Minions for Scaling Microservices](#)”, which is full of interesting nuggets of information.

Daniel Borkmann也讨论了BPF的历史以及它在支持网络和追踪功能方面的发展。我强烈推荐他的演讲“eBPF和Kubernetes: 协助微服务扩展的小助手”，其中包含了很多有趣的信息。

Naming Is Hard

选择一个合适的名字很难

eBPF’s applications range so far beyond packet filtering that the acronym is essentially meaningless now, and it has become a standalone term. And since the Linux kernels in widespread use these days all have support for the “extended” parts, the terms *eBPF* and *BPF* are largely used interchangeably. In the kernel source code and in eBPF programming, the common terminology is *BPF*. For example, as we’ll see in [Chapter 4](#), the system call for interacting with eBPF is `bpf()`, helper functions start with `bpf_`, and the different types of (e)BPF programs are identified with names that start with `BPF_PROG_TYPE`. Outside the kernel community, the name “eBPF” seems to have stuck, for example, in the community site [ebpf.io](#) and in the name of the [eBPF Foundation](#).

eBPF的应用范围远远超出了数据包过滤，以至于这个缩写基本上已经变得毫无意义，它已经成为一个独立的术语。由于目前广泛使用的Linux内核都支持"扩展"部分，因此eBPF和BPF这两个术语通常被互换使用。在内核源代码和eBPF编程中，常用的术语是BPF。例如，在第4章中我们会看到，与eBPF交互的系统调用是**bpf()**，辅助函数以**bpf_**开头，不同类型的(e)BPF程序以**BPF_PROG_TYPE**开头的名称进行标识。在内核社区之外，"eBPF"这个名称似乎已经被大家接受了，例如在社区网站**ebpf.io**和eBPF Foundation的名称中。

The Linux Kernel

Linux内核

To understand eBPF you'll need a solid grasp of the difference between the kernel and user space in Linux. I covered this in my report "What Is eBPF?"⁶ and I've adapted some of that content for the next few paragraphs.

要理解eBPF，你需要对Linux的内核空间和用户空间的区别有扎实的掌握。我在我的报告“什么是eBPF？”中提到了这一点，并且我已经为接下来的几段内容做了一些调整。

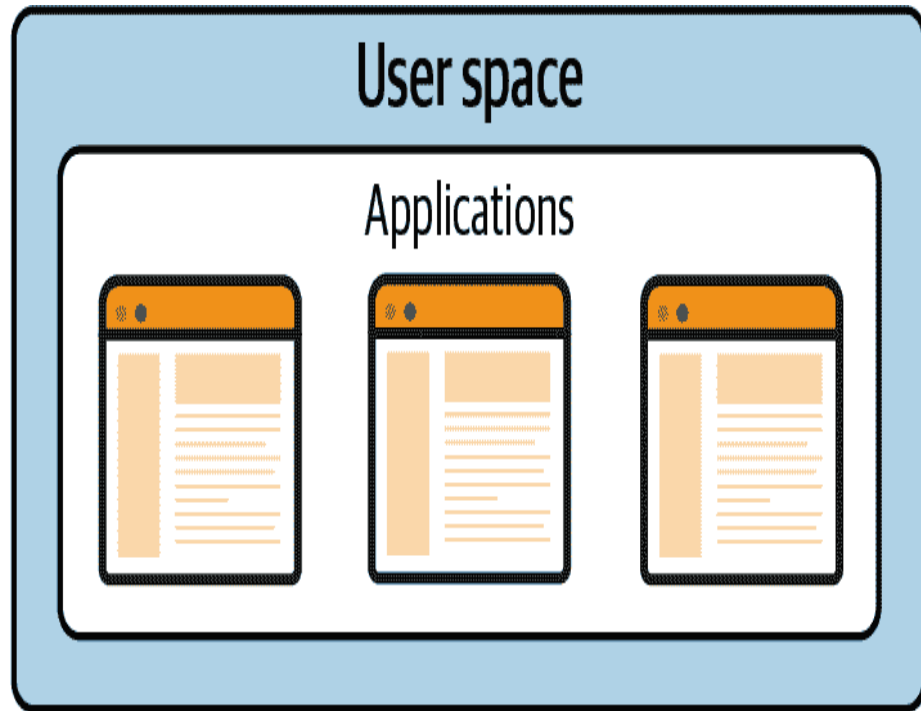
The Linux kernel is the software layer between your applications and the hardware they're running on. Applications run in an unprivileged layer called *user space*, which can't access hardware directly. Instead, an application makes requests using the system call (syscall) interface to request the kernel to act on its behalf. That hardware access can involve reading and writing to files, sending or

receiving network traffic, or even just accessing memory. The kernel is also responsible for coordinating concurrent processes, enabling many applications to run at once. This is illustrated in [Figure 1-1](#).

Linux内核是应用程序和硬件之间的软件层。应用程序在一个称为用户空间的非特权层中运行，无法直接访问硬件。相反，应用程序使用系统调用（syscall）接口发出请求，请求内核代表其采取行动。该硬件访问可能涉及读写文件，发送或接收网络流量，甚至只是访问内存。内核还负责协调并发进程，使许多应用程序能够同时运行。这在图1-1中有所示。

As application developers, we typically don't use the system call interface directly, because programming languages give us high-level abstractions and standard libraries that are easier interfaces to program. As a result, a lot of people are blissfully unaware of how much the kernel is doing while our programs run. If you want to get a sense of how often the kernel is invoked, you can use the `strace` utility to show all the system calls an application makes.

作为应用程序开发者，我们通常不直接使用系统调用接口，因为编程语言为我们提供了更高级的抽象和标准库，这些更容易编程的接口。因此，许多人对内核在我们的程序运行时做了多少事情毫不知情。如果你想了解内核被调用的频率，你可以使用strace工具来显示应用程序所进行的所有系统调用。



System calls - - - - -

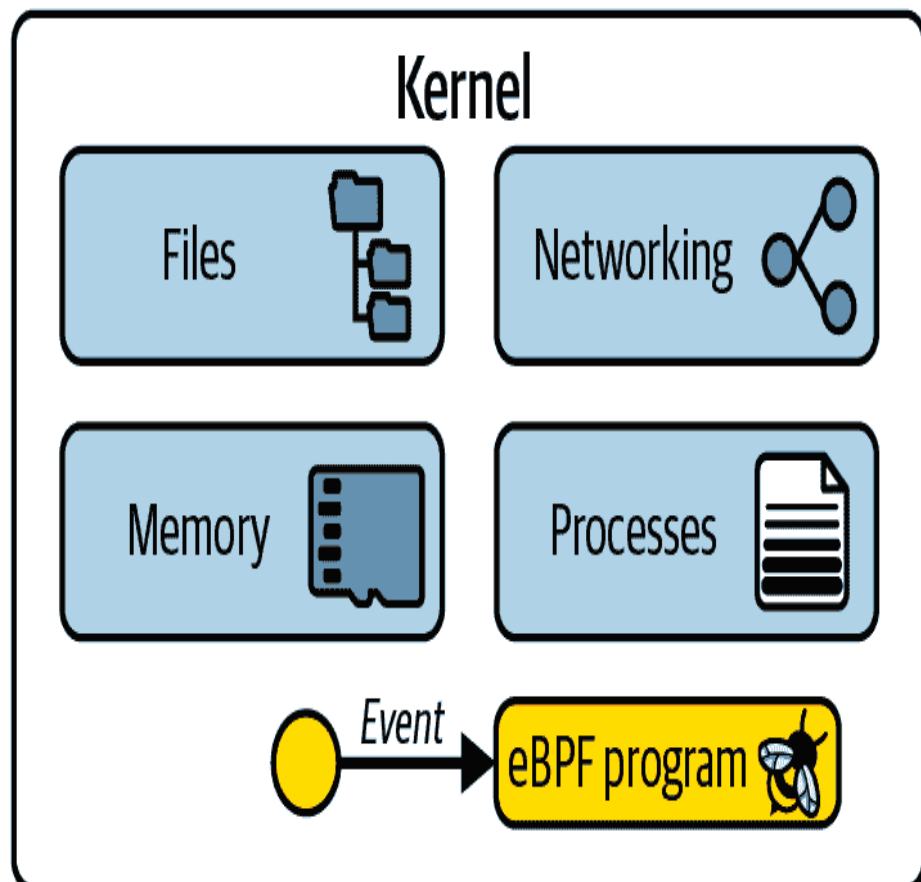


Figure 1-1. Applications in user space use the syscall interface to make requests to the kernel

Here’s an example, where using `cat` to echo the word *hello* to the screen involves more than 100 system calls:

这是一个例子，使用cat命令在屏幕上打印单词hello涉及到100多次系统调用。

\$ strace -c echo "hello"					
hello					
% time	seconds	usecs/call	calls	errors	syscall
-----	-----	-----	-----	-----	-----

24.62	0.001693	56	30	12	openat
17.49	0.001203	60	20		mmap
15.92	0.001095	57	19		
newfstatat					
15.66	0.001077	53	20		close
10.35	0.000712	712	1		execve
3.04	0.000209	52	4		
mprotect					
2.52	0.000173	57	3		read
2.33	0.000160	53	3		brk
2.09	0.000144	48	3		munmap
1.11	0.000076	76	1		write
0.96	0.000066	66	1	1	
faccessat					
0.76	0.000052	52	1		
getrandom					
0.68	0.000047	47	1		rseq
0.65	0.000045	45	1		
set_robust_list					
0.63	0.000043	43	1		
prlimit64					
0.61	0.000042	42	1		
set_tid_address					
0.58	0.000040	40	1		futex
-----	-----	-----	-----	-----	-----

100.00	0.006877	61	111	13 total

Because applications rely so heavily on the kernel, it means we can learn a lot about how an application behaves if we can observe its interactions with the kernel. With eBPF we can add instrumentation into the kernel to get these insights.

因为应用程序在很大程度上依赖于内核，所以如果我们能观察其与内核的交互，就可以了解应用程序的行为。使用eBPF，我们可以在内核中添加仪器来获得这些见解。

For example, if you are able to intercept the system call for opening files, you can see exactly which files any application accesses. But how could you do that interception? Let’s consider what would be involved if we wanted to modify the kernel, adding new code to create some kind of output whenever that system call is invoked.

例如，如果您能够截取打开文件的系统调用，您就可以准确地看到任何应用程序访问的文件。但是如何进行截取呢？让我们考虑一下，如果我们想要修改内核，在每次调用该系统调用时添加新代码以创建某种输出，将涉及哪些内容。

Adding New Functionality to the Kernel

向内核添加新功能

The Linux kernel is complex, with around 30 million lines of code at the time of this writing.⁷ Making a change to any codebase requires

some familiarity with the existing code, so unless you're a kernel developer already, this is likely to present a challenge.

Linux内核很复杂，在撰写本文时大约有三千万行代码。对任何代码库进行修改都需要对现有代码有一定的了解，所以除非你已经是一名内核开发者，否则这可能会很有挑战性。

Additionally, if you want to contribute your change upstream, you'll be facing a challenge that isn't purely technical. Linux is a general-purpose operating system, used in all manner of environments and circumstances. This means that if you want your change to become part of an official Linux release, it's not simply a matter of writing code that works. The code has to be accepted by the community (and more specifically by Linus Torvalds, creator and main developer of Linux) as a change that will be for the greater good of all. This isn't a given—only one-third of submitted kernel patches are accepted.⁸

此外，如果您想向上游贡献您的更改，您将面临一个不仅仅是技术方面的挑战。Linux是一个通用的操作系统，用于各种环境和情况。这意味着，如果您希望您的更改成为官方Linux版本的一部分，仅仅编写可运行的代码是不够的。代码必须被社区接受（更具体地说是由Linux的创始人和主要开发者Linus Torvalds接受），并被认为对所有人都有更大的好处的更改。这并非是一定能实现的事情 - 只有三分之一的提交的内核补丁会被接受。

Let's suppose you've figured out a good technical approach for intercepting the system call for opening files. After some months of discussion and some hard development work on your part, let's

imagine the change is accepted into the kernel. Great! But how long will it be until it arrives on everyone's machines?

假设您已经找到了一个良好的技术方法来拦截打开文件的系统调用。经过几个月的讨论和您的努力开发工作，假设这个改变被内核接受了。太好了！但是需要多久才能在每个人的机器上实现这个改变呢？

There's a new release of the Linux kernel every two or three months, but even when a change has made it into one of these releases, it's still some time away from being available in most people's production environments. This is because most of us don't just use the Linux kernel directly—we use Linux distributions like Debian, Red Hat, Alpine, and Ubuntu that package up a version of the Linux kernel with various other components. You may well find that your favorite distribution is using a kernel release that's several years old.

每两三个月就会有新的Linux内核发布，但即使一个变动被纳入了其中一个发布版本，它在大多数人的生产环境中仍需要一些时间才能使用。这是因为我们大多数人直接使用Linux内核—我们使用像Debian、Red Hat、Alpine和Ubuntu这样打包了Linux内核某个版本及其他各种组件的Linux发行版。你很可能会发现你最喜欢的发行版正在使用几年前的内核版本。

For example, a lot of enterprise users employ Red Hat Enterprise Linux (RHEL). At the time of this writing, the current release is RHEL 8.5, dated November 2021, and it uses version 4.18 of the Linux kernel. This kernel was released in August 2018.

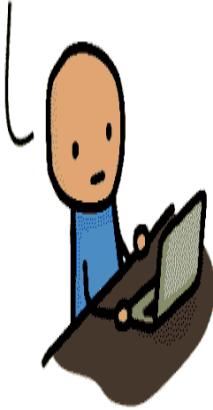
例如，许多企业用户使用Red Hat Enterprise Linux (RHEL)。截至本文撰写时，当前版本是2021年11月发布的RHEL 8.5，它使用Linux内核的4.18版本。该内核于2018年8月发布。

As illustrated in the cartoon in [Figure 1-2](#), it takes literally years to get new functionality from the idea stage into a production environment Linux kernel.⁹

正如图1-2中所示的卡通漫画，将新功能从构想阶段推进到生产环境的Linux内核需要几年的时间。

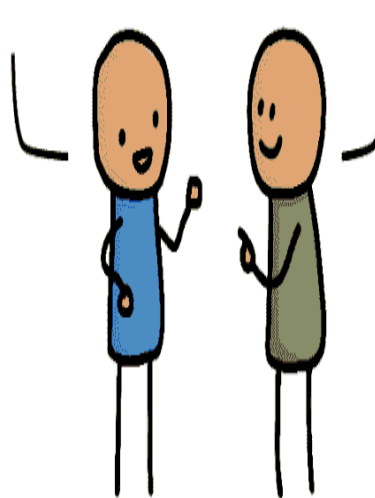
Application Developer:

I want this new feature to observe my app



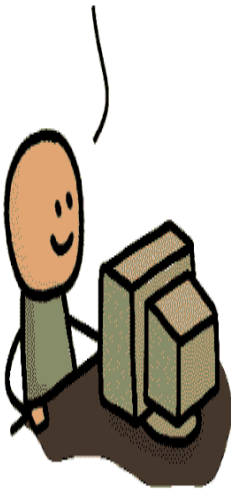
Hey kernel developer! Please add this new feature to the Linux kernel

OK! Just give me a year to convince the entire community that this is good for everyone.



1 year later...

I'm done. The upstream kernel now supports this.



But I need this in my Linux distro



5 years later...

Good news. Our Linux distribution now ships a kernel with your required feature

OK but my requirements have changed since...

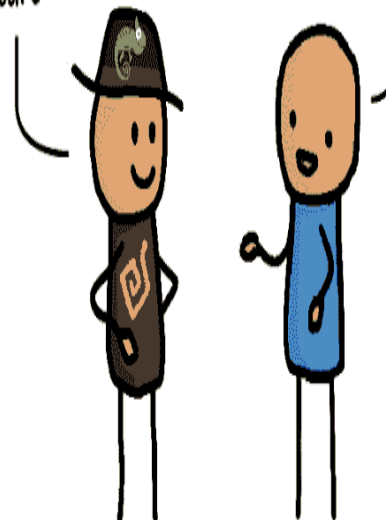


Figure 1-2. Adding features to the kernel (cartoon by Vadim Shchekoldin, Isovalent)

Kernel Modules

内核模块

If you don't want to wait years for your change to make it into the kernel, there is another option. The Linux kernel was designed to accept kernel modules, which can be loaded and unloaded on demand. If you want to change or extend kernel behavior, writing a module is certainly one way to do it. A kernel module can be distributed for others to use independent of the official Linux kernel release, so it doesn't have to be accepted into the main upstream codebase.

如果您不想等待数年才能将您的更改纳入内核，还有另一种选择。Linux 内核设计成可以接受内核模块，这些模块可以根据需求加载和卸载。如果您想要改变或扩展内核行为，编写一个模块当然是一种方法。内核模块可以分发给其他人使用，与官方 Linux 内核发布独立，因此不必将其纳入主要的上游代码库中。

The biggest challenge here is that this is still full-on kernel programming. Users have historically been very cautious about using kernel modules, for one simple reason: if kernel code crashes, it takes down the machine and everything running on it. How can a user be confident that a kernel module is safe to run?

这里最大的挑战是仍然需要进行全面的内核编程。用户对于使用内核模块一直持谨慎态度，这有一个简单的原因：如果内核代码崩溃，会导致整个计算机和其上运行的所有东西都崩溃。用户如何确保一个内核模块是安全可靠的？

Being “safe to run” doesn’t just mean not crashing—the user wants to know that a kernel module is safe from a security perspective.

Does it include vulnerabilities that an attacker could exploit? Do we trust the authors of the module not to put malicious code in it?

Because the kernel is privileged code, it has access to everything on the machine, including all the data, so malicious code in the kernel would be a serious cause for concern. This applies to kernel modules too.

“安全运行”并不仅意味着不会崩溃——用户希望了解一个内核模块在安全方面是安全的。它是否存在攻击者可以利用的漏洞？我们是否相信模块的作者没有插入恶意代码？因为内核是特权代码，它可以访问机器上的所有东西，包括所有数据，所以内核中的恶意代码将是一个严重的问题。这同样适用于内核模块。

The safety of the kernel is one important reason why Linux distributions take so long to incorporate new releases. If other people have been running a kernel version in a variety of circumstances for months or years, this should have flushed out issues. The distribution maintainers can have some confidence that the kernel they ship to their users/customers is *hardened*—that is, it is safe to run.

内核的安全性是Linux发行版需要花费很长时间来整合新版本的一个重要原因之一。如果其他人已经在各种情况下运行了数月或数年的内核版本，那么这应该可以发现问题。发行版的维护者可以相对确信他们提供给用户/客户的内核经过强化，也就是说，可以安全运行。

eBPF offers a very different approach to safety: the *eBPF verifier*, which ensures that an eBPF program is loaded only if it's safe to run—it won't crash the machine or lock it up in a hard loop, and it won't allow data to be compromised. We'll discuss the verification process in more detail in [Chapter 6](#).

eBPF提供了一种非常不同的安全方法：eBPF验证器，它确保仅在加载eBPF程序是安全运行时才加载它——它不会导致机器崩溃或陷入死循环，并且不允许数据被泄露。我们将在第6章更详细地讨论验证过程。

Dynamic Loading of eBPF Programs

eBPF程序的动态加载

eBPF programs can be loaded into and removed from the kernel dynamically. Once they are attached to an event, they'll be triggered by that event regardless of what caused that event to occur. For example, if you attach a program to the syscall for opening files, it will be triggered whenever any process tries to open a file. It doesn't matter whether that process was already running when the program was loaded. This is a huge advantage compared to upgrading the

kernel and then having to reboot the machine to use its new functionality.

eBPF程序可以动态地加载到内核并被移除。一旦它们附加到一个事件上，无论是什么引起了该事件的发生，它们都会被触发。例如，如果你将一个程序附加到打开文件的系统调用上，它将在任何进程尝试打开文件时被触发。无论这个进程在程序加载时是否已经运行，都无关紧要。这与升级内核然后不得不重新启动机器来使用新功能相比，是一个巨大的优势。

This leads to one of the great strengths of observability or security tooling that uses eBPF—it instantly gets visibility over everything that’s happening on the machine. In environments running containers, that includes visibility over all processes running inside those containers as well as on the host machine. I’ll dig into the consequences of this for cloud native deployments later in this chapter.

使用eBPF的观察性或安全工具之一的重要优点是可以立即对机器上发生的一切进行监控。在使用容器的环境中，这包括对容器内部以及主机上运行的所有进程的可见性。我将在本章后面详细探讨这对云原生部署的影响。

Additionally, as illustrated in [Figure 1-3](#), people can create new kernel functionality very quickly through eBPF without requiring every other Linux user to accept the same changes.

通过eBPF，人们可以迅速创建新的内核功能，而无需要求其他Linux用户接受相同的更改（如图1-3所示）。

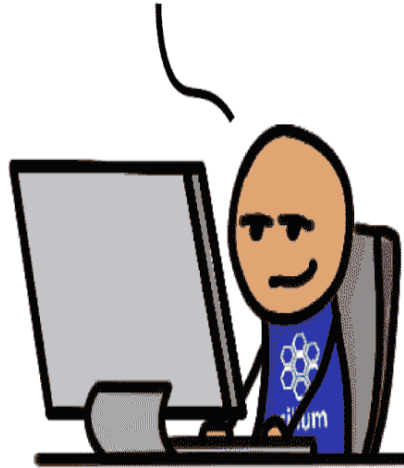
Application Developer:

i want this new feature
to observe my app



eBPF Developer:

OK! The kernel can't do this so let
me quickly solve this with eBPF.



A couple of days later...

Here is a release of our eBPF project that has this feature
now. BTW, you don't have to reboot your machine.

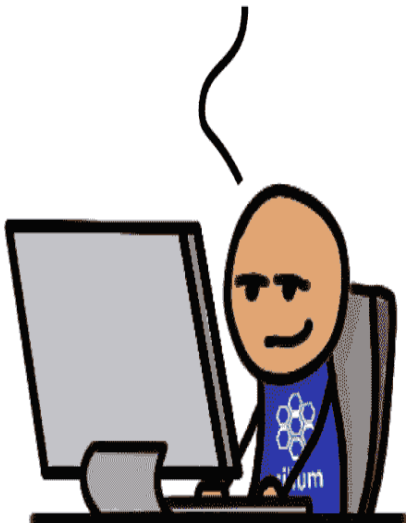


Figure 1-3. Adding kernel features with eBPF (cartoon by Vadim Shchekoldin, Isovalent)

High Performance of eBPF Programs

eBPF程序的高性能

eBPF programs are a very efficient way to add instrumentation.

Once loaded and JIT-compiled (which you'll see in [Chapter 3](#)), the program runs as native machine instructions on the CPU.

Additionally, there's no need to incur the cost of transitioning between kernel and user space (which is an expensive operation) to handle each event.

eBPF程序是一种非常高效的添加仪器的方式。一旦加载并进行即时编译（在第三章中介绍），该程序将作为本地机器指令在CPU上运行。此外，无需承担在内核和用户空间之间进行切换的开销（这是一项昂贵的操作），以处理每个事件。

The 2018 paper¹⁰ that describes the eXpress Data Path (XDP) includes some illustrations of the kinds of performance improvements eBPF enables in networking. For example, implementing routing in XDP “improves performance with a factor of 2.5” compared to the regular Linux kernel implementation, and “XDP offers a performance gain of 4.3x over IPVS” for load balancing.

2018年的论文描述了eXpress Data Path（XDP），其中包含了一些示意图，展示了eBPF在网络方面所能带来的性能改进。例如，在

XDP中实现路由与普通Linux内核实现相比，性能提升了2.5倍，“XDP在负载均衡方面比IPVS提供了4.3倍的性能增益”。

For performance tracing and security observability, another advantage of eBPF is that relevant events can be filtered within the kernel before incurring the costs of sending them to user space. Filtering only certain network packets was, after all, the point of the original BPF implementation. Today eBPF programs can collect information about all manner of events across a system, and they can use complex, customized programmatic filters to send only the relevant subset of information to user space.

在性能跟踪和安全可观察性方面，eBPF的另一个优点是可以在内核中对相关事件进行过滤，而不需要将它们发送到用户空间后再进行成本计算。毕竟，只过滤特定的网络数据包是原始BPF实现的初衷。如今，eBPF程序可以收集系统中各种事件的信息，并使用复杂的、定制的程序化过滤器，将只有相关子集信息发送到用户空间。

eBPF in Cloud Native Environments

在云原生环境中的eBPF

These days lots of organizations choose not to run applications by executing programs directly on servers. Instead, many use cloud native approaches: containers, orchestrators such as Kubernetes or ECS, or serverless approaches like Lambda, cloud functions, Fargate, and so on. These approaches all use automation to choose

the server where each workload will run; in serverless, we're not even aware what server is running each workload.

这些天，许多组织选择不直接在服务器上执行程序来运行应用程序。相反，许多人采用云原生方法：容器、编排工具如Kubernetes或ECS，或者使用Lambda、云函数、Fargate等无服务器方法。所有这些方法都使用自动化来选择每个工作负载将在哪个服务器上运行；在无服务器中，我们甚至不知道每个工作负载运行在哪个服务器上。

Nevertheless, there are servers involved, and each of those servers (whether it's a virtual machine or bare-metal machine) runs a kernel. Where applications run in a container, if they're running on the same (virtual) machine, they share the same kernel. In a Kubernetes environment, this means all the containers in all the pods on a given node are using the same kernel. When we instrument that kernel with eBPF programs, all the containerized workloads on that node are visible to those eBPF programs, as illustrated in [Figure 1-4](#).

然而，涉及到服务器，每个服务器（无论是虚拟机还是物理机器）都运行着一个内核。当应用程序运行在一个容器中时，如果它们运行在同一（虚拟）机器上，它们共享同一个内核。在Kubernetes环境中，这意味着在一个特定节点上的所有容器中的所有Pod都使用相同的内核。当我们用eBPF程序装设这个内核时，所有这个节点上的容器化工作负载对这些eBPF程序都是可见的，如图 1-4 所示。

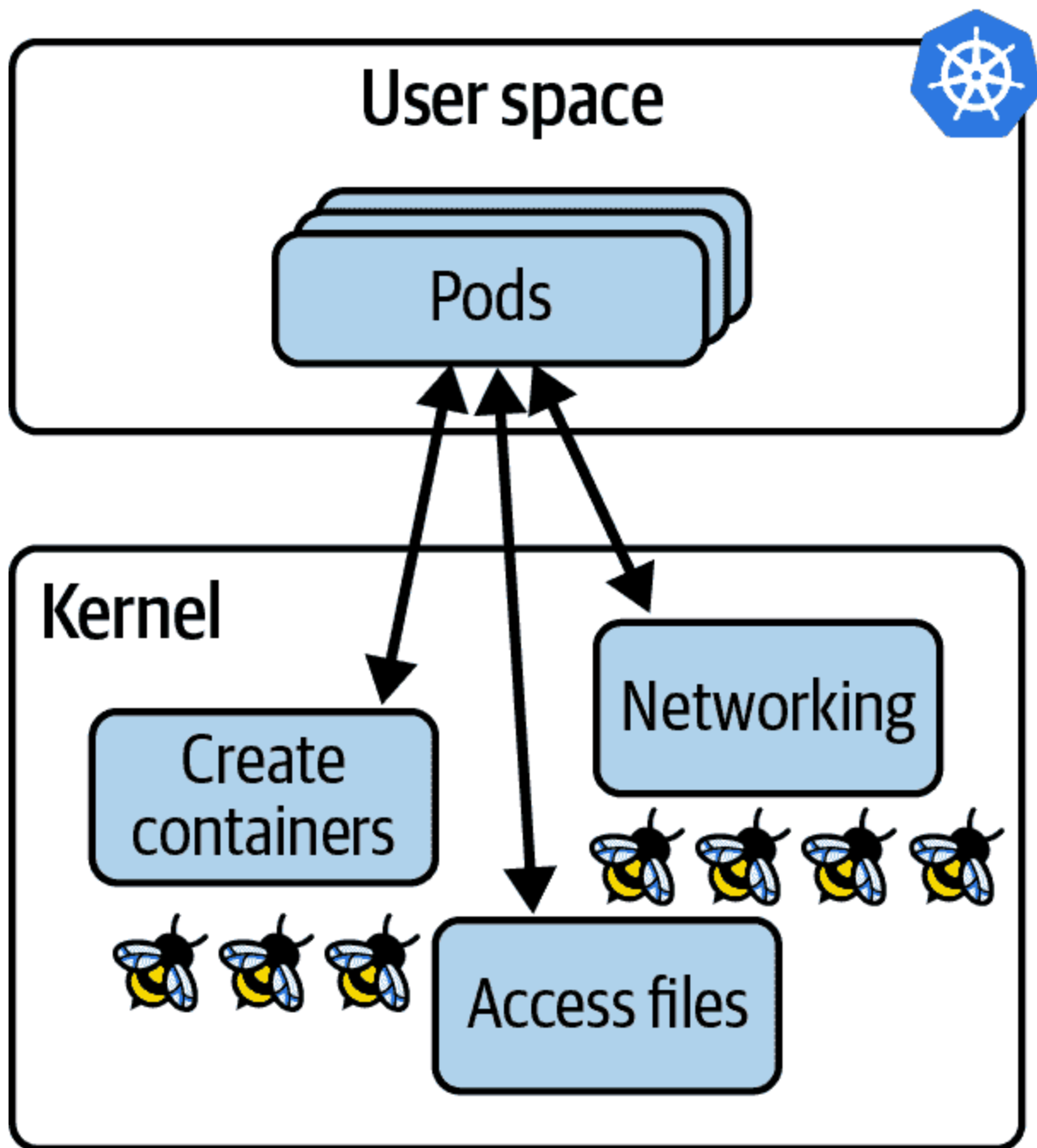


Figure 1-4. eBPF programs in the kernel have visibility of all applications running on a Kubernetes node

Visibility of all the processes on the node, combined with the ability to load eBPF programs dynamically, gives us the real superpowers

of eBPF-based tooling in cloud native computing:

节点上所有进程的可见性，结合动态加载eBPF程序的能力，使得eBPF基于云原生计算工具具备了真正的超级能力。

- We don't need to change our applications, or even the way they are configured, to instrument them with eBPF tooling.

我们不需要改变我们的应用程序，甚至不需要改变它们的配置，就可以使用eBPF工具对其进行追踪。

- As soon as it's loaded into the kernel and attached to an event, an eBPF program can start observing preexisting application processes.

一旦被加载到内核并附加到一个事件上，一个eBPF程序就可以开始观察已经存在的应用程序进程。

Contrast this with the *sidecar model*, which has been used to add functionality like logging, tracing, security, and service mesh functionality into Kubernetes apps. In the sidecar approach, the instrumentation runs as a container that is “injected” into each application pod. This process involves modifying the YAML that defines the application pods, adding in the definition of the sidecar container. This approach is certainly more convenient than adding the instrumentation into the source code of the application (which is what we had to do before the sidecar approach; for example, including a logging library in our application and making calls into that library at appropriate points in the code). Nevertheless, the sidecar approach has a few downsides:

对比一下与sidecar模型，该模型已被用于将日志记录、追踪、安全性和服务网格功能添加到Kubernetes应用程序中。在sidecar方法中，仪表化作为一个容器运行，它被「注入」到每个应用程序容器中。这个过程涉及修改定义应用程序容器的YAML文件，添加sidecar容器的定义。这种方法肯定比将仪表化添加到应用程序源代码中方便得多（这是我们在采用sidecar方法之前必须做的事情；例如，在我们的应用程序中包含一个日志记录库，并在代码中适当的位置调用该库）。然而，sidecar方法有一些缺点：

- The application pod has to be restarted for the sidecar to be added.

应用程序 Pod 必须重新启动才能添加 Sidecar。

- Something has to modify the application YAML. This is generally an automated process, but if something goes wrong, the sidecar won't be added, which means the pod doesn't get instrumented. For example, a deployment might be annotated to indicate that an admission controller should add the sidecar YAML to the pod spec for that deployment. But if the deployment isn't labeled correctly, the sidecar won't get added, and it's therefore not visible to the instrumentation.

必须修改应用程序的YAML文件。一般来说，这是一个自动化的过程，但如果出现问题，就无法添加sidecar，这意味着无法对pod进行仪表化。例如，一个部署可能会被注释以指示准入控制器应该将sidecar的YAML文件添加到该部署的pod规范中。但是，如果部署没有正确标记，sidecar就不会被添加，因此对仪表化来说是不可见的。

- When there are multiple containers within a pod, they might reach readiness at different times, the ordering of which may not be predictable. Pod start-up time can be significantly slowed by the injection of sidecars, or worse, it can cause race conditions or other instabilities. For example, the [Open Service Mesh documentation](#) describes how application containers have to be resilient to all traffic being dropped until the Envoy proxy container is ready.

当一个 pod 中存在多个容器时，它们可能以不同的时间达到准备状态，其顺序可能不可预测。通过注入 sidecar 可能会极大地降低 pod 的启动时间，或者更糟糕的是，可能会引起竞争条件或其他不稳定性。例如，Open Service Mesh 的文档描述了应用容器必须对所有流量被丢弃直到 Envoy 代理容器准备就绪的情况下保持可靠性。

- Where networking functionality such as service mesh is implemented as a sidecar, it necessarily means that all traffic to and from the application container has to travel through the network stack in the kernel to reach a network proxy container, adding latency to that traffic; this is illustrated in [Figure 1-5](#). We'll talk about improving network efficiency with eBPF in [Chapter 9](#).

当网络功能（如服务网格）作为一个边车实施时，这必然意味着所有进出应用容器的流量都必须通过内核中的网络堆栈传输到网络代理容器，从而增加了流量的延迟。这在图1-5中有所说明。我们将在第9章中讨论如何用eBPF提高网络效率。

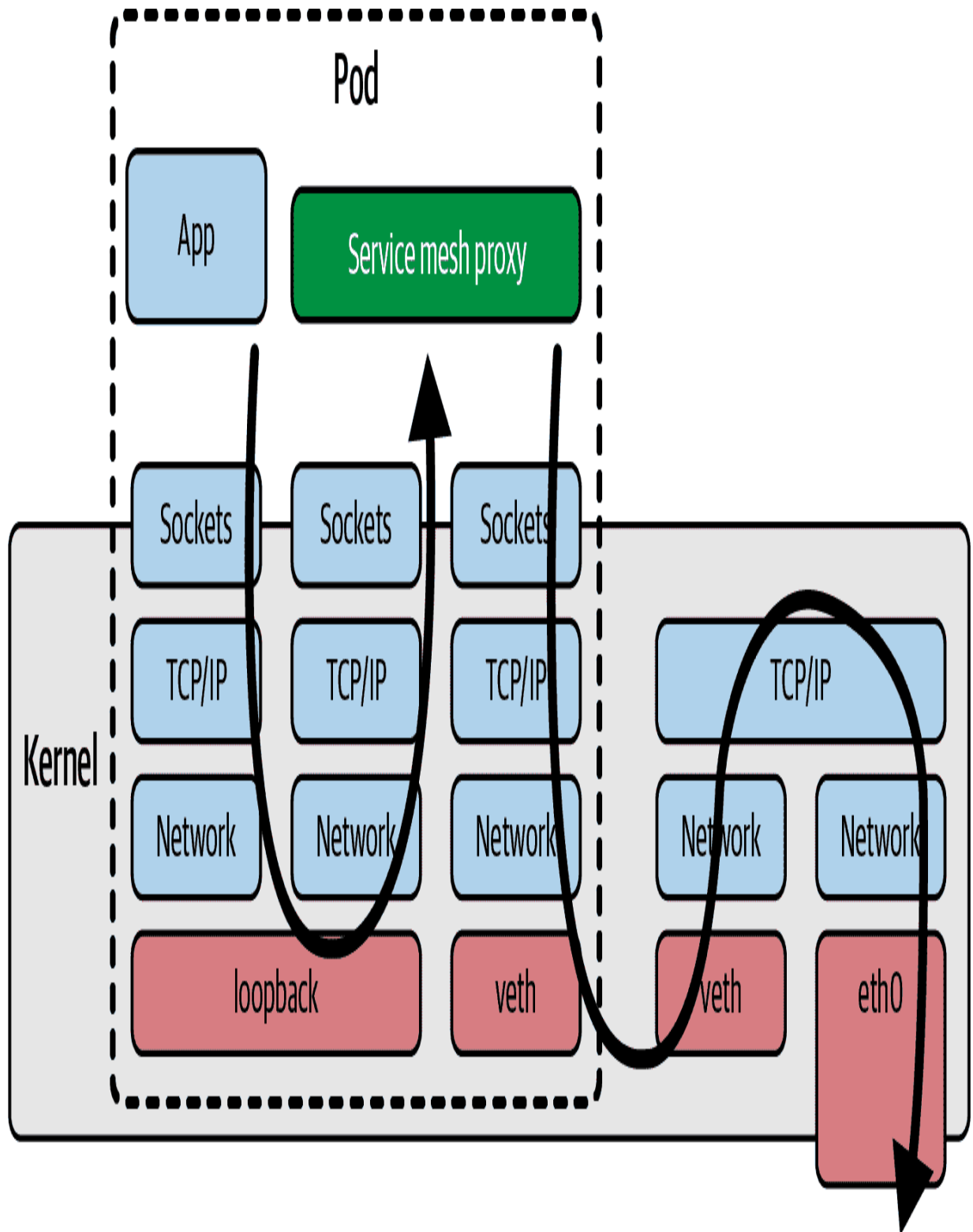


Figure 1-5. Path of a network packet using a service mesh proxy sidecar container

All these issues are inherent problems with the sidecar model. Fortunately, now that eBPF is available as a platform, we have a new model that can avoid these issues. Additionally, because eBPF-based tools can see everything that's happening on a (virtual) machine, they are harder for bad actors to sidestep. For example, if an attacker manages to deploy a cryptocurrency mining app on one of your hosts, they probably won't do you the courtesy of instrumenting it with the sidecars you're using on your application workloads. If you're relying on a sidecar-based security tool to prevent apps from making unexpected network connections, that tool isn't going to spot the mining app connecting to its mining pool if the sidecar isn't injected. In contrast, network security implemented in eBPF can police all traffic on the host machine, so this cryptocurrency mining operation could easily be stopped. The ability to drop network packets for security reasons is something we'll come back to in [Chapter 8](#).

所有这些问题都是旁路模式固有的问题。幸运的是，现在eBPF可用作平台，我们有了一个可以避免这些问题的新模型。此外，由于基于eBPF的工具可以看到（虚拟）机器上发生的一切，它们对于不良行为者来说更难规避。例如，如果攻击者设法在您的主机之一上部署了一个加密货币挖矿应用程序，他们可能不会客气地在您的应用工作负载上使用旁路工具进行仪表化。如果您依赖基于旁路的安全工具来防止应用程序进行意外的网络连接，如果没有注入旁路，该工具将无法发现挖矿应用程序连接到其挖矿池。相比之下，使用eBPF实施的网络安全

全可以监控主机机器上的所有流量，因此可以轻松停止此加密货币挖矿操作。我们将在第8章详细讨论出于安全原因丢弃网络数据包的能力。

Summary

摘要

I hope this chapter has given you some insight into why eBPF as a platform is so powerful. It allows us to change the behavior of the kernel, providing us the flexibility to build bespoke tools or customized policies. eBPF-based tools can observe any event across the kernel, and hence across all applications running on a (virtual) machine, whether they are containerized or not. eBPF programs can also be deployed dynamically, allowing behavior to be changed on the fly.

我希望这一章给你带来了>eBPF作为一个平台为何如此强大的一些见解。它允许我们改变内核的行为，为我们提供了构建定制工具或定制策略的灵活性。基于eBPF的工具可以观察内核中的任何事件，因此也可以观察到在（虚拟）机器上运行的所有应用程序，无论它们是否容器化。eBPF程序还可以动态部署，允许在运行过程中改变行为。

So far we've discussed eBPF at a relatively conceptual level. In the next chapter we'll make it more concrete and explore the constituent parts of an eBPF-based application.

到目前为止，我们在相对概念层面上讨论了eBPF。在下一章中，我们将更具体地探索基于eBPF的应用程序的组成部分。

¹ “The BSD Packet Filter: A New Architecture for User-level Packet Capture” by Steven McCanne and Van Jacobson.

“BSD数据包过滤器：一种用于用户级数据包捕获的新架构” 由Steven McCanne和Van Jacobson著。

² These and other details come from Alexei Starovoitov’s 2015 NetDev presentation, “BPF – in-kernel virtual machine”.

这些和其他细节来自Alexei Starovoitov于2015年的NetDev演讲，“BPF - 内核虚拟机”。

³ There is a good description of how kprobes work in [the kernel documentation](#).

内核文档中有一个很好的描述，说明了kprobes如何工作。

⁴ This wonderful fact comes from Daniel Borkmann’s KubeCon 2020 talk titled “eBPF and Kubernetes: Little Helper Minions for Scaling Microservices”.

这个美妙事实来自于Daniel Borkmann在KubeCon 2020上的演讲，题为“eBPF和Kubernetes：用于扩展微服务的小助手”。

⁵ For more details on the instruction limit and “complexity limit,” see <https://oreil.ly/0iVer>.

有关指令限制和“复杂性限制”的更多详细信息，请参阅<https://oreil.ly/0iVer>。

⁶ Extract from “What Is eBPF?” by Liz Rice. Copyright © 2022 O’Reilly Media. Used with permission.

《什么是eBPF? 》一文摘录，作者为Liz Rice。版权所有©2022 O'Reilly Media。经授权使用。

⁷ [“Linux 5.12 Coming In At Around 28.8 Million Lines”](#). Phoronix, March 2021.

Linux 5.12的代码量大约为28.8百万行。——来自《Phoronix》，2021年3月。

⁸ Jiang Y, Adams B, German DM. 2013. [“Will My Patch Make It? And How Fast?”](#) (2013). According to this research paper, 33% of patches are accepted, and most take three to six months.

根据这篇研究论文，有33%的补丁被接受，而且大部分的补丁需要三到六个月的时间。

⁹ Thankfully, security patches to existing functionality are made available more quickly.

幸运的是，对现有功能的安全补丁可以更快地提供。

¹⁰ Høiland-Jørgensen T, Brouer JD, Borkmann D, et al. [“The eXpress data path: fast programmable packet processing in the operating system kernel”](#). *Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '18)*. Association for Computing Machinery; 2018:54–66.

Høiland-Jørgensen T, Brouer JD, Borkmann D等。“eXpress数据路径：操作系统内核中的快速可编程数据包处理”。第14届新兴网络实验与技术国际会议论文集（CoNEXT '18）。计算机协会; 2018:54-66.

Chapter 2. eBPF's “Hello World”

第二章 eBPF的“Hello World”

In the previous chapter I discussed why eBPF is so powerful, but it's OK if you don't yet feel you have a concrete grasp of what it really means to run eBPF programs. In this chapter I'll use a simple “Hello World” example to give you a better feel for it.

在上一章节中，我讨论了为什么eBPF如此强大，但是如果你还没有对运行eBPF程序真正理解的感觉，也没有关系。在本章中，我将使用一个简单的“Hello World”示例来帮助你更好地理解它。

As you'll learn while you read through this book, there are several different libraries and frameworks for writing eBPF applications. As a warm-up, I'll show you what is probably the most accessible approach from a programming point of view: the [BCC Python framework](#). This offers a very easy way to write basic eBPF programs. For reasons that I'll cover in [Chapter 5](#), it's not necessarily an approach I would recommend these days for production apps that you're intending to distribute to other users, but it's great for taking your first steps.

正如您在阅读本书时会了解到的那样，有多种不同的库和框架可以用于编写eBPF应用程序。作为一个热身，我将向您展示从编程角度来看可能最易于掌握的方法：使用BCC Python框架。这提供了一种非常简单的编写基本eBPF程序的方式。出于我将在第5章中介绍的原因，它

并不一定是我现在推荐用于生产应用程序并分发给其他用户的方法，但是它非常适合初学者入门。

Note

If you want to try this code for yourself, it is available at <https://github.com/lizrice/learning-ebpf> in the *chapter2* directory.

如果你想自己尝试这段代码，可以在 <https://github.com/lizrice/learning-ebpf> 的 *chapter2* 目录中找到。

You'll find the BCC project at <https://github.com/iovisor/bcc>, and the instructions for installing BCC are at <https://github.com/iovisor/bcc/blob/master/INSTALL.md>.

你可以在 <https://github.com/iovisor/bcc> 找到 BCC 项目，安装 BCC 的指南说明在 <https://github.com/iovisor/bcc/blob/master/INSTALL.md> 中。

BCC's “Hello World”

嗨，世界！

The following is the full source code of *hello.py*, an eBPF “Hello World” application¹ written using BCC's Python library:

以下是 *hello.py* 的完整源代码，这是一个使用 BCC 的 Python 库编写的 eBPF “Hello World” 应用程序。

```
#!/usr/bin/python
from bcc import BPF

program = r"""
int hello(void *ctx) {
```

```

    bpf_trace_printk("Hello World!");
    return 0;
}
"""

b = BPF(text=program)
syscall = b.get_syscall_fnname("execve")
b.attach_kprobe(event=syscall, fn_name="hello")

b.trace_print()

```

This code consists of two parts: the eBPF program itself that will run in the kernel, and some user space code that loads the eBPF program into the kernel and reads out the trace that it generates. As you can see in [Figure 2-1](#), *hello.py* is the user space part of this application, and `hello()` is the eBPF program that runs in the kernel.

这段代码由两部分组成：eBPF程序本身在内核中运行，还有一些用户空间代码将eBPF程序加载到内核并读取其生成的跟踪信息。如图2-1所示，*hello.py*是这个应用程序的用户空间部分，`hello()`是在内核中运行的eBPF程序。

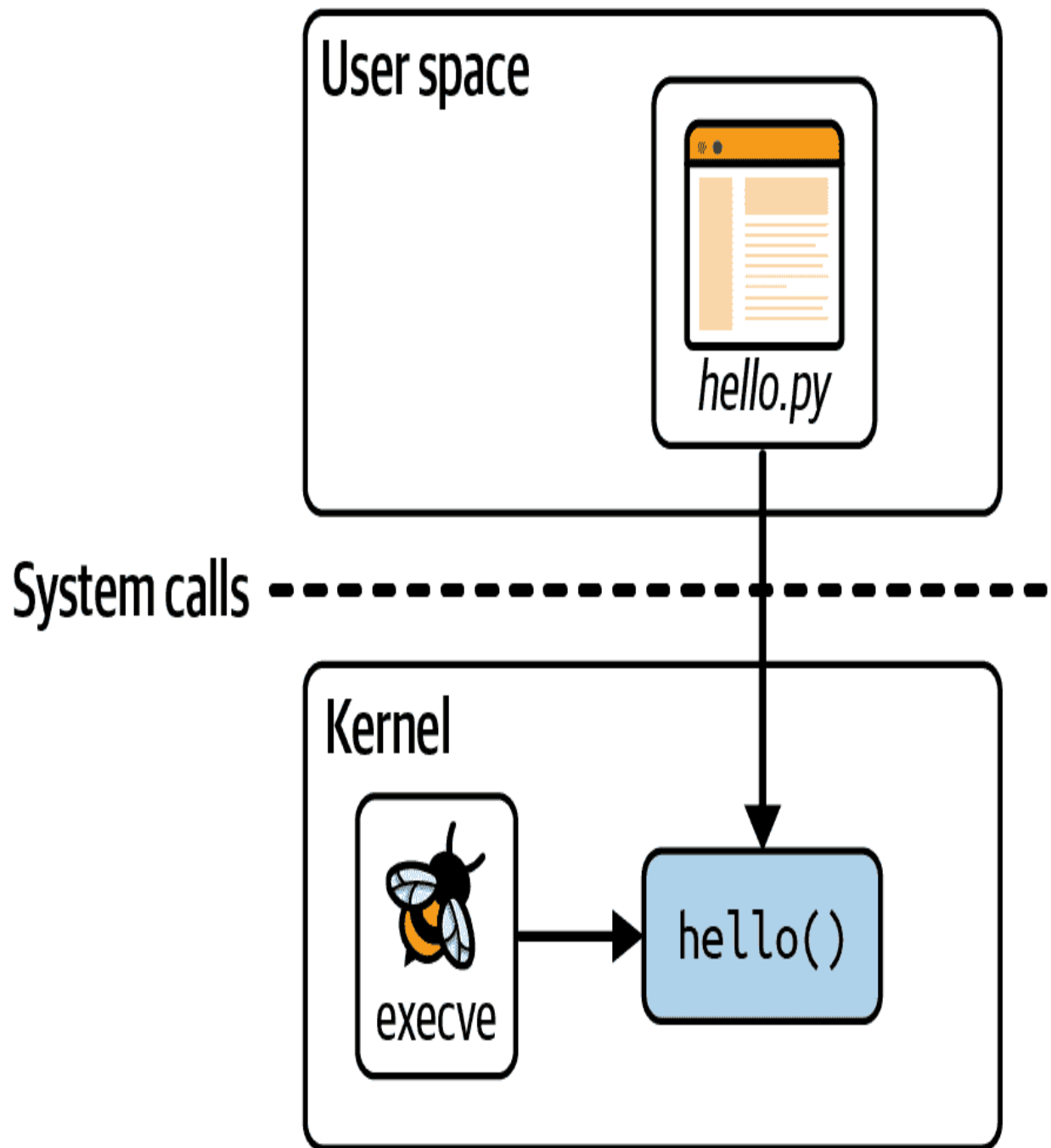


Figure 2-1. The user space and kernel components of “Hello World”

Let's dig into each line of the source code to understand it better.

让我们深入源代码的每一行来更好地理解它。

The first line tells you this is Python code, and the program that can run it is the Python interpreter (*/usr/bin/python*).

第一行告诉您这是Python代码，能够运行它的程序是Python解释器(*/usr/bin/python*)。

The eBPF program itself is written in C code, and it's this part:

eBPF程序本身是用C代码编写的，这部分是：

```
int hello(void *ctx) {
    bpf_trace_printk("Hello World!");
    return 0;
}
```

All the eBPF program does is use a helper function, `bpf_trace_printk()`, to write a message. Helper functions are another feature that distinguishes “extended” BPF from its “classic” predecessor. They are a set of functions that eBPF programs can call to interact with the system; I’ll discuss them further in [Chapter 5](#). For now you can just think of this as printing a line of text.

所有eBPF程序所做的只是使用一个辅助函数，即**`bpf_trace_printk()`**，来写入一条消息。辅助函数是区分“扩展”BPF与其“经典”前身的另一个特性。它们是一组eBPF程序可以调用以与系统交互的函数；我将在第5章进一步讨论它们。现在你可以将其简单地理解为打印一行文本。

The entire eBPF program is defined as a string called `program` in the Python code. This C program needs to be compiled before it can be executed, but BCC takes care of that for you. (You’ll see how to compile eBPF programs yourself in the next chapter.) All you need to

do is pass this string as a parameter when creating a BPF object, as in the following line:

整个eBPF程序在Python代码中被定义为一个名为program的字符串。这个C程序在执行之前需要被编译，但是BCC会为你处理这个过程（在下一章中你将会看到如何自己编译eBPF程序）。你只需要将这个字符串作为参数传递给创建BPF对象的时候，就像下面的这行代码一样。

```
b = BPF(text=program)
```

eBPF programs need to be attached to an event, and for this example I've chosen to attach to the system call `execve`, which is the syscall used to execute a program. Whenever anything or anyone starts a new program executing on this machine, that will call `execve()`, which will trigger the eBPF program. Although the “execve()” name is a standard interface in Linux, the name of the function that implements it in the kernel depends on the chip architecture, but BCC gives us a convenient way to look up the function name for the machine we're running on:

eBPF程序需要附加到一个事件上，对于这个例子，我选择了附加到系统调用execve上。execve是用于执行程序的系统调用。每当在这台机器上启动一个新程序时，都会调用execve()，从而触发eBPF程序。虽然“execve()”在Linux中是一个标准接口，但在内核中实现它的函数的名称取决于芯片架构，但是BCC为我们提供了一种方便的方法来查找正在运行的机器上的函数名称。

```
syscall = b.get_syscall_fname("execve")
```


Now, `syscall` represents the name of the kernel function I'm going to attach to, using a kprobe (you were introduced to the concept of kprobes in [Chapter 1](#)).² You can attach the `hello` function to that event, like this:

现在，`syscall`表示我将要连接的内核函数的名称，使用kprobe（你在第1章中了解到了kprobe的概念）。您可以将`hello`函数附加到该事件上，如下所示：

```
b.attach_kprobe(event=syscall, fn_name="hello")
```

At this point, the eBPF program is loaded into the kernel and attached to an event, so the program will be triggered whenever a new executable gets launched on the machine. All that's left to do in the Python code is to read the tracing that is output by the kernel and write it on the screen:

在这一点上，eBPF程序已加载到内核并附加到一个事件上，因此每当机器上启动一个新的可执行文件时，程序就会被触发。在Python代码中只需要读取内核输出的跟踪信息并将其显示在屏幕上就可以了。

```
b.trace_print()
```

This `trace_print()` function will loop indefinitely (until you stop the program, perhaps with Ctrl+C), displaying any trace.

这个`trace_print()`函数将无限循环（直到你停止程序，可能需要使用Ctrl+C），显示任何跟踪信息。

[Figure 2-2](#) illustrates this code. The Python program compiles the C code, loads it into the kernel, and attaches it to the `execve` syscall

kprobe. Whenever any application on this (virtual) machine calls `execve()`, it triggers the eBPF `hello()` program, which writes a line of trace into a specific pseudofile. (I'll cover where that pseudofile is later in this chapter.) The Python program reads the trace message from the pseudofile and displays it to the user.

图2-2说明了这段代码。Python程序编译C代码，将其加载到内核并将其附加到`execve`系统调用的kprobe。每当该（虚拟）机器上的任何应用程序调用`execve()`时，它都会触发eBPF `hello()`程序，该程序将一行跟踪写入特定的伪文件中（稍后在本章中我将介绍这个伪文件的位置）。Python程序从伪文件中读取跟踪消息，并将其显示给用户。

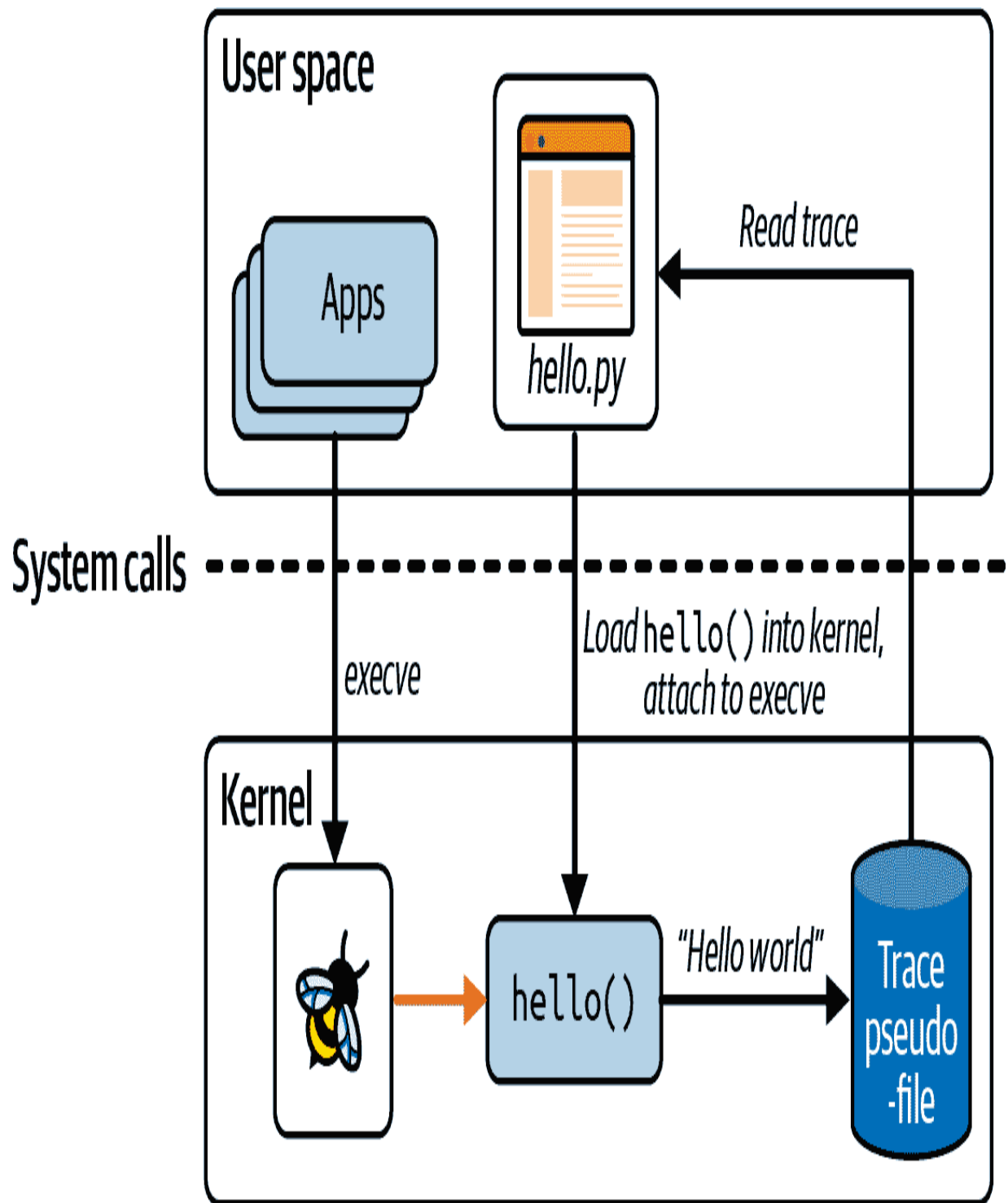


Figure 2-2. “Hello World” in operation

Running “Hello World”

运行“你好，世界”

Run this program, and depending on what is happening on the (virtual) machine you're using, you might see tracing being generated straightaway, because other processes could be executing programs³ with the `execve` syscall. If you don't see anything, open a second terminal and execute any commands you like,⁴ and you'll see the corresponding trace generated by “Hello World”:

运行此程序，根据您所使用的（虚拟）机器上发生的情况，您可能会立即看到生成的跟踪信息，因为其他进程可能正在执行使用`execve`系统调用的程序。如果您没有看到任何内容，请打开第二个终端并执行任何命令，您将看到生成的“Hello World”相应的跟踪信息。

```
$ hello.py
b'      bash-5412      [001] .... 90432.904952: 0:
bpf_trace_printk: Hello World'
```

Note

Since eBPF is so powerful, it requires special privileges to use it. Privileges are automatically assigned to the root user, so the easiest way to run eBPF programs is as root, perhaps by using `sudo`. For clarity I won't include `sudo` in the example commands in this book, but if you ever see an “Operation not permitted” error, the first thing to check is whether you're trying to run eBPF programs as an unprivileged user.

由于eBPF非常强大，因此需要特殊权限来使用它。特权会自动分配给root用户，所以以root用户运行eBPF程序是最简单的方式，可以通过使用sudo命令来实现。为了清晰起见，在本书的示例命令中我不会包含sudo，但是如果你遇到“操作不允许”的错误，首先要检查是否以非特权用户身份运行eBPF程序。

`CAP_BPF` was introduced in kernel version 5.8, and it gives sufficient privilege to perform some eBPF operations like creating certain types of map. However, you will probably need additional capabilities:

CAP_BPF是在5.8内核版本中引入的，它提供了足够的特权来执行一些eBPF操作，例如创建某些类型的映射。然而，您可能需要额外的功能：

- `CAP_PERFMON` and `CAP_BPF` are both required to load tracing programs.

CAP_PERFMON和CAP_BPF都是加载跟踪程序所需的。

- `CAP_NET_ADMIN` and `CAP_BPF` are both required for loading networking programs.

CAP_NET_ADMIN和CAP_BPF都是加载网络程序所需的。

There is a lot more detail on this in the blog post [“Introduction to CAP_BPF”](#) by Milan Landaverde.

在米兰·兰达贝德的博客文章“CAP_BPF简介”中有更详细的内容。

As soon as the *hello* eBPF program is loaded and attached to an event, it gets triggered by events that are being generated from preexisting processes. This should reinforce a couple of points that you learned in [Chapter 1](#):

一旦加载并附加到事件的hello eBPF程序，它就会被已有进程产生的事件所触发。这应该会强调一些你在第一章学到的内容。

- eBPF programs can be used to dynamically change the behavior of the system. There's no need to reboot the machine or restart existing processes. eBPF code starts taking effect as soon as it is attached to an event.

eBPF程序可以用于动态改变系统的行为。无需重新启动机器或重启现有进程。一旦eBPF代码附加到事件上，它就会立即生效。

- There's no need to change anything about other applications for them to be visible to eBPF. Wherever you have terminal access on that machine, if you run an executable in it, that will use the `execve()` syscall, and if you have the *hello* program attached to that syscall, it will be triggered to generate tracing output.

Likewise, if you have a script that runs executables, that will also trigger the *hello* eBPF program. You don't need to change anything about the terminal's shell, the script, or the executables you're running.

没有必要更改其他应用程序的任何内容，以使它们对eBPF可见。无论您在该机器上的终端访问到哪个位置，如果您在其中运行可执行文件，将使用execve()系统调用，并且如果您将hello程序附加到

该系统调用上，将会触发该程序以生成跟踪输出。同样，如果您有一个运行可执行文件的脚本，也会触发hello eBPF程序。您无需更改终端的Shell、脚本或要运行的可执行文件的任何内容。

The trace output shows not only the `"Hello World"` string, but also some additional contextual information about the event that triggered the *hello* eBPF program to run. In the example output shown at the beginning of this section, the process that made the `execve` system call had a process ID of 5412, and it was running the command `bash`. For trace messages, this contextual information is added as part of the kernel tracing infrastructure (which isn't specific to eBPF), but as you'll see later in this chapter, it's also possible to retrieve contextual information like this within the eBPF program itself.

跟踪输出不仅显示了“Hello World”字符串，还显示了触发hello eBPF程序运行的事件的一些附加上下文信息。在本节开始时显示的示例输出中，执行execve系统调用的进程的进程ID为5412，并且正在运行bash命令。对于跟踪消息，这些上下文信息作为内核跟踪基础设施的一部分添加（这与eBPF无关），但正如您将在本章后面看到的，还可以在eBPF程序内部检索类似的上下文信息。

You might be wondering how the Python code knows where to read the tracing output from. The answer is not very sophisticated—the `bpf_trace_printk()` helper function in the kernel always sends output to the same predefined pseudofile location: `/sys/kernel/debug/tracing/trace_pipe`. You can confirm this by using `cat` to view its contents; you'll need root privileges to access it.

你可能想知道Python代码如何知道从哪里读取跟踪输出。答案并不复杂-内核中的**bpf_trace_printk()**辅助函数总是将输出发送到同一个预定义的伪文件位置：`/sys/kernel/debug/tracing/trace_pipe`。您可以使用**cat**命令查看其内容以确认此问题；访问它需要**root**权限。

A single trace pipe location is fine for a simple “Hello World” example or for basic debugging purposes, but it’s very limited. There is very little flexibility in the format of the output, and it only supports the output of strings, so it’s not terribly useful for passing structured information. Perhaps most importantly, there is just this one location on the (virtual) machine. If you had multiple eBPF programs running simultaneously, they would all write trace output to the same trace pipe, which could get very confusing for a human operator.

仅一个追踪管道位置对于一个简单的“Hello World”例子或基本的调试目的来说是可以的，但这非常有限。输出的格式几乎没有灵活性，只支持字符串的输出，因此在传递结构化信息方面并不是非常有用。可能最重要的是，在（虚拟）机器上只有这一个位置。如果同时运行多个eBPF程序，它们都会将追踪输出写入同一个追踪管道，这可能对于操作人员来说非常混乱。

There’s a much better way to get information out of an eBPF program: use an eBPF map.

有一种更好的方法可以从一个eBPF程序中获取信息：使用一个eBPF映射。

BPF Maps

BPF地图

A *map* is a data structure that can be accessed from an eBPF program and from user space. Maps are one of the really significant features that distinguish extended BPF from its classic predecessor. (You might think this would mean they are commonly referred to as “eBPF maps,” but you’ll frequently see “BPF maps.” As is generally the case, both terms are used interchangeably.)

地图是一种数据结构，可以从eBPF程序和用户空间访问。地图是区分扩展BPF和经典前身的非常重要的功能之一（你可能会认为这意味着它们通常被称为“eBPF地图”，但你经常会看到“BPF地图”。通常情况下，两个术语可以互换使用）。

Maps can be used to share data among multiple eBPF programs or to communicate between a user space application and eBPF code running in the kernel. Typical uses include the following:

地图可用于在多个eBPF程序之间共享数据，或在用户空间应用程序和运行在内核中的eBPF代码之间进行通信。典型用途包括以下内容：

- User space writing configuration information to be retrieved by an eBPF program

用户空间将配置信息写入以供eBPF程序检索

- An eBPF program storing state, for later retrieval by another eBPF program (or a future run of the same program)

一个eBPF程序存储状态，以供另一个eBPF程序（或将来的运行相同的程序）检索。

- An eBPF program writing results or metrics into a map, for retrieval by the user space app that will present results

一个 eBPF 程序将结果或指标写入一个映射中，以使用户空间应用程序检索和展示结果。

There are various types of BPF maps defined in Linux's [uapi/linux/bpf.h file](#), and there is some information about them in the [kernel docs](#). In general they are all key–value stores, and in this chapter you'll see examples of maps for hash tables, perf and ring buffers, and arrays of eBPF programs.

在Linux的uapi/linux/bpf.h文件中定义了各种类型的BPF映射，并且内核文档中包含了一些关于它们的信息。总的来说，它们都是键-值存储，本章节将展示哈希表、perf和环形缓冲区以及eBPF程序数组的映射示例。

Some map types are defined as arrays, which always have a 4-byte index as the key type; other maps are hash tables that can use some arbitrary data type as the key.

一些地图类型被定义为数组，其中键值始终为4字节索引数据类型；其他地图则是哈希表，可以使用任何数据类型作为键值。

There are map types that are optimized for particular types of operations, such as [first-in-first-out queues](#), [first-in-last-out stacks](#), [least-recently-used data storage](#), [longest-prefix matching](#), and [Bloom filters](#) (a probabilistic data structure designed to provide very fast results on whether an element exists).

有针对特定操作进行优化的地图类型，如先进先出队列、后进先出堆栈、最近最少使用的数据存储、最长前缀匹配和Bloom过滤器（一种概率数据结构，旨在提供非常快速的元素存在性结果）。

Some eBPF map types hold information about specific types of objects. For example, [sockmaps](#) and [devmaps](#) hold information about sockets and network devices and are used by network-related eBPF programs to redirect traffic. A program array map stores a set of indexed eBPF programs, and (as you'll see later in this chapter) this is used to implement tail calls, where one program can call another. There's even a [map-of-maps type](#) to support storing information about maps.

一些eBPF地图类型保存特定类型对象的信息。例如，sockmaps和devmaps保存关于套接字和网络设备的信息，并被网络相关的eBPF程序用于重定向流量。程序数组地图存储一组索引的eBPF程序，并且（正如您稍后在本章中将看到的）用于实现尾调用，其中一个程序可以调用另一个程序。甚至还有一个地图映射类型来支持存储有关地图的信息。

Some map types have per-CPU variants, which is to say that the kernel uses a different block of memory for each CPU core's version of that map. This might have you wondering about concurrency concerns for maps that are *not* per-CPU, where multiple CPU cores could be accessing the same map simultaneously. Spin lock support for (some) maps was added in kernel version 5.1, and we'll return to this subject in [Chapter 5](#).

有些地图类型有每个CPU的变体，也就是说内核对该地图的每个CPU核心版本使用不同的内存块。这可能会让你对非每个CPU的地图的并发问题产生疑问，多个CPU核心可以同时访问同一个地图。内核版本5.1中添加了对某些地图的自旋锁支持，我们将在第5章中回到这个主题。

The next example (*chapter2/hello-map.py* in the [GitHub repository](#)) shows some basic operations using a hash table map. It also demonstrates some of BCC's convenient abstractions that make it very easy to use maps.

下一个示例（GitHub存储库中的chapter2/hello-map.py）展示了一些使用哈希表映射的基本操作。它还展示了一些BCC方便的抽象，使得使用映射非常容易。

Hash Table Map

哈希表映射

Like the previous example in this chapter, this eBPF program will be attached to a kprobe at the entry to the `execve` system call. It's going to populate a hash table with key-value pairs, where the key is a user ID and the value is a counter for the number of times `execve` is called by a process running under that user ID. In practice, this example will show how many times each different user has run programs.

与本章的先前示例一样，这个eBPF程序将附加到执行execve系统调用的入口处的kprobe上。它将使用键值对填充一个哈希表，其中键是

用户ID，值是在该用户ID下运行的进程调用execve的次数的计数器。在实践中，该示例将显示每个不同用户运行程序的次数。

First, let's look at the C code for the eBPF program itself:

首先，让我们来看一下 eBPF 程序本身的 C 代码：

```
BPF_HASH(counter_table);  
  
int hello(void *ctx) {  
    u64 uid;  
  
    u64 counter = 0;  
    u64 *p;  
  
    uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;  
    p = counter_table.lookup(&uid);  
    if (p != 0) {  
        counter = *p;  
    }  
    counter++;  
  
    counter_table.update(&uid, &counter);  
    return 0;  
}
```

❶

`BPF_HASH()` is a BCC macro that defines a hash table map.

❷

`bpf_get_current_uid_gid()` is a helper function used to obtain the user ID that is running the process that triggered this kprobe event. The user ID is held in the lowest 32 bits of the 64-bit value

that gets returned. (The top 32 bits hold the group ID, but that part is masked out.)

③

Look for an entry in the hash table with a key matching the user ID. It returns a pointer to the corresponding value in the hash table.

④

If there is an entry for this user ID, set the `counter` variable to the current value in the hash table (pointed to by `p`). If there is no entry for this user ID in the hash table, the pointer will be `0`, and the counter value will be left at `0`.

⑤

Whatever the current counter value is, it gets incremented by one.

⑥

Update the hash table with the new counter value for this user ID.

Take a closer look at the lines of code that access the hash table:

仔细查看访问哈希表的代码行。

```
p = counter_table.lookup(&uid);
```

And later:

后来:

```
counter_table.update(&uid, &counter);
```

If you're thinking "that's not proper C code!" you're absolutely right. C doesn't support defining methods on structures like that.⁵ This is a great example where BCC's version of C is very loosely a C-like language that BCC rewrites before it sends the code to the compiler. BCC offers some convenient shortcuts and macros that it converts into "proper" C.

如果你觉得“这不是合适的C代码！”那你是完全正确的。C不支持像这样在结构体上定义方法。这是一个很好的例子，BCC的C版本非常松散地类似于C语言，BCC在将代码发送到编译器之前会对其进行重写。BCC提供了一些方便的快捷方式和宏，它将它们转换为“适当”的C代码。

Just like in the previous example, the C code is defined as a string called `program`. The program is compiled, loaded into the kernel, and attached to the `execve` kprobe, in exactly the same way as the previous "Hello World" example:

就像之前的例子一样，C代码被定义为一个名为program的字符串。程序被编译、加载到内核中，并以与之前的“Hello World”示例完全相同的方式附加到execve kprobe。

```
b = BPF(text=program)
syscall = b.get_syscall_fnname("execve")
b.attach_kprobe(event=syscall, fn_name="hello")
```

This time a little more work is required on the Python side to read the information out of the hash table:

这次在Python端需要更多的工作来从哈希表中读取信息。

```
while True:
    sleep(2)
    s = ""
    for k,v in b["counter_table"].items():
        s += f"ID {k.value}: {v.value}\t"
    print(s)
```

❶

This part of the code loops indefinitely, looking for output to display every two seconds.

❷

BCC automatically creates a Python object to represent the hash table. This code loops through any values and prints them to the screen.

When you run this example, you'll want a second terminal window where you can run some commands. Here's some example output I obtained, annotated on the right side with the commands I ran in another terminal:

当您运行此示例时，您将需要第二个终端窗口来运行一些命令。以下是我在另一个终端窗口中运行的命令的示例输出，右侧带有注释：当您运行此示例时，您将需要第二个终端窗口来运行一些命令。以下是我在另一个终端窗口中运行的命令的示例输出，右侧带有注释：

```
Terminal 1
$ ./hello-map.py

run something]
ID 501: 1
ID 501: 1
```

```
Terminal 2

[blank line(s) until I
ls
```


ID 501: 2		ls
ID 501: 3	ID 0: 1	sudo ls
ID 501: 4	ID 0: 1	ls
ID 501: 4	ID 0: 1	
ID 501: 5	ID 0: 2	sudo ls

This example generates a line of output every two seconds, whether anything has happened or not. At the end of this output, the hash table contains two entries:

这个例子每两秒生成一行输出，无论是否发生了任何事情。在输出的末尾，哈希表包含两个条目。

- `key=501, value=5`

键=501, 值=5

- `key=0, value=2`

键=0, 值=2

In the second terminal, I have the user ID of 501. Running the `ls` command with this user ID increments the `execve` counter. When I run `sudo ls`, this results in two calls to `execve`: one is the execution of `sudo`, under user ID 501; the other is the execution of `ls`, under root's user ID of 0.

在第二个终端上，我有一个用户ID为501。使用这个用户ID运行ls命令会增加execve计数器。当我运行sudo ls时，会导致两次execve调用：一次是以用户ID 501运行sudo，另一次是以root的用户ID 0运行ls。

In this example, I used a hash table to convey data from the eBPF program to user space. (I could also have used an array type of map

here, since the key was an integer; hash tables let you use an arbitrary type as the key.) Hash tables are very convenient when the data is naturally in key–value pairs, but the user space code has to keep polling the table on a regular basis. The Linux kernel already supported the [perf subsystem](#) for sending data from the kernel to user space, and eBPF includes support for using perf buffers and their successor, BPF ring buffers. Let’s take a look.

在此示例中，我使用哈希表将数据从eBPF程序传递到用户空间。（在这里，我也可以使用数组类型的映射，因为键是整数；哈希表允许您使用任意类型作为键。）当数据自然地以键值对形式存在时，哈希表非常方便，但是用户空间代码必须定期轮询该表。Linux内核已经支持将数据从内核发送到用户空间的perf子系统，并且eBPF包括使用perf缓冲区及其后续BPF环形缓冲区的支持。让我们来看一下。

Perf and Ring Buffer Maps

性能和环形缓冲区映射

In this section I’m going to describe a slightly more sophisticated version of “Hello World” that uses BCC’s `BPF_PERF_OUTPUT` capabilities, which let you write data in a structure of your choosing into a perf ring buffer map.

在本节中，我将描述一个稍微复杂一些的“Hello World”版本，它使用了BCC的BPF_PERF_OUTPUT功能，该功能允许您将数据以您选择的结构写入perf环形缓冲区映射中。

Note

There is a newer construct called “BPF ring buffers” that are now generally preferred over BPF perf buffers, if you have a kernel of version 5.8 or above. Andrii Nakryiko discusses the difference in his [BPF ring buffer](#) blog post. You’ll see an example of BCC’s `BPF_RINGBUF_OUTPUT` in [Chapter 4](#).

有一个更新的结构体称为“BPF环形缓冲区”，现在通常比BPF性能缓冲区更受青睐，如果你的内核版本是5.8或更高。Andrii Nakryiko在他的BPF环形缓冲区博文中讨论了它们之间的区别。你将在第四章中看到BCC的BPF_RINGBUF_OUTPUT的一个例子。

Ring Buffers

环形缓冲器

Ring buffers are by no means unique to eBPF, but I’ll explain them just in case you haven’t come across them before. You can think of a ring buffer as a piece of memory logically organized in a ring, with separate “write” and “read” pointers. Data of some arbitrary length gets written to wherever the write pointer is, with the length information included in a header for that data. The write pointer moves to after the end of that data, ready for the next write operation.

环形缓冲器并不是eBPF独有的，但我会解释一下，以防你之前没见过。你可以将环形缓冲器想象成一个逻辑上组织成环的内存块，其中包括独立的“写入”和“读取”指针。某个任意长度的数据被写入到写入指

针所在的位置，该数据的长度信息包含在其标头中。写入指针移动到该数据结束后的位置，准备进行下一次写入操作。

Similarly, for a read operation, data gets read from wherever the read pointer is, using the header to determine how much data to read. The read pointer moves along in the same direction as the write pointer so that it points to the next available piece of data. This is illustrated in [Figure 2-3](#), showing a ring buffer with three items of different length available for reading.

同样地，对于读操作，数据从读指针所在位置读取，使用标头确定要读取的数据量。读指针沿着写指针的方向移动，以指向下一个可用的数据片段。图2-3显示了一个环形缓冲区，可用于读取的是三个不同长度的项。

If the read pointer catches up with the write pointer, it simply means there's no data to read. If a write operation would make the write pointer overtake the read pointer, the data doesn't get written and a *drop counter* gets incremented. Read operations include the drop counter to indicate whether data has been lost since the last successful read.

如果读指针追上写指针，意味着没有数据可读。如果写操作使得写指针超过读指针，数据不会被写入，并且丢弃计数器会增加。读操作包括丢弃计数器，以指示自上次成功读取以来是否丢失了数据。

If read and write operations happened at precisely the same rate with no variability, and they always contained the same amount of data, you could at least in theory get away with a ring buffer just big

enough to accommodate that data size. In most applications there will be some variation in the time between reads, writes, or both, so the buffer size needs to be tuned to account for this.

如果读写操作以完全相同的速度进行且没有变化，而且它们总是包含相同数量的数据，至少在理论上，您可以使用一个足够大的环形缓冲区来容纳该数据大小。在大多数应用中，读取、写入或两者之间的时间之间会有一些变化，因此缓冲区大小需要调整以考虑这一点。

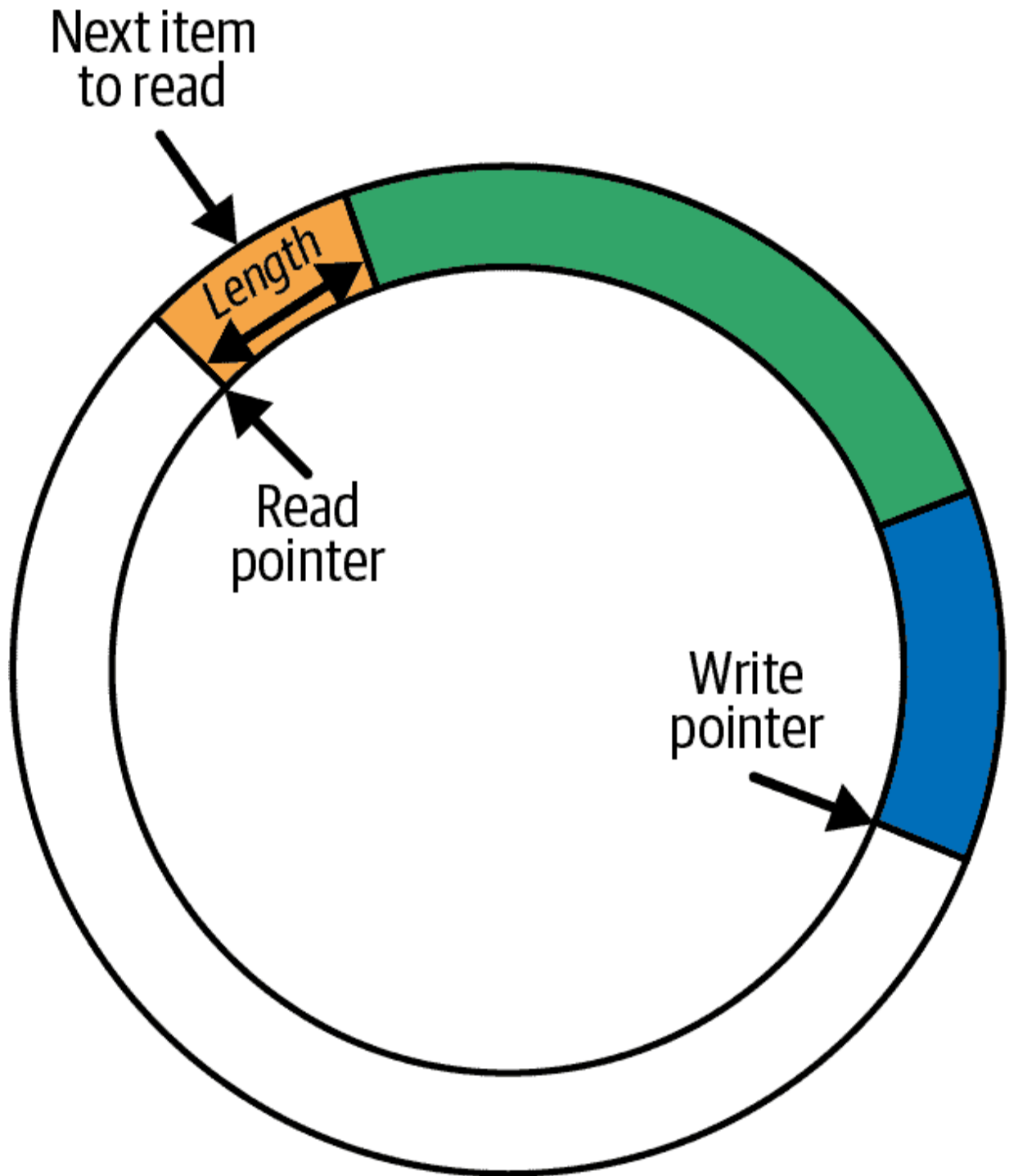


Figure 2-3. A ring buffer

You'll find the source code for this example in *chapter2/hello-buffer.py* in the *Learning eBPF* [GitHub repository](#). As in the first

“Hello World” example you saw early in this chapter, this version will write the string `"Hello World"` to the screen every time the `execve()` syscall is used. It will also look up the process ID and the name of the command that makes each `execve()` call so that you’ll get similar output to the first example. This gives me the opportunity to show you a couple more examples of BPF helper functions.

你可以在Learning eBPF GitHub仓库的chapter2/hello-buffer.py中找到这个示例的源代码。就像你在本章早期看到的第一个“Hello World”示例一样，这个版本将在每次使用`execve()`系统调用时将字符串“Hello World”写入屏幕。它还会查找每个`execve()`调用所使用的进程ID和命令的名称，以便你能够获得与第一个示例类似的输出。这为我提供了展示一些BPF辅助函数的机会。

Here’s the eBPF program that will be loaded into the kernel:

这是将被加载到内核的eBPF程序：

```
BPF_PERF_OUTPUT(output); ❶

struct data_t { ❷
    int pid;
    int uid;
    char command[16];
    char message[12];
};

int hello(void *ctx) {
    struct data_t data = {}; ❸

    char message[12] = "Hello World";

    data.pid = bpf_get_current_pid_tgid() >> 32; ❹
}
```

```

    data.uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;
    5

    bpf_get_current_comm(&data.command, sizeof(data.command
    ));
    6
    bpf_probe_read_kernel(&data.message, sizeof(data.
    message), message);
    7

    output.perf_submit(ctx, &data, sizeof(data));
    8

    return 0;
}

```

❶

BCC defines the macro `BPF_PERF_OUTPUT` for creating a map that will be used to pass messages from the kernel to user space. I've called this map `output`.

❷

Every time `hello()` is run, the code will write a structure's worth of data. This is the definition of that structure, which has fields for the process ID, the name of the currently running command, and a text message.

❸

`data` is a local variable that holds the data structure to be submitted, and `message` holds the `"Hello World"` string.

❹

`bpf_get_current_pid_tgid()` is a helper function that gets the ID of the process that triggered this eBPF program to run. It returns a 64-bit value with the process ID in the top 32 bits.⁶

5

`bpf_get_current_uid_gid()` is the helper function you saw in the previous example for obtaining the user ID.

6

Similarly, `bpf_get_current_comm()` is a helper function for getting the name of the executable (or “command”) that’s running in the process that made the `execve` syscall. This is a string, not a numeric value like the process and user IDs, and in C you can’t simply assign a string using `=`. You have to pass the address of the field where the string should be written, `&data.command`, as an argument to the helper function.

7

For this example, the message is `"Hello World"` every time. `bpf_probe_read_kernel()` copies it into the right place in the data structure.

8

At this point the data structure is populated with the process ID, command name, and message. This call to `output.perf_submit()` puts that data into the map.

Just as in the first “Hello World” example, this C program is assigned to a string called `program` in the Python code. What follows is the rest of the Python code:

就像第一个“Hello World”示例中那样，这个C程序在Python代码中被分配给一个叫做program的字符串。接下来是Python代码的剩余部

分。

```
b = BPF(text=program) ❶
syscall = b.get_syscall_fnname("execve")
b.attach_kprobe(event=syscall, fn_name="hello")

def print_event(cpu, data, size): ❷
    data = b["output"].event(data)
    print(f"
{data.pid} {data.uid} {data.command.decode()} " + \
        f"{data.message.decode()}")

b["output"].open_perf_buffer(print_event) ❸
while True: ❹
    b.perf_buffer_poll()
```

❶

The lines that compile the C code, load it into the kernel, and attach it to the syscall event are unchanged from the version of “Hello World” you saw earlier.

❷

`print_event` is a callback function that will output a line of data to the screen. BCC does some heavy lifting so that I can refer to the map simply as `b["output"]` and grab data from it using `b["output"].event()`.

❸

`b["output"].open_perf_buffer()` opens the perf ring buffer. The function takes `print_event` as an argument to define that this is the callback function to be used whenever there is data to read from the buffer.

❹

The program will now loop indefinitely,⁷ polling the perf ring buffer. If there is any data available, `print_event` will get called.

Running this code gives us output that's fairly similar to the original "Hello World":

运行此代码会得到与原始的“Hello World”相当相似的输出结果。

```
$ sudo ./hello-buffer.py
11654 node Hello World
11655 sh Hello World
...
```

As before, you might need to open a second terminal to the same (virtual) machine and run some commands to trigger some output.

与之前一样，您可能需要打开第二个终端到相同的（虚拟）机器，并运行一些命令来触发一些输出。

The big difference between this and the original “Hello World” example is that instead of using a single, central trace pipe, the data is now being passed via a ring buffer map called `output` that was created by this program for its own use, as shown in [Figure 2-4](#).

这个和原始的“Hello World”例子之间的主要区别在于，不再使用单一的中央追踪管道，而是通过一个名为output的环形缓冲区映射传递数据，这个映射是由这个程序自己创建并用于自身使用，如图2-4所示。

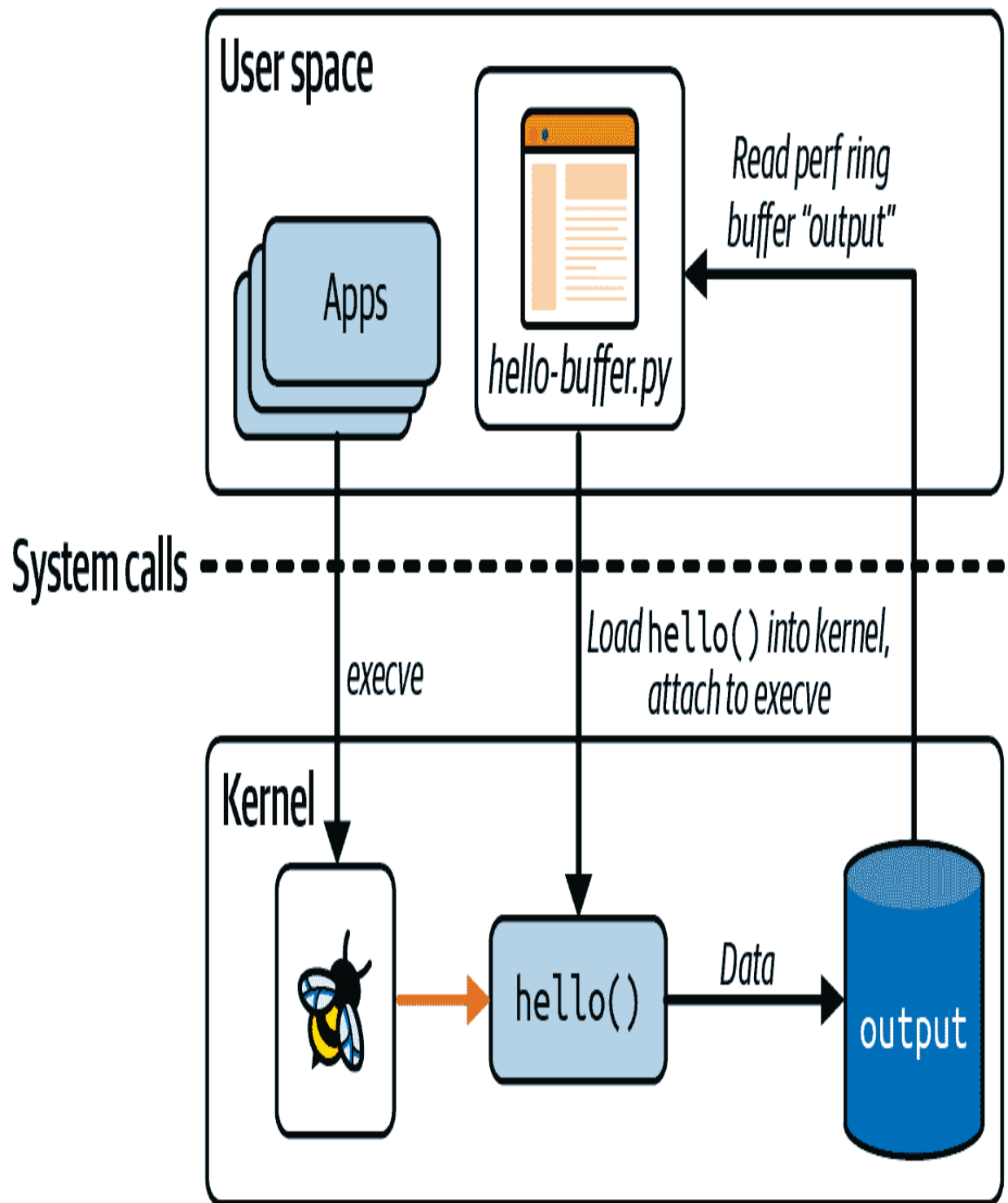


Figure 2-4. Using a perf ring buffer for passing data from the kernel to user space

You can verify that the information isn't going to the trace pipe by using `cat /sys/kernel/debug/tracing/trace_pipe`.

您可以通过使用`cat /sys/kernel/debug/tracing/trace_pipe` 来验证信息是否未发送到跟踪管道。

As well as demonstrating the use of a ring buffer map, this example shows some eBPF helper functions for retrieving contextual information about the event that triggered the eBPF program to run. Here you've seen helper functions getting the user ID, the process ID, and the name of the current command. As you'll see in [Chapter 7](#), the set of contextual information that's available and the set of valid helper functions that can be used to retrieve it depend on what type of program it is and what event triggered it.

除了展示了环形缓冲区映射的使用，此示例还展示了一些eBPF辅助函数，用于获取触发eBPF程序运行的事件的上下文信息。在这里，您已经看到了辅助函数获取用户ID、进程ID和当前命令的名称。正如您将在第7章中看到的那样，可用的上下文信息以及可用于检索它的有效辅助函数的集合取决于程序的类型和触发它的事件。

The fact that contextual information like this is available to the eBPF code is what makes it so valuable for observability. Whenever an event occurs, an eBPF program can report not only the fact that the event happened but also relevant information about what happened to trigger the event. It's also highly performant, since all this information can be gathered within the kernel, without the need for any synchronous context switching to user space.

上下文信息对于eBPF代码的可用性是使其如此有价值的事实。每当发生事件时，eBPF程序不仅可以报告事件发生的事实，还可以提供有关触发事件的相关信息。而且它还具有高性能，因为所有这些信息可以在内核中收集，而无需进行任何同步的上下文切换到用户空间。

You'll see further examples in this book where eBPF helper functions are used to gather other contextual data, as well as examples where eBPF programs change the contextual data or even block events from happening altogether.

在这本书中，您将看到更多使用eBPF辅助函数收集其他上下文数据的例子，以及使用eBPF程序改变上下文数据甚至完全阻止事件发生的例子。

Function Calls

函数调用

You've seen that eBPF programs can call helper functions provided by the kernel, but what if you want to split the code you're writing into functions? Generally, in software development it's considered good practice⁸ to pull common code into a function that you can call from multiple places, rather than duplicating the same lines over and over again. But in the early days, eBPF programs were not permitted to call functions other than helper functions. To work around this, programmers have directed the compiler to “always inline” their functions, like this:

你可能已经注意到eBPF程序可以调用内核提供的帮助函数，但是如果你想将你编写的代码分割成函数怎么办？通常，在软件开发中，将通

用的代码放入一个函数中，可以从多个地方调用该函数，而不是重复相同的行。但在早期，除了帮助函数之外，eBPF程序是不允许调用其他函数的。为了解决这个问题，程序员们指示编译器“总是内联”他们的函数，像这样：

```
static __always_inline void my_function(void *ctx, int val
)
```

Generally, a function in source code results in the compiler emitting a jump instruction, which causes execution to jump to the set of instructions that make up the called function (and then to jump back again when that function has completed). You can see this illustrated on the left side of [Figure 2-5](#). The right side shows what happens when a function is inlined: there is no jump instruction; instead, a copy of the function's instructions is emitted directly within the calling function.

通常，在源代码中的函数会导致编译器发出一条跳转指令，这会导致执行跳转到组成调用函数的一组指令（然后在函数完成后再跳回）。您可以在图2-5的左侧看到这个示例。右侧显示了当函数内联时会发生什么：没有跳转指令；相反，函数的指令副本会直接在调用函数中发出。

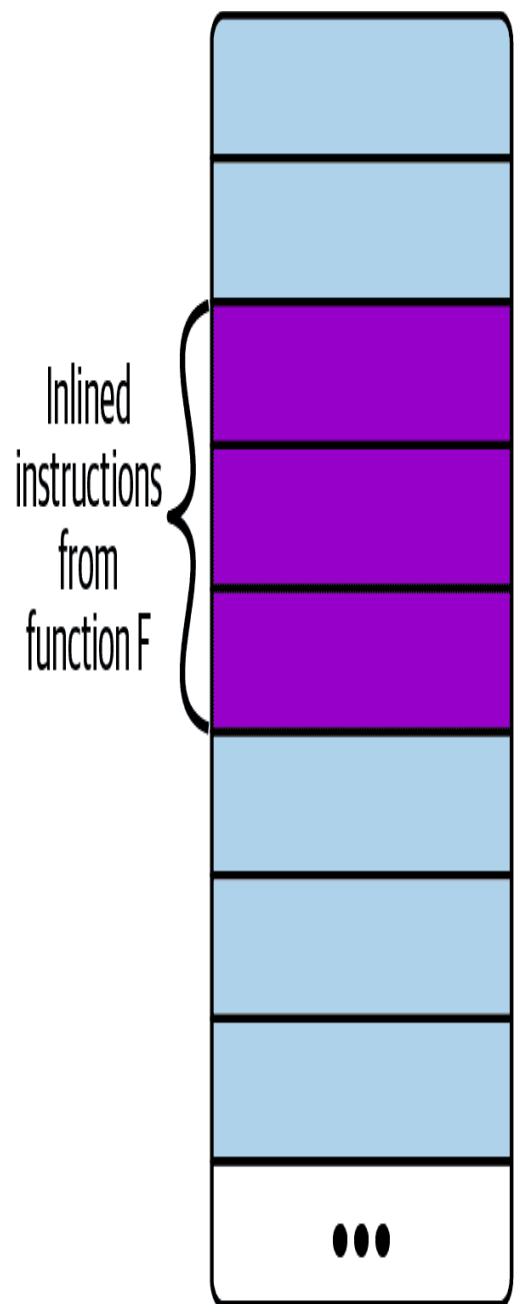
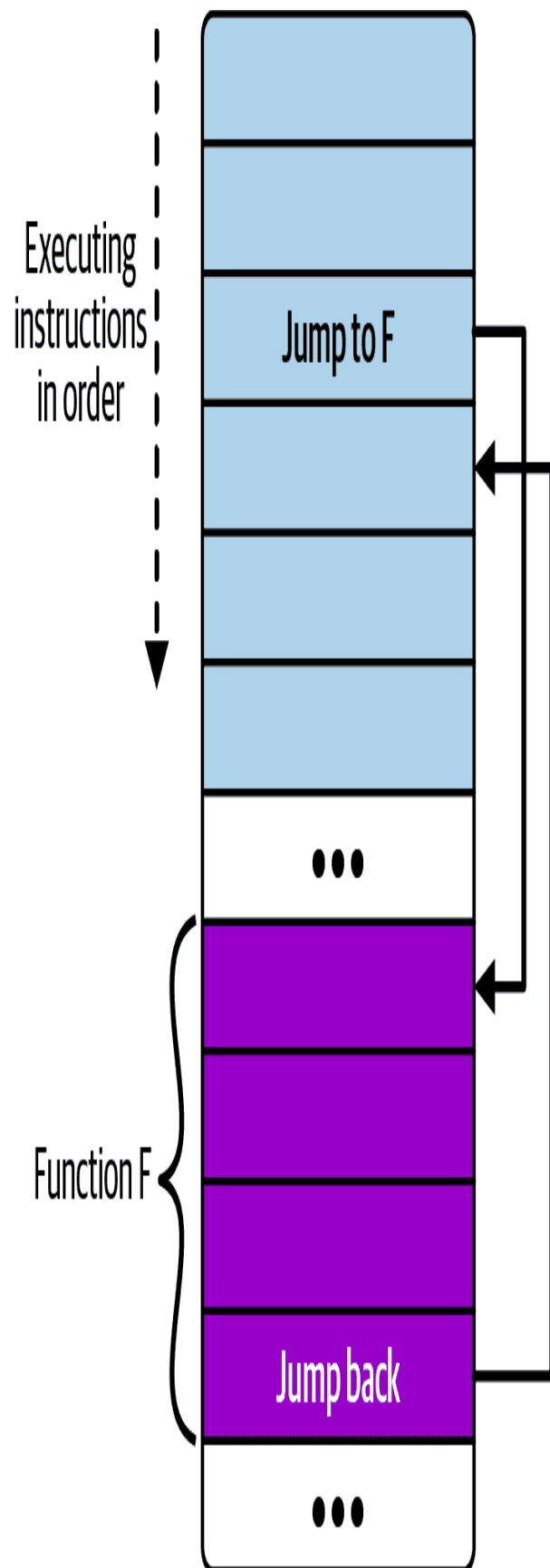


Figure 2-5. Layout of noninlined and inlined function instructions

If the function is called from multiple places, that results in multiple copies of that function’s instructions in the compiled executable. (Sometimes the compiler might choose to inline a function for optimization purposes, and that is one reason why you might not be able to attach a kprobe to certain kernel functions. I’ll come back to this in [Chapter 7](#).)

如果从多个地方调用该函数，会导致编译后的可执行文件中有多个该函数指令的副本。（有时编译器可能选择内联函数以进行优化，这也是为什么您可能无法将kprobe附加到某些内核函数的原因。在第7章中我会详细解释这个问题。）

Starting from Linux kernel 4.16 and LLVM 6.0, the restriction requiring functions to be inlined was lifted so that eBPF programmers could write function calls more naturally. However, this feature, called “BPF to BPF function calls” or “BPF subprograms,” isn’t currently supported by the BCC framework, so let’s come back to it in the next chapter. (You can, of course, continue to use functions with BCC if they are inlined.)

从Linux内核4.16和LLVM 6.0开始，不再要求函数必须内联，这样eBPF程序员可以更自然地编写函数调用。但是，目前BCC框架不支持这个称为“BPF到BPF函数调用”或“BPF子程序”的特性，所以让我们在下一章中回过头来讨论它。（当然，如果函数是内联的，你仍然可以继续使用BCC中使用函数。）

There is another mechanism for decomposing complex functionality into smaller parts in eBPF: tail calls.

eBPF有另一种将复杂功能分解成较小部分的机制：尾调用。

Tail Calls

尾调用

As described at ebpf.io, “tail calls can call and execute another eBPF program and replace the execution context, similar to how the `execve()` system call operates for regular processes.” In other words, execution doesn’t return to the caller after a tail call completes.

如ebpf.io所描述：“尾调用可以调用和执行另一个eBPF程序并替换执行上下文，类似于execve()系统调用对于常规进程的操作。”换句话说，尾调用完成后不会返回给调用者执行。

Note

[Tail calls](#) are by no means exclusive to eBPF programming. The general motivation behind tail calls is to avoid adding frames to the stack over and over again as a function is called recursively, which can eventually lead to stack overflow errors. If you can arrange your code to call a recursive function as the last thing it does, the stack frame associated with the calling function isn’t really doing anything useful. Tail calls allow for calling a series of functions without growing the stack. This is particularly useful in eBPF where the [stack is limited to 512 bytes](#).

尾调用绝非仅限于eBPF编程。尾调用的一般动机是避免在函数被递归调用时一次又一次地向堆栈中添加帧，这最终可能导致堆栈溢出错误。如果您可以安排代码在最后调用递归函数，与调用函数相关联的堆栈帧实际上并没有起到任何有用的作用。尾调用允许在不增加堆栈的情况下调用一系列函数。在eBPF中特别有用，因为堆栈仅限于512字节。

Tail calls are made using the `bpf_tail_call()` helper function, which has the following signature:

使用**bpf_tail_call()**助手函数来进行尾调用，该函数具有以下签名：

```
long bpf_tail_call(void *ctx, struct bpf_map *
prog_array_map, u32 index)
```

The three arguments to this function have the following meanings:

这个函数的三个参数具有以下含义：

- `ctx` allows passing the context from the calling eBPF program to the callee.

`ctx`允许将上层的eBPF程序上下文传递给被调用者。

- `prog_array_map` is an eBPF map of type `BPF_MAP_TYPE_PROG_ARRAY`, which holds a set of file descriptors that identify eBPF programs.

`prog_array_map`是一个类型为**BPF_MAP_TYPE_PROG_ARRAY**的eBPF映射，它保存了一组用于标识eBPF程序的文件描述符。

- `index` indicates which of that set of eBPF programs should be invoked.

索引表示应调用该集中的哪个eBPF程序。

This helper is somewhat unusual in that if it succeeds, it never returns. The currently running eBPF program is replaced on the stack by the program being called. The helper could fail, for example, if the indicated program doesn't exist in the map, in which case the calling program carries on executing.

这个帮助程序有点不寻常，如果成功的话，它永远不会返回。当前运行的eBPF程序会被调用的程序替换在堆栈上。如果指定的程序在地图中不存在，这个帮助程序可能会失败，此时调用程序会继续执行。

User space code has to load all the eBPF programs into the kernel (as usual), and it also sets up the program array map.

用户空间代码必须将所有的eBPF程序加载到内核中（如往常一样），它还设置了程序数组映射。

Let's look at a simple example written in Python using BCC; you'll find the code in the [GitHub repo](#) as *chapter2/hello-tail.py*. The main eBPF program is attached to a tracepoint at the common entry point for all syscalls. This program uses tail calls to trace out specific messages for certain syscall opcodes. If there isn't a tail call for a given opcode, the program traces out a generic message.

让我们来看一个使用BCC编写的Python简单示例；您可以在GitHub存储库的章节2/hello-tail.py中找到代码。主要的eBPF程序附加到了所

有系统调用的公共入口点的跟踪点上。该程序使用尾调用来追踪特定系统调用操作码的特定消息。如果给定操作码没有尾调用，程序将追踪一个通用消息。

If you're using the BCC framework, to make a [tail call](#) you can use a line of the slightly simpler form:

如果你正在使用BCC框架，想要进行尾调用，你可以使用一个稍微简化的形式的代码行。

```
prog_array_map.call(ctx, index)
```

Before passing the code to the compilation step, BCC will rewrite the preceding line to this:

在将代码传递给编译步骤之前，BCC将把前一行重写成这样：

```
bpf_tail_call(ctx, prog_array_map, index)
```

Here is the source code for the eBPF program and its tail calls:

这是eBPF程序及其尾部调用的源代码。

```
BPF_PROG_ARRAY(syscall, 300);  
  
int hello(struct bpf_raw_tracepoint_args *ctx) {  
    int opcode = ctx->args[1];  
    syscall.call(ctx, opcode);  
    bpf_trace_printk("Another syscall: %d", opcode);  
    return 0;  
}
```

```

int hello_execve(void *ctx) {
    bpf_trace_printk("Executing a program");6
    return 0;
}

int hello_timer(struct bpf_raw_tracepoint_args *ctx) {7
    if (ctx->args[1] == 222) {
        bpf_trace_printk("Creating a timer");
    } else if (ctx->args[1] == 226) {
        bpf_trace_printk("Deleting a timer");
    } else {
        bpf_trace_printk("Some other timer operation");
    }
    return 0;
}

int ignore_opcode(void *ctx) {8
    return 0;
}

```

❶

BCC provides a `BPF_PROG_ARRAY` macro for easily defining maps of type `BPF_MAP_TYPE_PROG_ARRAY`. I have called the map `syscall` and allowed for 300 entries,⁹ which is going to be sufficient for this example.

❷

In the user space code that you'll see shortly, I'm going to attach this eBPF program to the `sys_enter` raw tracepoint, which gets hit whenever any syscall is made. The context passed to an eBPF program attached to a raw tracepoint takes the form of this `bpf_raw_tracepoint_args` structure.

3

In the case of `sys_enter`, the raw tracepoint arguments include the opcode identifying which syscall is being made.

4

Here we make a tail call to the entry in the program array whose key matches the opcode. This line of code will be rewritten by BCC to a call to the `bpf_tail_call()` helper function before it passes the source code to the compiler.

5

If the tail call succeeds, this line tracing out the opcode value will never be hit. I've used this to provide a default line of trace for opcodes for which there isn't a program entry in the map.

6

`hello_exec()` is a program that will be loaded into the syscall program array map, to be executed as a tail call when the opcode indicates it's an `execve()` syscall. It's just going to generate a line of trace to tell the user a new program is being executed.

7

`hello_timer()` is another program that will be loaded into the syscall program array. In this case it's going to be referred to by more than one entry in the program array.

8

`ignore_opcode()` is a tail call program that does nothing. I'll use this for syscalls where I don't want any trace to be generated at all.

Now let's look at the user space code that loads and manages this set of eBPF programs:

现在让我们来看一下加载和管理这组eBPF程序的用户空间代码：

```
b = BPF(text=program)

b.attach_raw_tracepoint(tp="sys_enter", fn_name="hello")
❶

ignore_fn = b.load_func("ignore_opcode", BPF.
RAW_TRACEPOINT)
❷
exec_fn = b.load_func("hello_exec", BPF.RAW_TRACEPOINT)
timer_fn = b.load_func("hello_timer", BPF.RAW_TRACEPOINT)

prog_array = b.get_table("syscall")
❸

prog_array[ct.c_int(59)] = ct.c_int(exec_fn.fd)
prog_array[ct.c_int(222)] = ct.c_int(timer_fn.fd)
prog_array[ct.c_int(223)] = ct.c_int(timer_fn.fd)
prog_array[ct.c_int(224)] = ct.c_int(timer_fn.fd)
prog_array[ct.c_int(225)] = ct.c_int(timer_fn.fd)
prog_array[ct.c_int(226)] = ct.c_int(timer_fn.fd)

# Ignore some syscalls that come up a lot
prog_array[ct.c_int(21)] = ct.c_int(ignore_fn.fd)
prog_array[ct.c_int(22)] = ct.c_int(ignore_fn.fd)
prog_array[ct.c_int(25)] = ct.c_int(ignore_fn.fd)
...

b.trace_print()
❺
```


Instead of attaching to a kprobe, as you saw earlier, this time the user space code attaches the main eBPF program to the `sys_enter` tracepoint.

2

These calls to `b.load_func()` return a file descriptor for each tail call program. Notice that tail calls need to have the same program type as their parent—`BPF_RAW_TRACEPOINT` in this case. Also, it bears pointing out that each tail call program is an eBPF program in its own right.

3

The user space code creates entries in the `syscall` map. The map doesn't have to be fully populated for every possible opcode; if there is no entry for a particular opcode, it simply means no tail call will be executed. Also, it's perfectly fine to have multiple entries that point to the same eBPF program. In this case, I want the `hello_timer()` tail call to be executed for any of a set of timer-related syscalls.

4

Some syscalls get run so frequently by the system that a line of trace for each of them clutters up the trace output to the point of unreadability. I've used the `ignore_opcode()` tail call for several syscalls.

5

Print the trace output to the screen, until the user terminates the program.

Running this program generates trace output for every syscall that runs on the (virtual) machine, unless the opcode has an entry that links it to the `ignore_opcode()` tail call. Here's some example output from running `ls` in another terminal (some details have been omitted for readability):

运行该程序会为在（虚拟）机器上运行的每个系统调用生成跟踪输出，除非该操作码有一个将其链接到 `ignore_opcode()` 尾调用的条目。下面是在另一个终端运行 `ls` 后的一些示例输出（为了可读性，省略了一些细节）：

```
./hello-tail.py
b'    hello-tail.py-2767    ... Another syscall: 62'
b'    hello-tail.py-2767    ... Another syscall: 62'
...
b'          bash-2626    ... Executing a program'
b'          bash-2626    ... Another syscall: 220'
...
b'          <...>-2774    ... Creating a timer'
b'          <...>-2774    ... Another syscall: 48'
b'          <...>-2774    ... Deleting a timer'
...
b'          ls-2774    ... Another syscall: 61'
b'          ls-2774    ... Another syscall: 61'
...
```

The particular syscalls being executed are beside the point, but you can see that the different tail calls are getting called and are generating trace messages. You can also see the default message `Another syscall` for opcodes that don't have an entry in the tail call program map.

特定的系统调用正在被执行并不是重点，但你可以看到不同的尾调用被调用并生成跟踪消息。你还可以看到默认消息，另一个系统调用用于没有尾调用程序映射条目的操作码。

Note

Check out Paul Chaignon's blog post about the [cost of BPF tail calls](#) on various different kernel versions.

请查看Paul Chaignon关于不同内核版本上BPF尾部调用成本的博客文章。

Tail calls have been supported in eBPF since kernel version 4.2, but for a long time they were incompatible with making BPF to BPF function calls. This restriction was lifted in kernel 5.10.¹⁰

自内核版本4.2起，eBPF已支持尾调用，但长时间以来它们与进行BPF到BPF函数调用不兼容。这个限制在内核5.10中解除了。

The fact that you can chain up to 33 tail calls together, combined with the instruction complexity limit per eBPF program of 1 million instructions, means that today's eBPF programmers have a lot of leeway to write very complex code to run entirely in the kernel.

当下的eBPF程序员有很大的自由度，可以将最多33个尾调用链接在一起，并且每个eBPF程序的指令复杂度限制为100万条指令。这意味着他们可以编写非常复杂的代码，完全在内核中运行。

Summary

摘要

I hope that by showing some concrete examples of an eBPF program, this chapter helped you consolidate your mental model of eBPF code running in the kernel, triggered by events. You've also seen examples of data being passed from the kernel to user space using BPF maps.

我希望通过展示一些具体的eBPF程序示例，这一章帮助您巩固对eBPF代码在内核中运行并由事件触发的思维模型。您还看到了将数据从内核传递到用户空间的BPF映射的示例。

Using the BCC framework hides many of the details of how the program is built, loaded into the kernel, and attached to events. In the next chapter I'll show you a different approach to writing "Hello World," and we'll dive deeper into those hidden details.

使用BCC框架隐藏了程序构建、加载到内核和附加到事件的许多细节。在下一章中，我将向您展示一种不同的编写"Hello World"的方法，并深入探讨这些隐藏的细节。

Exercises

练习

Here are some optional activities you might like to try (or think about) if you want to explore "Hello World" a bit further:

以下是一些可选的活动，如果你想进一步探索“Hello World”，你可能会喜欢尝试（或考虑）的活动：

1. Adapt the *hello-buffer.py* eBPF program to output different trace messages for odd and even process IDs.

将hello-buffer.py eBPF程序适应为对于奇偶进程ID输出不同的跟踪信息。

2. Modify *hello-map.py* so that the eBPF code gets triggered by more than one syscall. For example, `openat()` is commonly called to open files, and `write()` is called to write data to a file. You can start by attaching the *hello* eBPF program to multiple syscall kprobes. Then try having modified versions of the *hello* eBPF program for different syscalls, demonstrating that you can access the same map from multiple different programs.

请帮助我修改hello-map.py，使得eBPF代码可以被多于一个系统调用触发。例如，`openat()` 常用于打开文件，而 `write()` 则用于向文件中写入数据。你可以先将 *hello* eBPF 程序附加到多个系统调用 kprobes 上。然后尝试对不同的系统调用使用已修改版本的 *hello* eBPF 程序，以证明你可以从多个不同的程序中访问同一个 map。

3. The *hello-tail.py* eBPF program is an example of a program that attaches to the `sys_enter` raw tracepoint that is hit whenever *any* syscall is called. Change *hello-map.py* to show the total number of syscalls made by each user ID, by attaching it to that same `sys_enter` raw tracepoint.

hello-tail.py eBPF程序是一个示例程序，它附加到sys_enter原始跟踪点，每当调用任何系统调用时都会触发。将hello-map.py更改

为通过附加到相同的sys_enter原始跟踪点来显示每个用户ID所执行的总系统调用次数。

Here's some example output I got after making that change:

这是在做出改变后我得到的一些示例输出结果。

```
$ ./hello-map.py
ID 104: 6      ID 0: 225
ID 104: 6      ID 101: 34      ID 100: 45      ID 0: 332
ID 501: 19
ID 104: 6      ID 101: 34      ID 100: 45      ID 0: 368
ID 501: 38
ID 104: 6      ID 101: 34      ID 100: 45      ID 0: 533
ID 501: 57
```

4. The `RAW_TRACEPOINT_PROBE` macro provided by BCC simplifies attaching to raw tracepoints, telling the user space BCC code to automatically attach it to a specified tracepoint. Try it in *hello-tail.py*, like this:

BCC提供的RAW_TRACEPOINT_PROBE宏简化了附加到原始跟踪点的过程，告知用户空间的BCC代码自动将其附加到指定的跟踪点。在hello-tail.py中试一试，就像这样：

- Replace the definition of the `hello()` function with `RAW_TRACEPOINT_PROBE(sys_enter)`.

请将`将hello()函数的定义替换为RAW\_TRACEPOINT\_PROBE(sys\_enter)。`翻译为简体中文，只返回翻译后的内容，不包括原始文本。

- Remove the explicit attachment call `b.attach_raw_tracepoint()` from the Python code.

从Python代码中删除明确附件调用**b.attach_raw_tracepoint()**。

You should see that BCC automatically creates the attachment and the program works exactly the same. This is an example of the many convenient macros that BCC provides.

你应该看到BCC自动创建附件并且程序工作的方式完全相同。这是BCC提供的许多便利宏的一个例子。

5. You could further adapt *hello_map.py* so that the key in the hash table identifies a particular syscall (rather than a particular user). The output will show how many times that syscall has been called across the whole system.

您可以进一步调整*hello_map.py*，使哈希表中的键标识特定的系统调用（而不是特定的用户）。输出将显示该系统调用在整个系统中被调用的次数。

¹ I originally wrote this for a talk titled “The Beginner’s Guide to eBPF Programming.” You can find the original code along with links to the slides and video at <https://github.com/lizrice/ebpf-beginners>.

我最初为一场名为“eBPF编程入门指南”的讲座撰写了这篇文章。您可以在<https://github.com/lizrice/ebpf-beginners>找到原始代码以及幻灯片和视频的链接。

² There is a more performant way to attach eBPF programs to functions, available from kernel version 5.5 onward, that uses *fentry* (and the corresponding *fexit* instead of *kretprobe* for the exit from a function). I’ll discuss this later in the book, but for now I’m using *kprobe* to keep the example in this chapter as simple as possible.

连接eBPF程序到函数的更高性能方式，从内核版本5.5开始可用，使用fentry（以及相应的fexit而非kretprobe在函数退出时）。我会在本书的后面部分讨论这个问题，但出于简单起见，本章我使用kprobe作为示例。

³ I quite often use VScode remote to connect to a virtual machine in the cloud. This runs lots of node scripts on the virtual machine, which generates lots of tracing from this “Hello World” app.

我经常使用VScode远程连接到云上的虚拟机。这在虚拟机上运行很多的Node脚本，这些脚本生成了许多来自这个“Hello World”应用程序的跟踪数据。

⁴ Some commands (`echo` is a common example) might be shell built-ins that run as part of the shell process, rather than executing a new program. These won't trigger the `execve()` event, so no trace will be generated.

某些命令（例如echo）可能是作为shell进程的一部分运行的内置命令，而不是执行一个新程序。这些命令不会触发execve()事件，因此不会生成任何追踪。

⁵ C++ does, but not C.

C++ 可以，但 C 不可以。

⁶ The lower 32 bits are the *thread group ID*. For a single-threaded process, this is the same as the process ID, but additional threads for the process would be given different IDs. The docs for the GNU C

library have a good description of the difference between [process and thread group IDs](#).

低32位是线程组ID。对于单线程进程，它与进程ID相同，但是为该进程添加的额外线程会有不同的ID。GNU C库的文档对进程ID和线程组ID之间的区别有很好的描述。

⁷ This is just example code, so I'm not worrying about cleaning up on keyboard interrupt or any other niceties!

这只是一个例子代码，所以我不担心在键盘中断或其他细节上进行清理！

⁸ This principle is often called “DRY” (“Don’t Repeat Yourself”), as popularized by [The Pragmatic Programmer](#).

这个原则通常被称为“DRY”（“不要重复自己”），由《实用程序员》所普及。

⁹ There are some 300 syscalls in Linux, and since I'm not using any recently added syscalls for this example, this is good enough.

Linux中有大约300个系统调用，由于我在这个例子中并没有使用任何最近添加的系统调用，所以已经足够了。

¹⁰ Making tail calls from a BPF subprogram requires support from the JIT compiler, which you'll meet in the next chapter. In the kernel version I used to write the examples in this book, only the JIT compiler on x86 has this support, although [support has been added to ARM in kernel 6.0](#).

从BPF子程序进行尾调用需要JIT编译器的支持，在下一章中我们将介绍到它。在我撰写本书示例时使用的内核版本中，只有x86架构上的JIT编译器具有此支持，但在6.0内核中还新增了对ARM架构的支持。

Chapter 3. Anatomy of an eBPF Program

第三章：eBPF程序的解剖 eBPF

(Extended Berkeley Packet Filter, 扩展伯克利数据包过滤器)程序的解剖，是理解eBPF的基础。在本章中，我们将深入了解eBPF程序的结构和组成部分。

eBPF程序是用C语言编写的，并通过特定的编译器转换为可以在Linux内核中运行的字节码。这些eBPF程序通过网络套接字与内核通信，以实现网络流量的处理和分析。一个eBPF程序由多个指令组成。每个指令都有对应的操作码和参数。这些指令按照顺序依次执行，实现特定的功能。eBPF程序还包括一组注册的内存映射，用于存储程序执行期间的变量和数据。这些内存映射可以被eBPF程序中的指令访问和修改。eBPF程序还可以调用一组特定的内核函数，以实现与内核的交互。这些内核函数提供了访问系统调用、

网络数据包和其他内核资源的能力。除了具体的指令和内存映射外，eBPF程序还可以定义一组过滤条件和钩子点。这些过滤条件和钩子点决定了eBPF程序在何时和何地执行。通过理解eBPF程序的结构和组成部分，我们可以更好地利用eBPF技术来解决网络和系统性能调优等问题。在后续章节中，我们将深入探讨如何编写和使用eBPF程序来实现各种功能。

In the previous chapter you saw a simple eBPF “Hello World” program written using the BCC framework. In this chapter there’s an example version of a “Hello World” program written entirely in C so that you can see some of the details BCC took care of behind the scenes.

在前一章中，您看到了一个使用BCC框架编写的简单eBPF“Hello World”程序。在本章中，这里有一个完全用C语言编写的“Hello World”程序的示例版本，这样您就可以看到BCC在幕后为您处理的一些细节。

This chapter also shows you the stages an eBPF program goes through on its journey from source code to execution, as illustrated in [Figure 3-1](#).

本章还向您展示了eBPF程序从源代码到执行所经历各个阶段，如图3-1所示。



Figure 3-1. C (or Rust) source code is compiled into eBPF bytecode, which is either JIT-compiled or interpreted into native machine code instructions

An eBPF program is a set of eBPF bytecode instructions. It's possible to write eBPF code directly in this bytecode, much as it's possible to program in assembly language. Humans typically find a higher-level programming language easier to deal with, and at least at the time of this writing, I'd say the vast majority of eBPF code is written in C¹ and then compiled to eBPF bytecode.

eBPF程序是一组eBPF字节码指令。可以直接用这种字节码编写eBPF代码，就像可以使用汇编语言编程一样。人们通常发现使用高级编程语言更容易处理，至少在撰写本文时，我认为绝大多数eBPF代码是用C语言编写的，然后编译成eBPF字节码。

Conceptually, this bytecode runs in an eBPF virtual machine within the kernel.

在概念上，这段字节码在内核中的eBPF虚拟机中运行。

The eBPF Virtual Machine

eBPF虚拟机

The eBPF virtual machine, like any virtual machine, is a software implementation of a computer. It takes in a program in the form of eBPF bytecode instructions, and these have to be converted to native machine instructions that run on the CPU.

eBPF虚拟机，像任何虚拟机一样，是计算机的软件实现。它接收以eBPF字节码指令形式表示的程序，并且需要将其转换为在CPU上运行的本地机器指令。

In early implementations of eBPF, the bytecode instructions were interpreted within the kernel—that is, every time an eBPF program runs, the kernel examines the instructions and converts them into machine code, which it then executes. Interpreting has since been largely replaced by JIT (just-in-time) compilation for performance reasons and to avoid the possibility of some Spectre-related vulnerabilities in the eBPF interpreter. *Compilation* means the conversion to native machine instructions happens just once, when the program is loaded into the kernel.

在早期的eBPF实现中，字节码指令是在内核中进行解释的，也就是说，每次运行eBPF程序时，内核会检查指令并将其转换为机器代码，然后执行。由于性能原因和为了避免eBPF解释器中某些与Spectre相关的漏洞的可能性，解释已经在很大程度上被JIT（即时编译）取代。编译意味着当程序加载到内核时，将其转换为本地机器指令，这个过程只会发生一次。

eBPF bytecode consists of a set of instructions, and those instructions act on (virtual) eBPF registers. The eBPF instruction set and register model were designed to map neatly to common CPU

architectures so that the step of compiling or interpreting from bytecode to machine code is reasonably straightforward.

eBPF字节码由一组指令组成，这些指令作用于（虚拟）eBPF寄存器。eBPF指令集和寄存器模型的设计旨在与常见的CPU架构相匹配，以便从字节码编译或解释为机器代码的步骤相对简单。

eBPF Registers

eBPF寄存器

The eBPF virtual machine uses 10 general-purpose registers, numbered 0 to 9. Additionally, Register 10 is used as a stack frame pointer (and can only be read, but not written). As a BPF program is executed, values get stored in these registers to keep track of state.

eBPF虚拟机使用10个通用寄存器，编号为0到9。此外，寄存器10用作堆栈帧指针（只能读取，不能写入）。当执行BPF程序时，值被存储在这些寄存器中以跟踪状态。

It's important to understand that these eBPF registers in the eBPF virtual machine are implemented in software. You can see them enumerated from `BPF_REG_0` to `BPF_REG_10` in the [include/uapi/linux/bpf.h header file](#) of the Linux kernel's source code.

这些eBPF虚拟机中的eBPF寄存器是在软件中实现的，理解这一点非常重要。您可以在Linux内核源代码的include/uapi/linux/bpf.h头文件中从BPF_REG_0到BPF_REG_10中看到它们枚举出来。

The context argument to an eBPF program is loaded into Register 1 before its execution begins. The return value from the function is

stored in Register 0.

eBPF程序的上下文参数在执行之前加载到寄存器1中。函数的返回值存储在寄存器0中。

Before calling a function from eBPF code, the arguments to that function are placed in Register 1 through Register 5 (not all the registers are used if there are fewer than five arguments).

在从eBPF代码调用函数之前，将函数的参数放置在寄存器1到寄存器5中（如果参数少于五个，则不使用所有寄存器）。

eBPF Instructions

eBPF指令

The same [linux/bpf.h header file](#) defines a structure called `bpf_insn`, which represents a BPF instruction:

相同的linux/bpf.h头文件定义了一个称为bpf_insn的结构体，它表示了一个BPF指令。

```
struct bpf_insn {
    __u8 code;                /* opcode */
    __u8 dst_reg:4;           /* dest register */
    __u8 src_reg:4;           /* source register */
    __s16 off;                /* signed offset */
    __s32 imm;                /* signed immediate constant */
};
```

❶

Each instruction has an opcode, which defines what operation the instruction is to perform: for example, adding a value to the contents of a register, or jumping to a different instruction in the program.² The lovisor project’s “[Unofficial eBPF spec](#)” has a list of the valid instructions.

②

Different operations might involve up to two registers.

③

Depending on the operation, there might be an offset value and/or an “immediate” integer value.

This `bpf_insn` structure is 64 bits (or 8 bytes) long. However, sometimes an instruction might need to span more than 8 bytes. If you want to set a register to a 64-bit value, you can’t somehow squeeze all 64 bits of that value into the structure, along with the opcode and register information. In these cases, the instruction uses *wide instruction encoding* that is 16 bytes long in total. You’ll see an example of this in this chapter.

这个**bpf_insn**结构体是64位（或8个字节）长。然而，有时候一个指令可能需要跨越超过8个字节的空间。如果你想要将一个寄存器设置为一个64位的值，你不能够将那个值的全部64位都挤进这个结构体中，同时还包括操作码和寄存器信息。在这些情况下，该指令使用宽指令编码，总共16个字节长。在本章中你将看到一个例子。

When loaded into the kernel, the bytecode of an eBPF program is represented by a series of these `bpf_insn` structures. The verifier

performs several checks on this information to ensure that the code is safe to run. You'll learn more about the verification process in [Chapter 6](#).

当加载到内核时，eBPF程序的字节码由一系列bpf_insn结构表示。验证器对这些信息执行几个检查，以确保代码可以安全运行。您将在第6章中了解更多关于验证过程的内容。

Most of the different opcodes fall into the following categories:

大多数不同的操作码都属于以下类别：

- Loading a value into a register (either an immediate value or a value read from memory or from another register)

将一个值加载到寄存器中（可以是立即值，也可以是从内存或其他寄存器中读取的值）。

- Storing a value from a register into memory

将寄存器中的值存储到内存中

- Performing arithmetic operations such as adding a value to the contents of a register

执行算术运算，如将一个值加到寄存器的内容中

- Jumping to a different instruction if a particular condition is satisfied

如果满足特定条件，跳转到另一条指令

Note

For an overview of eBPF architecture, I recommend the [BPF and XDP Reference Guide](#) that's included as part of the Cilium project's documentation. If you'd like more details, the [kernel documentation](#) describes the eBPF instructions and encoding quite clearly.

我建议您查看Cilium项目文档中包含的BPF和XDP参考指南，以了解eBPF架构的概述。如果您想了解更多详细信息，内核文档清楚地描述了eBPF指令和编码。

Let's use another simple example of an eBPF program and follow its journey from C source code, through eBPF bytecode, to machine code instructions.

让我们使用另一个简单的eBPF程序来举例，并跟随它从C源代码到eBPF字节码再到机器代码指令的过程。

Note

If you want to build and run this code yourself, you'll find the code along with instructions for setting up an environment to do so at github.com/lizrice/learning-ebpf. The code for this chapter is in the *chapter3* directory.

如果您想要自行构建和运行这段代码，您可以在 github.com/lizrice/learning-ebpf 找到代码以及设置环境的说明。本章的代码位于chapter3目录下。

The examples in this chapter are written in C using a library called *libbpf*. You'll learn more about this library in [Chapter 5](#).

本章的示例是用一个叫做libbpf的库编写的C代码。你将在第5章中学习更多关于这个库的内容。

eBPF “Hello World” for a Network Interface

eBPF 网络接口的“Hello World”

The examples in the previous chapter emitted the trace “Hello World” triggered by a system call kprobe; this time I’m going to show an eBPF program that writes a line of trace when triggered by the arrival of a network packet.

在上一章中的示例中，通过系统调用kprobe触发了“Hello World”的跟踪结果；这一次我将展示一个eBPF程序，在接收到网络数据包时可以触发并输出一行跟踪结果。

Packet processing is a very common application of eBPF. I’ll cover this in a lot more detail in [Chapter 8](#), but for now it might be helpful to be aware of the basic idea of an eBPF program that is triggered for every packet of data that arrives on a network interface. The program can inspect and even modify the contents of that packet, and it makes a decision (or *verdict*) on what the kernel should do with that packet. The verdict could tell the kernel to carry on processing it as usual, drop it, or redirect it elsewhere.

数据包处理是eBPF的一种常见应用。在第8章中，我将更详细地介绍这个问题，但是现在知道关于eBPF程序的基本概念可能会很有帮助。该程序在每个到达网络接口的数据包上触发，并且可以检查甚至修改

该数据包的内容，并且对内核对该数据包的处理做出决策（或者裁决）。决策可以告诉内核继续按照通常的方式处理它，丢弃它，或者重定向到其他位置。

In the simple example I'm showing here, the program doesn't do anything with the network packet; it simply writes out the words *Hello World* and a counter to the trace pipe every time a network packet is received.

在这里我展示的简单例子中，程序没有对网络数据包做任何操作，它只是每收到一个网络数据包就将单词“你好世界”和一个计数器写入到跟踪管道。

The example program is in *chapter3/hello.bpf.c*. It's a fairly common convention to put eBPF programs into filenames ending with *bpf.c* to distinguish them from user space C code that might live in the same source code directory. Here's the entire program:

示例程序位于第3章/ *hello.bpf.c*。将eBPF程序放入以**bpf.c**结尾的文件中，是一种非常常见的约定，用于区分可能位于同一源代码目录中的用户空间C代码。以下是整个程序：

```
#include <linux/bpf.h>                                ❶
#include <bpf/bpf_helpers.h>

int counter = 0;                                       ❷

SEC("xdp")                                           ❸
int hello(void *ctx) {                                ❹
    bpf_printk("Hello World %d", counter);
    counter++;
    return XDP_PASS;
}
```

```
char LICENSE[] SEC("license") = "Dual BSD/GPL"; ❺
```

❶

This example starts by including some header files. Just in case you're not familiar with C coding, every program has to include the header files that define any structures or functions the program is going to use. You can guess from the names that these header files are related to BPF.

❷

This example shows how eBPF programs can use global variables. This counter will get incremented every time the program runs.

❸

The macro `SEC()` defines a section called `xdp` that you'll be able to see in the compiled object file. I'll come back to how the section name is used in [Chapter 5](#), but for now you can simply think of it as defining that it's an eXpress Data Path (XDP) type of eBPF program.

❹

Here you can see the actual eBPF program. In eBPF, the program name is the function name, so this program is called `hello`. It uses a helper function, `bpf_printk`, to write a string of text, increments the global variable `counter`, and then returns the value `XDP_PASS`. This is the verdict indicating to the kernel that it should process this network packet as normal.

5

Finally there is another `SEC()` macro that defines a license string, and this is a crucial requirement for eBPF programs. Some of the BPF helper functions in the kernel are defined as “GPL only.” If you want to use any of these functions, your BPF code has to be declared as having a GPL-compatible license. The verifier (which we will discuss in [Chapter 6](#)) will object if the declared license is not compatible with the functions a program uses. Certain eBPF program types, including those that use BPF LSM (which you’ll learn about in [Chapter 9](#)), are also [required to be GPL compatible](#).

Note

You might be wondering why the previous chapter used `bpf_trace_printk()` and this version uses `bpf_printk()`. The short answer is that BCC’s version is called `bpf_trace_printk()` and *libbpf*’s version is `bpf_printk()`, but both of those are wrappers around the kernel function `bpf_trace_printk()`. Andrii Nakryiko wrote a [good post](#) on this on his blog.

你可能会想为什么之前的章节使用了**bpf_trace_printk()**而这个版本使用**bpf_printk()**。简单来说，BCC版本称为**bpf_trace_printk()**，而libbpf版本称为**bpf_printk()**，但这两个都是对内核函数**bpf_trace_printk()**的封装。Andrii Nakryiko在他的博客上写了一篇很好的文章解释了这个**问题**。

This is an example of an eBPF program that attaches to the XDP hook point on a network interface. You can think of the XDP event being triggered the moment a network packet arrives inbound on a (physical or virtual) network interface.

这是一个示例的eBPF程序，它会附加到网络接口的XDP挂钩点。你可以认为当一个网络数据包从（物理或虚拟）网络接口进站时，XDP事件会被触发。

Note

Some network cards support offloading XDP programs so that they can be executed on the network card itself. This means each network packet that arrives can be processed on the card, before it gets anywhere near the machine's CPU. XDP programs can inspect and even modify each network packet, so this is very useful for doing things like DDoS protection, firewalling, or load balancing in a highly performant way. You'll learn more about this in [Chapter 8](#).

某些网络适配卡支持卸载XDP程序，因此可以在网络适配卡上执行这些程序。这意味着每个到达的网络数据包在接触到计算机的CPU之前就可以在适配卡上进行处理。XDP程序可以检查甚至修改每个网络数据包，因此在高性能的方式进行DDoS防护、防火墙或负载均衡等操作非常有用。你将在第8章中了解更多相关内容。

You have seen the C source code, so the next step is to compile it into an object the kernel can understand.

你已经看到了C源代码，所以下一步是将其编译成内核可以理解的目标文件。

Compiling an eBPF Object File

编译一个eBPF对象文件

Our eBPF source code needs to be compiled into the machine instructions that the eBPF virtual machine can understand: eBPF bytecode. The Clang compiler from the [LLVM project](#) will do this if you specify `-target bpf`. The following is an extract from a Makefile that will do the compilation:

我们的eBPF源代码需要编译成eBPF虚拟机可以理解的机器指令：eBPF字节码。LLVM项目中的Clang编译器可以通过指定`-target bpf`来实现这一点。以下是一个Makefile中的代码段，用于进行编译：

```
hello.bpf.o: %.o: %.c
    clang \
        -target bpf \
        -I/usr/include/$(shell uname -m)-linux-gnu \
        -g \
        -O2 -c $< -o $@
```

This generates an object file called *hello.bpf.o* from the source code in *hello.bpf.c*. The `-g` flag is optional here,³ but it generates debug information so that you can see the source code alongside the bytecode when you inspect the object file. Let's inspect this object file to better understand the eBPF code it contains.

这将从`hello.bpf.c`源代码生成一个名为`hello.bpf.o`的目标文件。`-g`标志在这里是可选的，但它会生成调试信息，以便在检查对象文件时可以看到源代码与字节码并排。让我们检查一下这个对象文件，以更好地理解它所包含的eBPF代码。

Inspecting an eBPF Object File

检查eBPF对象文件

The file utility is commonly used to determine the contents of a file:

文件实用工具通常用于确定文件的内容。

```
$ file hello.bpf.o
hello.bpf.o: ELF 64-bit LSB relocatable, eBPF, version 1
(SYSV), with debug_info,
not stripped
```

This shows it's an ELF (Executable and Linkable Format) file, containing eBPF code, for a 64-bit platform with LSB (least significant bit) architecture. It includes debug information if you used the `-g` flag at the compilation step.

这显示它是一个ELF（可执行和可链接格式）文件，包含了64位平台上LSB（最低有效位）架构的eBPF代码。如果在编译步骤中使用了-g标志，它还包括调试信息。

You can inspect this object further with `llvm-objdump` to see the eBPF instructions:

您可以使用llvm-objdump进一步检查此对象，以查看eBPF指令：

```
$ llvm-objdump -S hello.bpf.o
```

Even if you're not familiar with disassembly, the output from this command isn't too hard to understand:

即使您对反汇编不熟悉，执行此命令后的输出也不太难理解。

```

hello.bpf.o:      file format elf64-bpf                                ❶

Disassembly of section xdp:                                           ❷

0000000000000000 <hello>:                                             ❸
; bpf_printk("Hello World %d", counter);                             ❹
  0:   18 06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
r6 = 0 ll
  2:   61 63 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  3:   18 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
r1 = 0 ll
  5:   b7 02 00 00 0f 00 00 00 00 00 00 00 00 00 00 00 00 00
  6:   85 00 00 00 06 00 00 00 00 00 00 00 00 00 00 00 00 00
; counter++;                                                         ❺
  7:   61 61 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  8:   07 01 00 00 01 00 00 00 00 00 00 00 00 00 00 00 00 00
  9:   63 16 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
; return XDP_PASS;                                                  ❻
10:   b7 00 00 00 02 00 00 00 00 00 00 00 00 00 00 00 00 00
11:   95 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
exit

```

❶

The first line gives further confirmation that *hello.bpf.o* is a 64-bit ELF file with eBPF code (there's no particular rhyme or reason why some tools use the term *BPF* and others *eBPF*; as I said earlier, these terms are now practically interchangeable).

❷

Next comes the disassembly of the section labeled `xdp`, which matches the `SEC()` definition in the C source code.

❸

This section is a function called `hello`.

❹

There are five lines of eBPF bytecode instructions that correspond to the source line

```
bpf_printk("Hello World %d", counter);.
```

5

Three lines of eBPF bytecode instructions increment the `counter` variable.

6

And another two lines of bytecode are generated from the source code `return XDP_PASS;`.

Unless you're particularly keen to do so, there's no real need to understand exactly how each line of bytecode relates to the source. The compiler takes care of generating the bytecode so that you don't have to think about it! But let's examine the output in a little more detail so you can get a feel for how this output relates to the eBPF instructions and registers you learned about earlier in this chapter.

除非你特别想这样做，否则没有必要确切地理解每行字节码与源码的关系。编译器会负责生成字节码，这样你就不必去考虑了！但是，让我们稍微详细地检查一下输出，这样你就能对这个输出与前面本章中学到的eBPF指令和寄存器之间的关系有所了解。

To the left of each line of bytecode you can see the offset of that instruction from wherever `hello` is located in memory. As described earlier in this chapter, eBPF instructions are generally 8 bytes long, and since on a 64-bit platform each memory location can hold 8 bytes, the offset is usually incremented by one for each instruction.

However, the first instruction in this program happens to be a wide instruction encoding that requires 16 bytes in order to set Register 6 to a 64-bit value of 0. That places the instruction in the second line of output at offset 2. After that there is another 16-byte instruction, setting Register 1 to a 64-bit value of 0. And after that, the remaining instructions each fit in 8 bytes, so the offset increments by one in each line.

每行字节码的左侧显示该指令与内存中hello所在位置的偏移量。如前面的章节中所述，eBPF指令通常是8个字节长，在64位平台上，每个内存位置可以容纳8个字节，因此偏移量通常每个指令增加1。然而，这个程序中的第一条指令恰好是一个宽指令编码，需要16个字节来将寄存器6设置为64位值0。这将导致第二行输出中的指令位于偏移量2处。之后还有另一个16字节的指令，将寄存器1设置为64位值0。然后，其余指令每行适合8个字节，所以偏移量每行增加1。

The first byte of each line is the opcode that tells the kernel what operation to perform, and on the right side of each instruction line is the human-readable interpretation of the instruction. At the time of this writing, the [lovisor project](#) has the most complete [documentation](#) of the eBPF opcodes, but the official [Linux kernel documentation](#) is catching up, and the eBPF Foundation is working on [standard documentation](#) that is not tied to a specific operating system.

每行的第一个字节是操作码，它告诉内核执行什么操作，每条指令行的右侧是可读的指令解释。在撰写本文时，lovisor项目拥有最完整的eBPF操作码文档，但官方Linux内核文档正在赶上，并且eBPF基金会正在制定与特定操作系统无关的标准文档。

For example, let's take the instruction at offset 5, which looks like this:

例如，让我们来看一下偏移量为5的指令，它是这样的：

```
5:    b7 02 00 00 0f 00 00 00 r2 = 15
```

The opcode is `0xb7`, and the documentation tells us the pseudocode corresponding to this is `dst = imm`, which can be read as “Set the destination to the immediate value.” The destination is defined by the second byte, `0x02`, which means “Register 2.” The “immediate” (or literal) value here is `0x0f`, which is 15 in decimal. So we can understand that this instruction tells the kernel to “set Register 2 to value 15.” This corresponds to the output we see on the right side of the instruction: `r2 = 15`.

操作码是0xb7，文档告诉我们与之对应的伪代码是 `dest = imm`，可以理解为“将目标设置为立即值”。目标由第二个字节0x02定义，意思是“寄存器2”。这里的“立即值”（字面值）是0x0f，转换为十进制是15。因此，我们可以理解这条指令告诉内核“将寄存器2设置为值15”。这对应于我们在指令右侧看到的输出：`r2 = 15`。

The instruction at offset 10 is similar:

指令在偏移量10处相似：

```
10:    b7 00 00 00 02 00 00 00 r0 = 2
```

This line also has opcode `0xb7`, and this time it's setting the value of Register 0 to 2. When an eBPF program finishes running, Register 0

holds the return code, and `XDP_PASS` has the value `2`. This matches the source code, which always returns `XDP_PASS`.

这一行也有操作码为0xb7，并且这次它将寄存器0的值设为2。当eBPF程序运行完成时，寄存器0保存着返回码，并且XDP_PASS的值为2。这与源代码相符，源代码总是返回XDP_PASS。

You now know that *hello.bpf.o* contains an eBPF program in bytecode. The next step is to load it into the kernel.

现在您已经知道hello.bpf.o中包含了一个以字节码形式表示的eBPF程序。下一步是将它加载到内核中。

Loading the Program into the Kernel

将程序加载到内核中

For this example we'll use a utility called `bpftool`. You can also load programs programmatically, and you'll see examples of that later in the book.

对于这个例子，我们将使用一个名为bpftool的实用程序。您也可以以编程方式加载程序，稍后在本书中将会看到相关的示例。

Note

Some Linux distributions provide a package that includes `bpftool`, or you can [compile it from source code](#). You can find more details about installing or building this tool on [Quentin Monnet's blog](#), as well as additional documentation and usage on the [Cilium site](#).

一些Linux发行版提供了一个包，其中包括bpftool，或者您可以从源代码编译它。您可以在Quentin Monnet的博客上找到有关安装或构建此工具的更多详细信息，以及在Cilium网站上的其他文档和用法。

The following is an example of using `bpftool` to load a program into the kernel. Note that you'll likely need to be root (or use `sudo`) to get the BPF privileges that `bpftool` requires.

以下是使用bpftool将程序加载到内核的示例。请注意，您可能需要以root身份（或使用sudo）获取bpftool所需的BPF权限。

```
$ bpftool prog load hello.bpf.o /sys/fs/bpf/hello
```

This loads the eBPF program from our compiled object file and “pins” it to the location `/sys/fs/bpf/hello`.⁴ No output response to this command indicates success, but you can confirm that the program is in place using `ls`:

这将从我们编译的对象文件加载eBPF程序，并将其“固定”在`/sys/fs/bpf/hello`位置上。没有输出响应表示成功，但您可以使用`ls`命令确认程序已经安装在相应位置。

```
$ ls /sys/fs/bpf
hello
```

The eBPF program has been successfully loaded. Let's use the `bpftool` utility to find out more about the program and its status within the kernel.

eBPF程序已成功加载。让我们使用bpftool实用程序来了解更多关于该程序及其在内核中的状态的信息。

Inspecting the Loaded Program

查看加载的程序

The `bpftool` utility can list all the programs that are loaded into the kernel. If you try this yourself you'll probably see several preexisting eBPF programs in this output, but for clarity I will just show the lines that relate to our “Hello World” example:

`bpftool`实用程序可以列出加载到内核中的所有程序。如果您自己尝试，您可能在输出中看到几个预先存在的eBPF程序，但为了清晰起见，我只会显示与我们的“Hello World”示例相关的行。

```
$ bpftool prog list
...
540: xdp  name hello  tag d35b94b4c0c10efb  gpl
      loaded_at 2022-08-02T17:39:47+0000  uid 0
      xlated 96B  jited 148B  memlock 4096B  map_ids
165,166
      btf_id 254
```

The program has been assigned the ID 540. This identity is a number assigned to each program as it's loaded. Knowing the ID, you can ask `bpftool` to show more information about this program. This time, let's get the output in prettified JSON format so that the field names are visible, as well as the values:

该程序已被分配ID 540。该身份是在加载每个程序时分配的一个号码。通过知道该ID，您可以要求`bpftool`显示有关该程序的更多信息。这次，让我们以美化的JSON格式获得输出，以便字段名称和值都可见。

```
$ bpftool prog show id 540 --pretty
{
  "id": 540,
  "type": "xdp",
  "name": "hello",
  "tag": "d35b94b4c0c10efb",
  "gpl_compatible": true,
  "loaded_at": 1659461987,
  "uid": 0,
  "bytes_xlated": 96,
  "jited": true,
  "bytes_jited": 148,
  "bytes_memlock": 4096,
  "map_ids": [165,166
],
  "btf_id": 254
}
```

Given the field names, a lot of this is straightforward to understand:

考虑到字段名称，很多内容都很容易理解。

- The program's ID is 540.

该程序的ID是540。

- The `type` field tells us this program can be attached to a network interface using the XDP event. Several other types of BPF programs can be attached to different sorts of events, and we'll discuss this more in [Chapter 7](#).

`type`字段告诉我们这个程序可以使用XDP事件附加到网络接口上。还有其他几种类型的BPF程序可以附加到不同类型的事件上，我们将在第七章中详细讨论这个问题。

- The name of the program is `hello`, which is the function name from the source code.

程序的名称是hello，这是源代码中的函数名。

- The `tag` is another identifier for this program, which I'll describe in more detail shortly.

这个标签是这个程序的另一个标识符，我将在稍后详细描述。

- The program is defined with a GPL-compatible license.

该程序以GPL兼容许可协议进行定义。

- There's a timestamp showing when the program was loaded.

程序加载时显示了一个时间戳。

- User ID 0 (which is root) loaded the program.

用户 ID 0（即 root）加载了该程序。

- There are 96 bytes of translated eBPF bytecode in this program, which I'll show you shortly.

这个程序中有96字节的被翻译的eBPF字节码，我马上会展示给你看。

- This program has been JIT-compiled, and the compilation resulted in 148 bytes of machine code. I'll cover this shortly too.

该程序已经经过JIT编译，并且编译结果产生了148字节的机器代码。稍后我会对此进行解释。

- The `bytes _memlock` field tells us this program reserves 4,096 bytes of memory that won't be paged out.

字节_memlock字段告诉我们这个程序保留了4,096字节的内存，不会被分页。

- This program refers to BPF maps with IDs 165 and 166. This might seem surprising, since there is no obvious reference to maps in the source code. You'll see later in this chapter how map semantics are used to handle global data in eBPF programs.

这个程序涉及到ID为165和166的BPF图。这可能令人惊讶，因为在源代码中没有明显的对图的引用。在本章的后面，你将会看到如何使用图语义来处理eBPF程序中的全局数据。

- You'll learn about BTF in [Chapter 5](#), but for now just know that `btf_id` indicates there is a block of BTF information for this program. This information is included in the object file only if you compile with the `-g` flag.

您将在第5章中了解到BTF，但现在只需知道btf_id表示该程序存在一块BTF信息的块。只有在使用-g标志进行编译时，这些信息才会包含在目标文件中。

The BPF Program Tag

BPF程序标签

The `tag` is a SHA (Secure Hashing Algorithm) sum of the program's instructions, which can be used as another identifier for the program. The ID can vary every time you load or unload the program, but the tag will remain the same. The `bpftool` utility accepts references to a BPF program by ID, name, tag, or pinned path, so in the example here, all of the following would give the same output:

标签是程序指令的SHA（安全散列算法）的总和，可用作程序的另一个标识符。每次加载或卸载程序时，ID 可能会变化，但标签将保持不变。bpftool 实用程序可通过 ID、名称、标签或固定路径接受对 BPF 程序的引用，因此在此示例中，以下所有内容将给出相同的输出：

- `bpftool prog show id 540`

bpftool prog show id 540 bpftool程序显示ID 540

- `bpftool prog show name hello`

显示bpftool程序名称hello

- `bpftool prog show tag d35b94b4c0c10efb`

请帮我翻译成简体中文，“bpftool prog show tag d35b94b4c0c10efb”，请只返回翻译的内容，不包含原始文本。

- `bpftool prog show pinned /sys/fs/bpf/hello`

请帮我翻译为简化中文：`bpftool prog show pinned /sys/fs/bpf/hello`

You could have multiple programs with the same name, and even multiple instances of programs with the same tag, but the ID and pinned path will always be unique.

你可以有多个同名的程序，甚至可以有多个带有相同标签的程序实例，但ID和固定路径始终是唯一的。

The Translated Bytecode

请帮我翻译成简体中文：“The Translated Bytecode”。请回答仅包含翻译后的内容，不包含原文内容。

The `bytes_xlated` field tells us how many bytes of “translated” eBPF code there are. This is the eBPF bytecode after it has passed through the verifier (and possibly been modified by the kernel for reasons I’ll discuss later in this book).

字节_已翻译字段告诉我们“已翻译”的eBPF代码有多少字节。这是经过验证器验证后的eBPF字节码（可能由于本书后面将讨论的原因，经过内核修改的字节码也可能包括在内）。

Let’s use `bpftool` to show this translated version of our “Hello World” code:

让我们使用bpftool显示我们的“Hello World”代码的转换版本：

```
$ bpftool prog dump xlated name hello
int hello(struct xdp_md * ctx):
; bpf_printk("Hello World %d", counter);
  0: (18) r6 = map[id:165][0]+0
  2: (61) r3 = *(u32 *)(r6 +0)
  3: (18) r1 = map[id:166][0]+0
  5: (b7) r2 = 15
  6: (85) call bpf_trace_printk#-78032
; counter++;
  7: (61) r1 = *(u32 *)(r6 +0)
  8: (07) r1 += 1
  9: (63) *(u32 *)(r6 +0) = r1
; return XDP_PASS;
10: (b7) r0 = 2
11: (95) exit
```

This looks very similar to the disassembled code you saw earlier in the output from `llvm-objdump`. The offset addresses are the same,

and the instructions look similar—for example, we can see the instruction at offset 5 is `r2=15`.

这看起来与您之前从llvm-objdump的输出中看到的反汇编代码非常相似。偏移地址相同，指令看起来也很相似——例如，我们可以看到偏移地址为5的指令是r2=15。

The JIT-Compiled Machine Code

JIT 编译的机器码

The translated bytecode is pretty low level, but it's not quite machine code yet. eBPF uses a JIT compiler to convert eBPF bytecode to machine code that runs natively on the target CPU. The `bytes_jited` field shows that after this conversation the program is 108 bytes long.

翻译的字节码相当底层，但尚未成为机器码。eBPF 使用即时编译器将 eBPF 字节码转换为在目标 CPU 上本机运行的机器码。bytes_jited 字段表明，在此转换之后，程序的长度为108字节长。

Note

For higher performance, eBPF programs are generally JIT-compiled. The alternative is to interpret the eBPF bytecode at runtime. The eBPF instruction set and registers were designed to map fairly closely to native machine instructions to make this interpretation straightforward and therefore relatively fast, but compiled programs will be faster, and most architectures now support JIT.⁵

为了获得更高的性能，eBPF程序通常会进行即时编译（JIT-compiled）。另一种选择是在运行时解释eBPF的字节码。eBPF的指令集和寄存器的设计是为了与本地机器指令相对应，以使解释变得简单直接和相对较快，但编译后的程序将更快，并且现在大多数架构都支持JIT。

The `bpftool` utility can generate a dump of this JITed code in assembly language. Don't worry if you're not familiar with assembly language and this looks entirely incomprehensible! I have included it only to illustrate all the transformations the eBPF code goes through from source code to the executable machine instructions. Here is the command and its output:

bpftool工具可以生成这个JIT代码的汇编语言转储。如果你不熟悉汇编语言，并且这些内容看起来完全难以理解，不要担心！我只是为了说明从源代码到可执行机器指令的所有变换过程，才包含了它。下面是命令及其输出：请帮忙将此翻译为简体中文。

```
$ bpftool prog dump jited name hello
int hello(struct xdp_md * ctx):
bpf_prog_d35b94b4c0c10efb_hello:
; bpf_printk("Hello World %d", counter);
  0:   hint      #34
  4:   stp       x29, x30, [sp, #-16]!
  8:   mov       x29, sp
  c:   stp       x19, x20, [sp, #-16]!
 10:   stp       x21, x22, [sp, #-16]!
 14:   stp       x25, x26, [sp, #-16]!
 18:   mov       x25, sp
 1c:   mov       x26, #0
 20:   hint      #36
 24:   sub       sp, sp, #0
 28:   mov       x19, #-140733193388033
 2c:   movk      x19, #2190, lsl #16
 30:   movk      x19, #49152
```



```

34:    mov     x10, #0
38:    ldr     w2, [x19, x10]
3c:    mov     x0, #-205419695833089
40:    movk    x0, #709, lsl #16
44:    movk    x0, #5904
48:    mov     x1, #15
4c:    mov     x10, #-6992
50:    movk    x10, #29844, lsl #16
54:    movk    x10, #56832, lsl #32
58:    blr     x10
5c:    add     x7, x0, #0
; counter++;
60:    mov     x10, #0
64:    ldr     w0, [x19, x10]
68:    add     x0, x0, #1
6c:    mov     x10, #0
70:    str     w0, [x19, x10]
; return XDP_PASS;
74:    mov     x7, #2
78:    mov     sp, sp
7c:    ldp     x25, x26, [sp], #16
80:    ldp     x21, x22, [sp], #16
84:    ldp     x19, x20, [sp], #16
88:    ldp     x29, x30, [sp], #16
8c:    add     x0, x7, #0
90:    ret

```

Note

Some packaged distributions of `bpftool` don't yet include support for dumping the JITed output, and if that's the case, you'll see "Error: No libbfd support." You can build `bpftool` for yourself by following the instructions at <https://github.com/libbpf/bpftool>.

一些打包的bpftool发行版尚不包括对JIT输出的支持，如果是这种情况，您会看到“错误：无libbfd支持。”您可以按照<https://github.com/libbpf/bpftool>上的说明自行构建bpftool。

You’ve seen that the “Hello World” program has been loaded into the kernel, but at this point it’s not yet associated with an event, so nothing will trigger it to run. It needs to be attached to an event.

您已经看到“Hello World”程序已加载到内核中，但在这一点上尚未与事件关联，因此没有任何触发条件来运行它。它需要与一个事件关联。

Attaching to an Event

附加到事件

The program type has to match the type of event it’s being attached to; you’ll learn more about this in [Chapter 7](#). In this case it’s an XDP program, and you can use `bpftool` to attach the example eBPF program to the XDP event on a network interface, like this:

程序类型必须匹配其附加到的事件类型；您将在第7章中了解更多相关信息。在这种情况下，它是一个XDP程序，您可以使用bpftool将示例eBPF程序附加到网络接口上的XDP事件，例如：

```
$ bpftool net attach xdp id 540 dev eth0
```

Note

At the time of this writing, the `bpftool` utility doesn’t support the ability to attach all program types, but it has [recently been extended](#) to auto-attach k(ret)probes, u(ret)probes, and tracepoints.

在撰写本文时，bpftool 实用程序不支持附加所有程序类型的能力，但最近已扩展为自动附加 k(ret)probes、u(ret)probes 和 tracepoints。

Here I have used the program's ID of 540, but you can also use the name (provided it is unique) or tag to identify the program being attached. In this example, I have attached the program to the network interface `eth0`.

在这里，我使用了程序的ID为540，但你也可以使用名称（前提是它是唯一的）或标签来标识所附加的程序。在这个例子中，我将程序附加到了网络接口eth0。

You can view all the network-attached eBPF programs using `bpftool`:

您可以使用bpftool来查看所有的网络附加的eBPF程序：

```
$ bpftool net list
xdp:
eth0(2) driver id 540

tc:

flow_dissector:
```

The program with ID 540 is attached to the XDP event on the `eth0` interface. This output also gives some clues about some other potential events in the network stack that you can attach eBPF programs to: `tc` and `flow_dissector`. More on this in [Chapter 7](#).

程序ID为540的程序附加到eth0接口的XDP事件上。此输出还提供了有关网络堆栈中其他可能的事件的一些线索，您可以将eBPF程序附加到：tc和flow_dissector。有关详细信息，请参阅第7章。

You can also inspect the network interfaces using `ip link`, and you'll see output that looks something like this (some details have been

removed for clarity):

您还可以使用 `ip link` 检查网络接口，您将看到类似于下面的输出（为了清晰起见，一些细节已被删除）：

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue
state UNKNOWN mode DEFAULT
group default qlen 1000
...
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 xdp
qdisc fq_codel state UP
mode DEFAULT group default qlen 1000
...
    prog/xdp id 540 tag 9d0e949f89f1a82c jited
...
```

In this example there are two interfaces: the loopback interface `lo`, which is used to send traffic to processes on this machine; and the `eth0` interface, which connects this machine to the outside world. This output also shows that `eth0` has a JIT-compiled eBPF program, with identity `540` and tag `9d0e949f89f1a82c`, attached to its XDP hook.

在这个示例中有两个接口：回环接口 `lo`，用于将流量发送到本机的进程；以及 `eth0` 接口，将该机器连接到外部世界。该输出还显示了，`eth0` 具有一个 JIT 编译的 eBPF 程序，其身份为 `540`，标记为 `9d0e949f89f1a82c`，附加到其 XDP hook 上。

Note

You can also use `ip link` to attach and detach XDP programs to a network interface. I have included this as an exercise at the end of this chapter, and there are further examples in [Chapter 7](#).

您还可以使用`ip link`命令将XDP程序附加到和分离出网络接口。我已在本章末尾包含了这个作为一个练习，并在第7章中还有更多的示例。

At this point, the *hello* eBPF program should be producing trace output every time a network packet is received. You can check this out by running `cat /sys/kernel/debug/tracing/trace_pipe`. This should show a lot of output that looks similar to this:

此时，`hello` eBPF程序应该在每次接收到网络数据包时产生跟踪输出。您可以通过运行`cat /sys/kernel/debug/tracing/trace_pipe`来检查。这应该会显示很多类似于以下内容的输出：

```
<idle>-0      [003] d.s.. 655370.944105:
bpf_trace_printk: Hello World 4531
<idle>-0      [003] d.s.. 655370.944587:
bpf_trace_printk: Hello World 4532
<idle>-0      [003] d.s.. 655370.944896:
bpf_trace_printk: Hello World 4533
```

If you're struggling to remember the location of the trace pipe, you can get the same output using the command

```
bpftool prog tracelog.
```

如果您无法记住追踪管道的位置， 您可以使用命令`bpftool prog tracelog`获得相同的输出结果。

In comparison to the output you saw in [Chapter 2](#), this time there is no command or process ID associated with each of these events; instead, you see `<idle>-0` at the start of each line of trace. In [Chapter 2](#), each syscall event happened because a process executing a command in user space made a call to the syscall API.

That process ID and command are part of the context in which the eBPF program was executed. But in the example here, the XDP event happens due to the arrival of a network packet. There is no user space process associated with this packet—at the point the *hello* eBPF program is triggered, the system hasn't done anything with the packet other than receive it in memory, and it has no idea what the packet is or where it's going.

与您在第2章看到的输出相比，这次每个事件都没有与之相关联的命令或进程ID；相反，您在每行跟踪的开头看到的是<空闲>-0。在第2章中，每个syscall事件发生的原因是一个正在用户空间执行命令的进程调用了syscall API。这个进程ID和命令是执行eBPF程序的上下文的一部分。但在这个例子中，XDP事件是由网络数据包的到达引发的。这个数据包没有与之关联的用户空间进程-在触发hello eBPF程序时，系统除了将数据包接收到内存中之外，对该数据包没有做任何操作，也不知道数据包是什么或者它要去向何方。

You can see that the counter value that is traced out is being incremented by one each time, as expected. In the source code, `counter` is a global variable. Let's see how that is implemented in eBPF using a map.

您可以看到追踪出的计数器值每次增加一，正如预期。在源代码中，`counter`是一个全局变量。让我们看看如何在eBPF中实现这一点，使用一个映射。

Global Variables

全局变量

As you learned in the previous chapter, an eBPF map is a data structure that can be accessed from an eBPF program or from user space. Since the same map can be accessed repeatedly by different runs of the same program, it can be used to hold state from one execution to the next. Multiple programs can also access the same map. Because of these characteristics, map semantics can be repurposed for use as global variables.

正如您在上一章中学到的那样，eBPF映射是一种可以从eBPF程序或用户空间访问的数据结构。由于同一个映射可以被同一个程序的不同运行重复访问，它可以用于保存从一次执行到下一次的状态。多个程序也可以访问同一个映射。由于这些特性，映射语义可以被重新用作全局变量。

Note

Before support for [global variables was added in 2019](#), eBPF programmers had to write maps explicitly to perform the same task.

在2019年添加对全局变量的支持之前，eBPF程序员必须显式地编写地图来执行相同的任务。

You saw earlier that `bpftool` shows this example program using two maps with the identities 165 and 166. (You will probably see different identities if you try this for yourself, as the identities are assigned when the maps are created in the kernel.) Let's explore what is in those maps.

你之前看到bpftool显示了一个使用两个映射的示例程序，它们的标识分别是165和166。（如果你自己尝试这个示例，你可能会看到不同的标识，因为标识是在内核中创建映射时分配的。）让我们来探索这些映射中的内容。

The `bpftool` utility can show the maps loaded into the kernel. For clarity I will only show the entries 165 and 166 that relate to the example “Hello World” program:

bpftool工具可以显示加载到内核中的映射。为了清晰起见，我只会显示与示例“Hello World”程序相关的条目165和166。

```
$ bpftool map list
165: array name hello.bss flags 0x400
      key 4B value 4B max_entries 1 memlock 4096B
      btf_id 254
166: array name hello.rodata flags 0x80
      key 4B value 15B max_entries 1 memlock 4096B
      btf_id 254 frozen
```

A `bss`⁶ section in an object file compiled from a C program typically holds global variables, and you can use `bpftool` to inspect its contents, like this:

在从C程序编译的目标文件中，一个 `bss` 段通常保存全局变量，您可以使用 `bpftool` 检查其内容，就像这样：

```
$ bpftool map dump name hello.bss
[{"value": {"bss": [{"counter": 11127}]}]}
```



```
}  
]
```

I could also have used `bpftool map dump id 165` to retrieve the same information. If I run either of these commands again, I'll see that the counter has increased, as the program has been run every time a network packet is received.

我也可以使用`bpftool map dump id 165`来获取相同的信息。如果我再次运行任何一个命令，我会看到计数器增加，因为每次接收到网络数据包时都会运行该程序。

As you'll learn in [Chapter 5](#), `bpftool` is able to pretty-print the field names from a map (here, the variable name `counter`) only if BTF information is available, and that information is included only if you compile with the `-g` flag. If you omitted that flag during the compilation step, you'd see something that looks more like this:

正如你在第5章中将学到的那样，只有在BTF信息可用且包含在编译时使用-g标志编译时，bpftool才能从映射中漂亮地打印字段名称（在这里是变量名counter）。如果在编译步骤中省略了该标志，你会看到类似以下内容：

```
$ bpftool map dump name hello.bss  
key: 00 00 00 00 value: 19 01 00 00  
Found 1 element
```

Without BTF information, `bpftool` has no way of knowing what variable name was used in the source code. You can infer that since there is only one item in this map, the hex value `19 01 00 00` must

be the current value of `counter` (281 in decimal, since the bytes are ordered starting with the least significant byte).

没有BTF信息，bpftool无法知道在源代码中使用了哪个变量名。你可以推断出，由于这个映射中只有一个项目，十六进制值19 01 00 00必须是counter的当前值（十进制为281，因为字节的顺序是从最低有效字节开始的）。

You've seen here that the eBPF program uses the semantics of a map to read and write to a global variable. Maps are also used to hold static data, as you can see by inspecting the other map.

你在这里看到eBPF程序使用映射的语义读写全局变量。映射也用于保存静态数据，通过检查其他映射可以看到。

The fact that the other map is named `hello.rodata` gives a hint that this could be read-only data related to our *hello* program. You can dump the contents of this map to see that it holds the string used by the eBPF program for tracing:

另一个地图被命名为hello.rodata这一事实表明这可能是与我们的hello程序相关的只读数据。您可以转储这个地图的内容，以查看它保存了eBPF程序用于跟踪的字符串。

```
$ bpftool map dump name hello.rodata
[{"value": {"rodata": [{"hello.____fmt": "Hello World %d"}]
}]
```

```
}  
]
```

If you didn't compile the object with the `-g` flag, you'll see output that looks like this:

如果你没有使用-g标志编译对象，你会看到类似于这样的输出：

```
$ bpftool map dump id 166  
key: 00 00 00 00 value: 48 65 6c 6c 6f 20 57 6f 72 6c 64  
20 25 64 00  
Found 1 element
```

There is one key-value pair in this map, and the value contains 12 bytes of data ending with a 0. It probably won't surprise you that those bytes are the ASCII representation of the string

```
"Hello World %d".
```

这个地图中有一个键值对，值包含12个字节的数据，以0结尾。这些字节很可能是字符串"Hello World %d"的ASCII表示形式。

Now that we've finished inspecting this program and its maps, it's time to clean it up. We'll start by detaching it from the event that triggers it.

既然我们已经完成了对这个程序及其地图的检查，现在是时候对其进行清理了。我们将从将其与触发它的事件分离开始。

Detaching the Program

分离程序

You can detach the program from the network interface like this:

您可以这样将程序从网络接口中分离开：

```
$ bpftool net detach xdp dev eth0
```

There is no output if this command runs successfully, but you can confirm that the program is no longer attached by the lack of XDP entries in the output from `bpftool net list`:

如果此命令成功运行，则没有任何输出，但是您可以通过`bpftool net list`的输出中缺少XDP条目来确认程序不再附加。

```
$ bpftool net list
xdp:

tc:

flow_dissector:
```

However, the program is still loaded into the kernel:

然而，程序仍然加载到内核中：

```
$ bpftool prog show name hello
395: xdp name hello tag 9d0e949f89f1a82c gpl
      loaded_at 2022-12-19T18:20:32+0000 uid 0
      xlated 48B jited 108B memlock 4096B map_ids 4
```

Unloading the Program

卸载程序

There's no inverse of `bpftool prog load` (at least not at the time of this writing), but you can remove the program from the kernel by deleting the pinned pseudofile:

目前为止，`bpftool prog load` 没有对应的反向操作，但是您可以通过删除固定的伪文件来从内核中移除该程序。

```
$ rm /sys/fs/bpf/hello
$ bpftool prog show name hello
```

There is no output from this `bpftool` command because the program is no longer loaded in the kernel.

因为该程序不再加载到内核中，所以从此bpftool命令中没有输出结果。

BPF to BPF Calls

BPF到BPF调用

In the previous chapter you saw tail calls in action, and I mentioned that now there is also the ability to call functions from within an eBPF program. Let's take a look at a simple example, which, like the tail call example, can be attached to the `sys_enter` tracepoint, except this time it will trace out the opcode for the syscall. You'll find the code in *chapter3/hello-func.bpf.c*.

在前面的章节中，你看到了尾调用的实际操作，并且我提到现在还可以在eBPF程序中调用函数。让我们看一个简单的示例，与尾调用示例类似，可以将其附加到sys_enter跟踪点上，但这一次它将跟踪系统调用的操作码。你可以在第三章的hello-func.bpf.c文件中找到这段代码。

For illustrative purposes I have written a very simple function that extracts the syscall opcode from the tracepoint arguments:

为了说明目的，我编写了一个非常简单的函数，从跟踪参数中提取系统调用的操作码。

```
static __attribute__((noinline)) int get_opcode(struct
bpf_raw_tracepoint_args

ctx) {
    return ctx->args[1];
}
```

Given the choice, the compiler would probably inline this very simple function that I'm only going to call from one place. Since that would defeat the point of this example, I have added

`__attribute__((noinline))` to force the compiler's hand. In normal circumstances you should probably omit this and allow the compiler to optimize as it sees fit.

如果给予选择，编译器可能会将此非常简单的函数内联，因为我只会在一个地方调用它。由于这将破坏这个示例的目的，我添加了 `__attribute__((noinline))` 以强制编译器不进行内联优化。在正常情况下，您应该省略此选项，让编译器根据需要进行优化。

The eBPF function that calls this function looks like this:

这个函数调用的eBPF函数是这样的:

```
SEC("raw_tp")
int hello(struct bpf_raw_tracepoint_args *ctx) {
    int opcode = get_opcode(ctx);
    bpf_printk("Syscall: %d", opcode);
}
```

```
    return 0;
}
```

After compiling this to an eBPF object file, you can load it into the kernel and confirm that it is loaded with `bpftool`:

编译生成eBPF对象文件后，您可以将其加载到内核中，并使用bpftool确认其是否已加载。

```
$ bpftool prog load hello-func.bpf.o /sys/fs/bpf/hello
$ bpftool prog list name hello
893: raw_tracepoint name hello tag 3d9eb0c23d4ab186 gpl
      loaded_at 2023-01-05T18:57:31+0000 uid 0
      xlated 80B jited 208B memlock 4096B map_ids 204
      btfl_id 302
```

The interesting part of this exercise is inspecting the eBPF bytecode to see the `get_opcode()` function:

这个练习中有趣的部分是检查eBPF字节码来查看get_opcode()函数。

```
$ bpftool prog dump xlated name hello
int hello(struct bpf_raw_tracepoint_args * ctx):
; int opcode = get_opcode(ctx);
❶
    0: (85) call pc+7#bpf_prog_cbacc90865b1b9a5_get_opcode
; bpf_printk("Syscall: %d", opcode);
    1: (18) r1 = map[id:193][0]+0
    3: (b7) r2 = 12
    4: (bf) r3 = r0
    5: (85) call bpf_trace_printk#-73584
; return 0;
    6: (b7) r0 = 0
    7: (95) exit
int get_opcode(struct bpf_raw_tracepoint_args * ctx):
❷
; return ctx->args[1];
    8: (79) r0 = *(u64 *) (r1 +8)
; return ctx->args[1];
    9: (95) exit
```

❶

Here you can see the `hello()` eBPF program making a call to `get_opcode()`. The eBPF instruction at offset 0 is `0x85`, which from the instruction set documentation corresponds to “Function call.” Instead of executing the next instruction, which would be at offset 1, execution will jump seven instructions ahead (`pc+7`), which means the instruction at offset 8.

❷

Here’s the bytecode for `get_opcode()`, and as you might hope, the first instruction is at offset 8.

The function call instruction necessitates putting the current state on the eBPF virtual machine’s stack so that when the called function exits, execution can continue in the calling function. Since the stack size is limited to 512 bytes, BPF to BPF calls can’t be very deeply nested.

函数调用指令需要将当前状态放入eBPF虚拟机的堆栈中，这样当被调用的函数退出时，执行可以继续从调用函数中进行。由于堆栈大小限制为512字节，BPF到BPF的调用不能太深嵌套。

Note

For a lot more detail on tail calls and BPF to BPF calls, there’s an excellent post by Jakub Sitnicki on Cloudflare’s blog: [“Assembly within! BPF tail calls on x86 and ARM”](#).

关于尾调用和BPF到BPF调用的更多细节，Cloudflare的博客上有Jakub Sitnicki撰写的一篇很棒的文章：“汇编内涵！x86和ARM上的BPF尾调用”。

Summary

总结

In this chapter you saw some example C source code transformed into eBPF bytecode and then compiled to machine code so that it's ready to be executed in the kernel. You also learned how to use `bpftool` to inspect programs and maps loaded into the kernel, and to attach to XDP events.

在本章中，您看到了一些示例C源代码转换为eBPF字节码，然后编译为机器代码，以便在内核中准备好执行。您还学会了如何使用bpftool来检查加载到内核中的程序和映射，并附加到XDP事件。

In addition, you saw examples of different types of eBPF programs triggered by different kinds of events. An XDP event is triggered by the arrival of a packet of data on a network interface, whereas kprobe and tracepoint events are triggered by hitting some particular point in kernel code. I'll discuss some other eBPF program types in [Chapter 7](#).

此外，您还看到了由不同类型的事件触发的不同类型的eBPF程序示例。XDP事件通过在网络接口上收到数据包来触发，而kprobe和tracepoint事件通过命中内核代码中的特定点来触发。我将在第七章讨论其他一些eBPF程序类型。

You also learned how maps are used to implement global variables for eBPF programs, and you saw BPF to BPF function calls.

您还学习了如何使用地图来实现eBPF程序的全局变量，并且您还看到了BPF到BPF的函数调用。

The next chapter goes into another level of detail as I show you what's happening at the system call level when `bpftool`—or any other user space code—loads programs and attaches them to events.

下一章将进一步详细介绍，在此我将展示在系统调用级别下发生的情况，当bpftool或任何其他用户空间代码加载程序并将其附加到事件上时。

Exercises

练习

Here are a few things to try if you want to explore BPF programs further:

如果你想进一步探索BPF程序，可以尝试以下几件事：

1. Try using `ip link` commands like the following to attach and detach the XDP program:

尝试使用以下的ip link命令来附加和分离XDP程序：

```
$ ip link set dev eth0 xdp obj hello.bpf.o sec xdp
$ ip link set dev eth0 xdp off
```

2. Run any of the BCC examples from [Chapter 2](#). While the program is running, use a second terminal window to inspect the loaded program using `bpftool`. Here's an example of what I saw by running the *hello-map.py* example:

运行来自第2章的任何BCC示例。在程序运行时，使用第二个终端窗口使用bpftool来检查加载的程序。这是运行hello-map.py示例时我看到的一个例子：

```
$ bpftool prog show name hello
197: kprobe name hello tag ba73a317e9480a37 gpl
      loaded_at 2022-08-22T08:46:22+0000 uid 0
      xlated 296B jited 328B memlock 4096B map_ids
65
      btf_id 179
      pids hello-map.py(2785)
```

You can also use `bpftool prog dump` commands to see the bytecode and machine code versions of those programs.

你也可以使用`bpftool prog dump`命令来查看这些程序的字节码和机器码版本。

3. Run *hello-tail.py* from the *chapter2* directory, and while it's running, take a look at the programs it loaded. You'll see that each tail call program is listed individually, like this:

在chapter2目录中运行hello-tail.py，并在其运行时查看加载的程序。您会看到每个尾递归调用程序被单独列出，像这样：

```
$ bpftool prog list
...
120: raw_tracepoint name hello tag b6bfd0e76e7f9aac
gpl
```

```

loaded_at 2023-01-05T14:35:32+0000 uid 0
xlated 160B jited 272B memlock 4096B map_ids
29
    btf_id 124
    pids hello-tail.py(3590)
121: raw_tracepoint name ignore_opcode tag
a04f5eef06a7f555 gpl
    loaded_at 2023-01-05T14:35:32+0000 uid 0
    xlated 16B jited 72B memlock 4096B
    btf_id 124
    pids hello-tail.py(3590)
122: raw_tracepoint name hello_exec tag
931f578bd09da154 gpl
    loaded_at 2023-01-05T14:35:32+0000 uid 0
    xlated 112B jited 168B memlock 4096B
    btf_id 124
    pids hello-tail.py(3590)
123: raw_tracepoint name hello_timer tag
6c3378ebb7d3a617 gpl
    loaded_at 2023-01-05T14:35:32+0000 uid 0
    xlated 336B jited 356B memlock 4096B
    btf_id 124
    pids hello-tail.py(3590)

```

You could also use `bpftool prog dump xlated` to look at the bytecode instructions and compare them to what you saw in “BPF to BPF Calls”.

你也可以使用`bpftool prog dump xlated`来查看字节码指令，并将其与“BPF to BPF Calls”中看到的内容进行比较。

4. *Be careful with this one, as it may be best to simply think about why this happens rather than trying it!* If you return a `0` value from an XDP program, this corresponds to `XDP_ABORTED`, which tells the kernel to abort any further processing of this packet. This might seem a bit counterintuitive given that the `0` value usually indicates success in C, but that’s how it is. So, if you try modifying the program to return `0` and attach it to a virtual

machine's `eth0` interface, all network packets will get dropped. This will be somewhat unfortunate if you're using SSH to attach to that machine, and you'll likely have to reboot the machine to regain access!

小心这一点，最好只是考虑为什么会发生这种情况，而不是尝试它！如果你从一个XDP程序返回一个0值，这对应于 `XDP_ABORTED`，它告诉内核终止对这个数据包的进一步处理。这可能在某种程度上有些违反直觉，因为在C语言中，0值通常表示成功，但事实就是如此。因此，如果你尝试修改程序以返回0，并将其附加到虚拟机的eth0接口上，所有的网络包都将被丢弃。如果你正在使用SSH连接到该机器，这将是相当不幸的，你可能需要重新启动机器才能恢复访问！

You could run the program within a container so that the XDP program is attached to a virtual Ethernet interface that only affects that container and not the whole virtual machine. There's an example of doing this at <https://github.com/lizrice/lb-from-scratch>.

您可以在容器中运行程序，以便将XDP程序附加到虚拟以太网接口，该接口仅影响该容器而不影响整个虚拟机。具体操作示例请参考<https://github.com/lizrice/lb-from-scratch>。

¹ Increasingly, eBPF programs are also being written in Rust, since the Rust compiler supports eBPF bytecode as a target.

越来越多的eBPF程序也开始使用Rust编写，因为Rust编译器支持eBPF字节码作为目标。

² There are a few instructions where the operation is “modified” by the value of other fields in the instruction. For example, there are a set of [atomic instructions](#) introduced in kernel 5.12 that include an arithmetic operation (`ADD`, `AND`, `OR`, `XOR`) that is specified in the `imm` field.

有一些指令的操作根据指令中其他字段的值而“修改”。例如，在内核 5.12 中引入了一组原子指令，其中包括一个在 `imm` 字段中指定的算术操作（`ADD`、`AND`、`OR`、`XOR`）。

³ The `-g` flag is required to generate BTF information that you’ll need for CO-RE eBPF programs, which I’ll cover in [Chapter 5](#).

`-g` 标志是生成 BTF 信息所必需的，这些信息在第 5 章中会涉及到 CO-RE eBPF 程序。

⁴ In general, this is optional—eBPF programs can be loaded into the kernel without being pinned to a file location—but it’s not optional for `bpftool`, which always has to pin the programs it loads. The reason for this is covered further in [“BPF Program and Map References”](#).

一般来说，这是可选项 - eBPF 程序可以在不固定文件位置的情况下加载到内核中，但对于 `bpftool` 而言，这是不可选的，它必须始终固定加载的程序。这样做的原因在“BPF 程序和映射引用”中有进一步解释。

⁵ The kernel setting `CONFIG_BPF_JIT` needs to be enabled to take advantage of JIT compilation, and it can be enabled or disabled at runtime with the `net.core.bpf_jit_enable sysctl` setting. See [the](#)

[docs](#) for more information on JIT support on different chip architectures.

内核设置CONFIG_BPF_JIT需要启用以利用JIT编译，可以在运行时使用net.core.bpf_jit_enable sysctl设置来启用或禁用。有关不同芯片架构上JIT支持的更多信息，请参阅文档。

⁶ Here, *bss* stands for “block started by symbol.”

这里，*bss*代表"以符号开始的块"。

Chapter 4. The bpf() System Call

第四章 bpf() 系统调用。

As you saw in [Chapter 1](#), when user space applications want the kernel to do something on their behalf, they make requests using the system call API. It therefore makes sense that if a user space application wants to load an eBPF program into the kernel, there must be some system calls involved. In fact, there's a system call named `bpf()`, and in this chapter I'll show you how it's used to load and interact with eBPF programs and maps.

正如您在第一章中所看到的那样，当用户空间应用程序希望内核代表它们执行某些操作时，它们会使用系统调用API发出请求。因此，如果用户空间应用程序想要将一个eBPF程序加载到内核中，就必须涉及一些系统调用。实际上，有一个名为bpf()的系统调用，在本章中我将展示如何使用它来加载和与eBPF程序和映射进行交互。

It's worth noting that the eBPF code running in the kernel does not use syscalls to access maps. The syscall interface is only used by user space applications. Instead, eBPF programs use helper functions to read and write to maps; you already saw examples of this in the previous two chapters.

值得注意的是，运行在内核中的 eBPF 代码并不使用系统调用来访问映射。系统调用接口仅由用户空间应用程序使用。相反，eBPF 程序使用辅助函数来读取和写入映射；你在前两章中已经看到了这方面的示例。

If you go on to write eBPF programs yourself, there's a good chance you won't directly call these `bpf()` system calls yourself. There are libraries that I'll discuss later in the book that offer higher-level abstractions to make things easier. That said, those abstractions generally map pretty directly to the underlying syscall commands you'll see in this chapter. Whatever library you're using, you'll need a grasp of the underlying operations—loading a program, creating and accessing maps, and so on—that you'll see in this chapter.

如果您继续自己编写eBPF程序，很有可能您不会直接调用这些bpf()系统调用。本书后面将会讨论一些库，这些库提供了更高级的抽象，使事情变得更容易。尽管如此，这些抽象通常与本章中所见到的底层系统调用命令非常相似。无论您使用哪个库，您都需要掌握本章中所见到的底层操作，如加载程序、创建和访问映射等。

Before I show you examples of `bpf()` system calls, let's consider what the [manpage for `bpf\(\)`](#) says, which is that `bpf()` is used to “perform a command on an extended BPF map or program.” It also tells us that `bpf()`'s signature is as follows:

在展示bpf()系统调用的示例之前，让我们先看看bpf()的manpage（手册页）上所说的，即bpf()被用于“在扩展BPF映射或程序上执行命令”。它还告诉我们bpf()的签名如下：

```
int bpf(int cmd, union bpf_attr *attr, unsigned int size);
```


The first argument to `bpf()`, `cmd`, specifies which command to perform. The `bpf()` syscall doesn't just do one thing—there are lots of different commands that can be used to manipulate eBPF programs and maps. Figure 4-1 shows an overview of some of the common commands the user space code might use to load eBPF programs, create maps, attach programs to events, and access the key–value pairs in a map.

`bpf()` 的第一个参数`cmd`指定要执行的命令。`bpf()` 系统调用不仅仅执行一个任务 - 有许多不同的命令可用于操作eBPF程序和映射。图4-1概述了用户空间代码可能用于加载eBPF程序，创建映射，将程序附加到事件并访问映射中的键值对的一些常见命令。

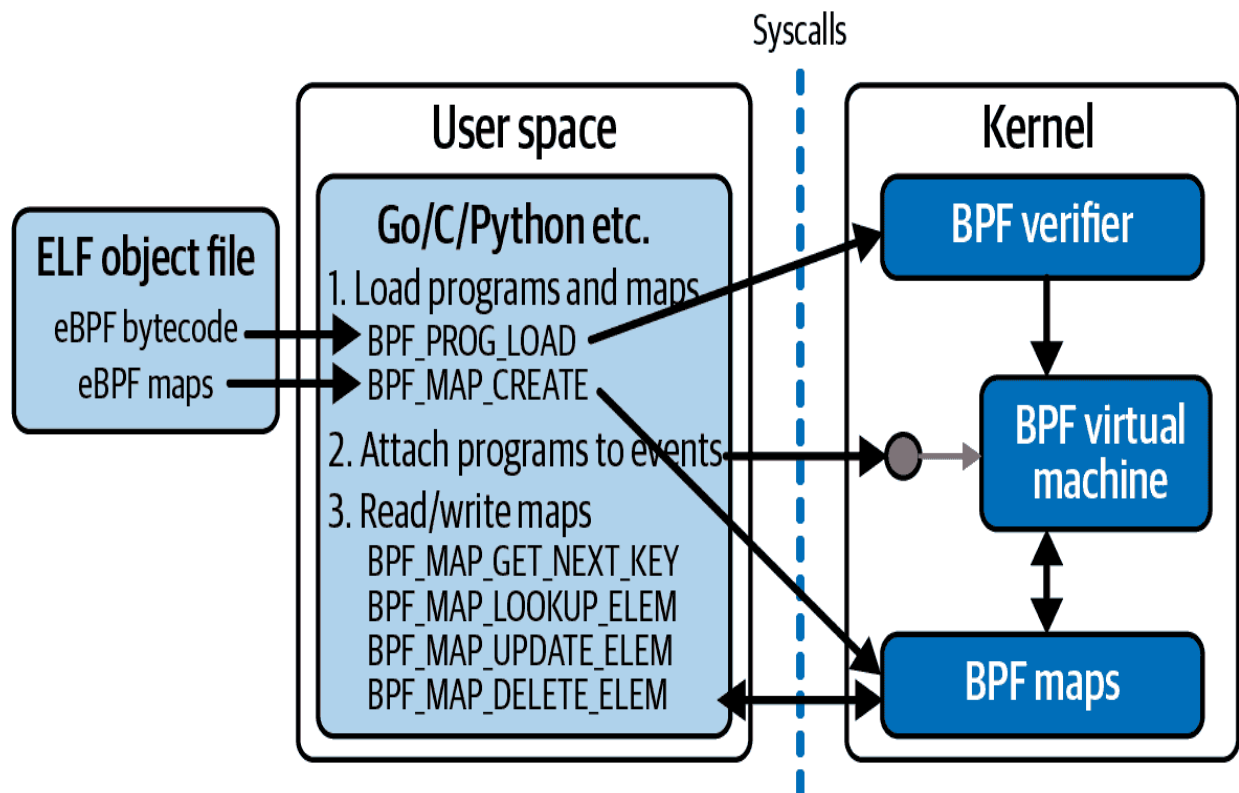


Figure 4-1. A userspace program interacts with eBPF programs and maps in the kernel using syscalls

The `attr` argument to the `bpf()` syscall holds whatever data is needed to specify the parameters for the command, and `size` indicates how many bytes of data there are in `attr`.

`bpf()` 系统调用的`attr`参数保存着用于指定命令参数所需的任何数据，`size`参数表示`attr`中有多少个字节的数据。

You already met `strace` in Chapter 1, when I used it to show how user space code makes many requests across the syscall API. In this chapter I'll use it to demonstrate how the `bpf()` syscall is used. The output from `strace` includes the arguments to each syscall, but to keep the example output in this

chapter from being too cluttered, I'll omit lots of the details from the `attr` arguments unless they are particularly interesting.

你已经在第一章中遇到过`strace`，那时我使用它来展示用户空间代码如何通过系统调用API发出多个请求。在本章中，我将使用它来演示如何使用`bpf()`系统调用。`strace`的输出包括每个系统调用的参数，但为了避免本章的示例输出过于杂乱，除非特别有趣，我会省略`attr`参数的许多细节。

Note

You'll find the code, along with instructions for setting up an environment to run it in, at github.com/lizrice/learning-ebpf. The code for this chapter is in the *chapter4* directory.

您可以在github.com/lizrice/learning-ebpf找到代码，以及设置运行环境的说明。本章的代码位于chapter4目录中。

For this example I'm going to use a BCC program called *hello-buffer-config.py*, which builds on the examples you saw in [Chapter 2](#). Like the *hello-buffer.py* example, this program sends a message to the perf buffer whenever it runs, conveying information from the kernel to user space about `execve()` syscall events. What's new in this version is that it allows for different messages to be configured for each user ID.

在这个例子中，我将使用一个名为`hello-buffer-config.py`的BCC程序，它基于你在第2章看到的示例。像`hello-buffer.py`示例一样，这个程序在每次运行时都会向perf缓冲区发送一条消息，将内核中`execve()`系统调用事件的信息传递到用户空间。这个版本的新功能是允许为每个用户ID配置不同的消息。

Here's the eBPF source code:

这是eBPF源代码：

```
struct user_msg_t {
    char message[12];
};

BPF_HASH(config, u32, struct user_msg_t);

BPF_PERF_OUTPUT(output);

struct data_t {
    int pid;
    int uid;
    char command[16];
    char message[12];
};

int hello(void *ctx) {
    struct data_t data = {};
    struct user_msg_t *p;
    char message[12] = "Hello World";

    data.pid = bpf_get_current_pid_tgid() >> 32;
    data.uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;

    bpf_get_current_comm(&data.command, sizeof(data.command));

    p = config.lookup(&data.uid);
    if (p != 0) {
```

```

    bpf_probe_read_kernel(&data.message, sizeof(data.message), p->message);
} else {
    bpf_probe_read_kernel(&data.message, sizeof(data.message), message);
}

output.perf_submit(ctx, &data, sizeof(data));
return 0;
}

```

❶

This line indicates that there's a structure definition, `user_msg_t`, for holding a 12-character message.

❷

The BCC macro `BPF_HASH` is used to define a hash table map called `config`. It will hold values of type `user_msg_t`, indexed by keys of type `u32`, which is the right size for a user ID. (If you don't specify the types for the keys and values, BCC defaults to `u64` for both.)

❸

The perf buffer output is defined in exactly the same way as in [Chapter 2](#). You can submit arbitrary data to a buffer, so there's no need to specify any data types here...

❹

...although in practice, in this example the program always submits a `data_t` structure. This is also unchanged from the [Chapter 2](#) example.

❺

Most of the rest of the eBPF program is unchanged from the `hello()` version you saw earlier.

❻

The only difference is that having used a helper function to get the user ID, the code looks for an entry in the `config` hash map with that user ID as the key. If there is a matching entry, the value contains a message that gets used instead of the default "Hello World."

The Python code has two additional lines:

Python代码有两行额外的内容。

```

b["config"][ct.c_int(0)] = ct.create_string_buffer(b"Hey root!")
b["config"][ct.c_int(501)] = ct.create_string_buffer(b"Hi user 501!")

```

These define messages in the `config` hash table for user IDs 0 and 501, which correspond to the root user and my user ID on this virtual machine. This code uses Python's `ctypes` package to ensure that the key and value have the same types as those used in the C definition of `user_msg_t`.

这些在配置哈希表中定义了用户ID为0和501的消息，这两个ID分别对应于根用户和我的用户ID在这个虚拟机上。该代码使用Python的ctypes包来确保键和值具有与C语言中的user_msg_t定义相同的类型。

Here's some illustrative output from this example, along with the commands I ran in a second terminal to obtain it:

以下是这个例子的一些说明性输出，以及我在第二个终端中运行的命令来获取它：

Terminal 1	Terminal 2
\$./hello-buffer-config.py	
37926 501 bash Hi user 501!	ls
37927 501 bash Hi user 501!	sudo ls
37929 0 sudo Hey root!	
37931 501 bash Hi user 501!	sudo -u daemon ls
37933 1 sudo Hello World	

Now that you've got an idea of what this program does, I'd like to show you the `bpf()` system calls that are used when it runs. I'll run it again using `strace`, specifying `-e bpf` to indicate that I am only interested in seeing the `bpf()` syscall:

既然你已经对这个程序的功能有了一个了解，我想向你展示运行时使用的**bpf()**系统调用。我将再次使用**strace**来运行它，并指定**-e bpf**来表示我只对看到**bpf()**系统调用感兴趣。

```
$ strace -e bpf ./hello-buffer-config.py
```

The output you'll see if you try this yourself shows several calls to this syscall. For each, you'll see the command indicating what the `bpf()` syscall should do. The broad outline looks like this:

如果您自己尝试，您将看到的输出会显示对此系统调用的多个调用。对于每个调用，您将看到指示**bpf()**系统调用应执行的指令。大致概述如下：

```
bpf(BPF_BTF_LOAD, ...) = 3
bpf(BPF_MAP_CREATE, {map_type=BPF_MAP_TYPE_PERF_EVENT_ARRAY...}) = 4
bpf(BPF_MAP_CREATE, {map_type=BPF_MAP_TYPE_HASH...}) = 5
bpf(BPF_PROG_LOAD, {prog_type=BPF_PROG_TYPE_KPROBE,...prog_name="hello",...}) = 6
bpf(BPF_MAP_UPDATE_ELEM, ...)
...
```

Let's examine them one by one. Neither you, the reader, nor I have infinite patience, so I won't discuss every single argument to every single call! I'll focus on the parts that I think really help to tell the story of what's happening when a user space program interacts with an eBPF program.

让我们逐个研究它们。无论是您，读者，还是我，都没有无限的耐心，所以我不会讨论每个调用的每个参数的每个论点！我将专注于那些我认为真正有助于讲述用户空间程序与eBPF程序交互时发生的故事的部分。

Loading BTF Data

加载BTF数据

The first call to `bpf()` that I see looks like this:

我看到的第一个调用**bpf()**的样子是这样的：

```
bpf(BPF_BTF_LOAD, {btf="\237\353\1\0..."}, 128) = 3
```

In this case the command you can see in the output is `BPF_BTF_LOAD`. This is just one of a set of valid commands that are (at least at the time of this writing) most comprehensively documented within the kernel source code.¹

在这种情况下，你可以在输出中看到的命令是BPF_BTF_LOAD。这只是一组有效命令中的一个，这些命令在内核源代码中（至少在撰写本文时）有最全面的文档记录。

It's possible that you won't see a call with this command if you're using a relatively old Linux kernel, as it relates to BTF, or BPF Type Format.² BTF allows eBPF programs to be portable across different kernel versions so that you can compile a program on one machine and use it on another, which might be using a different kernel version and hence have different kernel data structures. I'll discuss this in more detail in [Chapter 5](#).

如果您使用的是相对较旧的Linux内核，可能看不到这个命令的调用，因为它涉及BTF或BPF类型格式。BTF允许eBPF程序在不同的内核版本之间可移植，这样您可以在一台计算机上编译程序并在另一台计算机上使用，后者可能使用不同的内核版本并且具有不同的内核数据结构。我将在第5章中详细讨论这个问题。

This call to `bpf()` is loading a blob of BTF data into the kernel, and the return code from the `bpf()` system call (3 in my example) is a file descriptor that refers to that data.

这次对bpf()的调用正在将一个BTF数据块加载到内核中，bpf()系统调用的返回代码（在我的例子中为3）是一个指向该数据的文件描述符。

Note

A *file descriptor* is an identifier for an open file (or file-like object). If you open a file (with the `open()` or `openat()` system call) the return code is a file descriptor, which is then passed as an argument to other syscalls such as `read()` or `write()` to perform operations on that file. Here the blob of data isn't exactly a file, but it is given a file descriptor as an identifier that can be used for future operations that refer to it.

文件描述符是对打开文件（或类似文件对象）的标识符。如果您使用`open()`或`openat()`系统调用打开文件，则返回值是文件描述符，然后将其作为参数传递给其他系统调用（如`read()`或`write()`）以对该文件执行操作。这里的数据块并不完全是一个文件，但它被赋予了一个文件描述符作为标识符，用于将来引用它的操作。

Creating Maps

制作地图

The next `bpf()` creates the `output` perf buffer map:

下一个bpf()函数创建了输出的perf缓冲区映射:

```
bpf(BPF_MAP_CREATE, {map_type=BPF_MAP_TYPE_PERF_EVENT_ARRAY, , key_size=4,
value_size=4, max_entries=4, ... map_name="output", ...}, 128) = 4
```

You can probably guess from the command name `BPF_MAP_CREATE` that this call creates an eBPF map. You can see that the type of this map is `PERF_EVENT_ARRAY` and it is called `output`. The keys and values in this perf event map are 4 bytes long. There's also a limit of four key–value pairs that can be held in this map, defined by the field `max_entries`; I'll explain why there are four entries in this map later in this chapter. The return value of `4` is the file descriptor for the user space code to access the `output` map.

从命令名称BPF_MAP_CREATE，你可以猜到这个调用是用来创建一个eBPF映射的。你可以看到这个映射的类型是PERF_EVENT_ARRAY，它被称为output。在这个perf事件映射中，键和值的长度都是4字节。这个映射中最多可以容纳四对键值对，这个限制由max_entries字段定义；我将在本章后面解释为什么这个映射中有四个条目。返回值4是用户空间代码访问output映射的文件描述符。

The next `bpf()` system call in the output creates the `config` map:

下一个在输出中的bpf()系统调用创建配置映射：

```
bpf(BPF_MAP_CREATE, {map_type=BPF_MAP_TYPE_HASH, key_size=4, value_size=12,
max_entries=10240... map_name="config", ...btf_fd=3,...}, 128) = 5
```

This map is defined to be a hash table map, with keys that are 4 bytes long (which corresponds to a 32-bit integer that can be used to hold a user ID) and values that are 12 bytes long (which matches the length of the `msg_t` structure). I didn't specify the size of the table, so it has been given BCC's default size of 10,240 entries.

这张地图被定义为一个哈希表映射，其键长为4字节（对应可用于保存用户ID的32位整数），值为12字节（与msg_t结构的长度相匹配）。我没有指定表的大小，因此它被赋予了BCC的默认大小，即10,240个条目。

This `bpf()` system call also returns a file descriptor, `5`, which will be used to refer to this `config` map in future syscalls.

这个bpf()系统调用还会返回一个文件描述符，编号为5，将用于以后的系统调用中引用该配置映射。

You can also see the field `btf_fd=3`, which tells the kernel to use the BTF file descriptor `3` that was obtained earlier. As you'll see in [Chapter 5](#), BTF information describes the layout of data structures, and including this in the definition of the map means there's information about the layout of the key and value types used in this map. This is used by tools like `bpftool` to pretty-print map dumps, making them human readable—you saw an example of this in [Chapter 3](#).

您还可以查看字段btf_fd=3，它告诉内核使用之前获取到的BTF文件描述符3。正如您在第5章中所看到的，BTF信息描述了数据结构的布局，并在地图的定义中包含了关于此地图中使用的键和值类型布局的信息。这被诸如bpftool之类的工具用于漂亮打印地图转储，使其易于阅读 - 您在第3章中已经看到了一个例子。

Loading a Program

装载程序

So far you have seen the example program using syscalls to load BTF data into the kernel and create some eBPF maps. The next thing it does is load the eBPF program being loaded into the kernel with the following `bpf()` syscall:

到目前为止，您已经看到了使用系统调用加载BTF数据到内核并创建一些eBPF映射的示例程序。接下来它做的是使用以下**bpf()**系统调用将eBPF程序加载到内核中：

```
bpf(BPF_PROG_LOAD, {prog_type=BPF_PROG_TYPE_KPROBE, insn_cnt=44,
  insns=0xfffffa836abe8, license="GPL", ... prog_name="hello", ...
  expected_attach_type=BPF_CGROUP_INET_INGRESS, prog_btf_fd=3,...}, 128) = 6
```

Quite a few of the fields here are interesting:

这里有相当多的领域都很有趣。

- The `prog_type` field describes the program type, which here indicates that it's intended to be attached to a kprobe. You'll learn more about program types in [Chapter 7](#).

`prog_type`字段描述了程序类型，在这里表示它是针对kprobe的附加程序。在第7章中，您将了解有关程序类型的更多信息。

- The `insn_cnt` field means “instruction count.” This is the number of bytecode instructions in the program.

`insn_cnt`字段表示“指令计数”。这是程序中字节码指令的数量。

- The bytecode instructions that make up this eBPF program are held in memory at the address specified in the `insns` field.

这个eBPF程序组成的字节码指令保存在内存中，其地址由insns字段指定。

- This program was specified as GPL licensed so that it can use GPL-licensed BPF helper functions.

本程序被指定为GPL许可，以便使用GPL许可的BPF助手函数。

- The program name is `hello`.

程序的名称是你好。

- The `expected_attach_type` of `BPF_CGROUP_INET_INGRESS` might seem surprising, because that sounds like something to do with ingress network traffic, but you know this eBPF program is going to be attached to a kprobe. In fact, the `expected_attach_type` field is only used for some program types, and `BPF_PROG_TYPE_KPROBE` isn't one of them. `BPF_CGROUP_INET_INGRESS` just happens to be the first in the list of BPF attachment types,³ so it has the value `0`.

`BPF_CGROUP_INET_INGRESS`的`expected_attach_type`可能会让人感到惊讶，因为听起来像是与入口网络流量有关的内容，但你要知道这个eBPF程序将被附加到一个kprobe上。实际上，`expected_attach_type`字段仅用于某些程序类型，而`BPF_PROG_TYPE_KPROBE`不是其中之一。`BPF_CGROUP_INET_INGRESS`只是BPF附加类型列表中的第一个，所以它的值为0。

- The `prog_btf_fd` field tells the kernel which blob of previously loaded BTF data to use with this program. The value `3` here corresponds to the file descriptor you saw returned from the `BPF_BTF_LOAD` syscall (and it's the same blob of BTF data used for the `config` map).

`prog_btf_fd`字段告诉内核使用先前加载的BTF数据块来与此程序配对。此处的值3对应于从 `BPF_BTF_LOAD`系统调用中返回的文件描述符（并且它是与配置映射使用的相同的BTF数据块）。

If the program had failed verification (which I'll discuss in [Chapter 6](#)), this syscall would have returned a negative value, but here you can see it returned the file descriptor 6. To recap, at this point the file descriptors have the meanings shown in [Table 4-1](#).

如果程序未经验证失败（我将在第6章讨论此问题），此系统调用将返回一个负值，但在此处你可以看到它返回了文件描述符6。简要回顾一下，在这一点上，文件描述符具有表4-1所示的含义。

Table 4-1. File descriptors when running `hello-buffer-config.py` after loading the program

File descriptor	Represents
3	BTF data
4	output perf buffer map
5	config hash table map
6	hello eBPF program

Modifying a Map from User Space

从用户空间修改地图

You already saw the line in the Python user space source code that configures special messages that will be displayed for the root user with user ID 0, and for the user with ID 501:

您已经看到了在Python用户空间源代码中配置特殊消息的行，这些消息将显示给用户ID为0的root用户和用户ID为501的用户。

```
b["config"][ct.c_int(0)] = ct.create_string_buffer(b"Hey root!")
b["config"][ct.c_int(501)] = ct.create_string_buffer(b"Hi user 501!")
```

You can see these entries being defined in the map through syscalls like this:

您可以通过类似这样的系统调用在地图中看到这些条目的定义。

```
bpf(BPF_MAP_UPDATE_ELEM, {map_fd=5, key=0xfffffa7842490, value=0xfffffa7a2b410,
flags=BPF_ANY}, 128) = 0
```


The `BPF_MAP_UPDATE_ELEM` command updates the key–value pair in a map. The `BPF_ANY` flag indicates that if the key doesn’t already exist in this map, it should be created. There are two of these calls, corresponding to the two entries configured for two different user IDs.

`BPF_MAP_UPDATE_ELEM`命令用于更新地图中的键值对。`BPF_ANY`标志表示，如果该地图中不存在该键，则应创建它。这个命令有两个，对应配置了两个不同用户ID的两个条目。

The `map_fd` field identifies which map is being operated on. You can see that in this case it’s `5`, which is the file descriptor value returned earlier when the `config` map was created.

`map_fd`字段用于标识正在操作的地图。在这种情况下，您可以看到它是5，这是之前创建配置地图时返回的文件描述符值。

File descriptors are assigned by the kernel for a particular process, so this value of `5` is only valid for this particular user space process in which the Python program is running. However, multiple user space programs (and multiple eBPF programs in the kernel) can all access the same map. Two user space programs that access the same map structure in the kernel might very well be assigned different file descriptor values; equally, two user space programs might have the same file descriptor value for entirely different maps.

文件描述符是由内核为特定进程分配的，因此该值为5仅对该特定用户空间进程（即运行Python程序的进程）有效。然而，多个用户空间程序（和内核中的多个eBPF程序）可以访问同一张地图。访问内核中相同地图结构的两个用户空间程序很可能被分配不同的文件描述符值；同样，两个用户空间程序可能对应完全不同地图的相同文件描述符值。

Both the key and the value are pointers, so you can’t tell the numeric value of either the key or the value from this `strace` output. You could, however, use `bpftool` to view the map’s contents and see something like this:

密钥和值都是指针，因此您无法从这个`strace`输出中获取密钥或值的数值。但是，您可以使用`bpftool`查看映射的内容，看到类似以下内容：

```
$ bpftool map dump name config
[ {
    "key": 0,
    "value": {
        "message": "Hey root!"
    }
}, {
    "key": 501,
    "value": {
        "message": "Hi user 501!"
    }
}
]
```

How does `bpftool` know how to format this output? For example, how does it know the value is a structure, with a field called `message` that contains a string? The answer is that it uses the definitions in the BTF information included on the `BPF_MAP_CREATE` syscall that defined this map. You’ll see more details on how BTF conveys this information in the next chapter.

`bpftool`是如何知道如何格式化这个输出的？例如，它是如何知道该值是一个包含字符串的结构体，且字段名为`message`？答案是它使用了在定义此映射的`BPF_MAP_CREATE`系统调用中包含的BTF信息中的定义。在下一章节中，您将看到关于BTF如何传达这些信息的更多细节。

You’ve now seen how user space interacts with the kernel to load programs and maps and to update the information in a map. In the sequence of syscalls you have seen up to this point, the program hasn’t yet been attached to an event. This step has to happen; otherwise, the program will never be triggered.

你已经看到了用户空间如何与内核进行交互来加载程序和映射，并更新映射中的信息。在你到目前为止看到的系统调用序列中，程序还没有被附加到事件上。这一步必须发生；否则，程序将永远不会被触发。

Fair warning: different types of eBPF programs get attached to different events in a variety of different ways! Later in this chapter I’ll show you the syscalls used in this example to attach to the kprobe event, and in this case it doesn’t involve `bpf()`. In contrast, in the exercises at the end of this chapter I will show you another example where a `bpf()` syscall is used to attach a program to a raw tracepoint event.

提前警告：不同类型的eBPF程序以各种不同的方式附加到不同的事件上！本章后面我会向您展示在本示例中用于附加到kprobe事件的系统调用，这种情况下不涉及`bpf()`。相比之下，在本章末尾的练习中，我将向您展示另一个示例，在该示例中使用`bpf()`系统调用将程序附加到原始tracepoint事件上。

Before we get to those details, I’d like to discuss what happens when you quit running the program. You’ll find that the program and maps are automatically unloaded, and this happens because the kernel is keeping track of them using *reference counts*.

在我们讨论这些细节之前，我想先讨论一下当您退出运行程序时会发生什么。您会发现程序和地图会自动被卸载，这是因为内核使用引用计数来跟踪它们。

BPF Program and Map References

BPF程序和Map参考

You know that loading a BPF program into the kernel with the `bpf()` syscall returns a file descriptor. Within the kernel, this file descriptor is a *reference* to the program. The user space process that made the syscall owns this file descriptor; when that process exits, the file descriptor gets released, and the reference count to the program is decremented. When there are no references left to a BPF program, the kernel removes the program.

你知道使用`bpf()`系统调用将BPF程序加载到内核中会返回一个文件描述符。在内核中，这个文件描述符是对程序的引用。调用该系统调用的用户空间进程拥有这个文件描述符；当该进程退出时，文件描述符会被释放，程序的引用计数会被递减。当没有对BPF程序的引用时，内核会删除该程序。

An additional reference is created when you *pin* a program to the filesystem.

把一个程序固定到文件系统时会创建一个额外的引用。

Pinning

固定

You already saw pinning in action in [Chapter 3](#), with the following command:

在第三章中，您已经看到了固定作用的示例，使用以下命令：

```
bpftool prog load hello.bpf.o /sys/fs/bpf/hello
```

Note

These pinned objects aren't real files persisted to disk. They are created on a *pseudo filesystem*, which behaves like a regular disk-based filesystem with directories and files. But they are held in memory, which means they will not remain in place over a system reboot.

这些被固定的对象并不是真正保存在磁盘上的文件。它们是在一个伪文件系统上创建的，它的行为就像一个普通的基于磁盘的文件系统，有目录和文件。但它们被保存在内存中，这意味着它们在系统重新启动后不会保留在原处。

If `bpftool` were to allow you to load the program without pinning it, that would be pointless, because the file descriptor gets released when `bpftool` exits, and if there are zero references, the program would get deleted, so nothing useful would have been achieved. But pinning it to the filesystem means there is an additional reference to the program, so the program remains loaded after the command completes.

如果bpftool允许您在不固定程序的情况下加载它，那将是无意义的，因为当bpftool退出时，文件描述符将被释放，如果没有任何引用，程序将被删除，因此不会实现任何有用的效果。但是将程序钉到文件系统意味着对程序有一个额外的引用，因此在命令完成后程序将仍然加载。

The reference counter also gets incremented when a BPF program is attached to a hook that will trigger it. The behavior of these reference counts depends on the BPF program type. You'll learn more about these program types in [Chapter 7](#), but there are some that relate to tracing (like kprobes and tracepoints) and are always associated with a user space process; for these types of eBPF programs, the kernel's reference count gets decremented when that process exits. Programs that are attached within the network stack or cgroups (short for "control groups") aren't associated with any user space process, so they stay in place even after the user space program that loads them exits. You already saw an example of this when loading an XDP program with the `ip link` command:

参考计数器在将BPF程序附加到触发它的钩子时也会递增。这些引用计数的行为取决于BPF程序的类型。在第7章中，您将更多地了解这些程序类型，但其中一些与跟踪相关（例如kprobes和tracepoints），并且始终与用户空间进程关联；对于这些类型的eBPF程序，内核的引用计数会在该进程退出时减少。附加在网络堆栈或cgroups（缩写为“控制组”）中的程序不与任何用户空间进程关联，因此即使加载它们的用户空间程序退出，它们仍然保持不变。当使用`ip link`命令加载XDP程序时，您已经看到了这种情况的示例：参考计数器在将BPF程序附加到触发它的钩子时也会递增。引用计数的行为取决于BPF程序的类型。在第7章中，您将了解更多有关这些程序类型的信息，但其中一些与跟踪相关（例如kprobes和tracepoints），始终与用户空间进程相关联；对于这些类型的eBPF程序，进程退出时减少内核的引用计数。附加在网络堆栈或cgroups（即“控制组”）中的程序不与任何用户空间进程相关联，因此即使加载它

们的用户空间程序退出，它们也会保持不变。当使用`ip link`命令加载XDP程序时，您已经看到了这种情况的示例。

```
ip link set dev eth0 xdp obj hello.bpf.o sec xdp
```

The `ip` command has completed, and there is no definition of a pinned location, but nevertheless, `bpftool` will show you that the XDP program is loaded in the kernel:

ip命令已完成，没有对固定位置的定义，但是bpftool可以显示XDP程序已加载到内核中：

```
$ bpftool prog list
...
1255: xdp  name hello  tag 9d0e949f89f1a82c  gpl
      loaded_at 2022-11-01T19:21:14+0000  uid 0
      xlated 48B  jited 108B  memlock 4096B  map_ids 612
```

The reference count for this program is nonzero, because of the attachment to the XDP hook that persisted after the `ip link` command completed.

由于在ip link命令完成后仍然保留了与XDP hook的连接，所以此程序的引用计数为非零。

eBPF maps also have reference counters, and they get cleaned up when their reference count drops to zero. Each eBPF program that uses a map increments the counter, as does each file descriptor that user space programs might hold to the map.

eBPF映射也有引用计数器，当其引用计数降为零时会清理。每个使用映射的eBPF程序都会递增计数器，用户空间程序持有的每个文件描述符也会递增计数器。

It's possible that the source code for an eBPF program might define a map that the program doesn't actually reference. Suppose you wanted to store some metadata about a program; you could define it as a global variable, and as you saw in the previous chapter, this information gets stored in a map. If the eBPF program doesn't do anything with that map, there won't automatically be a reference count from the program to the map. There's a `BPF(BPF_PROG_BIND_MAP)` syscall that associates a map with a program so that the map doesn't get cleaned up as soon as the user space loader program exits and is no longer holding a file descriptor reference to the map.

这是可能的，eBPF程序的源代码可能定义了一个程序实际上没有引用的映射。假设您想要存储一些关于程序的元数据；您可以将其定义为全局变量，在前一章节中您已经看到，此信息会存储在一个映射中。如果eBPF程序对该映射没有任何操作，程序与映射之间将不会自动存在引用计数。有一种BPF

(`BPF_PROG_BIND_MAP`) 系统调用可以将一个映射与一个程序关联起来，这样映射在用户空间加载程序退出并且不再持有映射的文件描述符引用时不会被清除。

Maps can also be pinned to the filesystem, and user space programs can gain access to the map by knowing the path to the map.

地图也可以被固定到文件系统中，用户空间程序可以通过知道地图的路径来访问地图。

Note

Alexei Starovoitov wrote a good description of BPF reference counters and file descriptors in his blog post [“Lifetime of BPF Objects”](#).

Alexei Starovoitov在他的博客文章“BPF对象的生命周期”中对BPF参考计数器和文件描述符进行了很好的描述。

Another way to create a reference to a BPF program is with a BPF link.

另一种创建对BPF程序的引用的方法是使用BPF链接。

BPF Links

BPF链接

BPF links provide a layer of abstraction between an eBPF program and the event it's attached to. A BPF link itself can be pinned to the filesystem, which creates an additional reference to the program. This means the user space process that loaded the program into the kernel can terminate, leaving the program loaded. The file descriptor for the user space loader program gets freed up, decreasing the count of references to the program, but the reference count will be nonzero because of the BPF link.

BPF链接提供了一个在eBPF程序和其所附加事件之间的抽象层。BPF链接本身可以固定到文件系统上，从而为该程序创建了一个额外的引用。这意味着将程序加载到内核中的用户空间进程可以终止，但程序仍然保持加载状态。用户空间加载程序的文件描述符被释放，减少了对程序的引用计数，但由于BPF链接的存在，引用计数仍将为非零值。

You'll get an opportunity to see BPF links in action if you follow the exercises at the end of this chapter. For now, let's get back to the sequence of `bpf()` syscalls used by *hello-buffer-config.py*.

如果你按照本章末尾的练习来进行，你将有机会看到BPF链接的运作。现在，让我们回到hello-buffer-config.py中使用的bpf()系统调用的顺序。

Additional Syscalls Involved in eBPF

附加的系统调用参与 eBPF

To recap, so far you have seen `bpf()` syscalls that add the BTF data, program and maps, and map data to the kernel. The next thing the `strace` output shows relates to setting up the perf buffer.

总结一下，到目前为止，你已经看到了添加BTF数据、程序和映射以及映射数据到内核的bpf()系统调用。接下来，strace输出显示的是设置性能缓冲区的相关内容。

Note

The rest of this chapter dives relatively deeply into the syscall sequences involved when using perf buffers, ring buffers, kprobes, and map iterations. Not all eBPF programs need to do these things, so if you're in a hurry or you're finding it a bit too detailed, feel free to skip to the chapter summary. I won't be offended!

本章剩余部分相对深入地讲解了使用perf缓冲区、环形缓冲区、kprobe和映射迭代时涉及的系统调用序列。并非所有的eBPF程序都需要执行这些操作，所以如果你匆忙或者觉得内容太详细的话，可以跳到章节总结部分。我不会生气的！

Initializing the Perf Buffer

初始化性能缓冲

You have seen the `bpf(BPF_MAP_UPDATE_ELEM)` calls that add entries into the `config` map. Next, the output shows some calls that look like this:

你已经看到了调用BPF_MAP_UPDATE_ELEM的代码行，它们向配置映射中添加条目。接下来，输出结果会出现一些类似这样的调用：

```
bpf(BPF_MAP_UPDATE_ELEM, {map_fd=4, key=0xfffffa7842490, value=0xfffffa7a2b410, flags=BPF_ANY}, 128) = 0
```

These look very similar to the calls that defined the `config` map entries, except in this case the map's file descriptor is `4`, which represents the `output` perf buffer map.

这些看起来与定义配置映射条目的调用非常相似，但在这种情况下，映射的文件描述符是4，表示输出perf缓冲区映射。

As before, the key and the value are pointers, so you can't tell the numeric value of either the key or the value from this `strace` output. I see this syscall repeated four times with identical values for all the parameters, though there's no way of knowing whether the values the pointers hold have changed between each call. Looking at these `BPF_MAP_UPDATE_ELEM bpf()` calls leaves some unanswered questions about how the buffer is set up and used:

与之前一样，键和值都是指针，因此无法从此strace输出中确定键或值的数值。我看到这个syscall重复了四次，所有参数的值都相同，尽管无法知道指针所持有的值在每次调用之间是否发生了变化。查看这些BPF_MAP_UPDATE_ELEM bpf()调用在如何设置和使用缓冲区方面留下了一些未解答的问题。

- Why are there four calls to `BPF_MAP_UPDATE_ELEM`? Does this relate to the fact that the `output` map was created with a maximum of four entries?

为什么有四个调用BPF_MAP_UPDATE_ELEM的操作？这是否与输出映射被创建为最多四个条目有关？

- After these four instances of `BPF_MAP_UPDATE_ELEM`, no more `bpf()` syscalls appear in the `strace` output. That might seem a little odd, because the map is there so that the eBPF program can write data every time it is triggered, and you've seen data being displayed by the user space code. That data is clearly not being retrieved from the map with `bpf()` syscalls, so how is it obtained?

在这四次 BPF_MAP_UPDATE_ELEM 之后，strace 输出中不再出现 bpf() 系统调用。这看起来有些奇怪，因为映射是为了让 eBPF 程序在每次触发时写入数据，并且你已经看到用户空间代码显示了数据。这些数据显然不是通过 bpf() 系统调用从映射中获取的，那么它是如何获取的呢？

You’ve also yet to see any evidence of how the eBPF program is getting attached to the kprobe event that triggers it. To get the explanation for all these concerns, I need `strace` to show a few more syscalls when running this example, like this:

你还没有看到任何证据表明eBPF程序如何附加到触发它的kprobe事件。为了了解所有这些问题的解释，我需要strace在运行此示例时显示更多的系统调用，就像这样：

```
$ strace -e bpf,perf_event_open,ioctl,ppoll ./hello-buffer-config.py
```

For brevity, I’m going to ignore calls to `ioctl()` that aren’t specifically related to the eBPF functionality of this example.

为了简洁起见，我将忽略那些与此示例的eBPF功能无关的ioctl()调用。

Attaching to Kprobe Events

附加到Kprobe事件

You’ve seen that file descriptor 6 was assigned to represent the eBPF program *hello* once it was loaded into the kernel. To attach the eBPF program to an event, you also need a file descriptor representing that particular event. The following line from the `strace` output shows the creation of the file descriptor for the `execve()` kprobe:

你已经看到文件描述符6被分配为代表eBPF程序hello在加载到内核后。要将eBPF程序连接到事件，您还需要一个表示该特定事件的文件描述符。以下来自strace输出的一行显示了为execve() kprobe创建文件描述符：

```
perf_event_open({type=0x6 /* PERF_TYPE_??? */, ...},...) = 7
```

According to the [manpage for the `perf\_event\_open\(\)` syscall](#), it “creates a file descriptor that allows measuring performance information.” You can see from the output that `strace` doesn’t know how to interpret the type parameter with the value 6, but if you examine that manpage further, it describes how Linux supports dynamic types of Performance Measurement Unit:

根据perf_event_open()系统调用的manpage，它“创建一个文件描述符，允许测量性能信息。”从输出可以看出，strace不知道如何解释值为6的类型参数，但如果进一步查看该manpage，可以发现Linux支持动态的性能测量单元类型。

...there is a subdirectory per PMU instance under `/sys/bus/event_source/devices`. In each subdirectory there is a type file whose content is an integer that can be used in the type field.

每个PMU实例在/sys/bus/event_source/devices下都有一个子目录。在每个子目录中，都有一个类型文件，其内容是一个整数，可以在类型字段中使用。

Sure enough, if you look under that directory, you’ll find a *kprobe/type* file:

确定了，如果您在该目录下查找，您会找到一个kprobe/type文件：

```
$ cat /sys/bus/event_source/devices/kprobe/type
6
```

From this, you can see that the call to `perf_event_open()` has a type set to the value `6` to indicate that it's a kprobe type of perf event.

从这里您可以看到，对`perf_event_open()`的调用的类型被设置为6，表示它是一种kprobe类型的性能事件。

Unfortunately, `strace` doesn't output the details that would conclusively show that the kprobe is attached to the `execve()` syscall, but I hope there is enough evidence here to convince you that that's what the file descriptor returned here represents.

很不幸，`strace`无法输出能够确切显示kprobe已连接到`execve()`系统调用的详细信息，但我希望这里有足够的证据，可以说服您这就是所返回的文件描述符所代表的含义。

The return code from `perf_event_open()` is `7`, and this represents the file descriptor for the kprobe's perf event, and you know that file descriptor `6` represents the *hello* eBPF program. The manpage for `perf_event_open()` also explains how to use `ioctl()` to create the attachment between the two:

`perf_event_open()`的返回代码为7，这代表了kprobe的perf事件的文件描述符，而你知道文件描述符6代表了hello eBPF程序。`perf_event_open()`的manpage还解释了如何使用`ioctl()`创建两者之间的附加关系。

```
PERF_EVENT_IOC_SET_BPF [...] allows attaching a Berkeley Packet Filter (BPF) program to an existing
kprobe tracepoint event. The argument is a BPF program file descriptor that was created by a
previous bpf(2) system call.
```

```
`PERF_EVENT_IOC_SET_BPF[...]' 允许将一个Berkeley Packet Filter (BPF)程序附加到现有的
kprobe跟踪点事件。参数是通过之前的bpf(2)系统调用创建的BPF程序文件描述符。
```

This explains the following `ioctl()` syscall that you'll see in the `strace` output, with arguments referring to the two file descriptors:

这解释了你在`strace`输出中会看到的下面的`ioctl()`系统调用，其参数引用了两个文件描述符。

```
ioctl(7, PERF_EVENT_IOC_SET_BPF, 6)      = 0
```

There is also another `ioctl()` call that turns the kprobe event on:

还有另外一个`ioctl()`调用将kprobe事件打开：

```
ioctl(7, PERF_EVENT_IOC_ENABLE, 0)      = 0
```

With this in place, the eBPF program should be triggered whenever `execve()` is run on this machine.

有了这个设置，每当在这台机器上运行`execve()`时，eBPF程序都会被触发。

Setting Up and Reading Perf Events

建立和阅读 Perf 事件

I already mentioned that I see four calls to `bpf(BPF_MAP_UPDATE_ELEM)` related to the output perf buffer. With the additional syscalls being traced, the `strace` output shows four sequences, like this:

我已经提到了我看到了四个与输出perf缓冲区相关的**bpf(BPF_MAP_UPDATE_ELEM)**调用。随着其他系统调用的跟踪，`strace`输出显示了四个类似于这样的序列：

```
perf_event_open({type=PERF_TYPE_SOFTWARE, size=0 /* PERF_ATTR_SIZE_?? */,  
config=PERF_COUNT_SW_BPF_OUTPUT, ...}, -1, X, -1, PERF_FLAG_FD_CLOEXEC) = Y  
  
ioctl(Y, PERF_EVENT_IOC_ENABLE, 0)          = 0  
  
bpf(BPF_MAP_UPDATE_ELEM, {map_fd=4, key=0xfffffa7842490, value=0xfffffa7a2b410,  
flags=BPF_ANY}, 128) = 0
```

I've used `x` to indicate where the output shows values `0`, `1`, `2`, and `3` in the four instances of this call. Referring to the manpage for the `perf_event_open()` syscall, you'll see that this is the `cpu`, and the field before it is `pid` or process ID. From the manpage:

我在这个调用的四次实例中使用X来表示输出显示值0、1、2和3。在**perf_event_open()**系统调用的man页面上，您会看到这是CPU，而它之前的字段是进程ID或PID。

```
pid == -1 and cpu >= 0
```

```
pid == -1 and cpu >= 0 -> pid == -1 并且 cpu >= 0
```

```
This measures all processes/threads on the specified CPU.
```

```
这个测量了指定 CPU 上的所有进程/线程。
```

The fact that this sequence happens four times corresponds to there being four CPU cores in my laptop. This, at last, is the explanation for why there are four entries in the “output” perf buffer map: there is one for each CPU core. It also explains the “array” part of the map type name

`BPF_MAP_TYPE_PERF_EVENT_ARRAY`, as the map doesn't just represent one perf ring buffer but an array of buffers, one for each core.

这个序列发生四次的事实对应于我的笔记本电脑有四个CPU核心。最后，这解释了“output” perf buffer映射中有四个条目的原因：每个CPU核心都有一个映射。这也解释了

`BPF_MAP_TYPE_PERF_EVENT_ARRAY`映射类型名称中的“array”部分，因为映射不仅代表一个perf环形缓冲区，而是一个缓冲区数组，每个核心一个。

If you write eBPF programs, you won't need to worry about details like handling the number of cores, as this will be taken care of for you by any of the eBPF libraries discussed in [Chapter 10](#), but I think it's an interesting aspect of the syscalls you see when you use `strace` on this program.

如果你编写eBPF程序，你就不需要担心像处理核心数量这样的细节，因为在第10章中讨论的任何eBPF库都会为你处理这个问题，但我认为当你在这个程序上使用**strace**时，你会看到syscalls的一个有趣方面。

The `perf_event_open()` calls each return a file descriptor, which I've represented as `Y`; these have the values `8`, `9`, `10`, and `11`. The `ioctl()` syscalls enable the perf output for each of these file descriptors.

The `BPF_MAP_UPDATE_ELEM bpf()` syscalls set the map entry to point to the perf ring buffer for each CPU core to indicate where it can submit data.

`perf_event_open()` 函数调用每次返回一个文件描述符，我表示为 Y；这些文件描述符分别为 8、9、10 和 11。`ioctl()` 系统调用用于启用每个文件描述符的 perf 输出。`BPF_MAP_UPDATE_ELEM bpf()` 系统调用用于将映射条目设置为指向每个 CPU 核心的 perf 环形缓冲区，以指示在哪里可以提交数据。

User space code can then use `ppoll()` on all four of these output stream file descriptors so that it can get the data output, whichever core happens to run the eBPF program *hello* for any given `execve()` kprobe event. Here's the syscall to `ppoll()`:

用户空间代码可以使用 `ppoll()` 在这四个输出流文件描述符中选择任何一个来获取数据输出，无论哪个核心运行 eBPF 程序 *hello* 的 `execve()` kprobe 事件。这是 `ppoll()` 的系统调用：

```
ppoll([{{fd=8, events=POLLIN}}, {{fd=9, events=POLLIN}}, {{fd=10, events=POLLIN}},
{{fd=11, events=POLLIN}}], 4, NULL, NULL, 0) = 1 ({{fd=8, revents=POLLIN}})
```

As you'll see if you try running the example program yourself, these `ppoll()` calls block until there is something to read from one of the file descriptors. You won't see the return code written to the screen until something triggers `execve()`, which causes the eBPF program to write data that user space retrieves using this `ppoll()` call.

正如你自己运行示例程序时会看到的那样，这些 `ppoll()` 调用会被阻塞，直到有东西从其中一个文件描述符中读取到。只有当触发 `execve()` 函数时，你才会看到返回码写入屏幕，因为这会导致 eBPF 程序写入数据，而用户空间会使用 `ppoll()` 调用来检索这些数据。

In [Chapter 2](#) I mentioned that if you have a kernel of version 5.8 or above, BPF ring buffers are now preferred over perf buffers.⁴ Let's take a look at a modified version of the same example code that uses a ring buffer.

在第2章中，我提到如果您有5.8版本或更高版本的内核，那么BPF环形缓冲区现在比perf缓冲区更受欢迎。让我们看一下使用环形缓冲区的同一个示例代码的修改版本。

Ring Buffers

环形缓冲区

As discussed in the [kernel documentation](#), ring buffers are preferred over perf buffers partly for performance reasons, but also to ensure that the ordering of data is preserved, even if the data is submitted by different CPU cores. There is just one buffer, shared across all cores.

如内核文档所讨论的那样，环形缓冲区比性能缓冲区更受青睐，部分原因是为了提高性能，同时也可以确保数据的顺序被保留下来，即使这些数据是由不同的CPU核心提交的。这里只有一个缓冲区，共享于所有核心。

There aren't many changes needed to convert *hello-buffer-config.py* to use a ring buffer. In the accompanying GitHub repo you'll find this example as *chapter4/hello-ring-buffer-config.py*. [Table 4-2](#)

shows the differences.

不需要太多修改即可将 `hello-buffer-config.py` 转换为使用环形缓冲区。在附带的GitHub仓库中，您将找到这个例子作为第4章/`hello-ring-buffer-config.py`。表4-2显示了它们之间的差异。

Table 4-2. Differences between example BCC code using a perf buffer and a ring buffer

<i>hello-buffer-config.py</i>	<i>hello-ring-buffer-config.py</i>
<code>BPF_PERF_OUTPUT(output);</code>	<code>BPF_RINGBUF_OUTPUT(output, 1);</code>
<code>output.perf_submit(ctx, &data, sizeof(data));</code>	<code>output.ringbuf_output(&data, sizeof(data),</code>
<code>b["output"].</code> <code>open_perf_buffer(print_event)</code>	<code>b["output"].</code> <code>open_ring_buffer(print_event)</code>
<code>b.perf_buffer_poll()</code>	<code>b.ring_buffer_poll()</code>

As you'd expect, since these changes relate only to the `output` buffer, the syscalls related to loading the program and the `config` map and attaching the program to the kprobe event all remain unchanged.

正如你所期望的那样，由于这些更改仅涉及输出缓冲区，与加载程序、配置映射以及将程序附加到kprobe事件相关的系统调用都保持不变。

The `bpf()` syscall that creates the `output` ring buffer map looks like this:

用简体中文来翻译：创建输出环形缓冲区映射的 `bpf()` 系统调用如下：

```
bpf(BPF_MAP_CREATE, {map_type=BPF_MAP_TYPE_RINGBUF, key_size=0, value_size=0,
max_entries=4096, ... map_name="output", ...}, 128) = 4
```

The major difference in the `strace` output is that there is no sign of the series of four different `perf_event_open()`, `ioctl()`, and `bpf(BPF_MAP_UPDATE_ELEM)` system calls that you observed during the setup of a perf buffer. For a ring buffer, there's just the one file descriptor shared across all CPU cores.

`strace`输出的主要区别是，在perf buffer的设置过程中，没有你观察到的一系列四个不同的 `perf_event_open()`、`ioctl()`和`bpf(BPF_MAP_UPDATE_ELEM)`系统调用的迹象。对于ring buffer，只有一个文件描述符在所有CPU核心之间共享。

At the time of this writing, BCC is using the `ppoll` mechanism I showed earlier for perf buffers, but it uses the newer `epoll` mechanism to wait for data from the ring buffer. Let's use this as an opportunity to understand the difference between `ppoll` and `epoll`.

在撰写本文时，BCC正在使用我之前展示过的`ppoll`机制来处理perf缓冲区，但它使用较新的`epoll`机制来等待环形缓冲区中的数据。让我们利用这个机会来了解`ppoll`和`epoll`之间的区别。

In the perf buffer example, I showed `hello-buffer-config.py` generating a `ppoll()` syscall, like this:

在perf缓冲区示例中，我展示了`hello-buffer-config.py`生成一个`ppoll()`系统调用，就像这样：

```
ppoll([{fd=8, events=POLLIN}, {fd=9, events=POLLIN}, {fd=10, events=POLLIN},
{fd=11, events=POLLIN}], 4, NULL, NULL, 0) = 1 ([{fd=8, revents=POLLIN}])
```

Notice that this passes in the set of file descriptors 8, 9, 10, and 11 from which the user space process wants to retrieve data. Every time this poll event returns data, another call has to be made to `ppoll()` to set up the same set of file descriptors all over again. When using `epoll`, the file descriptor set is managed in a kernel object.

注意，这将在文件描述符集合8、9、10和11中传递，用户空间进程希望从中检索数据。每次此轮询事件返回数据时，都必须再次调用`ppoll()`来设置相同的文件描述符集合。在使用`epoll`时，文件描述符集合由内核对象管理。

You can see this in the following sequence of `epoll`-related system calls made when *hello-ring-buffer-config.py* is setting up access to the `output` ring buffer.

在`hello-ring-buffer-config.py`设置访问输出环形缓冲区时，您可以在以下`epoll`相关系统调用的序列中看到。

First, the user space program asks for a new `epoll` instance to be created in the kernel:

首先，用户空间程序要求在内核中创建一个新的 `epoll` 实例。

```
epoll_create1(EPOLL_CLOEXEC) = 8
```

This returns file descriptor 8. Then there is a call to `epoll_ctl()`, which tells the kernel to add file descriptor 4 (the `output` buffer) to the set of file descriptors in that `epoll` instance:

这返回文件描述符8。然后调用`epoll_ctl()`，告诉内核将文件描述符4（输出缓冲区）加入到该`epoll`实例的文件描述符集合中。

```
epoll_ctl(8, EPOLL_CTL_ADD, 4, {events=EPOLLIN, data={u32=0, u64=0}}) = 0
```

The user space program uses `epoll_pwait()` to wait until data is available in the ring buffer. This call only returns when data is available:

用户空间程序使用`epoll_pwait()`来等待直到环形缓冲区中有可用数据。只有当有可用数据时，此调用才会返回。

```
epoll_pwait(8, [{events=EPOLLIN, data={u32=0, u64=0}}], 1, -1, NULL, 8) = 1
```

Of course, if you're writing code using a framework like BCC (or *libbpf* or any of the other libraries I'll describe later in this book), you really don't need to know these underlying details about how your user space application gets information from the kernel via `perf` or ring buffers. I hope you've found it interesting to get a peek under the covers to see how these things work.

当然，如果你正在使用像BCC（或`libbpf`或本书后面将描述的其他库）这样的框架编写代码，你实际上不需要了解有关用户空间应用程序通过`perf`或环形缓冲区从内核获取信息的这些底层细节。我希望你发现这些工作原理对你来说很有趣。

However, you might well find yourself writing code that accesses a map from user space, and it might be helpful to see an example of how this is achieved. Earlier in this chapter, I used `bpftool` to examine the contents of the `config` map. Since it's a utility that runs in user space, let's use `strace` to see what syscalls it's making to retrieve this information.

然而，你可能会发现自己需要编写访问用户空间中映射的代码，并且了解一个示例对于实现这一点可能会很有帮助。在本章的早些时候，我使用bpftool来检查配置映射的内容。由于bpftool是在用户空间中运行的实用工具，所以让我们使用strace来查看它所做的系统调用以获取这些信息。

Reading Information from a Map

从地图中阅读信息

The following command shows an extract of the `bpf()` syscalls that `bpftool` makes while reading the contents of the `config` map:

以下命令显示bpftool在读取配置映射内容时发出的bpf()系统调用的摘要：

```
$ strace -e bpf bpftool map dump name config
```

As you'll see, the sequence consists of two main steps:

如你所见，该序列包含两个主要步骤：

- Iterate through all the maps, looking for any with the name `config`.
遍历所有的地图，查找名称为config的地图。
- If a matching map is found, iterate through all the elements in that map.
如果找到匹配的地图，则遍历该地图中的所有元素。

Finding a Map

找地图

The output starts with a repeated sequence of similar calls, as `bpftool` walks through all the maps looking for any with the name `config`:

输出以重复的相似调用序列开始，因为bpftool在查找所有具有名称`config:`的映射时进行遍历。

```
bpf(BPF_MAP_GET_NEXT_ID, {start_id=0,...}, 12) = 0           ❶  
bpf(BPF_MAP_GET_FD_BY_ID, {map_id=48...}, 12) = 3           ❷  
bpf(BPF_OBJ_GET_INFO_BY_FD, {info={bpf_fd=3, ...}}, 16) = 0  ❸  
  
bpf(BPF_MAP_GET_NEXT_ID, {start_id=48, ...}, 12) = 0        ❹  
bpf(BPF_MAP_GET_FD_BY_ID, {map_id=116, ...}, 12) = 3  
bpf(BPF_OBJ_GET_INFO_BY_FD, {info={bpf_fd=3...}}, 16) = 0
```

❶

`BPF_MAP_GET_NEXT_ID` gets the ID of the next map after the value specified in `start_id`.

❷

`BPF_MAP_GET_FD_BY_ID` returns the file descriptor for the specified map ID.

❸

`BPF_OBJ_GET_INFO_BY_FD` retrieves information about the object (in this case, the map) referred to by the file descriptor. This information includes its name so `bpftool` can check whether this is the map it is looking for.

❹

The sequence repeats, getting the ID of the next map after the one in step 1.

There's a group of these three syscalls for each map loaded into the kernel, and you should also see that the values used for `start_id` and `map_id` match the IDs of those maps. The repeated pattern ends when there are no more maps left to look at, which results in `BPF_MAP_GET_NEXT_ID` returning a value of `ENOENT`, like this:

每个加载到内核中的映射都有一组这样的三个系统调用，你还应该看到用于 `start_id` 和 `map_id` 的值与这些映射的ID相匹配。当没有更多的映射需要查看时，重复模式就会结束，这会导致 `BPF_MAP_GET_NEXT_ID` 返回 `ENOENT` 值，如下所示：

```
bpf(BPF_MAP_GET_NEXT_ID, {start_id=133,...}, 12) = -1 ENOENT (No such file or directory)
```

If a matching map has been found, `bpftool` holds its file descriptor so that it can read the elements out of that map.

如果找到了匹配的映射，`bpftool`将保存其文件描述符，以便可以从该映射中读取元素。

Reading Map Elements

阅读地图要素

At this point `bpftool` has a file descriptor reference to the map(s) it's going to read from. Let's look at the syscall sequence for reading that information:

在这一点上，`bpftool`具有对要读取的映射文件描述符的引用。让我们看一下读取该信息的系统调用序列：

```
bpf(BPF_MAP_GET_NEXT_KEY, {map_fd=3, key=NULL, next_key=0xaaaaaf7a63960}, 24) = 0 ❶
bpf(BPF_MAP_LOOKUP_ELEM, {map_fd=3, key=0xaaaaaf7a63960, value=0xaaaaaf7a63980, flags=BPF_ANY}, 32) = 0 ❷
[{ ❸
    "key": 0,
    "value": {
        "message": "Hey root!"
    }
}]
bpf(BPF_MAP_GET_NEXT_KEY, {map_fd=3, key=0xaaaaaf7a63960, next_key=0xaaaaaf7a63960}, 24) = 0 ❹
bpf(BPF_MAP_LOOKUP_ELEM, {map_fd=3, key=0xaaaaaf7a63960, value=0xaaaaaf7a63980, flags=BPF_ANY}, 32) = 0
    }, {
        "key": 501,
```

```

        "value": {
            "message": "Hi user 501!"
        }
    }
    bpf(BPF_MAP_GET_NEXT_KEY, {map_fd=3, key=0xaaaf7a63960, ❸
    next_key=0xaaaf7a63960}, 24) = -1 ENOENT (No such file or directory) ❹
    }
}
+++ exited with 0 +++

```

❶

First, the application needs to find a valid key that is present in the map. It does this with the `BPF_MAP_GET_NEXT_KEY` flavor of the `bpf()` syscall. The `key` argument is a pointer to a key, and the syscall will return the next valid key *after* this one. By passing in a NULL pointer, the application is requesting the first valid key in the map. The kernel writes the key into the location specified by the `next_key` pointer.

❷

Given a key, the application requests the associated value, which gets written to the memory location specified by `value`.

❸

At this point, `bpftool` has the contents of the first key–value pair, and it writes this information to the screen.

❹

Here, `bpftool` moves on to the next key in the map, retrieves its value, and writes out this key–value pair to the screen.

❺

The next call to `BPF_MAP_GET_NEXT_KEY` returns `ENOENT` to indicate that there are no more entries in the map.

❻

Here, `bpftool` finalizes the output written to screen and exits.

Notice that here, `bpftool` has been assigned file descriptor `3` to correspond to the `config` map. This is the same map that *hello-buffer-config.py* refers to with file descriptor `4`. As I've mentioned already, file descriptors are process specific.

请注意，这里将bpftool分配给文件描述符3以对应于配置映射。这与hello-buffer-config.py所引用的文件描述符4相同。正如我已经提到的，文件描述符是进程特定的。

This analysis of how `bpftool` behaves shows how a user space program can iterate through the available maps and through the key–value pairs stored in a map.

这个bpftool行为的分析展示了用户空间程序如何通过可用的地图和地图中存储的键值对进行迭代。

Summary

摘要

In this chapter you saw how user space code uses the `bpf()` syscall to load eBPF programs and maps. You saw programs and maps being created using the `BPF_PROG_LOAD` and `BPF_MAP_CREATE` commands.

在本章中，您看到用户空间代码如何使用**bpf()**系统调用来加载eBPF程序和映射。您还看到使用**BPF_PROG_LOAD**和**BPF_MAP_CREATE**命令创建了程序和映射。

You learned that the kernel keeps track of the number of references to eBPF programs and maps, releasing them when the reference count drops to zero. You were also introduced to the concepts of pinning BPF objects to a filesystem and using BPF links to create additional references.

你了解到内核跟踪对eBPF程序和映射的引用数量，并在引用计数降至零时释放它们。你还了解到将BPF对象固定到文件系统和使用BPF链接创建额外引用的概念。

You saw an example of `BPF_MAP_UPDATE_ELEM` being used to create entries in a map from user space. There are similar commands—`BPF_MAP_LOOKUP_ELEM` and `BPF_MAP_DELETE_ELEM`—for retrieving and deleting values from a map. There is also the command `BPF_MAP_GET_NEXT_KEY` for finding the next key that's present in a map. You can use this to iterate through all the valid entries.

您看到了使用**BPF_MAP_UPDATE_ELEM**在用户空间中创建映射条目的示例。还有类似的命令——**BPF_MAP_LOOKUP_ELEM**和**BPF_MAP_DELETE_ELEM**，用于从映射中检索和删除值。还有一个命令**BPF_MAP_GET_NEXT_KEY**，用于查找映射中存在的下一个键。您可以使用它来迭代所有有效的条目。

You saw examples of user space programs making use of `perf_event_open()` and `ioctl()` for attaching eBPF programs to kprobe events. The attachment method can be very different for other types of eBPF programs, and some of them even use the `bpf()` system call. For example, there's a `bpf(BPF_PROG_ATTACH)` syscall that can be used to attach cgroup programs, and `bpf(BPF_RAW_TRACEPOINT_OPEN)` for raw tracepoints (see Exercise 5 at the end of this chapter).

你看到了使用**perf_event_open()**和**ioctl()**来将eBPF程序附加到kprobe事件的用户空间程序的示例。对于其他类型的eBPF程序，附加方法可能会有很大的不同，其中一些甚至使用了**bpf()**系统调用。例如，有一个**bpf(BPF_PROG_ATTACH)**系统调用可以用于附加cgroup程序，**bpf(BPF_RAW_TRACEPOINT_OPEN)**用于原始跟踪点（请参见本章末尾的练习5）。

I also showed how you can use `BPF_MAP_GET_NEXT_ID`, `BPF_MAP_GET_FD_BY_ID`, and `BPF_OBJ_GET_INFO_BY_FD` to locate map (and other) objects held by the kernel.

我还展示了如何使用**BPF_MAP_GET_NEXT_ID**、**BPF_MAP_GET_FD_BY_ID**和**BPF_OBJ_GET_INFO_BY_FD**来定位内核中保存的映射（和其他）对象。

There are some other `bpf()` commands that I haven't covered in this chapter, but what you have seen here is enough to get a good overview.

这一章中我没有涵盖到一些其他的**bpf()**命令，但您已经看到的足够让你对此有一个良好的概述了。

You also saw some BTF data being loaded into the kernel, and I mentioned that `bpftool` uses this information to understand the format of data structures so that it can print them out nicely. I didn't explain yet what BTF data looks like or how it's used to make eBPF programs portable across kernel versions. That's coming up in the next chapter.

你还看到一些BTF数据被加载到内核中，我提到bpftool使用这些信息来了解数据结构的格式，以便能够将其打印出来。我还没有解释BTF数据的形式如何以及如何使用它来使eBPF程序在不同内核版本间具有可移植性。这将在下一章中介绍。

Exercises

练习

Here are a few things you can try if you'd like to explore the `bpf()` syscall further:

以下是一些您可以尝试的事情，如果您想进一步探索bpf()系统调用：

1. Confirm that the `insn_cnt` field from a `BPF_PROG_LOAD` system call corresponds to the number of instructions that are output if you dump the translated eBPF bytecode for that program using `bpftool`. (This is as documented on the [manpage for the `bpf\(\)` system call](#).)

请确认通过使用bpftool转储已翻译的eBPF字节码来输出程序的insn_cnt字段与BPF_PROG_LOAD系统调用中的指令数量相对应。（这是根据bpf（）系统调用手册中的文档记录而来）。

2. Run two instances of the example program so that there are two maps called `config`. If you run `bpftool map dump name config`, the output will include information about the two different maps as well as their contents. Run this under `strace`, and follow the use of different file descriptors through the syscall output. Can you see where it's retrieving information about a map and where it's retrieving the key-value pairs stored within it?

运行两个示例程序实例，以便有两个名为config的地图。如果运行bpftool map dump name config命令，则输出将包括关于这两个不同地图以及它们的内容的信息。在strace下运行，并通过系统调用输出跟踪不同文件描述符的使用情况。您能看到它在哪里检索地图信息以及在哪里检索存储在其中的键值对吗？

3. Use `bpftool map update` to modify the `config` map while one of the example programs is running. Use `sudo -u username` to check that these configuration changes are picked up by the eBPF program.

使用`bpftool map update`命令在示例程序运行时修改配置项映射。使用`sudo -u username`命令来检查是否生效了这些配置更改，并被 eBPF 程序捕捉到。

4. While `hello-buffer-config.py` is running, use `bpftool` to pin the program to the BPF filesystem, like this:

在运行 hello-buffer-config.py 的同时，使用 bpftool 将程序固定到 BPF 文件系统上，就像这样：

```
bpftool prog pin name hello /sys/fs/bpf/hi
```

Quit the running program, and check that the *hello* program is still loaded in the kernel using `bpftool prog list`. You can clean up the link by removing the pin with `rm /sys/fs/bpf/hi`.

退出正在运行的程序，并使用`bpftool prog list`检查`hello`程序是否仍在内核中加载。您可以通过使用`rm /sys/fs/bpf/hi`命令来清除链接。

5. Attaching to a raw tracepoint is considerably more straightforward at the syscall level than attaching to a kprobe, as it simply involves a `bpf()` syscall. Try converting *hello-buffer-config.py* to attach to the raw tracepoint for `sys_enter`, using BCC's `RAW_TRACEPOINT_PROBE` macro (if you did the exercises in [Chapter 2](#), you'll already have a suitable program you can use). You won't need to explicitly attach the program in the Python code, as BCC will take care of it for you. Running this under `strace`, you should see a syscall similar to this:

将整个流程与系统调用级别的拦截比较，可以发现在原始跟踪点上的拦截要简单得多，因为它只涉及到一个`bpf()`系统调用。尝试将`hello-buffer-config.py`转换为使用BCC的`RAW_TRACEPOINT_PROBE`宏来连接到`sys_enter`的原始跟踪点（如果您在第2章中做过练习，您可能已经有一个适用的程序）。您不需要在Python代码中显式连接程序，因为BCC会为您处理。在`strace`下运行，您应该看到类似于以下的系统调用。

```
bpf(BPF_RAW_TRACEPOINT_OPEN, {raw_tracepoint={name="sys_enter",  
prog_fd=6}}, 128) = 7
```

The tracepoint in the kernel has the name `sys_enter`, and the eBPF program with file descriptor `6` is being attached to it. From now on, whenever execution in the kernel reaches that tracepoint, it will trigger the eBPF program.

内核中的跟踪点名称为`sys_enter`，并且文件描述符`6`的eBPF程序正在附加到它。从现在开始，每当内核中的执行到达该跟踪点时，将触发eBPF程序。

6. Run the opensnoop application from [BCC's set of libbpf tools](#). This tool sets up some BPF links that you can see with `bpftool`, like this:

从BCC的libbpf工具集中运行opensnoop应用程序。此工具将设置一些BPF链接，您可以使用`bpftool`查看，就像这样：

```
$ bpftool link list  
116: perf_event prog 1849  
    bpf_cookie 0  
    pids opensnoop(17711)  
117: perf_event prog 1851  
    bpf_cookie 0  
    pids opensnoop(17711)
```

Confirm that the program IDs (1849 and 1851 in my example output here) match the output from listing the loaded eBPF programs:

请确认程序ID（在此示例输出中为1849和1851）与列出加载的eBPF程序的输出相匹配：

```
$ bpftool prog list
...
1849: tracepoint name tracepoint__syscalls__sys_enter_openat
      tag 8ee3432dcd98ffc3 gpl run_time_ns 95875 run_cnt 121
      loaded_at 2023-01-08T15:49:54+0000 uid 0
      xlated 240B jited 264B memlock 4096B map_ids 571,568
      btf_id 710
      pids opensnoop(17711)
1851: tracepoint name tracepoint__syscalls__sys_exit_openat
      tag 387291c2fb839ac6 gpl run_time_ns 8515669 run_cnt 120
      loaded_at 2023-01-08T15:49:54+0000 uid 0
      xlated 696B jited 744B memlock 4096B map_ids 568,571,569
      btf_id 710
      pids opensnoop(17711)
```

7. While opensnoop is running, try pinning one of these links with

`bpftool link pin id 116 /sys/fs/bpf/mylink` (using one of the link IDs you see output from `bpftool link list`). You should see that even after you terminate opensnoop, both the link and the corresponding program remain loaded in the kernel.

在`opensnoop`运行时，尝试使用`bpftool link pin id 116 /sys/fs/bpf/mylink`中的一个链接ID（使用从`bpftool link list`输出的链接ID之一）来固定住其中一个链接。你会发现即使在终止`opensnoop`之后，链接和相应的程序仍然加载在内核中。

8. If you skip ahead to the example code for [Chapter 5](#), you'll find a version of *hello-buffer-config.py* written using the *libbpf* library. This library automatically sets up a BPF link to the program that it loads into the kernel. Use `strace` to inspect the `bpf()` system calls that it makes, and see `bpf(BPF_LINK_CREATE)` system calls.

如果你直接跳到第五章的示例代码，你会发现有一个使用libbpf库编写的hello-buffer-config.py的版本。这个库会自动设置一个BPF连接到它加载到内核的程序上。使用strace来检查它所进行的bpf()系统调用，并查看bpf(BPF_LINK_CREATE)系统调用。

¹ If you want to see the full set of BPF commands, they're documented in the [linux/bpf.h](#) header file.

如果你想看到完整的BPF命令集，它们会在linux/bpf.h头文件中有文档记录。

² BTF was introduced upstream in the 5.1 kernel, but it has been back-ported on some Linux distributions, as you can see from [this discussion](#).

BTF在5.1内核中被引入，但它已经被一些Linux发行版进行了回移，你可以从这次讨论中看到。

³ These are defined in the `bpf_attach_type` enumerator in [linux/bpf.h](#).

这些在linux/bpf.h中的bpf_attach_type枚举器中定义。

⁴ A reminder that for more information on the difference, read Andrii Nakryiko's "[BPF ring buffer](#)" blog post.

请注意，要了解更多关于差异的信息，请阅读Andrii Nakryiko的“BPF环形缓冲区”博客文章。

Chapter 5. CO-RE, BTF, and Libbpf

第五章。CO-RE，BTF和Libbpf。

In the previous chapter you encountered BTF (BPF Type Format) for the first time. This chapter discusses why it exists and how it's used to make eBPF programs portable across different versions of the kernel. It's a key part of BPF's compile once, run everywhere (CO-RE) approach, which solves the problem of making eBPF programs portable across different kernel versions.

在前一章中，您第一次遇到了BTF（BPF类型格式）。本章讨论它的存在原因以及如何使用它使eBPF程序可以在不同版本的内核中进行移植。它是BPF的编译一次，在任何地方运行（CO-RE）方法的关键部分，可以解决在不同内核版本之间使eBPF程序移植的问题。

Many eBPF programs access kernel data structures, and an eBPF programmer would need to include relevant Linux header files so that their eBPF code can correctly locate fields within those data structures. However, the Linux kernel is under continuous development, which means internal data structures can change between different kernel versions. If you were to take an eBPF object file compiled on one machine¹ and load it onto a machine with a different kernel version, there would be no guarantee that the data structures would be the same.

许多eBPF程序访问内核数据结构，eBPF程序员需要包含相关的Linux头文件，以便其eBPF代码可以正确地定位这些数据结构中的字段。然而，Linux内核正在持续开发中，这意味着在不同的内核版本之间，内部数据结构可能会发生变化。如果您将在一台机器上编译的eBPF对象文件加载到具有不同内核版本的机器上，无法保证数据结构是相同的。

The CO-RE approach is a huge step forward in addressing this portability issue in an efficient way. It allows eBPF programs to include information about the data structure layouts they were compiled with, and it provides a mechanism for adjusting how fields are accessed if the data structure layout is different on the target machine where they run. Provided the program doesn't want to access a field or data structure that simply doesn't exist in the target machine's kernel, the program is portable across different kernel versions.

CO-RE方法是在高效处理该可移植性问题方面的一大步。它允许eBPF程序包含有关它们编译所使用的数据结构布局的信息，并提供了一种机制，用于调整字段的访问方式，以适应目标机器上的数据结构布局是否不同。只要程序不想访问在目标机器的内核中根本不存在的字段或数据结构，该程序就能在不同的内核版本之间实现可移植。

But before we dive into the details of how CO-RE works, let's discuss why it was so desirable, by looking at the previous approach to kernel portability as originally implemented in the BCC project.

但在我们深入了解CO-RE工作原理的细节之前，让我们通过观察BCC项目中最初实施的内核可移植性的先前方法来讨论它的吸引力。

BCC's Approach to Portability

BCC对可移植性的方法

In [Chapter 2](#) I used [BCC](#) to show a basic “Hello World” example of an eBPF program. The BCC project was the first popular project for implementing eBPF programs, providing a framework for both the user space and kernel aspects that’s relatively accessible to programmers without much kernel experience. To address portability across kernels, BCC took the approach of compiling eBPF code at runtime, in situ on the destination machine. There are a number of issues with this approach:

第2章中，我使用了BCC来展示一个基本的“Hello World”的eBPF程序示例。BCC项目是第一个流行的用于实现eBPF程序的项目，为没有太多内核经验的程序员提供了一个相对易于访问的用户空间和内核部分的框架。为了解决内核之间的可移植性问题，BCC采取了在目标机器上即时编译eBPF代码的方法。这种方法存在一些问题：

- The compilation toolchain needs to be installed on every destination machine where you want the code to run, as well as the kernel header files (which aren’t always present by default).

编译工具链需要安装在每个目标机器上，您希望代码能运行的地方，以及内核头文件（这些文件通常不会默认存在）。

- You have to wait for the compilation to complete before the tool starts, which could mean a delay of several seconds, every time the tool is launched.

每次工具启动前，你需要等待编译完成，这可能意味着几秒钟的延迟。

- If you're running the tool on a large fleet of identical machines, repeating the compilation on each machine is a waste of compute resources.

如果您在一大批相同的机器上运行该工具，每台机器上重复编译将浪费计算资源。

- Some BCC-based projects package their eBPF source code and the toolchain into a container image, which makes distribution to each machine easier. But it doesn't solve the problem of ensuring that the kernel headers are present, and it can even mean more duplication if several of these BCC containers are installed on each machine.

一些基于BCC的项目将其eBPF源代码和工具链封装到容器镜像中，这样可以更方便地分发到每台机器。但是，这并不能解决确保内核头文件存在的问题，甚至可能意味着如果在每台机器上安装了多个这样的BCC容器，则会导致更多的重复。

- Embedded devices might not have sufficient memory resources to run the compilation step.

嵌入式设备可能没有足够的内存资源来运行编译步骤。

Because of these issues, if you're planning to embark on developing a significant new eBPF project, I would not recommend using this legacy BCC approach for it, especially if you're planning to distribute it for others to use. In this book I've given some examples based on

BCC because it's a good approach for learning about the basic concepts of eBPF, particularly because the Python user space code is so compact and easy to read. It's also a perfectly good choice if you're more comfortable with it and you want to put something together quickly. But it's not the best approach for serious modern eBPF development.

因为这些问题，如果你计划着手开发一个重要的新eBPF项目，我不建议使用这种传统的BCC方法，特别是如果你计划将其分发给其他人使用。在本书中，我给出了一些基于BCC的例子，因为这是学习eBPF基本概念的一个很好的方法，特别是因为Python用户空间代码非常简洁易读。如果你对它更熟悉并且想快速组建一些东西，那么这也是一个完全不错的选择。但对于严肃的现代eBPF开发来说，这并不是最佳方法。

The CO-RE approach offers a much better solution to the problem of cross-kernel portability for eBPF programs.

CO-RE方法为eBPF程序跨内核可移植性问题提供了更好的解决方案。

Note

The BCC project at github.com/iovisor/bcc includes a wide range of command-line tools for observing all sorts of information about how a Linux machine is behaving. The original versions located in the *tools* directory are mostly implemented in Python using this legacy approach to portability that I have described in this section.

BCC项目位于github.com/iovisor/bcc，包含了一系列的命令行工具，用于观察Linux机器的各种行为信息。在tools目录下的原始版本大部分是用Python实现的，采用了我在本章节中描述的传统可移植性方法。

In BCC's *libbpf-tools* directory, you'll find updated versions of these tools written in C that take advantage of *libbpf* and CO-RE and that don't suffer from the problems I've just listed. They are an incredibly useful set of utilities!

在BCC的libbpf-tools目录中，您会找到使用C语言编写的更新版本的这些工具，利用libbpf和CO-RE，并且不会遇到我刚刚列出的问题。它们是一套非常有用的工具！

CO-RE Overview

CO-RE 概述

The CO-RE approach consists of a few elements:^{2,3}

CO-RE方法包括几个要素：

BTF

BTF is a format for expressing the layout of data structures and function signatures. In CO-RE it's used to determine any differences between the structures used at compilation time and at runtime. BTF is also used by tools like `bpftool` to dump data structures in human-readable formats. Linux kernels from 5.4 onward support BTF.

Kernel headers

The Linux kernel source code includes header files that describe the data structures it uses, and these headers can change between versions of Linux. eBPF programmers can choose to include individual header files, or, as you'll see in this chapter, you can use `bpftool` to generate a header file called `vmlinux.h` from a running system, containing all the data structure information about a kernel that a BPF program might need.

Compiler support

The [Clang compiler was enhanced](#) so that when it compiles eBPF programs with the `-g` flag, it includes what are known as *CO-RE relocations*, derived from the BTF information describing the kernel data structures. The GCC compiler also added CO-RE support for BPF targets in [version 12](#).

Library support for data structure relocations

At the point where a user space program loads an eBPF program into the kernel, the CO-RE approach requires the bytecode to be adjusted to compensate for any differences between the data structures present when it was compiled, and what's on the destination machine where it's about to run, based on the CO-RE relocation information compiled into the object. There are a few libraries that will take care of this: [libbpf](#) was the original C library that includes this relocation capability, the Cilium eBPF library provides the same capability for Go programmers, and Aya does it for Rust.

Optionally, a BPF skeleton

A skeleton can be auto-generated from a compiled BPF object file, containing handy functions that user space code can call to manage the lifecycle of BPF programs—loading them into the kernel, attaching them to events, and so on. If you’re writing the user space code in C, you can generate the skeleton with `bpftool gen skeleton`. These functions are higher-level abstractions that can be more convenient for the developer than using the underlying library (*libbpf*, *cilium/ebpf*, etc.) directly.

Note

Andrii Nakryiko wrote an [excellent blog post](#) that describes the background of CO-RE, as well as laying out how it works and how to use it. He also wrote the canonical [BPF CO-RE Reference Guide](#), so please do read that if you’re embarking on writing code yourself. His [libbpf-bootstrap guide](#) to building an eBPF app from scratch with CO-RE + *libbpf* + skeletons is another must-read.

安德里·纳克里科撰写了一篇出色的博客文章，描述了CO-RE的背景，并介绍了它的工作原理和使用方法。他还撰写了权威的BPF CO-RE参考指南，如果你打算自己编写代码，请务必阅读该指南。他的libbpf-bootstrap指南则教你如何从头开始构建一个带有CO-RE + libbpf + 骨架的eBPF应用程序，也是一本必读的指南。

Now that you have an overview of the elements of CO-RE, let’s dig in to see how they work, starting with an exploration of BTF.

既然你已经对CO-RE的要素有了一个概述，现在让我们深入了解它们的工作原理，首先探索一下BTF。

BPF Type Format

BPF类型格式

BTF information describes how data structures and code are laid out in memory. This information can be put to a variety of different uses.

BTF信息描述了数据结构和代码在内存中的布局。这些信息可以被用于各种不同的用途。

BTF Use Cases

BTF 使用案例

The main reason for discussing BTF in this chapter on CO-RE is that knowing the differences between a structure's layout where an eBPF program was compiled and where it is about to run allows for the appropriate adjustments to be made as the program is loaded into the kernel. I'll discuss the relocation process later in this chapter, but for now, let's also consider some of the other uses to which BTF information can be put.

讨论BTF在CO-RE中的主要原因是因为了解在编译eBPF程序的结构布局和即将运行的位置之间的差异，可以在程序加载到内核时进行适当的调整。我将在本章后面讨论重定位过程，但现在，让我们也考虑一下BTF信息可以用于的其他一些用途。

Knowing how a structure is laid out, and the type of every field in that structure, makes it possible to pretty-print a structure's contents in human-readable form. For example, a string is just a series of bytes

from the computer's point of view, but converting those bytes into characters makes the string much easier for humans to understand. You already saw an example of this in the previous chapter, where `bpftool` used BTF information to format the output of map dumps.

了解结构的布局方式和每个字段的类型，可以将结构的内容以人可读的形式美观地打印出来。例如，从计算机的角度来看，字符串只是一系列字节，但将这些字节转换为字符使得字符串对人类更容易理解。在上一章节中，您已经看到了一个例子，即bpftool使用BTF信息来格式化map dumps的输出。

BTF information also includes the line and function information that enables `bpftool` to interleave source code within the output from translated or JITed program dumps, as you saw in [Chapter 3](#). When you come to [Chapter 6](#), you'll also see the source code information interleaved with the verifier log output, and again this comes from the BTF information.

BTF信息还包括用于使bpftool在翻译或JIT编译程序转储的输出中插入源代码的行和函数信息，就像你在第三章中看到的那样。当您到达第六章时，您还将看到源代码信息与验证器日志输出交织在一起，同样这也来自BTF信息。

BTF information is also required for BPF spin locks. *Spin locks* are used to stop two CPU cores from simultaneously accessing the same map values. The lock has to be part of the map's value structure, like this:

BTF信息也是BPF自旋锁所必需的。自旋锁用于防止两个CPU核心同时访问相同的映射值。该锁必须是映射值结构的一部分，如下所示：

```
struct my_value {  
    ... <other fields>  
    struct bpf_spin_lock lock;  
    ... <other fields>  
};
```

Within the kernel, eBPF programs use `bpf_spin_lock()` and `bpf_spin_unlock()` helper functions to acquire and release a lock. These helpers can be used only if BTF information is available to describe where the lock field is within the structure.

在内核中，eBPF程序使用**bpf_spin_lock()**和**bpf_spin_unlock()**助手函数来获取和释放锁。只有在BTF信息可用以描述锁字段在结构体中的位置时，才可以使用这些助手函数。

Note

Spin lock support was added in kernel version 5.1. There are lots of restrictions on the use of spin locks: they can only be used on hash or array map types, and they can't be used in tracing or socket filter type eBPF programs. Read more about spin locks in the [lwn.net article on concurrency management in BPF](#).

自核心版本5.1开始，增加了自旋锁的支持。对自旋锁的使用有很多限制：它们只能用于哈希或数组映射类型，并且不能在跟踪或套接字过滤类型的eBPF程序中使用。请在BPF的并发管理的lwn.net文章中了解更多关于自旋锁的信息。

Now that you know why BTF information is useful, let's make it more concrete by looking at some examples.

既然你知道BTF信息为什么有用，让我们通过一些例子让它更具体一些。

Listing BTF Information with bpftool

使用bpftool列出BTF信息

As with programs and maps, you can use the `bpftool` utility to show BTF information. The following command lists all the BTF data loaded into the kernel:

与程序和地图一样，您可以使用bpftool工具显示BTF信息。以下命令列出了加载到内核的所有BTF数据：

```
bpftool btf list
1: name [vmlinux] size 5843164B
2: name [aes_ce_cipher] size 407B
3: name [cryptd] size 3372B
...
149: name <anon> size 4372B prog_ids 319 map_ids 103
      pids hello-buffer-co(7660)
155: name <anon> size 37100B
      pids bpftool(7784)
```

(I've omitted many entries from the results for brevity.)

我为了简洁起见，省略了许多条目的结果。

The first entry in the list is `vmlinux`, and it corresponds to the *vmlinux* file I mentioned earlier that holds the BTF information about the currently running kernel.

列表中的第一个条目是vmlinux，它对应于我之前提到的vmlinux文件，该文件保存了有关当前运行的内核的BTF信息。

Note

Some of the examples early in this chapter reuse the programs from [Chapter 4](#), and then later in this chapter you'll find new examples for which the source is in the *chapter5* directory at github.com/lizrice/learning-ebpf.

本章的一些例子在早期重用了第四章的程序，然后在本章的后面，你会发现一些新的例子，它们的源代码位于github.com/lizrice/learning-ebpf的chapter5目录中。

To obtain this example output I ran this command while the `hello-buffer-config` example from [Chapter 4](#) was running. You can see the entry describing the BTF information that this process is using, on the line that starts with `149:`:

为了获得这个示例输出，我在运行第四章的hello-buffer-config示例时运行了这个命令。您可以在以149开头的那一行中看到描述这个进程正在使用的BTF信息的条目。

```
149: name <anon>  size 4372B  prog_ids 319  map_ids 103  
      pids hello-buffer-co(7660)
```

Here's what that line is telling us:

这句话告诉我们的是：

- This chunk of BTF information has ID 149.

这个BTF信息块的ID是149。

- It's an anonymous blob of around 4 KB of BTF information.

这是一个大约4 KB的BTM信息匿名块。

- It's used by the BPF program with `prog_id 319` and the BPF map with `map_id 103`.

它由具有prog_id 319的BPF程序使用，并由具有map_id 103的BPF映射使用。

- It's also used by the process with ID 7660 (shown within parentheses) running the `hello-buffer-config` executable (whose name has been truncated to 15 characters).

也被进程ID为7660（括号内显示）的运行hello-buffer-config可执行文件（文件名被截断为15个字符）使用。

These program, map, and BTF identifiers match with the following output that `bpftool` shows about `hello-buffer-config`'s program called `hello`:

这些程序、映射和BTF标识符与bpftool显示的关于hello-buffer-config程序称为hello的以下输出相匹配：

```
bpftool prog show name hello
319: kprobe name hello tag a94092da317ac9ba gpl
      loaded_at 2022-08-28T14:13:35+0000 uid 0
      xlated 400B jited 428B memlock 4096B map_ids
103,104
      btf_id 149
      pids hello-buffer-co(7660)
```

The only thing that doesn't appear to match completely between these two sets of information is that the program refers to an extra `map_id, 104`. That's the perf event buffer map, and it doesn't use BTF information; hence, it doesn't appear in the BTF-related output.

两组信息之间唯一不完全一致的地方是程序中引用了一个额外的地图ID，104。这是性能事件缓冲地图，并且它不使用BTF信息，因此在与BTF相关的输出中不会出现。

Much like `bpftool` can dump the contents of programs and maps, it can also be used to view the BTF type information contained in a blob of data.

与bpftool可以转储程序和映射的内容一样，它也可以用来查看包含在数据块中的BTF类型信息。

BTF Types

BTF类型

Knowing the ID of the BTF information, you can inspect its contents with the command `bpftool btf dump id <id>`. When I ran this using the ID 149 that I obtained earlier, I got 69 lines of output, each of which is a type definition. I'll just describe the first few lines, which should give you a good idea of how to interpret the rest. The BTF information from these first few lines relates to the `config` hash map, which was defined in the source code like this:

知道BTF信息的ID后，您可以使用命令`bpftool btf dump id <id>`来检查其内容。当我使用之前获得的ID 149运行时，得到了69行输出，每行

输出都是一个类型定义。我只描述前几行，这应该能给您一个很好的理解如何解释其余部分。这几行中的BTF信息与配置哈希映射相关，其在源代码中被定义如下：

```
struct user_msg_t {  
    char message[12];  
};  
  
BPF_HASH(config, u32, struct user_msg_t);
```

This hash table has keys of type `u32` and values of type `struct user_msg_t`. That structure holds a 12-byte `message` field. Let's see how these types are defined in the corresponding BTF information.

这个哈希表的键是u32类型，值是struct user_msg_t类型。该结构包含一个12字节的消息字段。让我们看一下这些类型在相应的BTF信息中是如何定义的。

The first three lines of the BTF output are as follows:

BTF输出的前三行如下所示：

```
[1] TYPEDEF 'u32' type_id=2  
[2] TYPEDEF '__u32' type_id=3  
[3] INT 'unsigned int' size=4 bits_offset=0 nr_bits=32  
encoding=(none)
```

The number in square brackets at the start of each line is the type ID (so the first line, starting with `[1]`, defines `type_id 1`, etc.). Let's dive into these three types in more detail:

每行开头方括号中的数字是类型ID（因此第一行从[1]开始，定义了类型ID为1，依此类推）。让我们更详细地了解这三种类型：

- Type 1 defines a type named `u32` and its type, defined by `type_id 2`, that is, the type defined in the line that starts with `[2]`. As you know, the keys in the hash table have this type `u32`.

类型1定义了一个名为u32的类型，及其由type_id 2定义的类型，即在以[2]开头的行中定义的类型。如你所知，哈希表中的键具有这种u32类型。

- Type 2 has the name `__u32` and the type defined by `type_id 3`.

类型2的名称为__u32，并由类型ID 3定义的类型。

- Type 3 is an integer type with the name `unsigned int`, which is 4 bytes long.

类型3是一个整数类型，名为无符号整型unsigned int，长度为4字节。

All three of these types are synonyms for a 32-bit unsigned integer type. In C, the lengths of integers are platform dependent, so Linux defines types like `u32` to explicitly define integers of specific lengths. On this machine, `u32` corresponds to an unsigned integer. User space code that refers to these should use the synonym prefixed with underscores, as in `__u32`.

所有这三种类型都是32位无符号整数类型的同义词。在C语言中，整数的长度与平台有关，所以Linux定义了像u32这样的类型，以显式地定义特定长度的整数。在这台机器上，u32对应无符号整数。引用这些类型的用户空间代码应使用带下划线前缀的同义词，例如__u32。

The next few types in the BTF output look like this:

下面几个类型在BTF输出中看起来像这样：

```
[4] STRUCT 'user_msg_t' size=12 vlen=1
    'message' type_id=6 bits_offset=0
[5] INT 'char' size=1 bits_offset=0 nr_bits=8 encoding=
(none)
[6] ARRAY '(anon)' type_id=5 index_type_id=7 nr_elems=12
[7] INT '__ARRAY_SIZE_TYPE__' size=4 bits_offset=0
nr_bits=32 encoding=(none)
```

These relate to the `user_msg_t` structure used for values in the `config` map:

与配置映射中用于值的 `user_msg_t` 结构相关。

- Type 4 is the `user_msg_t` structure itself, and in total it is 12 bytes long. It contains one field named `message`, which is defined by type 6. The `vlen` field indicates how many fields there are in this definition.

Type 4是`user_msg_t`结构本身，总共长度为12字节。它包含一个名为`message`的字段，由type 6定义。`vlen`字段表示此定义中有多少个字段。

- Type 5 is named `char` and is a 1-byte integer—exactly the definition a C programmer would expect for a type called “char.”

类型5被命名为字符类型，它是一个1字节的整数，正是C程序员对名为“char”的类型所期望的定义。

- Type 6 defines the type for that `message` field as an array with 12 elements. Each element has type 5 (it's a `char`), and the array is indexed by type 7.

Type 6将该消息字段的类型定义为包含12个元素的数组。每个元素的类型为5（它是一个字符），并且该数组由类型7索引。

- Type 7 is a 4-byte integer.

Type 7是一个4字节整数。

With these definitions, you can build a complete picture of how the `user_msg_t` structure is laid out in memory, as illustrated in [Figure 5-1](#).

通过这些定义，你可以构建出用户消息(`user_msg_t`)结构在内存中的完整图像，就像图5-1所示。

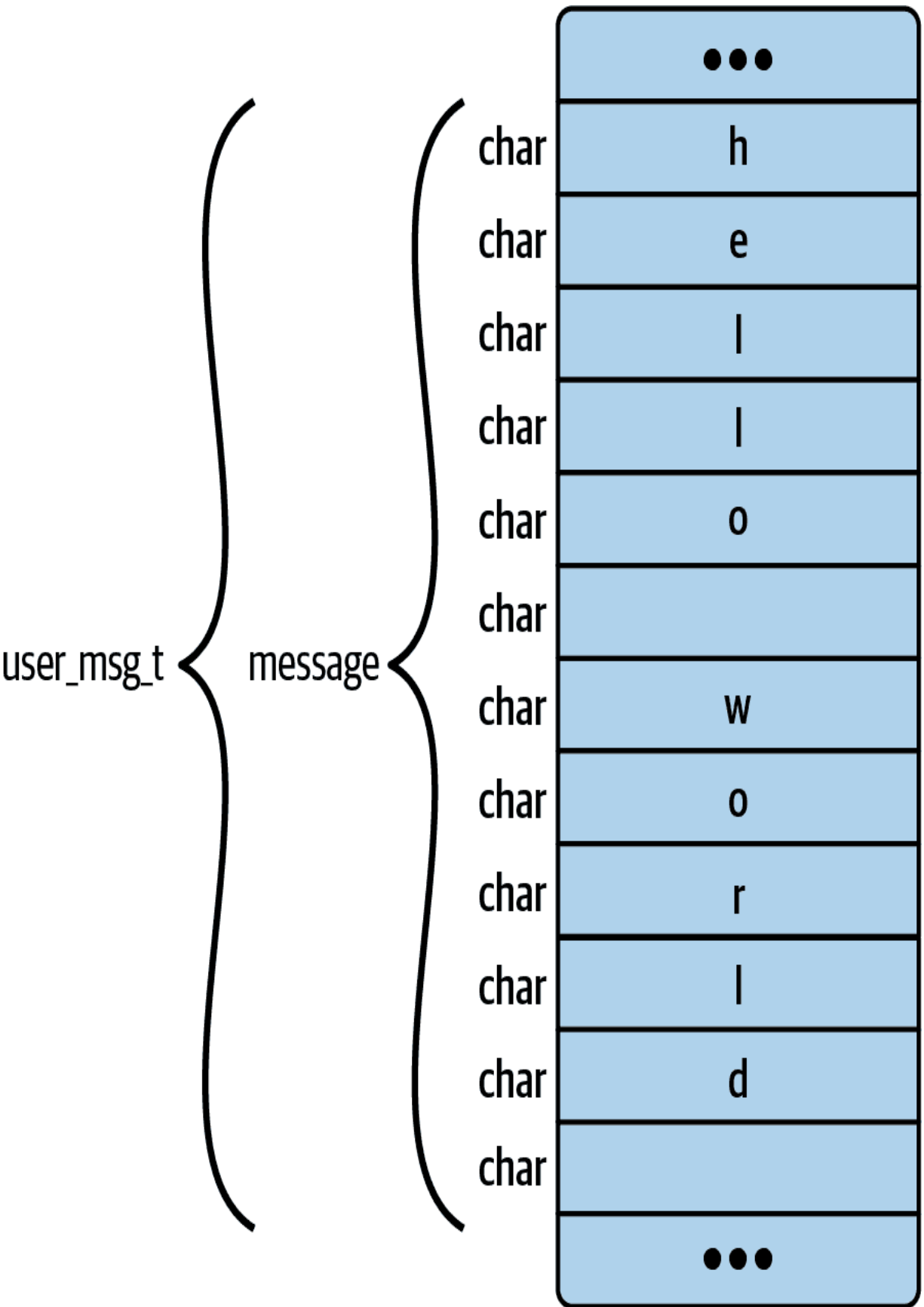


Figure 5-1. A `user_msg_t` structure takes 12 bytes of memory

So far, all the entries have `bits_offset` set to 0, but the next line of output has a structure with more than one field:

到目前为止，所有的条目的`bits_offset`都设置为0，但是下一行的输出有一个带有多个字段的结构。

```
[8] STRUCT '____btf_map_config' size=16 vlen=2
      'key' type_id=1 bits_offset=0
      'value' type_id=4 bits_offset=32
```

This is a structure definition for the key–value pairs stored in the map called `config`. I didn't define this `____btf_map_config` type myself in the source code, but it has been generated by BCC. The key is of type `u32`, and the value is the `user_msg_t` structure. These correspond to the types 1 and 4 that you saw earlier.

这是一个用于存储在名为`config`的映射中的键-值对结构定义。我没有在源代码中定义这个`____btf_map_config`类型，而是由BCC生成的。键的类型是`u32`，值是`user_msg_t`结构。这对应于之前你看到的类型1和类型4。

The other important part of the BTF information about this structure is that the `value` field starts 32 bits after the start of the structure. That completely makes sense because the first 32 bits are needed to hold the `key` field.

BTF 信息中关于这个数据结构的另一个重要部分是，值字段在结构开始后的 32 位处开始。这完全说得通，因为前 32 位用于保存键字段。

Note

In C, structure fields get automatically aligned to boundaries, so you can't simply assume that one field always follows directly after the previous one in memory. For example, consider a structure like this:

在C中，结构体字段会自动对齐到边界，因此您不能简单地假设一个字段总是直接跟在前一个字段之后的内存中。例如，考虑这样的一个结构体：

```
struct something {  
    char letter;  
    u64 number;  
}
```

There would be 7 bytes of unused memory after the field called `letter` before the `number` field so that the 64-bit number can be aligned to a memory location divisible by 8.

在名为`letter`的字段之前，会有7个字节的未使用内存，以便64位数字可以对齐到可被8整除的内存位置。

It's possible in some circumstances to turn on compiler packing to avoid this unused space, but it generally results in lower performance and—at least in my experience—it's unusual to do so. More often, C programmers will design structures by hand to make efficient use of space.

在某些情况下，可以启用编译器打包以避免这些未使用的空间，但通常会导致性能下降，在我个人的经验中，这样做是不常见的。更常见的是，C程序员会手动设计结构以有效地利用空间。

Maps with BTF Information

地图与BTF信息

You've just seen the BTF information associated with a map. Now let's see how this BTF data is passed to the kernel when the map is created.

现在让我们看一下这个与地图关联的BTF信息。当地图被创建时，我们将看到这些BTF数据是如何传递给内核的。

You saw in [Chapter 4](#) that maps are created using the `bpf(BPF_MAP_CREATE)` syscall. This takes a `bpf_attr` structure as a parameter, [defined in the kernel](#) like this (some details omitted):

之前在第四章中你看到了，地图是通过使用bpf（BPF_MAP_CREATE）系统调用创建的。它使用了bpf_attr结构作为参数，在内核中定义如下（省略了一些细节）：

```
struct {
    /* anonymous struct used by BPF_MAP_CREATE command */
    __u32    map_type;
            /* one of enum bpf_map_type */
    __u32    key_size;
            /* size of key in bytes */
    __u32    value_size;
            /* size of value in bytes */
    __u32    max_entries;
            /* max number of entries in a map */
    ...
    char     map_name[BPF_OBJ_NAME_LEN];
    ...
    __u32    btf_fd;
            /* fd pointing to a BTF type data */
    __u32    btf_key_type_id;
            /* BTF type_id of the key */
    __u32    btf_value_type_id;
}
```

```
    /* BTF type_id of the value */  
    ...  
};
```

Before the introduction of BTF, the `btf_*` fields weren't present in this `bpf_attr` structure, and the kernel had no knowledge of the structure of keys or values. The `key_size` and `value_size` fields defined how much memory was required for them, but they were just treated as so many bytes. By additionally passing in the BTF information defining the types of the keys and values, the kernel can introspect them, and utilities like `bpftool` can retrieve the type information for pretty-printing, as discussed earlier. However, it's interesting to note that separate BTF `type_ids` are passed in for the key and the value. The `__btf_map_config` structure that you just saw defined isn't used by the kernel for the map definition; it's just used by BCC on the user space side.

BTF引入之前，`bpf_attr`结构中不存在**`btf_*`**字段，内核对键和值的结构一无所知。`key_size`和**`value_size`**字段仅定义了所需内存量，但它们只被视为字节数。通过额外传递定义键和值类型的BTF信息，内核可以自省它们，并且像**`bpftool`**这样的工具可以检索类型信息以进行漂亮打印，就像之前讨论的那样。然而，有趣的是要注意，键和值分别传递了独立的BTF类型_id。刚才看到的**`__btf_map_config`**结构不被内核用于映射定义；它只在用户空间的BCC中使用。

BTF Data for Functions and Function Prototypes

函数和函数原型的BTF数据

So far the BTF data in this example output has related to data types, but the BTF data also contains information about functions and function prototypes. Here's the information from the same BTF data blob that describes the `hello` function:

到目前为止，这个示例输出中的BTF数据与数据类型有关，但BTF数据还包含有关函数和函数原型的信息。这是描述hello函数的同一BTF数据块中的信息：

```
[31] FUNC_PROTO '(anon)' ret_type_id=23 vlen=1
      'ctx' type_id=10
[32] FUNC 'hello' type_id=31 linkage=static
```

In type 32 you can see the function named `hello` is defined as having the type defined in the previous line. That's a *function prototype*, which returns a value of type ID `23` and takes a single parameter (`vlen=1`) called `ctx` with type ID `10`. For completeness, here are the definitions of those types from earlier in the output:

在32类型中，您可以看到名为hello的函数被定义为具有在前一行中定义的类型。这是一个函数原型，它返回类型为ID 23的值，并带有一个名为ctx且类型为ID 10的参数（vlen=1）。为了完整起见，这里是输出中先前那些类型的定义：

```
[10] PTR '(anon)' type_id=0

[23] INT 'int' size=4 bits_offset=0 nr_bits=32
      encoding=SIGNED
```

Type 10 is an anonymous pointer with the default type of `0`, which isn't explicitly included in the BTF output but is defined as a void pointer.⁴

类型10是一个匿名指针，其默认类型为0，在BTF输出中没有明确包含，但被定义为void指针。

The return value with type 23 is a 4-byte integer, and

`encoding=SIGNED` indicates that it's a signed integer; that is, it can have either a positive or negative value. This corresponds to the function definition in the source code of *hello-buffer-config.py*, which looks like this:

返回类型为23的返回值是一个4字节整数，`encoding=SIGNED`表示它是一个有符号整数；也就是说，它可以有正值或负值。这对应于*hello-buffer-config.py*源代码中的函数定义，如下所示：

```
int hello(void *ctx)
```

The example BTF information I've shown so far comes from listing the contents of a blob of BTF data. Let's see how to obtain just the BTF information that relates to a particular map or program.

我所展示的BTF示例信息是从列出BTF数据块的内容中得到的。现在让我们看看如何获取与特定地图或程序相关的BTF信息。

Inspecting BTF Data for Maps and Programs

检查BTF数据的地图和程序

If you want to inspect the BTF types associated with a particular map, `bpftool` makes that easy. For example, here's the output for the `config` map:

如果您想检查与特定地图相关联的BTF类型，bpftool可以轻松实现。例如，这是配置地图的输出：

```
bpftool btf dump map name config
[1] TYPEDEF 'u32' type_id=2
[4] STRUCT 'user_msg_t' size=12 vlen=1
      'message' type_id=6 bits_offset=0
```

Similarly, you can inspect the BTF information related to a particular program with `bpftool btf dump prog <prog identity>`. I'll leave you to check out the [manpage](#) for additional details.

同样，您可以使用`bpftool btf dump prog <prog identity>`检查与特定程序相关的BTF信息。我会让您自行查看manpage以获取更多详细信息。

Note

If you'd like to better understand how the BTF type data is generated and de-duplicated, there is another [excellent blog post from Andrii Nakryiko](#) on the subject.

如果您想更好地了解BTF类型数据是如何生成和去重的，可以参考Andrii Nakryiko的另一篇关于此主题的博客文章。

By this stage you should have an understanding of how BTF describes the format of data structures and functions. An eBPF program written in C needs header files that define the types and structures. Let's see how easy it is to generate a header file for any kernel data types that an eBPF program might need.

到此为止，你应该已经对BTF如何描述数据结构和函数有了一定的理解。用C语言编写的eBPF程序需要定义类型和结构的头文件。我们来看看为eBPF程序可能需要的任何内核数据类型生成头文件有多容易。

Generating a Kernel Header File

生成内核头文件

If you run `bpftool btf list` on a BTF-enabled kernel, you'll see lots of preexisting blobs of BTF data that look like this:

如果在启用BTF的内核上运行`bpftool btf list`，你会看到很多预先存在的BTF数据块，看起来像这样：

```
$ bpftool btf list
1: name [vmlinux] size 5842973B
2: name [aes_ce_cipher] size 407B
3: name [cryptd] size 3372B
...
```

The first item in this list, with ID 1 and named `vmlinux`, is the BTF information about all the data types, structures, and function definitions used by the kernel that's running on this (virtual) machine.⁵

该列表中的第一项，ID为1，名为`vmlinux`，是关于在此（虚拟）机器上运行的内核使用的所有数据类型、结构和函数定义的BTF信息。

An eBPF program needs the definitions of any kernel data structures and types that it is going to refer to. Before the days of CO-RE, you'd typically have to figure out which of the many individual header files in the Linux kernel source held the definition for the structures you

were interested in, but now there is a much easier way, as BTF-enabled tools can generate an appropriate header file from the BTF information included with the kernel.

在CO-RE之前，eBPF程序需要对其引用的任何内核数据结构和类型进行定义。在Linux内核源代码的众多单独的头文件中确定您感兴趣的结构的定义通常是必需的，但现在有一种更简单的方法，即启用BTF的工具可以从与内核一起包含的BTF信息中生成适当的头文件。

This header file is conventionally called *vmlinux.h*, and you can generate it with `bpftool` like this:

此头文件通常被称为vmlinux.h，您可以使用bpftool生成它，像这样：

```
bpftool btf dump file /sys/kernel/btf/vmlinux format c >
vmlinux.h
```

This file defines all the kernel's data types, so including this generated *vmlinux.h* file in your eBPF program source supplies the definitions of any Linux data structures you might need. When you compile the source into an eBPF object file, that object will include BTF information that matches the definitions used in this header file. Later, when the program is run on a target machine, the user space program that loads it into the kernel will make adjustments to account for differences between this build-time BTF information and the BTF information for the kernel that's running on that target machine.

这个文件定义了所有内核的数据类型，所以将这个生成的vmlinux.h文件包含在您的eBPF程序源代码中，可以提供您可能需要的任何Linux

数据结构的定义。当您将源代码编译成一个eBPF对象文件时，该对象将包含与该头文件中使用的定义相匹配的BTF信息。稍后，在目标机器上运行程序时，将载入它到内核的用户空间程序将根据该构建时的BTF信息和运行在目标机器上的内核的BTF信息之间的差异进行调整。

BTF information in the form of the `/sys/kernel/btf/vmlinux` file has been included in the Linux kernel since version 5.4,⁶ but raw BTF data that *libbpf* can make use of can also be generated for older kernels. In other words, if you want to run a CO-RE-enabled eBPF program on a target machine that doesn't have BTF information already, you might be able to provide the BTF data for that target yourself. There's information on how to generate BTF files, and an archive of files for a variety of Linux distributions, on the [BTFHub](#).

自Linux内核5.4版本起，BTF信息以`/sys/kernel/btf/vmlinux`文件的形式被包含在Linux内核中。但是，对于旧版本的内核，也可以生成libbpf可以利用的原始BTF数据。换句话说，如果您想在目标机器上运行一个启用了CO-RE的eBPF程序，但该机器上没有BTF信息，您可能可以为该目标提供BTF数据。在BTFHub上有关于如何生成BTF文件的信息，以及各种Linux发行版的文件存档。

Note

The BTFHub repo also includes further reading about [BTF internals](#) should you want to dive deeper into this topic.

BTFHub仓库还包括了关于BTF内部机制的进一步阅读材料，如果你想深入了解这个主题的话。

Next, let's look at how this and other tactics are used to write eBPF programs to be portable across kernels using CO-RE.

接下来，让我们看一下如何使用CO-RE来编写可在不同内核间具有可移植性的eBPF程序以及其他策略的应用。

CO-RE eBPF Programs

CO-RE eBPF程序

You'll recall that eBPF programs run in the kernel. Later in this chapter I'll show some user space code that will interact with the code running in the kernel, but in this section I'm concentrating on the kernel side.

eBPF程序运行在内核中，你可能还记得。本章后面我会展示一些用户空间代码，它们将与在内核中运行的代码进行互动。但在本节中，我把注意力集中在内核方面。

As you've already seen, eBPF programs are compiled to eBPF bytecode, and (at least at the time of this writing) the compilers that support this are Clang or gcc for compiling C code, and the Rust compiler. I'll discuss some of your options for using Rust in [Chapter 10](#), but for the purposes of this chapter I'll assume you're writing in C and using Clang, along with the *libbpf* library.

正如您已经看到的，eBPF程序被编译为eBPF字节码，而（至少在撰写本文时）支持此功能的编译器有Clang或gcc用于编译C代码以及Rust编译器。我将在第10章讨论一些使用Rust的选项，但在本章中，我将假设您使用C编写并使用Clang和libbpf库。

For the remainder of this chapter, let's consider an example application called *hello-buffer-config*. It's very similar to the *hello-buffer-config.py* example from the previous chapter that used the BCC framework, but this version is written in C to use *libbpf* and CO-RE.

在本章的其余部分中，让我们考虑一个名为hello-buffer-config的示例应用程序。它与前一章中使用BCC框架的hello-buffer-config.py示例非常相似，但此版本使用libbpf和CO-RE编写的C代码。

If you have BCC-based eBPF code that you want to migrate to *libbpf*, check out the excellent and comprehensive [guide by Andrii Nakryiko on his website](#). BCC provides some convenient shortcuts that aren't handled in quite the same way using *libbpf*; conversely, *libbpf* provides its own set of macros and library functions to make life easier for the eBPF programmer. As I walk through the example, I will point out a few differences between the BCC and *libbpf* approaches.

如果你有基于BCC的eBPF代码想要迁移到libbpf，可以访问Andrii Nakryiko在他的网站上提供的出色而全面的指南。BCC提供了一些方便快捷的快捷方式，而使用libbpf则没有以完全相同的方式处理；相反，libbpf提供了自己的一套宏和库函数来简化eBPF程序员的生活。在我介绍示例时，我会指出BCC和libbpf方法之间的一些差异。

Note

You'll find the example C eBPF program to accompany this section in the *chapter5* directory of the github.com/lizrice/learning-ebpf repo.

您可以在github.com/lizrice/learning-ebpf repo的chapter5目录中找到一个与本节相关的C eBPF程序示例。

First let's look at *hello-buffer-config.bpf.c*, which implements the eBPF program that runs in the kernel. Later in the chapter I'll show you the user space code in *hello-buffer-config.c* that loads the program and displays output, much as the Python code did in the BCC implementation of this example in [Chapter 4](#).

首先，让我们看一下hello-buffer-config.bpf.c，它实现了在内核中运行的eBPF程序。稍后在本章中，我将向您展示用户空间代码hello-buffer-config.c，该代码加载程序并显示输出，就像第4章中的BCC实现示例中的Python代码那样。

Like any C program, an eBPF program will need to include some header files.

与任何C程序一样，一个eBPF程序都需要包含一些头文件。

Header Files

头文件

The first few lines of *hello-buffer-config.bpf.c* specify the header files that it needs:

hello-buffer-config.bpf.c的前几行指定了它所需的头文件：

```
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>
#include "hello-buffer-config.h"
```

These five files are the *vmlinux.h* file, a few headers from *libbpf*, and an application-specific header file that I wrote myself. Let's see why this is a typical pattern for the header files needed for a *libbpf* program.

这五个文件是vmlinux.h文件、libbpf中的一些头文件，以及我自己编写的一个应用程序特定的头文件。让我们看看为什么这是一个libbpf程序所需的头文件的典型模式。

Kernel header information

内核头部信息

If you're writing an eBPF program that refers to any kernel data structures or types, the easiest option is to include the *vmlinux.h* file described earlier in this chapter. Alternatively, it's possible to include individual header files from the Linux source, or to define the types by hand in your own code if you really want to go to that trouble. If you're going to use any BPF helper functions from *libbpf*, you'll need to include either *vmlinux.h* or *linux/types.h* to get the definitions for types like `u32`, `u64`, and so on, that the BPF helper source refers to.

如果您正在编写一个引用任何内核数据结构或类型的eBPF程序，最简单的方法是在本章前面描述的vmlinux.h文件中包含它。另外，您也可以从Linux源中包含单独的头文件，或者在自己的代码中手动定义类型，如果您确实愿意这样做的话。如果您要使用libbpf中的任何BPF助手函数，您需要包含vmlinux.h或linux/types.h来获取BPF助手源代码中引用的类型，如u32、u64等的定义。

The *vmlinux.h* file is derived from the kernel source headers, but it doesn't include `#define`'d values from them. For example, if your eBPF program parses Ethernet packets, you'll probably need the constant definitions that tell you what protocol the packet contains (such as `0x0800` to indicate that it's an IP packet, or `0x0806` for an ARP packet). There is a series of constant values that you'll need to duplicate in your own code, if you don't include the *if_ether.h* file that defines these values for the kernel. I didn't need any of these value definitions for *hello-buffer-config*, but you'll see another example in [Chapter 8](#) where this is relevant.

"vmlinux.h文件是从内核源代码的头文件派生而来，但它不包括其中的#define定义的值。例如，如果你的eBPF程序解析以太网数据包，你可能需要常量定义来告诉你数据包包含的协议（比如0x0800表示它是一个IP数据包，0x0806表示一个ARP数据包）。如果你不包含定义这些值的if_ether.h文件，那么你需要在你自己的代码中复制一系列常量值。对于hello-buffer-config项目，我不需要这些值的定义，但你会在第8章中看到另一个相关的例子。"

Headers from libbpf

来自libbpf的标题

To use any BPF helper functions in your eBPF code, you'll need to include the header files from *libbpf* that give you their definitions.

在您的eBPF代码中使用任何BPF助手函数，您需要包含libbpf的头文件，这些头文件提供了它们的定义。

Note

One thing that can be slightly confusing about *libbpf* is that it's not just a user space library. You'll find yourself including header files from *libbpf* in both user space and eBPF C code.

关于libbpf有一件稍微令人困惑的事情是，它不仅仅是一个用户空间库。你会发现在用户空间和eBPF C代码中都要包含来自libbpf的头文件。

At the time of this writing, it is common to see eBPF projects including *libbpf* as a submodule and building/installing from source—this is what I have done in the example repository for this book. If you include it as a submodule, you'll simply need to run `make install` from the *libbpf/src* directory. I don't think it will be long before it's more common to see *libbpf* widely available as a package on common Linux distributions, particularly since *libbpf* has now passed the milestone of a [version 1.0 release](#).

在撰写本书时，目前普遍使用的做法是将libbpf作为子模块，从源代码进行构建/安装 - 这是我在本书示例存储库中所做的。如果您将其作为子模块包含进来，只需从libbpf/src目录运行make install即可。我认为不久之后，在常见的Linux发行版上，特别是自从libbpf已经发布1.0版本的重要里程碑后，我们会更常见地看到libbpf作为一个常用软件包广泛可用。

Application-specific headers

应用特定标头

It's very common to have an application-specific header file that defines any structures that are used by both the user space and eBPF parts of your app. In my example, the *hello-buffer-config.h* header file defines the `data_t` structure that I'm using to pass event data from the eBPF program to user space. It's almost the same structure you saw in the BCC version of this code, and it looks like this:

通常情况下，拥有一个应用特定的头文件是非常常见的，该头文件定义了用户空间和eBPF应用程序共用的任何结构体。在我的例子中，*hello-buffer-config.h*头文件定义了数据结构`data_t`，我使用它来将事件数据从eBPF程序传递到用户空间。它几乎与您在这段代码的BCC版本中看到的结构体相同，如下所示：

```
struct data_t {
    int pid;
    int uid;
    char command[16];
    char message[12];
    char path[16];
};
```

The only difference from the version you saw before is that I have added a field called `path`.

你之前看到的版本唯一的区别就是我增加了一个叫做"路径"的字段。

The reason to pull this structure definition into a separate header file is that I will also refer to it from the user space code in *hello-buffer-config.c*. In the BCC version, the kernel and user space code were both defined in a single file, and BCC did some work behind the

scenes to make the structure available to the Python user space code.

将这个结构定义文件拆分到一个单独的头文件中的原因是，我将在 `hello-buffer-config.c` 的用户空间代码中引用它。在BCC版本中，内核和用户空间代码都定义在一个文件中，BCC在幕后进行了一些处理，使结构对Python用户空间代码可用。

Defining Maps

定义地图

After including the header files, the next few lines of the source code in *hello-buffer-config.bpf.c* define the structures used for maps, like this:

包含了头文件后，源代码中 `hello-buffer-config.bpf.c` 的下几行定义了用于映射的结构，就像这样：

```
struct {
    __uint(type, BPF_MAP_TYPE_PERF_EVENT_ARRAY);
    __uint(key_size, sizeof(u32));
    __uint(value_size, sizeof(u32));
} output SEC(".maps");

struct user_msg_t {
    char message[12];
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 10240);
    __type(key, u32);
    __type(value, struct user_msg_t);
} my_config SEC(".maps");
```

This requires more lines of code than I needed in the equivalent BCC example! With BCC, the map called `config` was created with the following macro:

这比等效的BCC示例中我需要的代码行数更多！使用BCC，以下宏创建了名为config的映射：

```
BPF_HASH(config, u64, struct user_msg_t);
```

This macro isn't available when you're not using BCC, so in C you have to write it out longhand. You'll see that I have used `__uint` and `__type`. These are defined in [bpf/bpf_helpers_def.h](#) along with `__array`, like this:

这个宏在不使用BCC时不可用，因此在C中需要手动编写。你会看到我使用了__uint和__type。在bpf/bpf_helpers_def.h中定义了这些，还有__array，如下所示：

```
#define __uint(name, val) int (*name)[val]  
#define __type(name, val) typeof(val) *name  
#define __array(name, val) typeof(val) *name[]
```

These macros generally seem to be used by convention in *libbpf*-based programs, and I think they make the map definitions a little easier to read.

这些宏通常按照惯例在基于libbpf的程序中使用，我认为它们使得映射的定义更容易阅读。

Note

The name “config” clashed with a definition in *vmlinux.h*, so I renamed the map “my_config” for this example.

名称“config”与vmlinux.h中的定义冲突，因此我为此示例将该映射重命名为“my_config”。

eBPF Program Sections

eBPF程序部分

Use of *libbpf* requires each eBPF program to be marked with a `SEC()` macro that defines the program type, like this:

使用libbpf需要对每个eBPF程序进行标记，使用SEC（）宏来定义程序的类型，就像这样：

```
SEC("kprobe")
```

This results in a section called `kprobe` in the compiled ELF object, so *libbpf* knows to load this as a `BPF_PROG_TYPE_KPROBE`. We’ll discuss different program types further in [Chapter 7](#).

这将导致编译后的ELF对象中出现一个名为kprobe的部分，因此libbpf会将其加载为BPF_PROG_TYPE_KPROBE。在第7章中我们将进一步讨论不同的程序类型。

Depending on the program type, you can also use the section name to specify what event the program will be attached to. The *libbpf* library will use this information to set up the attachment automatically, rather than leaving you to do it explicitly in your user space code. So, for example, to auto-attach to the kprobe for the

`execve` syscall on an ARM-based machine, you could specify the section like this:

根据程序类型，您还可以使用部分名称来指定程序将附加到的事件。`libbpf`库将使用此信息自动设置附件，而不是让用户在用户空间代码中显式执行。因此，例如，要自动附加到ARM架构机器上的`execve`系统调用的kprobe，您可以像这样指定部分：

```
SEC("kprobe/__arm64_sys_execve")
```

This requires you to know the function name for the syscall on that architecture (or figure it out, perhaps by looking at the `/proc/kallsyms` file on your target machine, which lists all the kernel symbols, including its function names). But *libbpf* can make life even easier for you with the `k(ret)syscall` section name, which tells the loader to attach to the kprobe in the architecture-specific function automatically:

这要求你了解该体系结构上系统调用的函数名（或者通过查看目标机器上的`/proc/kallsyms`文件来找到它，该文件列出了所有内核符号，包括其函数名）。但是`libbpf`可以通过`k(ret)syscall`部分名称为你自动附加到体系结构特定函数中的kprobe，从而让你的生活更加轻松。

```
SEC("ksyscall/execve")
```

Note

The valid section names and formats are listed in the [libbpf documentation](#). In the past, the requirements for section names were much looser, so you may come across eBPF programs written

before *libbpf 1.0* with section names that don't match the valid set. Don't let them confuse you!

有效的节名和格式都列在libbpf文档中。过去，节名的要求更为宽松，因此你可能会遇到在libbpf 1.0之前编写的eBPF程序，它们的节名不符合有效的集合。不要被它们搞混了！

The section definition declares where the eBPF program should be attached, and then the program itself follows. As before, the eBPF program itself is written as a C function. In the example code it's called `hello()`, and it's extremely similar to the `hello()` function you saw in [Chapter 4](#). Let's consider the differences between that previous version and the version here:

章节定义声明了eBPF程序应该附加的位置，然后是程序本身。和之前一样，eBPF程序本身是以C函数的形式编写的。在示例代码中，它被称为`hello()`，与你在第四章见过的`hello()`函数非常相似。让我们来考虑一下之前版本和这个版本之间的区别：

```
SEC("ksyscall/execve")
int BPF_KPROBE_SYSCALL(hello, const char *pathname)
    ❶
{
    struct data_t data = {};
    struct user_msg_t *p;

    data.pid = bpf_get_current_pid_tgid() >> 32;
    data.uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;

    bpf_get_current_comm(&data.command, sizeof(data.command)
);
    bpf_probe_read_user_str(&data.path, sizeof(data.path),
pathname);    ❷

    p = bpf_map_lookup_elem(&my_config, &data.uid);
```

```

    ③
    if (p != 0) {
        bpf_probe_read_kernel(&data.message, sizeof(data.
message), p->message);
    } else {
        bpf_probe_read_kernel(&data.message, sizeof(data.
message), message);
    }

    bpf_perf_event_output(ctx, &output, BPF_F_CURRENT_CPU,
    ④
                        &data, sizeof(data));

    return 0;
}

```

①

I've taken advantage of a `BPF_KPROBE_SYSCALL` macro defined in *libbpf* that makes it easy to access the arguments to a syscall by name. For `execve()`, the first argument is the pathname for the program that's going to be executed. The eBPF program name is `hello`.

②

Since the macro has made it so easy to access that pathname argument to `execve()`, I'm including it in the data sent to the perf buffer output. Notice that copying memory requires the use of a BPF helper function.

③

Here, `bpf_map_lookup_elem()` is the BPF helper function for looking up values in a map, given a key. BCC's equivalent of this would be `p = my_config.lookup(&data.uid)`. BCC rewrites this to use the underlying `bpf_map_lookup_elem()` function before it

passes the C code to the compiler. When you're using *libbpf*, there is no rewriting of the code before compilation,⁷ so you have to write directly to the helper functions.

④

Here's another similar example where I have written directly to the helper function `bpf_perf_event_output()`, where BCC gave me the convenient equivalent

```
output.perf_submit(ctx, &data, sizeof(data)).
```

The only other difference is that in the BCC version, I defined the message string as a local variable within the `hello()` function. BCC doesn't (at least at the time of this writing) support global variables. In this version I have defined it as a global variable, like this:

唯一的其他区别是，在BCC版本中，我将消息字符串定义为`hello()`函数内的局部变量。BCC（至少在写这篇文章的时候）不支持全局变量。在这个版本中，我将其定义为全局变量，如下所示：

```
char message[12] = "Hello World";
```

In *chapter4/hello-buffer-config.py* the `hello` function was defined rather differently, like this:

在第四章/ `hello-buffer-config.py`中，`hello`函数被定义得有些不同，就像这样：

```
int hello(void *ctx)
```

The `BPF_KPROBE_SYSCALL` macro is one of the convenient additions from *libbpf* that I mentioned. You're not required to use the macro, but it makes life easier. It does all the heavy lifting to provide named arguments for all the parameters passed to a syscall. In this case, it supplies a `pathname` argument that points to a string holding the path of the executable that is about to be run, which is the first argument to the `execve()` syscall.

`BPF_KPROBE_SYSCALL` 宏是我之前提到的 *libbpf* 提供的便利之一。你不必使用这个宏，但它会让生活更方便。它会承担所有繁重的工作，为传递给系统调用的所有参数提供具名参数。在这个例子中，它提供了一个 `pathname` 参数，指向一个包含即将运行的可执行文件路径的字符串，这是传递给 `execve()` 系统调用的第一个参数。

If you're paying very close attention you might notice that the `ctx` variable isn't visibly defined in my source code for *hello-buffer-config.bpf.c*, but nevertheless, I've been able to use it when submitting data to the output perf buffer, like this:

如果你非常仔细注意，你可能会注意到`ctx`变量在我的`hello-buffer-config.bpf.c`源代码中没有明确定义，但是尽管如此，我仍然能够在向输出perf缓冲区提交数据时使用它，就像这样：

```
bpf_perf_event_output(ctx, &output, BPF_F_CURRENT_CPU, &data, sizeof(data));
```

The `ctx` variable does exist, hidden within the `BPF_KPROBE_SYSCALL` macro definition inside *bpf/bpf_tracing.h*, in *libbpf*, where you'll also find some commentary about this. It can be slightly confusing to use

a variable that's not visibly defined, but it's very helpful that it can be accessed.

在libbpf的bpf/bpf_tracing.h文件中的BPF_KPROBE_SYSCALL宏定义中，ctx变量是存在的，其中还有一些关于它的注释。使用一个不可见定义的变量可能会稍微令人困惑，但是能够访问它非常有帮助。

Memory Access with CO-RE

使用CO-RE进行内存访问

eBPF programs for tracing have restricted access to memory, via a BPF helper function from the `bpf_probe_read_*`() family.⁸ (There is also a `bpf_probe_write_user()` helper function, but it's only “[meant for experiments](#)”). The problem is that, as you'll see in the next chapter, the eBPF verifier generally won't let you simply read memory through a pointer as you usually can in C (e.g., `x = p->y`).⁹

eBPF跟踪程序对内存的访问受限，通过BPF助手函数 `bpf_probe_read_*`()系列来实现。（还有一个名为 `bpf_probe_write_user()`的辅助函数，但只是“用于实验”）。问题在于，正如您在下一章中将看到的那样，eBPF验证器通常不允许您像在C语言中那样简单地通过指针读取内存（例如，`x = p->y`）。

The *libbpf* library provides CO-RE wrappers around the `bpf_probe_read_*`() helpers to take advantage of the BTF information and make memory access calls portable across different kernel versions. Here's an example of one of those wrappers, as defined in the [bpf_core_read.h header file](#):

libbpf库提供了对bpf_probe_read_*(())辅助方法的CO-RE封装，以利用BTF信息，并使内存访问调用在不同内核版本中具有可移植性。以下是其中一个封装的示例，定义在bpf_core_read.h头文件中：

```
#define bpf_core_read(dst, sz, src) \
    bpf_probe_read_kernel(dst, sz, \
                           (const void *)__builtin_preserve_access_index(src))
```

As you can see, `bpf_core_read()` calls directly to `bpf_probe_read_kernel()`, the only difference being that it wraps the `src` field with `__builtin_preserve_access_index()`. This tells Clang to emit a CO-RE relocation entry along with the eBPF instruction that accesses this address in memory.

正如您所见，`bpf_core_read()` 直接调用 `bpf_probe_read_kernel()`，唯一的区别在于它用 `__builtin_preserve_access_index()` 包装了 `src` 字段。这告诉 Clang 在访问内存地址时生成 CO-RE 重定位条目和 eBPF 指令。

Note

This `__builtin_preserve_access_index()` instruction is an extension to “regular” C code, and adding it to eBPF also required changes to the Clang compiler to support it and emit these CO-RE relocation entries. Extensions like these are examples of why some C compilers cannot (today, at least) generate eBPF bytecode. Read more about the Clang changes required for eBPF CO-RE support on the [LLVM mailing list](#).

这个__builtin_preserve_access_index()指令是对“普通”C代码的扩展，将其添加到eBPF还需要对Clang编译器进行更改，以支持并生成这些CO-RE重定位条目。这样的扩展是为什么一些C编译器（至少目前）无法生成eBPF字节码的例子。详细了解eBPF CO-RE支持所需的Clang更改，请阅读LLVM邮件列表上的相关内容。

As you'll see later in this chapter, the CO-RE relocation entry tells *libbpf* to rewrite the address, as it's loading the eBPF program into the kernel, to take account of any BTF differences. If the offset of `src` within its containing structure is different on the target kernel, the rewritten instruction will take that into account.

正如您将在本章后面看到的那样，CO-RE重定位条目告诉libbpf在将eBPF程序加载到内核时重新编写地址，以考虑任何BTF差异。如果在目标内核中，src在其包含结构中的偏移量不同，重新编写的指令将考虑这一点。

The *libbpf* library provides a `BPF_CORE_READ()` macro so that you can write several `bpf_core_read()` calls in a single line rather than needing a separate helper function call for every pointer dereference. For example, if you wanted to do something like `d = a->b->c->d`, you could write the following code:

libbpf库提供了BPF_CORE_READ()宏，使您可以在一行代码中编写多个bpf_core_read()调用，而不需要为每个指针解引用调用单独的帮助函数。例如，如果您想要执行类似于d = a->b->c->d的操作，可以编写以下代码：

```
struct b_t *b;  
struct c_t *c;
```

```
bpf_core_read(&b, 8, &a->b);  
bpf_core_read(&c, 8, &b->c);  
bpf_core_read(&d, 8, &c->d);
```

But it's much more compact to use:

但使用起来更加紧凑：

```
d = BPF_CORE_READ(a, b, c, d);
```

You can then read from point `d` using the `bpf_probe_read_kernel()` helper function.

您可以使用**bpf_probe_read_kernel()**辅助函数从点d开始读取。

There's a good description of this in Andrii's [guide](#).

安德里的指南中有一个很好的描述。

License Definition

定义许可

As you already know from [Chapter 3](#), the eBPF program has to declare its license. The example code does it like this:

正如你已经从第三章知道的那样，eBPF程序必须声明其许可证。示例代码是这样做的：

```
char LICENSE[] SEC("license") = "Dual BSD/GPL";
```

You've now seen all the code in the *hello-buffer-config.bpf.c* example. Now let's compile it into an object file.

你现在已经看到了hello-buffer-config.bpf.c示例中的所有代码。现在让我们将其编译成一个目标文件。

Compiling eBPF Programs for CO-RE

为CO-RE编译eBPF程序

In [Chapter 3](#) you saw an extract from a Makefile that compiles C to eBPF bytecode. Let's dig into the options used and see why they are necessary for CO-RE/*libbpf* programs.

在第3章中，您看到了一个从Makefile中提取的将C编译为eBPF字节码的示例。让我们深入研究所使用的选项，并看看它们为CO-RE/*libbpf*程序所必需的原因。

Debug Information

调试信息

You have to pass the `-g` flag to Clang so that it includes debug information, which is necessary for BTF. However, the `-g` flag also adds DWARF debugging information to the output object file, but that's not needed by eBPF programs, so you can reduce the size of the object by running the following command to strip it out:

你必须给Clang传递-g标志，以便包含调试信息，这对BTF是必需的。然而，-g标志还会将DWARF调试信息添加到输出目标文件中，而eBPF程序不需要该信息，因此你可以通过运行以下命令剥离它来缩小目标文件的大小：

```
llvm-strip -g <object file>
```

Optimization

优化

The `-O2` optimization flag (level 2 or higher) is required for Clang to produce BPF bytecode that will pass the verifier. One example of this being necessary is that, by default, Clang will output

`callx <register>` to call helper functions, but eBPF doesn't support calling addresses from registers.

使用-O2优化标志（2级或更高级别）是要求Clang生成的BPF字节码能够通过验证器的必要条件。其中一个必要的示例是，Clang默认会输出`callx <register>`来调用辅助函数，但eBPF不支持从寄存器调用地址。

Target Architecture

目标架构

If you're using certain macros defined by *libbpf*, you'll need to specify the target architecture at compile time. The *libbpf* header file *bpf/bpf_tracing.h* defines several macros that are platform specific, such as `BPF_KPROBE` and `BPF_KPROBE_SYSCALL` that I've used in this example. The `BPF_KPROBE` macro can be used for eBPF programs that are being attached to kprobes, and `BPF_KPROBE_SYSCALL` is a variant specifically for syscall kprobes.

如果您正在使用libbpf定义的某些宏，您需要在编译时指定目标架构。libbpf头文件bpf/bpf_tracing.h定义了一些平台特定的宏，例如我在这

个示例中使用的BPF_KPROBE和BPF_KPROBE_SYSCALL。
BPF_KPROBE宏可用于将eBPF程序附加到kprobes，而
BPF_KPROBE_SYSCALL是专门针对系统调用kprobes的变体。

The argument to a kprobe is a `pt_regs` structure that holds a copy of the contents of the CPU registers. Since registers are architecture specific, the `pt_regs` structure definition depends on the architecture you're running on. This means that if you want to use these macros, you'll need to also tell the compiler what the target architecture is. You can do this by setting `-D __TARGET_ARCH_($ARCH)` where `$ARCH` is an architecture name like `arm64`, `amd64`, and so on.

kprobe的参数是一个pt_regs结构，它保存了CPU寄存器内容的副本。由于寄存器是与体系结构相关的，pt_regs结构的定义取决于您运行的体系结构。这意味着如果您想使用这些宏，您还需要告诉编译器目标体系结构是什么。您可以通过设置-D __TARGET_ARCH_(\$ARCH)来实现这一点，其中\$ ARCH是像arm64、amd64等的体系结构名称。

Also note that if you didn't use the macro, you'd need architecture-specific code to access the register information anyway for a kprobe.

请注意，如果你没有使用宏，你仍然需要使用特定于架构的代码来访问寄存器信息，以便进行 kprobe。

Perhaps “compile once *per architecture*, run everywhere” would have been a bit of a mouthful!

或许，“为每个体系结构编译一次，到处运行”可能有点啰嗦！

Makefile

The following is an example Makefile instruction for compiling CO-RE objects (taken from the Makefile in the *chapter5* directory of the GitHub repo for this book):

下面是一个编译CO-RE对象的示例Makefile指令（摘自本书的GitHub代码库中的chapter5目录下的Makefile）：

```
hello-buffer-config.bpf.o: %.o: %.c
    clang \
        -target bpf \
        -D __TARGET_ARCH_${ARCH} \
        -I/usr/include/$(shell uname -m)-linux-gnu \
        -Wall \
        -O2 -g \
        -c $< -o $@
    llvm-strip -g $@
```

If you're using the example code, you should be able to build the eBPF object file *hello-buffer-config.bpf.o* (and its companion user space executable that I'll describe shortly) by running `make` in the *chapter5* directory. Let's inspect that object file to see that it includes BTF information.

如果您正在使用示例代码，您应该能够在chapter5目录中运行make命令来构建eBPF对象文件hello-buffer-config.bpf.o（以及我即将描述的配套用户空间可执行文件）。让我们检查该对象文件，以确认它包含了BTF信息。

BTF Information in the Object File

目标文件中的BTF信息

The [kernel documentation for BTF](#) describes how BTF data is encoded in an ELF object file in two sections: *.BTF*, which contains the data and string information, and *.BTF.ext*, which covers function and line information. You can use `readelf` to see that these sections have been added to the object file, like this:

BTF内核文档描述了BTF数据在ELF目标文件中的两个部分的编码方式：*.BTF*包含数据和字符串信息，*.BTF.ext*包含函数和行信息。您可以使用`readelf`命令查看这些部分是否已添加到目标文件中，如下所示：

```
$ readelf -S hello-buffer-config.bpf.o | grep BTF
 [10] .BTF                                PROGBITS             0000000000000000
000002c0
 [11] .rel.BTF                            REL                  0000000000000000
00000e50
 [12] .BTF.ext                          PROGBITS             0000000000000000
00000b18
 [13] .rel.BTF.ext                      REL                  0000000000000000
00000ea0
```

The `bpftool` utility lets us examine the BTF data from an object file, like this:

`bpftool`实用工具允许我们从一个对象文件中检查BTF数据，就像这样：

```
bpftool btf dump file hello-buffer-config.bpf.o
```

The output looks just like the output you get from dumping BTF info from loaded programs and maps, as you saw earlier in this chapter.

输出看起来就像从加载的程序和映射中转储BTF信息时得到的输出，就像你在本章之前看到的那样。

Let's see how this BTF information can be used to allow the program to run on another machine with a different kernel version and different data structures.

让我们看看如何利用这个BTF信息使程序能够在具有不同内核版本和不同数据结构的另一台机器上运行。

BPF Relocations

BPF搬迁

The *libbpf* library adapts eBPF programs to work with the data structure layout on the target kernel where they run, even if this layout is different from the kernel where the code was compiled. To do this, *libbpf* needs the BPF CO-RE relocation information generated by Clang as part of the compilation process.

libbpf库将eBPF程序适应运行目标内核上的数据结构布局，即使这个布局与编译代码的内核不同。为了实现这一点，libbpf需要Clang在编译过程中生成的BPF CO-RE重定位信息。

You can learn more about how the relocations work from the definition of `struct bpf_core_relo` in the [linux/bpf.h](#) header file:

你可以从linux/bpf.h头文件中的struct bpf_core_relo的定义了解有关迁移工作原理的更多信息。

```
struct bpf_core_relo {
    __u32 insn_off;
    __u32 type_id;
    __u32 access_str_off;
    enum bpf_core_relo_kind kind;
};
```

The CO-RE relocation data for an eBPF program consists of one of these structures for each instruction that needs relocation. Suppose the instruction is setting a register to the value of a field within a structure. The `bpf_core_relo` structure for that instruction (identified by the `insn_off` field) encodes the BTF type of that structure (the `type_id` field) and also indicates how the field is accessed relative to that structure (`access_str_off`).

CO-RE迁移数据用于eBPF程序，每个需要迁移的指令都包含以下结构之一。假设指令是将寄存器设置为结构体中的字段值。该指令的**bpf_core_relo**结构（由**insn_off**字段标识）编码了该结构体的BTF类型（**type_id**字段）并指示了相对于该结构体访问该字段的方式（**access_str_off**字段）。

As you’ve just seen, the relocation data for the kernel data structures is generated automatically by Clang and encoded in the ELF object file. It’s the following line, which you’ll find near the start of the *vmlinux.h* file, that causes Clang to do this:

正如您刚才所见，内核数据结构的重定位数据是由Clang自动生成并编码在ELF目标文件中。在**vmlinux.h**文件的开头附近，有以下一行代码会导致Clang执行此操作：

```
#pragma clang attribute push (__attribute__((preserve_access
```

The `preserve_access_index` attribute tells Clang to generate BPF CO-RE relocations for a type definition. The `clang attribute push` part says that this attribute should be applied to all definitions until a `clang attribute pop`, which appears at the end of the file. That means Clang generates the relocation information for all the types defined in *vmlinux.h*.

`preserve_access_index`属性告诉Clang为类型定义生成BPF CO-RE重定位。`clang`属性`push`部分表示该属性应该应用于所有定义，直到出现位于文件末尾的`clang`属性`pop`。这意味着Clang为在`vmlinux.h`中定义的所有类型生成重定位信息。

You can see the relocations taking place when you load a BPF program, by using `bpftool` and turning on the debug information with the `-d` flag, like this:

在加载BPF程序时，您可以使用`bpftool`，并通过使用`-d`标志打开调试信息，看到正在发生的重新定位，就像这样：

```
bpftool -d prog load hello.bpf.o /sys/fs/bpf/hello
```

This generates a lot of output, but the parts relating to relocation look like this:

这会生成很多输出内容，但与重新定位有关的部分看起来像这样：

```
libbpf: CO-RE relocating [24] struct user_pt_regs: found
target candidate [205]
struct user_pt_regs in [vmlinux]
libbpf: prog 'hello': relo #0: <byte_off> [24] struct
user_pt_regs.regs[0]
(0:0:0 @ offset 0)
libbpf: prog 'hello': relo #0: matching candidate #0
```

```
<byte_off> [205] struct
user_pt_regs regs[0] (0:0:0 @ offset 0)
libbpf: prog 'hello': relo #0: patched insn #1
(LDX/ST/STX) off 0 -> 0
```

In this example you can see that type ID 24 from the `hello` program's BTF information refers to the structure called `user_pt_regs`. The `libbpf` library has matched this against a kernel structure, also called `user_pt_regs`, that has type ID 205 in the `vmlinux` BTF data set. In practice, because I compiled and loaded the program on the same machine, the type definitions are identical, so in this example the offset of 0 from the start of the structure remains unchanged, and the “patch” to instruction #1 leaves it unchanged.

在这个例子中，你可以看到hello程序的BTF信息中类型ID为24的结构体被称为user_pt_regs。libbpf库已经将它与内核结构体匹配，该内核结构体也被称为user_pt_regs，在vmlinux的BTF数据集中的类型ID为205。实际上，因为我在同一台机器上编译和加载了这个程序，所以类型定义是相同的，在这个例子中，结构体从开始位置的偏移量仍然保持不变，对指令1的“修补”也不会改变它。

In many applications you won't want to ask users to run `bpftool` to load an eBPF program. Instead, you'll want to build this functionality into a dedicated user space program that you supply as an executable. Let's consider how to write this user space code.

在许多应用中，您不会希望要求用户运行bpftool来加载eBPF程序。相反，您将希望将此功能构建到一个专用的用户空间程序中，该程序由您提供为可执行文件。让我们考虑如何编写这个用户空间代码。

CO-RE User Space Code

使用者空间代码

There are a few different frameworks in different programming languages that support CO-RE by implementing the relocations as they load eBPF programs into the kernel. In this chapter I'll show C code that uses *libbpf*; other options include the Go packages *cilium/ebpf* and *libbpfgo*, and Aya for Rust. I'll discuss those options further in [Chapter 10](#).

有一些不同的编程语言框架支持CO-RE，它们通过在将eBPF程序加载到内核时实现重定位。在本章中，我将展示使用libbpf的C代码；其他选项包括Go的cilium/ebpf和libbpfgo包，以及Rust的Aya。我将在第10章进一步讨论这些选项。

The Libbpf Library for User Space

用户空间的Libbpf库

The *libbpf* library is a user space library you can use directly if you're writing the user space part of your application in C. If you want to, you can use this library without using CO-RE. There's an example of this in [Andrii Nakryiko's excellent blog post on *libbpf-bootstrap*](#).

libbpf库是一个用户空间库，如果您使用C语言编写应用程序的用户空间部分，可以直接使用它。如果您愿意，您可以在不使用CO-RE的情况下使用此库。Andrii Nakryiko在他的博客文章“libbpf-bootstrap”中有一个例子。

This library provides functions that wrap the `bpf()` and related syscalls that you met in [Chapter 4](#) to perform operations like loading programs into the kernel and attaching them to events, or accessing map information from user space. The conventional and easiest way to use these abstractions is through auto-generated BPF skeleton code.

这个库提供了封装了在第4章中遇到的和相关系统调用的函数，可以用于将程序加载到内核并将其附加到事件，或者从用户空间访问映射信息等操作。使用这些抽象的传统和最简单的方法是通过自动生成的BPF框架代码。

BPF Skeletons

骨架BPF

You can use `bpftool` to auto-generate this skeleton code from existing eBPF objects in ELF file format, like this:

你可以使用**bpftool**来从现有的以ELF文件格式存在的eBPF对象中自动生成此框架代码，就像这样：

```
bpftool gen skeleton hello-buffer-config.bpf.o > hello-  
buffer-config.skel.h
```

Look into this skeleton header and you'll see that it contains structure definitions for the eBPF programs and maps, as well as several functions that all start with the name

`hello_buffer_config_bpf__` (based on the name of the object file).

These functions manage the lifecycle of the eBPF programs and maps. You don't have to use the skeleton code—you can make calls

to *libbpf* directly if you prefer—but the auto-generated code will typically save you some typing.

看看这个骨架头文件，你会看到它包含了eBPF程序和映射的结构定义，还有几个函数，它们的名字都以`hello_buffer_config_bpf__`开头（根据对象文件的名字而定）。这些函数管理着eBPF程序和映射的生命周期。你不一定要使用骨架代码——如果你愿意，你可以直接调用`libbpf`——但是自动生成的代码通常会节省你一些打字。

Toward the end of the generated skeleton file you'll see a function called `hello_buffer_config_bpf__elf_bytes` that returns the byte contents of the ELF object file *hello-buffer-config.bpf.o*. Once the skeleton has been generated, we don't really need that object file anymore. You can test this by running `make` to generate the `hello-buffer-config` executable and then deleting the `.o` file; the executable has the eBPF bytecode contained within it.

在生成的骨架文件的末尾，你会看到一个名为`hello_buffer_config_bpf__elf_bytes`的函数，它返回ELF目标文件`hello-buffer-config.bpf.o`的字节内容。一旦骨架文件生成完毕，我们就不再需要这个目标文件了。你可以通过运行`make`命令来生成`hello-buffer-config`可执行文件，然后删除`.o`文件来测试这一点；可执行文件中包含了eBPF字节码。

Note

If you prefer, you can use the *libbpf* function `bpf_object__open_file` to load the eBPF programs and maps from an ELF file rather than using the bytes from a skeleton file.

如果你愿意，你可以使用libbpf函数bpf_object__open_file来从ELF文件中加载eBPF程序和map，而不是使用骨架文件中的字节。

Here's the outline of the user space code that manages the lifecycle of the eBPF program and maps for this example, using the generated skeleton code. I have omitted some of the details and error handling for clarity, but you'll find the full source code in *chapter5/hello-buffer-config.c*.

以下是管理eBPF程序和映射的用户空间代码的概述，使用生成的骨架代码作为示例。为了清晰起见，我省略了一些细节和错误处理，但你可以在第5章的hello-buffer-config.c中找到完整的源代码。

```
... [other #includes]
#include "hello-buffer-config.h"
#include "hello-buffer-config.skel.h"
... [some callback functions]

int main()
{
    struct hello_buffer_config_bpf *skel;
    struct perf_buffer *pb = NULL;
    int err;

    libbpf_set_print(libbpf_print_fn);

    skel = hello_buffer_config_bpf__open_and_load();
    ...
    err = hello_buffer_config_bpf__attach(skel);
    ...
    pb = perf_buffer__new(bpf_map__fd(skel-
>maps.output), 8, handle_event,
```

los

```

, NULL, NULL);

...
    while (true) {
        err = perf_buffer__poll(pb, 100);
    ...}

    perf_buffer__free(pb);

    hello_buffer_config_bpf__destroy(skel);
    return -err;
}

```

❶

This file includes the auto-generated skeleton header, as well as the header file I wrote manually for data structures shared between the user space and kernel code.

❷

This code sets a callback function that will print any log messages generated by *libbpf*.

❸

Here a `skel` structure is created that represents all the maps and programs defined in the ELF bytes and loads them into the kernel.

❹

Programs are auto-attached to the appropriate events.

❺

This function creates a structure for handling the perf buffer output.

⑥

Here that perf buffer is continuously polled.

⑦

This is the clean-up code.

Let's dive into some of those steps in more detail.

让我们详细讨论其中的一些步骤。

Loading programs and maps into the kernel

将程序和地图加载到内核中

The first call to an auto-generated function is this one:

这是对一个自动生成的函数的第一个调用。

```
skel = hello_buffer_config_bpf__open_and_load();
```

As its name suggests, this function covers two phases: opening and loading. The “open” phase involves reading the ELF data and converting its sections into structures that represent eBPF programs and maps. The “load” phase loads those maps and programs into the kernel, performing any CO-RE fixups as necessary.

正如其名，此函数涉及两个阶段：打开和加载。“打开”阶段涉及读取 ELF 数据并将其转换为表示 eBPF 程序和映射的结构。“加载”阶段将这些映射和程序加载到内核中，并根据需要执行任何 CO-RE 修复。

These two phases can easily be handled separately, as the skeleton code provides separate `name__open()` and `name__load()` functions. This gives you the option to manipulate the eBPF information before loading it. This is commonly done to configure a program before loading it. For example, I could initialize a counter global variable `c` to some value, like this:

这两个阶段可以很容易分开处理，因为骨架代码提供了单独的 `name__open()` 和 `name__load()` 函数。这使您有机会在加载之前操作 eBPF 信息。这通常用于在加载程序之前进行配置。例如，我可以将计数器全局变量 `c` 初始化为某个值，像这样：

```
skel = hello_buffer_config_bpf__open();
if (!skel) {
    // Error ...
}
skel->data->c = 10;
err = hello_buffer_config_bpf__load(skel);
```

The data type returned by `hello_buffer_config_bpf__open()`, and also by `hello_buffer_config_bpf__load()`, is a structure called `hello_buffer_config_bpf` defined in the skeleton header to include information about all the maps, programs, and data defined in the object file.

`hello_buffer_config_bpf__open()` 和 `hello_buffer_config_bpf__load()` 返回的数据类型是一个名为 `hello_buffer_config_bpf` 的结构体，该结构体在骨架头文件中定义，包含了关于所有地图、程序和数据的信息。

Note

The skeleton object (`hello_buffer_config_bpf` in this example) is just a user space representation of information from the ELF bytes. Once it has been loaded into the kernel, if you change a value in the object, it won't have any effect on the kernel-side data. So, for example, changing `skel->data->c` after loading will not have any effect.

骨架对象（在此示例中为`hello_buffer_config_bpf`）只是 ELF 字节信息的用户空间表示。一旦加载到内核中，如果在对象中更改一个值，它不会影响内核端的数据。例如，在加载后更改 `skel->data->c` 不会产生任何效果。

Accessing existing maps

访问现有的地图

By default, *libbpf* will also create any maps that are defined in the ELF bytes, but sometimes you might want to write an eBPF program that reuses an existing map. You already saw an example of this in the previous chapter, where you saw `bpftool` iterating through all the maps, looking for the one that matched a specified name.

Another common reason to use a map is to share information between two different eBPF programs, so only one program should create the map. The `bpf_map__set_autocreate()` function allows you to override *libbpf*'s auto-creation.

默认情况下，*libbpf*还会创建在ELF字节中定义的任何映射，但有时您可能希望编写一个重用现有映射的eBPF程序。在上一章中，您已经看到了这种情况的一个示例，其中`bpftool`遍历所有映射，寻找与指定名

称匹配的映射。另一个常用的使用映射的原因是在两个不同的eBPF程序之间共享信息，因此只有一个程序应该创建映射。

`bpf_map__set_autocreate()`函数允许您覆盖libbpf的自动创建功能。

So how do you access an existing map? Maps can be pinned, and if you know the pinned path, you can get a file descriptor to an existing map with `bpf_obj_get()`. Here's a very simple example (available in the GitHub repository as *chapter5/find-map.c*):

那么如何访问现有的地图呢？地图可以被固定，如果你知道固定的路径，就可以使用**`bpf_obj_get()`**获取一个对现有地图的文件描述符。这里有一个非常简单的例子（可以在GitHub仓库的chapter5/find-map.c中找到）。

```
struct bpf_map_info info = {};  
unsigned int len = sizeof(info);  
  
int findme = bpf_obj_get("/sys/fs/bpf/findme");  
if (findme <= 0) {  
    printf("No FD\n");  
} else {  
    bpf_obj_get_info_by_fd(findme, &info, &len);  
    printf("Name: %s\n", info.name);  
}
```

To try this out you can create a map using `bpftool`, like this:

要尝试这个，你可以使用bpftool创建一个地图，像这样：

```
$ bpftool map create /sys/fs/bpf/findme type array key 4  
value 32 entries 4  
name findme
```

Running the find-map executable will print out:

运行find-map可执行文件会打印出：

```
Name: findme
```

Let's get back to the *hello-buffer-config* example and the skeleton code.

让我们回到`hello-buffer-config`示例和骨架代码。

Attaching to events

连接到事件 (Liánjiē dào shìjiàn)

The next skeleton function in the example attaches the program to the `execve` syscall function:

下面的示例函数将程序连接到execve系统调用函数。

```
err = hello_buffer_config_bpf__attach(skel);
```

The *libbpf* library automatically takes the attachment point from the `SEC()` definition for this program. If you didn't define the attachment point fully, there are a whole series of *libbpf* functions, such as `bpf_program__attach_kprobe`, `bpf_program__attach_xdp`, and so on, for attaching different program types.

libbpf库会自动从SEC()定义中获取此程序的附加点。如果您没有完全定义附加点，可以使用一系列的libbpf函数，例如 `bpf_program__attach_kprobe`、`bpf_program__attach_xdp`等，用于附加不同类型的程序。

Managing an event buffer

管理事件缓冲区

Setting up the perf buffer uses a function defined in *libbpf* itself, rather than in the skeleton:

设置perf缓冲区使用的函数是在libbpf本身中定义的，而不是在框架中定义的。

```
pb = perf_buffer__new(bpf_map__fd(skel->maps.output), 8,
    handle_event,
    , NULL, NULL);
```

You can see the `perf_buffer__new()` function takes the file descriptor for the “output” map as the first argument. The `handle_event` argument is a callback function that gets called when new data arrives in the perf buffer, and `lost_event` gets called if there isn’t enough room in the perf buffer for the kernel to write a data entry. In my example these functions just write messages to the screen.

您可以看到perf_buffer__new () 函数将“output”映射的文件描述符作为第一个参数。 handle_event参数是一个回调函数，在perf缓冲区中有新数据到达时调用，并且在perf缓冲区没有足够的空间让内核写入数据条目时调用lost_event。在我的示例中，这些函数只是将消息写到屏幕上。

Finally, the program has to poll the perf buffer repeatedly:

最后，程序必须反复轮询性能缓冲区：

```
while (true) {
    err = perf_buffer__poll(pb, 100);
```



```
    ...  
}
```

The 100 is a timeout in milliseconds. The callback functions previously set up will get called as appropriate when data arrives or when the buffer is full.

100是以毫秒为单位的超时时间。数据到达或缓冲区已满时，预先设置的回调函数将会根据需求被调用。

Finally, to clean up I free the perf buffer and destroy the eBPF programs and maps in the kernel, like this:

最后，为了清理，我释放了性能缓冲区并销毁了内核中的eBPF程序和映射，就像这样：

```
perf_buffer__free(pb);  
hello_buffer_config_bpf__destroy(skel);
```

There are a whole set of `perf_buffer_*`- and `ring_buffer_*`-related functions in *libbpf* to help you manage event buffers.

在libbpf中有一整套与perf_buffer_*和ring_buffer_*相关的函数，可帮助您管理事件缓冲区。

If you make and run this example `hello-buffer-config` program, you'll see the following output (that's very similar to what you saw in [Chapter 4](#)):

如果您制作并运行这个示例hello-buffer-config程序，您将看到以下输出（与第4章中看到的非常相似）：

23664	501	bash	Hello World
23665	501	bash	Hello World
23667	0	cron	Hello World
23668	0	sh	Hello World

Libbpf Code Examples

Libbpf 代码示例

There are lots of great examples of *libbpf*-based eBPF programs available that you can use as inspiration and guidance for writing your own:

在使用和撰写自己的libbpf-based eBPF程序时，有许多优秀的示例可以作为灵感和指导。

- The [libbpf-bootstrap](#) project is intended to help you get off the ground with a set of example programs.

‘libbpf-bootstrap项目旨在通过一套示例程序帮助您起步。’

- The BCC project has many of the original BCC-based tools migrated to a *libbpf* version. You'll find them in the [libbpf-tools directory](#).

BCC项目中的许多基于BCC的工具已迁移到libbpf版本。您可以在libbpf-tools目录中找到它们。

Summary

摘要

CO-RE enables eBPF programs that can run on kernel versions different from the versions on which they were built. This massively improves the portability of eBPF and makes life much easier for tool developers who want to deliver production-ready tooling to their users and customers.

CO-RE使得eBPF程序能够运行在与其构建的版本不同的内核版本上。这极大地提高了eBPF的可移植性，使工具开发者能更轻松地将生产就绪的工具交付给用户和客户。

In this chapter you saw how CO-RE achieves this by encoding type information into the compiled object file and using relocations to rewrite instructions as they are loaded into the kernel. You also had an introduction to writing code in C that uses *libbpf*: both the eBPF programs that run in the kernel and the user space programs that manage the lifecycle of those programs, based on auto-generated BPF skeleton code. In the next chapter you'll learn how the kernel verifies that eBPF programs are safe to run.

在这一章中，您看到了CO-RE如何通过将类型信息编码到编译的目标文件中，并使用重定位来在加载到内核时重写指令，实现了这一目标。您还介绍了使用libbpf编写代码的方法：包括在内核中运行的eBPF程序和管理这些程序的用户空间程序，这些程序是基于自动生成的BPF骨架代码的。在下一章中，您将了解内核如何验证eBPF程序的安全性。

Exercises

练习

Here are a few things you can do to further explore BTF, CO-RE, and *libbpf*.

以下是一些进一步探索BTF、CO-RE和libbpf的方法：

1. Experiment with `bpftool btf dump map` and `bpftool btf dump prog` to see the BTF information associated with maps and programs, respectively. Remember that you can specify individual maps and programs in more than one way.

请尝试使用`bpftool btf dump map`和`bpftool btf dump prog`来查看与地图和程序相关的BTF信息。请记住，您可以以多种方式指定单个地图和程序。

2. Compare the output from `bpftool btf dump file` and `bpftool btf dump prog` for the same program in its ELF object file form and after it has been loaded into the kernel. They should be identical.

请帮我翻译成简体中文：“比较从`bpftool btf dump`文件和`bpftool btf dump prog`命令得到的输出结果，对于相同的程序，一份是其ELF目标文件形式，另一份是加载到内核后的形式。它们应该是相同的。”

3. Examine the debug output from `bpftool -d prog load hello-buffer-config.bpf.o /sys/fs/bpf/hello`. You'll see each section being loaded, checks on the license, and relocations taking place, as well as output describing each BPF program instruction.

请帮我翻译：“检查来自**bpftool -d prog load hello-buffer-config.bpf.o /sys/fs/bpf/hello**的调试输出。您将看到加载每个部分，对许可证的检查以及发生的重定位，以及描述每个BPF程序指令的输出。”

4. Try building a BPF program against a different *vmlinux* header file from BTFHub, and look in the debug output from `bpftool` for relocations that change offsets.

请尝试使用BTFHub的不同vmlinux标题文件构建BPF程序，并在bpftool的调试输出中查找更改偏移量的迁移项。

5. Modify the *hello-buffer-config.c* program so that you can configure different messages for different user IDs using the map (similar to the *hello-buffer-config.py* example in [Chapter 4](#)).

修改`hello-buffer-config.c`程序，使您可以使用映射来为不同的用户ID配置不同的消息（类似于第4章中的hello-buffer-config.py示例）。

6. Try changing the section name in the `SEC()`, perhaps to your own name. When you come to load the program into the kernel you should see an error because *libbpf* doesn't recognize the section name. This illustrates how *libbpf* uses the section name to work out what kind of BPF program this is. You could try writing your own attachment code to explicitly attach to an event of your choice rather than relying on *libbpf*'s auto-attachment.

尝试在SEC()中更改节名称，可以改成您自己的名称。当您加载程序到内核中时，应该会出现错误，因为libbpf无法识别节名称。这

说明了libbpf如何使用节名称来确定这是什么类型的BPF程序。您可以尝试编写自己的附加代码，以明确附加到您选择的事件，而不是依赖于libbpf的自动附加功能。

¹ Strictly speaking, the data structure definitions come from kernel header files, and you could choose to compile based on a set of these header files that is different from what was used to build the kernel running on that machine. To work correctly (without the CO-RE mechanisms described in this chapter), the kernel headers have to be compatible with the kernel on the target machine where the eBPF program will run.

严格来说，数据结构定义来自内核头文件，您可以选择根据一组与在该设备上运行的内核构建所使用的不同的头文件进行编译。要正确工作（在本章中描述的CO-RE机制之外），内核头文件必须与目标机上将运行 eBPF程序的内核兼容。

² Part of this section is adapted from “What Is eBPF?” by Liz Rice. Copyright © 2022 O’Reilly Media. Used with permission.

该部分部分内容摘自Liz Rice的“What Is eBPF?”。版权所有 ©2022 O’Reilly Media。已获许可使用。

³ A small and unscientific survey suggests that most people pronounce this the same as the word *core* rather than in two syllables.

一个小而不科学的调查表明，大多数人发音与单词"core"相同，而不是两个音节。

⁴ See the kernel documentation at <https://docs.kernel.org/bpf/btf.xhtml#type-encoding>.

请帮我翻译成简体中文：“请查阅

<https://docs.kernel.org/bpf/btf.xhtml#type-encoding> 上的内核文档。”

⁵ The kernel needs to have been built with the `CONFIG_DEBUG_INFO_BTF` option enabled.

内核必须启用了 `CONFIG_DEBUG_INFO_BTF` 选项来构建。

⁶ Which is the oldest Linux kernel version that can support BTF? See <https://oreil.ly/HML9m>.

最旧的 Linux 内核版本能支持 BTF 是哪个？请查阅 <https://oreil.ly/HML9m>。

⁷ Well, normal C preprocessing applies so that you can do things like `#define`. But there's no *special* rewriting like there is when you use BCC.

好的，正常的C预处理适用，这意味着你可以使用诸如`#define`的功能。但是没有像使用BCC时那样的特殊重写。

⁸ eBPF programs handling network packets don't get to use this helper function and can only access the network packet memory.

eBPF程序处理网络数据包时不能使用此辅助函数，只能访问网络数据包内存。

⁹ It is permitted in certain BTF-enabled program types such as `tp_btf`, `fentry`, and `fexit`.

在某些启用BTF的程序类型中是允许的，例如tp_btf，fentry和fexit。

Chapter 6. The eBPF Verifier

第六章. eBPF验证器

I've mentioned the verification step a few times, so you already know that when you load an eBPF program into the kernel, this verification process ensures that the program is safe. In this chapter we'll dive into how the verifier works to achieve this goal.

我已经多次提到了验证步骤，所以你已经知道当你将一个eBPF程序加载到内核时，这个验证过程确保程序是安全的。在本章中，我们将深入了解验证器是如何工作以实现这个目标的。

Verification involves checking every possible execution path through the program and ensuring that every instruction is safe. The verifier also makes some updates to the bytecode to ready it for execution. In this chapter I'll show some examples of verification failures, by starting from an example that works and making modifications that render that code invalid to the verifier.

验证涉及对程序的每条可能执行路径进行检查，确保每条指令都是安全的。验证器还会对字节码进行一些更新，以准备执行。在本章中，我将展示一些验证失败的示例，方法是从一个正常工作的示例开始，然后进行修改，使得代码对验证器来说变得无效。

Note

The example code for this chapter is in the *chapter6* directory of the repository at github.com/lizrice/learning-ebpf.

本章的示例代码位于github.com/lizrice/learning-ebpf仓库的chapter6目录中。

This chapter doesn't attempt to cover every possible check the verifier makes. It's intended to be an overview, with illustrative examples that will help you deal with verification errors that you might run into when writing your own eBPF code.

本章不试图涵盖验证器进行的每一种可能检查。它旨在概述，并提供说明性示例，以帮助您在编写自己的eBPF代码时可能遇到的验证错误。

One thing to bear in mind is that the verifier works on eBPF bytecode, not directly on the source. That bytecode depends on the output from the compiler. Because of things like compiler optimization, a change in the source code might not always result in exactly what you expect in the bytecode, so correspondingly it might not give you the result you expect in the verifier's verdict. For example, the verifier will reject unreachable instructions, but the compiler might optimize them away before the verifier sees them.

需要记住的一件事是，验证器是基于eBPF字节码工作的，而不是直接在源代码上工作。该字节码取决于编译器的输出。由于诸如编译器优化之类的原因，源代码的更改可能不会始终产生预期中的字节码，因此相应地，它可能不会给您在验证器的判断中产生预期的结果。例

如，验证器将拒绝无法访问的指令，但编译器在验证器看到它们之前可能会对其进行优化。

The Verification Process

验证过程

The verifier analyzes the program to assess all possible execution paths. It steps through the instructions in order, evaluating them rather than actually executing them. As it goes along it keeps track of the state of each register in a structure called `bpf_reg_state`. (The registers I'm referring to here are the registers from the eBPF virtual machine that you met in [Chapter 3](#).) This structure includes a field called `bpf_reg_type`, which describes what type of value is held in that register. There are several possible types, including these:

验证器会分析程序以评估所有可能的执行路径。它按顺序逐步执行指令，而不是实际执行它们。在执行过程中，它会跟踪每个寄存器的状态，并存储在一个名为bpf_reg_state的结构中。（这里提到的寄存器是在第三章中介绍的eBPF虚拟机的寄存器。）该结构包含一个名为bpf_reg_type的字段，用于描述该寄存器中保存的值的类型。有几种可能的类型，包括：

- `NOT_INIT`, indicating that the register has not yet been set to a value.

`NOT_INIT`，表示寄存器尚未设置为某个值。

- `SCALAR_VALUE`, indicating that the register has been set to a value that doesn't represent a pointer.

指示该寄存器已设置为一个不代表指针的值，标识为 SCALAR_VALUE。

- Several `PTR_TO_*` types, indicating that the register holds a pointer to something. That something could be, for example:

几个PTR_TO_*类型，表示寄存器保存的是指向某个东西的指针。这个东西可以是，例如：

- `PTR_TO_CTX`: The register holds a pointer to the context passed as the argument to a BPF program.

`PTR_TO_CTX`：该寄存器保存一个指针，指向作为参数传递给BPF程序的上下文。

- `PTR_TO_PACKET`: The register points to a network packet (held in the kernel as `skb->data`).

指针指向网络数据包（在内核中保存为`skb->data`）。

- `PTR_TO_MAP_KEY` or `PTR_TO_MAP_VALUE`: I'm sure you can guess what these mean.

`PTR_TO_MAP_KEY`或`PTR_TO_MAP_VALUE`：我相信你能猜到这些的意思。

There are several other `PTR_TO_*` types, and you can find the full set enumerated in the [linux/bpf.h header file](#).

还有几种其他的PTR_TO_*类型，你可以在linux/bpf.h头文件中找到完整的枚举集合。

The `bpf_reg_state` structure also keeps track of the range of possible values the register might hold. This information is used by the verifier to determine when invalid actions are being attempted.

`bpf_reg_state`结构还记录了寄存器可能保持的可能值范围。这些信息被验证器用来判断是否尝试了无效操作。

Each time the verifier comes to a branch, where a decision has to be made on whether to carry on in sequence or jump to a different instruction, the verifier pushes a copy of the current state of all the registers onto a stack and explores one of the possible paths. It continues evaluating the instructions until it reaches the return at the end of the program (or reaches the limit on the number of instructions it will process, which is currently one million instructions¹), at which point it pops a branch off the stack to evaluate next. If it finds an instruction that could result in an invalid operation, it fails verification.

每次验证器来到一个分支，需要根据选择来决定是按顺序继续执行还是跳转到不同的指令时，验证器将当前所有寄存器状态的副本压入栈中，然后探索其中一条可能的路径。它会继续执行指令，直到到达程序末尾的返回语句（或达到当前处理指令数的上限，目前为一百万条指令），此时它会从栈中弹出一个分支进行下一步的评估。如果发现一个可能导致无效操作的指令，验证失败。

Verifying every single possibility could get computationally expensive, so in practice there are optimizations called *state pruning* that avoid reevaluating paths through the program that are essentially equivalent. As it works through the program, the verifier

records the state of all the registers at certain instructions within the program. If it later arrives at the same instruction with registers in a matching state, there is no need to continue to verify the rest of that path, as it's already known to be valid.

验证每种可能性都可能导致计算成本很高，因此在实践中有一些称为状态剪枝的优化方法，可以避免对程序中实质上等效的路径进行重新评估。当验证器遍历程序时，会记录在程序中某些指令的位置上所有寄存器的状态。如果它后来到达具有匹配状态寄存器的相同指令，则无需继续验证该路径的其余部分，因为已知其是有效的。

[Lots of work has gone into optimizing the verifier](#) and its pruning process. The verifier used to store pruning state before and after each jump instruction, but analysis showed that this results in storing state on average every four instructions or so, and the vast majority of these pruning states would never get matched. It turned out that it's more efficient to store pruning state every 10 instructions, regardless of branching.

大量的工作已经投入到优化验证程序和其减枝过程中。验证程序以前和每个跳转指令之后都会存储减枝状态，但分析显示这样做会导致平均每四个指令就存储一次状态，而绝大部分减枝状态都无法匹配。事实证明，每隔10个指令存储一次减枝状态无论是否有分支效率更高。

Note

You can read more details on how verification works in the [kernel documentation](#).

您可以在内核文档中阅读有关验证工作原理的更多详细信息。

The Verifier Log

核实者日志

When the verification of a program fails, the verifier generates a log showing how it reached the conclusion that the program is invalid. If you're using `bpftool prog load`, the verifier log gets output to `stderr`. When you're writing a program with *libbpf*, you can use the function `libbpf_set_print()` to set a handler that will display (or do something else useful with) any errors. (You'll see an example of this in the *hello-verifier.c* source code for this chapter.)

当程序的验证失败时，验证器会生成一份日志，展示程序被判定为无效的过程。如果您使用的是`bpftool prog load`命令，验证器日志将会输出到标准错误流(`stderr`)。如果您正在使用`libbpf`编写程序，您可以使用`libbpf_set_print()`函数来设置一个处理程序，用来显示（或执行其他有用的操作）任何错误。（您将在本章的`hello-verifier.c`源代码中看到一个示例。）

Note

If you really want to dig into what the verifier is doing, you can get it to generate the log on success as well as on failure. There is a basic example of this in the *hello-verifier.c* file too. It involves passing a buffer that will hold verifier log contents into the *libbpf* call that loads the program into the kernel and then writing the contents of that log to screen.

如果你真的想深入了解验证器的工作原理，你可以让它在成功和失败时生成日志。在hello-verifier.c文件中也有一个基本的示例。这涉及将一个用于保存验证器日志内容的缓冲区传递给加载程序到内核的libbpf调用，然后将该日志的内容写入屏幕。

The verifier log includes a summary of how much work the verifier did, which looks something like this:

验证者日志包括验证者所做的工作摘要，大致如下所示：

```
processed 61 insns (limit 1000000) max_states_per_insn 0
total_states 4
peak_states 4 mark_read 3
```

In this example, the verifier processed 61 instructions, including potentially processing the same instruction multiple times by arriving at it through different paths. Note that the complexity limit of one million is an upper bound on the number of instructions in a program; in practice, if there are branches in the code, the verifier will process some instructions more than once.

在本示例中，验证程序处理了61条指令，包括通过不同的路径到达相同的指令并可能多次处理它。请注意，一百万的复杂度限制是程序中指令数量的上限；在实践中，如果代码中存在分支，验证程序将会对某些指令进行多次处理。

The total number of states stored was four, which for this simple program matches the peak number of stored states. If some of the states had been pruned, the peak number might be lower than the total.

存储的状态总数为四个，对于这个简单的程序来说，它与存储状态的峰值数相匹配。如果其中一些状态被修剪了，峰值数可能会低于总数。

The log output includes the BPF instructions the verifier has analyzed, along with the corresponding C source code lines (if the object file was built with the `-g` flag to include debug information) and summaries of verifier state information. Here is an example extract of the verifier log relating to the first few lines of the program in *hello-verifier.bpf.c*:

日志输出包括验证程序分析的BPF指令，以及相应的C源代码行（如果对象文件使用-g标志构建以包含调试信息），以及验证状态信息摘要。以下是验证程序日志的示例摘录，与hello-verifier.bpf.c程序的前几行相关。

```
0: (bf) r6 = r1
; data.counter = c;
❶
1: (18) r1 = 0xffff800008178000
3: (61) r2 = *(u32 *)(r1 +0)
   R1_w=map_value(id=0,off=0,ks=4,vs=16,imm=0)
   R6_w=ctx(id=0,off=0,imm=0)
   R10=fp0
❷
; c++;
4: (bf) r3 = r2
5: (07) r3 += 1
6: (63) *(u32 *)(r1 +0) = r3
   R1_w=map_value(id=0,off=0,ks=4,vs=16,imm=0)
   R2_w=inv(id=1,umax_value=4294967295,
   var_off=(0x0; 0xffffffff))
   R3_w=inv(id=0,umin_value=1,umax_value=4294967296,
   var_off=(0x0; 0xffffffff)) R6_w=ctx(id=0,off=0,imm=0)
   R10=fp0 ❸
```

❶

The log includes source code lines to make it easier to understand how the output relates to the source. This source code is available because the `-g` flag was used to build in debug information during the compilation step.

❷

Here's an example of some register state information being output in the log. It tells us that at this stage Register 1 contains a map value, Register 6 holds the context, and Register 10 is the frame (or stack) pointer where local variables are held.

❸

This is another example of register state information. Here you can see not only the types of values that are held in each (initialized) register, but also the range of possible values for Register 2 and Register 3.

Let's dig further into the details of this. I said that Register 6 holds the context, and the verifier log indicates this with

`R6_w=ctx(id=0,off=0,imm=0)`. This was set up in the very first line of the bytecode, where Register 1 was copied to Register 6. When an eBPF program is called, Register 1 always holds the context argument passed to the program. Why copy it to Register 6? Well, when a BPF helper function is called, the arguments to that call are passed in Registers 1 through 5. Helper functions don't modify the contents of Registers 6 through 9, so saving the context off into

Register 6 means the code can call a helper function without losing access to the context.

让我们进一步深入讨论这个问题。我说寄存器6保存着上下文，并且验证者日志中的R6_w=ctx(id=0,off=0,imm=0)证明了这一点。这是在字节码的第一行设置的，在那里寄存器1的内容被复制到了寄存器6。当调用一个eBPF程序时，寄存器1始终保存着传递给程序的上下文参数。为什么要将它复制到寄存器6呢？嗯，在调用BPF帮助函数时，调用的参数通过寄存器1到5传递。帮助函数不会修改寄存器6到9的内容，因此将上下文保存到寄存器6意味着代码在调用帮助函数时不会失去对上下文的访问。

Register 0 is used for the return value from a helper function and also for the return value from an eBPF program. Register 10 always holds a pointer to the eBPF stack frame (and the eBPF program can't modify it).

寄存器0用于来自辅助函数和eBPF程序的返回值。寄存器10永远存储着指向eBPF堆栈帧的指针（eBPF程序无法修改它）。

Let's look at the register state information for Registers 2 and 3 after instruction 6:

让我们来看一下指令 6 执行后的寄存器 2 和寄存器 3 的状态信息。

```
R2_w=inv(id=1,umax_value=4294967295,var_off=(0x0;
0xffffffff))
R3_w=inv(id=0,umin_value=1,umax_value=4294967296,var_off=
(0x0; 0x1ffffffff))
```

Register 2 doesn't have a minimum value, and the `umax_value` shown here in decimal corresponds to 0xFFFFFFFF, which is the largest value that can be held in an 8-byte register. In other words, at this point the register could hold any of its possible values.

寄存器2没有最小值，这里以十进制表示的`umax_value`对应着0xFFFFFFFF，也就是一个8字节寄存器中能够容纳的最大值。换句话说，在这一点上，寄存器可以容纳它的任意可能值。

In instruction 4, the contents of Register 2 are copied into Register 3, and then instruction 5 adds one to that value. Therefore, Register 3 could have any value that's 1 or greater. You can see this in the state information for Register 3, which has `umin_value` set to 1, and a `umax_value` of 0xFFFFFFFF.

在第四条指令中，将寄存器2的内容复制到寄存器3中，然后第五条指令将该值加1。因此，寄存器3的值可以是任何大于等于1的值。这可以从寄存器3的状态信息中看出，其中`umin_value`设为1，`umax_value`设为0xFFFFFFFF。

The verifier uses the information about not just the states of each register, but also the range of values each can contain, to determine the possible paths through the program. This is also used for the state pruning that I mentioned before: if the verifier has been in the same position in the code, with the same types and possible ranges of values for each register, there's no need to evaluate this path further. What's more, if the current state is a subset of a state that was seen earlier, it can also be pruned.

验证器使用有关每个寄存器状态以及每个寄存器可能包含的值范围的信息，来确定程序可能的路径。这也用于之前提到的状态剪枝：如果验证器在代码中的相同位置，具有相同类型和每个寄存器可能的值范围，就没有必要进一步评估该路径。而且，如果当前状态是先前看到的状态的子集，也可以进行剪枝。

Visualizing Control Flow

可视化控制流程

The verifier explores all the possible paths through the eBPF program, and if you're trying to debug an issue, it can be helpful to see those paths for yourself. The `bpftool` utility can help with this by producing a control flow graph of the program in [DOT format](#), which you can then convert into an image format, like this:

验证工具会探索通过 eBPF 程序的所有可能路径，如果你想要调试一个问题，可以通过查看这些路径来帮助你。bpftool 实用程序可以协助完成这个任务，它可以生成一个以 DOT 格式表示程序的控制流图，你可以将它转换为图片格式，就像这样：

```
$ bpftool prog dump xlated name kprobe_exec visual >
out.dot
$ dot -Tpng out.dot > out.png
```

This produces a visual representation of the control flow like that shown in [Figure 6-1](#).

这将产生一个控制流的可视化表示，类似于图6-1所示。

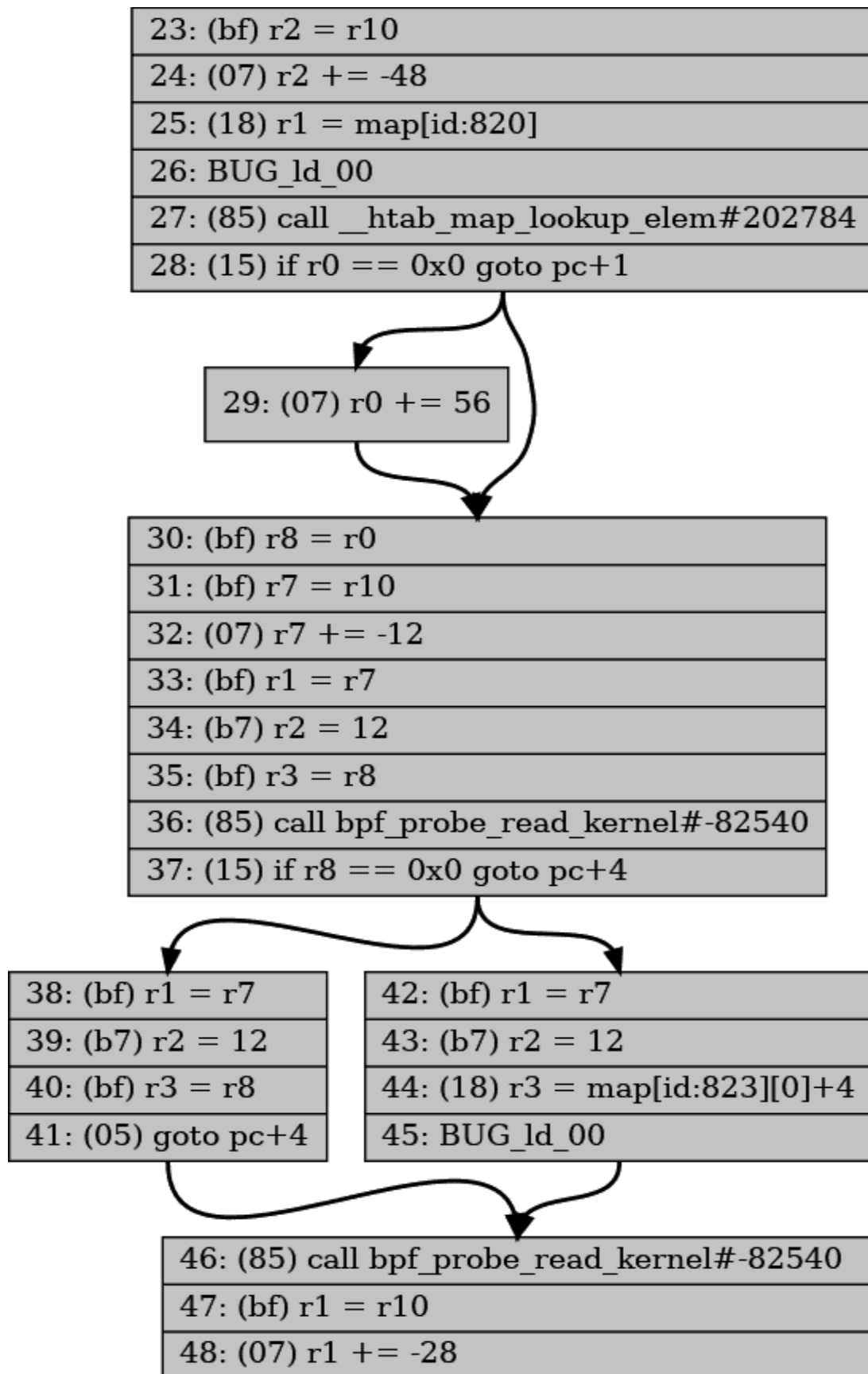


Figure 6-1. Extract from control flow graph (the full image can be found as `chapter6/kprobe_exec.png` in the [GitHub repo](#) for this book)

Validating Helper Functions

验证助手函数

You're not allowed to call directly from eBPF programs to any kernel function (unless it has been registered as a kfunc, which you'll meet in the next chapter), but eBPF provides a number of helper functions that enable programs to access information from the kernel. There's a [bpf-helpers manpage](#) that attempts to document them all.

你不能直接从eBPF程序调用任何内核函数（除非它已经被注册为kfunc，在下一章中会介绍），但是eBPF提供了许多辅助函数，使程序能够从内核访问信息。有一个名为bpf-helpers的手册试图记录它们所有的内容。

Different helper functions are valid for different BPF program types. For example, the helper function `bpf_get_current_pid_tgid()` retrieves the current user space process ID and thread ID, but it doesn't make sense to call this from an XDP program that is triggered by the receipt of a packet at a network interface, because there is no user space process involved. You can see an example of this by changing the `SEC()` definition for the *hello* eBPF program in *hello-verifier.bpf.c* from `kprobe` to `xdp`. On attempting to load this program the verifier output gives the following message:

不同的辅助函数对于不同类型的BPF程序是有效的。例如，辅助函数 `bpf_get_current_pid_tgid()` 检索当前用户空间进程ID和线程ID，但是从由网络接口接收数据包触发的XDP程序中调用它是没有意义的，因为没有涉及用户空间进程。您可以通过将 `hello-verifier.bpf.c` 中的 `SEC()` 定义从 `kprobe` 更改为 `xdp` 来看到这个例子。在尝试加载该程序时，验证输出会显示以下消息：

```
...
16: (85) call bpf_get_current_pid_tgid#14
unknown func bpf_get_current_pid_tgid#14
```

The `unknown func` doesn't mean the function is completely unknown, just that it is unknown *for this BPF program type*. (BPF program types are a topic for the next chapter; for now you can just think of them as being programs that are suitable for attaching to different types of event.)

未知的函数并不意味着该函数完全未知，只是对于这个BPF程序类型来说是未知的。(BPF程序类型是下一章的主题；现在你可以简单地将它们视为适用于不同类型事件的程序。)

Helper Function Arguments

辅助函数参数，

If you look, for example, in [kernel/bpf/helpers.c](#),² you'll find that each helper function has a `bpf_func_proto` structure similar to this example for the helper function `bpf_map_lookup_elem()`:

如果您仔细查看，例如在kernel/bpf/helpers.c文件中，您会发现每个辅助函数都有一个类似于bpf_map_lookup_elem()辅助函数的bpf_func_proto结构，就像下面这个例子一样。

```
const struct bpf_func_proto bpf_map_lookup_elem_proto = {
    .func          = bpf_map_lookup_elem,
    .gpl_only      = false,
    .pkt_access     = true,
    .ret_type       = RET_PTR_TO_MAP_VALUE_OR_NULL,
    .arg1_type      = ARG_CONST_MAP_PTR,
    .arg2_type      = ARG_PTR_TO_MAP_KEY,
};
```

This structure defines the constraints for arguments to and return values from the helper function. Because the verifier is keeping track of the type of value held in each register, it can spot if you try to pass the wrong kind of argument to a helper function. For example, try changing the argument to the call to `bpf_map_lookup_elem()` in the *hello* program, like this:

这个结构定义了对助手函数的参数和返回值的约束。因为验证器在跟踪每个寄存器中保存的值的类型，所以它可以发现如果您试图向助手函数传递错误类型的参数。例如，在hello程序中尝试将bpf_map_lookup_elem()的参数改变成这样：``

```
p = bpf_map_lookup_elem(&data, &uid);
```

Instead of passing `&my_config`, which is a pointer to a map, this now passes `&data`, which is a pointer to a local variable structure. This is valid from the compiler's point of view, so you can build the BPF object file *hello-verifier.bpf.o*, but when you try to load the program into the kernel, you'll see an error like this in the verifier log:

而不是传递指向映射的指针`&my_config`，现在传递指向本地变量结构的指针`&data`。从编译器的角度来看，这是有效的，因此您可以构建BPF对象文件`hello-verifier.bpf.o`，但是当您尝试将程序加载到内核时，您将在验证器日志中看到类似以下的错误。

```
27: (85) call bpf_map_lookup_elem#1
R1 type=fp expected=map_ptr
```

Here, `fp` stands for *frame pointer*, and it's the area of memory on the stack where local variables are stored. Register 1 was loaded with the address of the local variable called `data`, but the function expects a pointer to a map (as indicated by the `arg1_type` field in the `bpf_func_proto` structure shown earlier). By tracking the types of value stored in each register, the verifier was able to spot this discrepancy.

在这里，`fp`代表帧指针，它是存储局部变量的堆栈内存区域。寄存器1被加载了一个名为`data`的局部变量的地址，但是函数期望的是一个指向地图的指针（正如之前所示的`bpf_func_proto`结构中的`arg1_type`字段所示）。通过跟踪每个寄存器中存储的值的类型，验证器能够发现这个不一致。

Checking the License

查看许可证

The verifier also checks that if you are using a BPF helper function that's licensed under GPL, your program also has a GPL-compatible license. The last line in the [Chapter 6](#) example code *hello-*

`verifier.bpf.c` defines a “license” section that holds the string `Dual BSD/GPL`. If you remove this line, the output from the verifier will end like this:

审查者还会检查您是否使用了在GPL下许可的BPF帮助函数，以确保您的程序也具有GPL兼容的许可。第6章示例代码`hello-verifier.bpf.c`的最后一行定义了一个“license”部分，其中包含字符串“Dual BSD/GPL”。如果您删除此行，则审查者的输出将以这样的方式结束：

```
...
37: (85) call bpf_probe_read_kernel#113
cannot call GPL-restricted function from non-GPL
compatible program
```

That’s because the `gpl_only` field is set to `true` for the `bpf_probe_read_kernel()` helper function. There are other helper functions called earlier in this eBPF program, but they are not GPL licensed, so the verifier doesn’t object to their use.

这是因为`bpf_probe_read_kernel()`助手函数的`gpl_only`字段被设置为`true`。在这个eBPF程序中还有其他先前调用的助手函数，但它们不是GPL许可的，所以验证器不会反对使用它们。

The BCC project maintains a [list of helper functions](#), indicating whether they are GPL licensed or not. If you’re interested in more details on how helper functions are implemented, there’s a section on this in the [BPF and XDP reference guide](#).

BCC项目维护着一个助手函数清单，指明它们是否采用GPL许可证。如果您对助手函数的实现细节感兴趣，可以在BPF和XDP参考指南中找到相关内容。

Checking Memory Access

检查内存访问

The verifier performs a number of checks to make sure BPF programs only access memory they are supposed to have access to.

验证器执行一系列检查，以确保BPF程序只能访问它们被授权访问的内存。

For example, when processing a network packet, an XDP program is only permitted to access the memory locations that make up that network packet. Most XDP programs start with something very similar to the following:

例如，在处理网络数据包时，XDP程序只允许访问构成该网络数据包的内存位置。大多数XDP程序都以以下非常相似的内容开始：

```
SEC("xdp")
int xdp_load_balancer(struct xdp_md *ctx)
{
    void *data = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;
    ...
}
```

The `xdp_md` structure passed as the context to the program describes the network packet that has been received. The `ctx->data` field within that structure is the location in memory where the packet starts, and `ctx->data_end` is the last location in the packet. The verifier will ensure that the program doesn't exceed these bounds.

作为上下文传递给程序的xdp_md结构描述了接收到的网络数据包。该结构中的ctx->data字段是数据包起始位置在内存中的位置，而ctx->data_end是数据包的最后位置。验证程序将确保程序不超出这些边界。

For example, the following program in *hello_verifier.bpf.c* is valid:

例如，以下在 *hello_verifier.bpf.c* 中的程序是有效的：

```
SEC("xdp")
int xdp_hello(struct xdp_md *ctx) {
    void *data = (void *) (long) ctx->data;
    void *data_end = (void *) (long) ctx->data_end;
    bpf_printk("%x", data_end);
    return XDP_PASS;
}
```

The variables `data` and `data_end` are very similar, but the verifier is smart enough to recognize that `data_end` relates to the end of a packet. Your program is required to check that any values read from the packet aren't from beyond that location, and it won't let you "cheat" by modifying the `data_end` value. Try adding the following line just before the `bpf_printk()` call:

变量data和data_end非常相似，但该验证器足够聪明，可以识别出data_end与数据包的末尾有关。您的程序需要检查从数据包中读取的任何值是否不超过此位置，并且不允许您通过修改data_end值来“作弊”。请尝试在bpf_printk()调用之前添加以下行：

```
data_end++;
```

The verifier will complain, like this:

核查员会抱怨，就像这样：

```
; data_end++;  
1: (07) r3 += 1  
R3 pointer arithmetic on pkt_end prohibited
```

In another example, when accessing an array you need to make sure there's no possibility of accessing an index that is beyond the bounds of that array. In the example code there is a section that reads a character out of the `message` array, like this:

在另一个例子中，当访问数组时，您需要确保没有可能访问超出该数组边界的索引。在示例代码中，有一个部分从消息数组中读取一个字符，如下所示：

```
if (c < sizeof(message)) {  
    char a = message[c];  
    bpf_printk("%c", a);  
}
```

This is fine because of the explicit check to ensure that the counter variable `c` is no bigger than the size of the message array. Making a simple “off by one” error like the following renders it invalid:

因为明确检查以确保计数器变量`c`不大于消息数组的大小，所以这是好的。如果出现以下简单的“错位”错误，则使其无效。

```
if (c <= sizeof(message)) {  
    char a = message[c];  
    bpf_printk("%c", a);  
}
```

The verifier will fail this with an error message similar to this:

验证器将返回类似以下错误信息的失败提示：

```
invalid access to map value, value_size=16 off=16 size=1
R2 max value is outside of the allowed memory range
```

It's fairly clear from this message that there is an invalid access to a map value because Register 2 might hold a value that's too large for indexing the map. If you were debugging this error, you'd want to dig into the log to see what line in the source code was responsible. The log ends like this just before emitting the error message (I have removed some of the state information for clarity):

从这个消息中可以清楚地看出，访问地图值时存在无效访问，因为寄存器2可能持有的值对于地图索引来说太大了。如果你正在调试这个错误，你会想要仔细查看日志，看看源代码中哪一行有问题。在发出错误消息之前，日志最后是这样结束的（为了清晰起见，我已删除了一些状态信息）：

```
; if (c <= sizeof(message)) {
30: (25) if r1 > 0xc goto pc+10
❸
  R0_w=map_value_or_null(id=2,off=0,ks=4,vs=12,imm=0)
  R1_w=inv(id=0,
    umax_value=12,var_off=(0x0; 0xf))
  R6=ctx(id=0,off=0,imm=0) ...
  ; char a = message[c];
31: (18) r2 = 0xffff800008e00004
❷
33: (0f) r2 += r1
last_idx 33 first_idx 19
regs=2 stack=0 before 31: (18) r2 = 0xffff800008e00004
regs=2 stack=0 before 30: (25) if r1 > 0xc goto pc+10
regs=2 stack=0 before 29: (61) r1 = *(u32 *)(r8 +0)
34: (71) r3 = *(u8 *)(r2 +0)
❶
  R0_w=map_value_or_null(id=2,off=0,ks=4,vs=12,imm=0)
  R1_w=invP(id=0,
```

```
umax_value=12,var_off=(0x0; 0xf))  
R2_w=map_value(id=0,off=4,ks=4,vs=16,  
umax_value=12,var_off=(0x0;  
0xf),s32_max_value=15,u32_max_value=15)  
R6=ctx(id=0,off=0,imm=0) ...
```

❶

Working backward from the error, the last register state information shows that Register 2 could have a maximum value of 12.

❷

At instruction 31, Register 2 is set to an address in memory and then is incremented by the value of Register 1. The output shows that this corresponds to the line of code accessing `message[c]`, so it stands to reason that Register 2 is set to point to the message array and then to be incremented by the value of `c`, which is held in the Register 1 register.

❸

Working further back to find the value of Register 1, the log shows that it has a maximum value of 12 (which is hex 0x0c). However, `message` is defined as a 12-byte character array, so only indexes 0 through 11 are within its bounds. From this, you can see that the error springs from the source code testing for `c <= sizeof(message)`.

At step 2, I have inferred the relationship between some registers and the source code variables they represent, from the lines of source code the verifier has helpfully included in the log. You could

work back through the verifier log to check that this is true, and indeed you might have to if the code was compiled without debug information. Given the debug information is present, it makes sense to use it.

在第2步，我从验证器在日志中有用地包含的源代码行中推断出了一些寄存器与它们所代表的源代码变量之间的关系。你可以通过查看验证器日志回溯来检查这一点，如果代码是没有调试信息编译的话，确实可能需要这样做。鉴于调试信息是存在的，使用它是有意义的。

The `message` array is declared as a global variable, and you might recall from [Chapter 3](#) that global variables are implemented using maps. This explains why the error message talks about “invalid access to a map value.”

消息数组被声明为全局变量，你可能还记得在第3章中我们讲过全局变量是使用映射实现的。这就解释了为什么错误消息中提到了“无效访问映射值”。

Checking Pointers Before Dereferencing Them

在解引用指针之前检查它们

One easy way to make a C program crash is to dereference a pointer when the pointer has a zero value (also known as *null*). Pointers indicate where in memory a value is being held, and zero is not a valid memory location. The eBPF verifier requires all pointers

to be checked before they are dereferenced so that this type of crash can't happen.

让一个C程序崩溃的一个简单方式就是在指针的值为零（也称为空指针）时对其进行解引用。指针指示值在内存中的位置，而零不是一个有效的内存位置。eBPF验证器要求在对指针进行解引用之前对所有指针进行检查，以防止发生这种类型的崩溃。

The example code in *hello-verifier.bpf.c* looks for a custom message that might exist in the `my_config` hash table map for a user, with the following line:

hello-verifier.bpf.c文件中的示例代码查找my_config哈希表映射中可能存在的用户自定义消息，以下是代码行的内容：

```
p = bpf_map_lookup_elem(&my_config, &uid);
```

If there's no entry in this map corresponding to `uid`, this will set `p` (which is a pointer to the message structure `msg_t`) to zero. Here's a little bit of additional code that attempts to dereference this potentially null pointer:

如果在此映射中没有与uid对应的条目，这会将p（它是指向消息结构体msg_t的指针）设置为零。下面是一小段额外的代码，试图解引用这个可能为空的指针。

```
char a = p->message[0];  
bpf_printk("%c", a);
```

This compiles fine, but the verifier rejects it as follows:

这个编译没有问题，但是验证器拒绝它，并且给出以下提示：

```

; p = bpf_map_lookup_elem(&my_config, &uid);
25: (18) r1 = 0xffff263ec2fe5000
27: (85) call bpf_map_lookup_elem#1
28: (bf) r7 = r0
; char a = p->message[0];
29: (71) r3 = *(u8 *)(r7 +0)
R7 invalid mem access 'map_value_or_null'

```

❶

The return value from a helper function call gets stored in Register 0. Here, that value is being stored in Register 7. This means Register 7 now holds the value of the local variable `p`.

❷

This instruction attempts to dereference the pointer value `p`. The verifier has been keeping track of the state of Register 7 and knows that it may hold a pointer to a map value, or it might be null.

The verifier rejects this attempt to dereference a null pointer, but the program will pass if there is an explicit check, like this:

验证器拒绝了对空指针的解引用尝试，但如果有显式的检查，程序就会通过，如下：

```

if (p != 0) {
    char a = p->message[0];
    bpf_printk("%d", cc);
}

```

Some helper functions incorporate the pointer check for you. For example, if you look at the manpage for bpf-helpers, you'll find the function signature for `bpf_probe_read_kernel()` is as follows:

一些辅助函数为您内置了指针检查。例如，如果您查看bpf-helpers的manpage，您会发现bpf_probe_read_kernel()的函数签名如下：

```
long bpf_probe_read_kernel(void *dst, u32 size, const void
*unsafe_ptr)
```

The third argument to this function is called `unsafe_ptr`. This is an example of a BPF helper function that helps programmers write safe code by handling checks for you. You're allowed to pass a potentially null pointer—but only as the third argument called `unsafe_ptr`—and the helper function will check that it's not null before attempting to deference it.

这个函数的第三个参数叫做`unsafe_ptr`。这是一个BPF辅助函数的例子，帮助程序员编写安全的代码，通过处理检查来确保安全。你可以传递一个可能为空的指针，但只能作为叫做`unsafe_ptr`的第三个参数，并且辅助函数会在尝试解引用之前检查它是否为空。

Accessing Context

获取上下文

Every eBPF program is passed some context information as an argument, but depending on the program and attachment type, it may be allowed to access only some of that context information. For example, [tracepoint programs](#) receive a pointer to some tracepoint data. The format of that data depends on the particular tracepoint, but they all start with some common fields—yet those common fields are not accessible to eBPF programs. Only the tracepoint-specific

fields that follow can be accessed. Attempting to read or write the wrong fields leads to an `invalid bpf_context access` error. There is an example of this in the exercises at the end of this chapter.

每个eBPF程序作为参数传递一些上下文信息，但根据程序和附件类型的不同，它可能只能访问部分上下文信息。例如，跟踪点程序接收某个跟踪点数据的指针。该数据的格式取决于特定的跟踪点，但它们都以一些共同的字段开头，然而这些共同的字段不能被eBPF程序访问。只能访问其后面跟随的特定于跟踪点的字段。试图读取或写入错误的字段会导致无效的bpf_context访问错误。本章末尾的练习中有一个例子。

Running to Completion

运行到完成

The verifier ensures that the eBPF program will run to completion; otherwise, there is a risk that it might consume resources indefinitely. It does this by having a limit on the total number of instructions that it will process, which, as I mentioned earlier, is set at one million instructions at the time of this writing. That limit is [hard-coded into the kernel](#); it's not a configurable option. If the verifier hasn't reached the end of the BPF program before it has processed this many instructions, it rejects the program.

验证器确保eBPF程序能够完成运行；否则，存在着它可能持续消耗资源的风险。为了实现这一点，它对将被处理的指令总数有一个限制，在我前面提到的时候，这个限制被设置为一百万条指令。这个限制是

硬编码到内核中的，它不是一个可配置的选项。如果验证器在处理了这么多指令之后还没有达到BPF程序的结尾，它就会拒绝这个程序。

One easy way to create a program that never completes is to write a loop that never ends. Let's see how loops can be created in eBPF programs.

创建一个永不完成的程序的简单方法是编写一个永不结束的循环。让我们看看如何在eBPF程序中创建循环。

Loops

循环 (Xún huán)

To guarantee completion, until kernel version 5.3 there was a restriction on loops.³ Looping through the same instructions requires a jump backward to earlier instructions, and it used to be the case that the verifier would not permit this. eBPF programmers worked around this by using the `#pragma unroll` compiler directive to tell the compiler to write out a set of identical (or very similar) bytecode instructions for each time around the loop. This saved the programmer typing in the same lines many times, but you would see repeated instructions in the emitted bytecode.

为了保证完成，直到内核版本5.3，对循环有一项限制。循环遍历相同的指令需要跳回到较早的指令，而验证器过去不允许这种情况。eBPF程序员通过使用`#pragma unroll`编译器指令绕过了这个问题，告诉编译器为每次循环写出一组相同（或非常相似）的字节码指令。这样一

来，程序员无需重复输入相同的行，但在生成的字节码中会看到重复的指令。

From version 5.3 onward the verifier follows branches backward as well as forward as part of its process of checking all the possible execution paths. This means it can accept some loops, provided the execution path remains within the limit of one million instructions.

从5.3版本开始，验证器在检查所有可能的执行路径时，向前和向后跟踪分支。这意味着它可以接受一些循环，只要执行路径保持在一百万条指令的限制之内。

You can see an example of a loop in the example *xdp_hello* program. A version of the loop that passes verification looks like this:

您可以在xdp_hello程序的示例中看到一个循环的例子。一种经过验证的循环版本如下所示：

```
for (int i=0; i < 10; i++) {  
    bpf_printk("Looping %d", i);  
}
```

The (successful) verifier log will show that it has followed the execution path around this loop 10 times. In doing so, it doesn't hit the complexity limit of one million instructions. In the exercises for this chapter, there's another version of this loop that will hit that limit and will fail verification.

（成功的）验证器日志将显示它已经按照这个循环的执行路径循环了10次。在这样做的过程中，它没有达到一百万条指令的复杂度限制。

在本章的练习中，还有另一个版本的这个循环会达到那个限制，并且会验证失败。

In version 5.17 a new helper function, `bpf_loop()`, was introduced that makes it much easier for the verifier not only to accept loops but also to do it in a much more efficient way. This helper takes the maximum number of iterations as its first argument, and it is also passed a function that is called for each iteration. The verifier only has to validate the BPF instructions in that function once, however many times it might be called. That function can return a nonzero value to indicate that there is no need to call it again, which is used to terminate a loop early once the desired result is achieved.

在版本5.17中，引入了一个新的辅助函数**bpf_loop()**，它使得验证程序不仅更容易接受循环，而且以更高效的方式完成。这个辅助函数以最大迭代次数作为第一个参数，并传入一个在每次迭代中调用的函数。验证程序只需对该函数中的BPF指令进行一次验证，无论调用多少次。该函数可以返回非零值来表示无需再次调用，一旦达到期望的结果，就可以提前终止循环。

There's also a helper function `bpf_for_each_map_elem()` that calls a provided callback function for each item in a map.

还有一个辅助函数**bpf_for_each_map_elem()**，它会为地图中的每个项目调用一个提供的回调函数。

Checking the Return Code

检查返回代码

The return code from an eBPF program is stored in Register 0 (R0). If the program leaves R0 uninitialized, the verifier will fail, like this:

eBPF程序的返回代码存储在寄存器0 (R0)中。如果程序未对R0进行初始化，验证器将会失败，如下所示：

```
R0 !read_ok
```

You can try this by commenting out all the code in a function; for example, modify the `xdp_hello` example to be like this:

你可以用注释掉函数中的所有代码来尝试这个方法；例如，将 `xdp_hello` 示例修改为以下内容：

```
SEC("xdp")
int xdp_hello(struct xdp_md *ctx) {
    void *data = (void *) (long) ctx->data;
    void *data_end = (void *) (long) ctx->data_end;

    // bpf_printk("%x", data_end);
    // return XDP_PASS;
}
```

This will fail the verifier. However, if you put the line with the helper function `bpf_printf()` back in, the verifier won't complain, even though there's no explicit return value set by the source code!

这将使验证器失败。然而，如果将包含帮助函数 `bpf_printf()` 的行放回去，验证器将不会抱怨，尽管源代码中没有明确设置返回值！

This is because Register 0 is also used to hold the return code from a helper function. After returning from a helper function in an eBPF program, Register 0 is no longer uninitialized.

这是因为寄存器0还用于保存来自辅助函数的返回码。在eBPF程序中从辅助函数返回后，寄存器0不再是未初始化的状态。

Invalid Instructions

无效指令

As you know from the discussion of the eBPF (virtual) machine in [Chapter 3](#), eBPF programs consist of a set of bytecode instructions. The verifier checks that the instructions in a program are valid bytecode instructions—for example, using only known opcodes.

正如您从第3章中对eBPF（虚拟）机器的讨论所了解的那样，eBPF程序由一组字节码指令组成。验证器会检查程序中的指令是否为有效的字节码指令，比如只使用已知的操作码。

It would be considered a bug in the compiler if it emitted invalid bytecode, so you're not likely to see this kind of verifier error unless you choose (for some reason best known to yourself) to write eBPF bytecode by hand. However, there have been some instructions added more recently, such as the atomic operations. If your compiled bytecode uses these instructions, they would fail verification on an older kernel.

如果编译器产生无效的字节码，那么这将被视为编译器的一个错误，因此你不太可能看到这种类型的验证错误，除非你选择（出于某种原因最了解自己的）手动编写 eBPF 字节码。然而，最近有一些指令被添加进来，比如原子操作。如果你的编译字节码使用这些指令，它们在旧版本的内核上将无法通过验证。

Unreachable Instructions

到达不了的指示

The verifier also rejects programs that have unreachable instructions. Oftentimes, these will get optimized out by the compiler anyway.

验证器还会拒绝具有无法到达的指令的程序。通常情况下，这些指令会被编译器优化掉。

Summary

摘要

When I first got interested in eBPF, getting code through the verifier seemed like a dark art, where seemingly valid code would get rejected, throwing up what seemed to be arbitrary errors. Over time there have been *lots* of improvements to the verifier, and in this chapter you've seen several examples where the verifier log gives hints to help you figure out what the problem is.

当我开始对eBPF产生兴趣时，通过验证器获取代码似乎是一门黑暗的艺术，看似有效的代码会被拒绝，出现看似随意的错误。随着时间的推移，验证器有了许多改进，在本章中，您已经看到了几个例子，其中验证器日志给出了提示，帮助您找出问题所在。

These hints are more helpful when you have a mental model of how the eBPF (virtual) machine works, using a set of registers for

temporary value storage as it steps through an eBPF program. The verifier keeps track of the types and possible range of values for each register to ensure that eBPF programs are safe to run.

当你对eBPF（虚拟）机器的工作原理有一个精确的认知时，这些提示将更加有用。eBPF通过使用一组寄存器来临时存储值，并在执行eBPF程序时逐步运行。验证器会跟踪每个寄存器的类型和可能的值范围，以确保eBPF程序的运行安全。

If you try writing some eBPF code of your own, you might find yourself needing assistance to resolve verifier errors. The [eBPF community Slack channel](#) is a good place to ask for help, and lots of people have also found advice on [StackOverflow](#).

如果你尝试编写自己的eBPF代码，可能会发现自己需要帮助来解决验证器错误。eBPF社区的Slack频道是寻求帮助的好地方，很多人也在StackOverflow上找到了建议。

Exercises

练习

Here are some more ways to cause a verifier error. See if you can correlate the verifier log output to the errors you get:

这里有一些导致验证器错误的更多方法。看看你能否将验证器日志输出与你得到的错误相关联。

1. In “[Checking Memory Access](#)”, you saw the verifier rejecting access beyond the end of the global `message` array. In the

example code there's a section that accesses the local variable `data.message` in a similar way:

在“检查内存访问”中，你看到了验证器拒绝超出全局消息数组末尾的访问。在示例代码中，还有一部分以类似的方式访问局部变量 `data.message` 的部分。

```
if (c < sizeof(data.message)) {  
    char a = data.message[c];  
    bpf_printk("%c", a);  
}
```

Try adjusting the code to make the same out-by-one mistake by replacing the `<` with `<=`, and you'll see an error message about `invalid variable-offset read from stack R2`.

试着调整代码，通过将 `<` 替换为 `<=` 来制造相同的偏差错误，你将会看到关于从栈 R2 读取无效变量偏移的错误消息。

2. Find the commented-out loops in `xdp_hello` in the example code. Try adding in the first loop that looks like this:

在示例代码的 `xdp_hello` 中找到被注释掉的循环。尝试添加类似下面这样的第一个循环。

```
for (int i=0; i < 10; i++) {  
    bpf_printk("Looping %d", i);  
}
```

You should see in the verifier log a repeated series of lines that look something like this:

你应该在验证器日志中看到一系列重复的行，看起来类似于这样：

```

42: (18) r1 = 0xffff800008e10009
44: (b7) r2 = 11
45: (b7) r3 = 8
46: (85) call bpf_trace_printk#6
    R0=inv(id=0)
R1_w=map_value(id=0,off=9,ks=4,vs=26,imm=0) R2_w=inv11
    R3_w=inv8 R6=pkt_end(id=0,off=0,imm=0)
R7=pkt(id=0,off=0,r=0,imm=0)
    R10=fp0
last_idx 46 first_idx 42
regs=4 stack=0 before 45: (b7) r3 = 8
regs=4 stack=0 before 44: (b7) r2 = 11

```

From the log, work out which register is tracking the loop variable `i`.

从日志中找出跟踪循环变量*i*的寄存器。

3. Now try adding in a loop that will fail, which looks like this:

现在尝试添加一个会失败的循环，它的代码如下：

```

for (int i=0; i < c; i++) {
    bpf_printk("Looping %d", i);
}

```

You should see that the verifier tries to explore this loop to its conclusion, but it reaches the instruction complexity limit before it completes (because there is no upper bound on the global variable `c`).

你应该看到验证器试图探索这个循环直到结束，但是在完成之前达到了指令复杂性限制（因为全局变量*c*没有上限）。

4. Write a program that attaches to a tracepoint. (You may have done this already for the exercises in [Chapter 4](#).) Looking ahead

to “[Tracepoints](#)”, you can see a structure definition for the context argument that starts with these fields:

编写一个程序，附加到跟踪点。（您可能已经在第4章的练习中完成了这个操作。）在“跟踪点”部分，您可以看到一个以这些字段开始的上下文参数的结构定义：

```
unsigned short common_type;  
unsigned char common_flags;  
unsigned char common_preempt_count;  
int common_pid;
```

Create your own version of a structure that starts like this, and make the context argument in your program a pointer to this structure. In the program, try accessing any of these fields and see that the verifier fails with `invalid bpf_context access`.

请帮我翻译成简体中文，“创建自己的版本的结构，像这样开始，并将程序中的上下文参数设为指向这个结构的指针。在程序中，尝试访问这些字段中的任何一个，并查看验证器报告无效的 `bpf_context` 访问。”

¹ For a long time the limit was 4,096 instructions, which imposed significant restrictions on the complexity of eBPF programs. This limit still applies to unprivileged users running BPF programs.

长时间以来，限制为4,096条指令，这对eBPF程序的复杂性施加了重大限制。这个限制仍然适用于运行BPF程序的非特权用户。

² Helper functions are also defined in some other places in the source code, for example, [kernel/trace/bpf_trace.c](#) and

[net/core/filter.c](#).

在源代码的其他地方也定义了辅助函数，例如，
`kernel/trace/bpf_trace.c`和`net/core/filter.c`。

³ This release brought a number of significant optimizations and improvements to the BPF verifier, which are summarized nicely in the LWN article [“Bounded loops in BPF for the 5.3 kernel”](#).

此版本带来了一系列对BPF验证器的重大优化和改进，在LWN文章“5.3内核中的BPF有界循环”中得到了很好的总结。

Chapter 7. eBPF Program and Attachment Types

第七章：eBPF程序和附件类型

In the preceding chapters you saw lots of examples of eBPF programs, and you probably spotted the fact that they are attached to different types of events. Some of the examples I've shown attach to kprobes, but in other examples I've demonstrated XDP programs that handle a newly arrived network packet. These are just two of the many attachment points within the kernel. In this chapter we'll take a deeper look at different program types and how they can be attached to different events.

在前面的章节中，你看到了很多eBPF程序的例子，你可能也注意到它们被附加到不同类型的事件上。我展示的一些例子是附加到kprobe上的，但在其他例子中，我演示了处理新到达的网络数据包的XDP程序。这些只是内核中许多附加点的两个例子。在本章中，我们将更深入地了解不同的程序类型以及它们如何附加到不同的事件上。

Note

You can build and run the examples from this chapter using the code and instructions at github.com/lizrice/learning-ebpf. The code for this chapter is in the *chapter7* directory.

您可以使用github.com/lizrice/learning-ebpf上的代码和说明来构建和运行本章的示例。本章的代码位于chapter7目录中。

At the time of this writing, some of the examples are not supported on ARM processors. Check out the *README* file in the *chapter7* directory for more details and advice.

在撰写本文时，一些示例不支持ARM处理器。请查看第7章目录中的README文件以获取更多详细信息和建议。

There are currently around 30 program types enumerated in [uapi/linux/bpf.h](#), and more than 40 attachment types. The attachment type defines more specifically where the program gets attached; for lots of program types, the attachment type can be inferred from the program type, but some program types can be attached to multiple different points in the kernel, so an attachment type has to be specified as well.

当前在uapi/linux/bpf.h中列举了大约30种程序类型，以及40多种附着类型。附着类型更具体地定义了程序被附着的位置；对于许多程序类型来说，附着类型可以从程序类型推断出来，但是某些程序类型可以附着到内核中的多个不同点上，因此还必须指定附着类型。

As you know, this book isn't intended to be a reference manual, so I won't cover every single eBPF program type. There's a good chance that new types will have been added by the time you read this book anyway!

正如您所知，本书并不打算成为一本参考手册，所以我不会涵盖每一种eBPF程序类型。当您阅读本书时，很可能已经添加了新的类型！

Program Context Arguments

程序上下文参数

All eBPF programs take a context argument that is a pointer, but the structure it points to depends on the type of event that triggered it. eBPF programmers need to write programs that accept the appropriate type of context; there is no point in pretending that the context argument points to a network packet if the event is, say, a tracepoint. Defining different types of programs allows the verifier to ensure that the contextual information is handled appropriately and to enforce rules about what helper functions are permissible.

所有的eBPF程序都需要接受一个上下文参数作为指针，但它所指向的结构取决于触发它的事件类型。eBPF程序员需要编写接受适当类型上下文的程序；如果事件是跟踪点，那么假装上下文参数指向网络数据包是没有意义的。定义不同类型的程序可以让验证器确保上下文信息得到适当处理，并强制执行有关可使用的辅助函数的规则。

Note

To dive into the details of the context data passed to different BPF program types, check out [this post by Alan Maguire on Oracle's blog](#).

查看Alan Maguire在Oracle博客上发布的帖子，深入了解传递给不同BPF程序类型的上下文数据的细节。

Helper Functions and Return Codes

助手函数和返回码

As you saw in the previous chapter, the verifier checks that all helper functions used by a program are compatible with its program type. The example in the previous chapter demonstrated that the `bpf_get_current_pid_tgid()` helper function isn't permitted in an XDP program. There is no user space process or thread involved at the point where a packet is received and the XDP hook is triggered, so a call to discover the current process and thread ID doesn't make sense in that context.

正如您在前一章节中所看到的，验证器检查程序使用的所有辅助函数是否与其程序类型兼容。前一章节的例子证明了在XDP程序中不允许使用**bpf_get_current_pid_tgid()**辅助函数。在接收数据包并触发XDP钩子的时候，不存在用户空间进程或线程，因此在那种情况下调用获取当前进程和线程ID的函数是没有意义的。

The program type also determines the meaning of the return code from the program. Again using XDP as an example, the return code value tells the kernel what to do with the packet once the eBPF program has finished processing it—which could involve passing it to the network stack, dropping it, or redirecting it to a different interface. These return codes wouldn't make any sense when an eBPF program is triggered by, say, hitting a particular tracepoint, where there is no network packet involved.

程序类型还确定了程序返回码的含义。以XDP为例，返回码的数值告诉内核在eBPF程序完成处理后如何处理数据包，包括传递给网络栈、丢弃或重定向到不同的接口。当eBPF程序通过特定的跟踪点触发时，例如没有涉及网络数据包时，这些返回码是没有意义的。

There is a [manpage for helper functions](#) (with, quite reasonably, disclaimers that it might not be complete due to the ongoing development of the BPF subsystem).

有一个针对辅助函数的 man 页面（合理地说明，由于 BPF 子系统的持续开发，可能不完整）。

You can get a list of which helper functions are available for each program type in your version of the kernel with the `bpftool feature` command. This shows the system configuration and lists all the available program types and map types, and even lists all the helper functions that are supported for each program type.

您可以使用bpftool feature命令获取您当前版本内核中每种程序类型可用的辅助函数列表。该命令会显示系统配置并列出所有可用的程序类型和映射类型，甚至还会列出每种程序类型支持的所有辅助函数。

Helper functions are considered part of the *UAPI*, the Linux kernel's external, stable interface. As such, once a helper function has been defined in the kernel, it shouldn't change in the future, even though the kernel's internal functions and data structures can change.

助手函数被视为UAPI的一部分，它是Linux内核的外部稳定接口。因此，一旦在内核中定义了助手函数，它将来不应该发生变化，即使内核的内部函数和数据结构可能会变化。

Despite the risk of changes between kernel versions, there was demand from eBPF programmers to be able to access some internal functions from eBPF programs. This can be achieved using the mechanism called *BPF kernel functions*, or [kfuncs](#).

尽管内核版本之间存在风险变动，但eBPF程序员还是需求能够从eBPF程序中访问一些内部函数。这可以通过称为BPF内核函数或kfuncs的机制实现。

Kfuncs

功能函数

Kfuncs allow internal kernel functions to be registered with the BPF subsystem so that the verifier will allow them to be called from eBPF programs. There is a registration for each eBPF program type that is permitted to call a given kfunc.

Kfuncs允许将内部内核函数在BPF子系统中注册，以便验证程序允许它们被eBPF程序调用。每个允许调用特定kfunc的eBPF程序类型都有一个注册过程。

Unlike helper functions, kfuncs don't provide compatibility guarantees, so an eBPF programmer has to consider the possibility of changes between kernel versions.

与辅助函数不同，kfuncs没有提供兼容性保证，所以eBPF程序员必须考虑内核版本之间的变化可能性。

There is a set of “[core](#)” BPF kfuncs, which at the time of this writing consists of functions that allow eBPF programs to obtain and release kernel references to tasks and cgroups.

当前有一组“核心”BPF kfuncs，这些函数允许eBPF程序获取和释放内核对任务和cgroup的引用。

To recap, the type of an eBPF program determines what events it can be attached to, which in turn defines the type of context information it receives. The program type also defines the set of helper functions and kfuncs it can call.

简而言之，eBPF程序的类型确定了它可以连接到哪些事件，从而定义了它接收的上下文信息的类型。程序类型还定义了它可以调用的助手函数和kfuncs集合。

Program types are broadly considered to fall into two categories: tracing (or perf) program types and networking-related program types. Let's look at some examples.

程序类型被广泛分为两类：跟踪（或性能）程序类型和与网络相关的程序类型。让我们来看一些例子。

Tracing

追踪

Programs that attach to kprobes, tracepoints, raw tracepoints, fentry/fexit probes, and perf events were all designed to provide an efficient way for eBPF programs in the kernel to report tracing information about events into user space. These tracing-related types weren't expected to influence the way the kernel behaves in response to the events they are attached to (although, as you'll see in [Chapter 9](#), there have been some innovations in this area!).

那些附加到kprobes、tracepoints、原始tracepoints、fentry/fexit probes和perf事件的程序，都是为了提供一种高效的方式，让内核中

的eBPF程序向用户空间报告有关事件的跟踪信息。这些与跟踪相关的类型并不预期会影响内核对它们所附加的事件的响应方式（尽管，正如您将在第9章中看到的那样，这方面确实有一些创新！）。

These are sometimes referred to as “perf-related” programs. For example, the `bpftool perf` subcommand lets you view programs attached to perf-related events like this:

这些有时被称为“与性能相关的”程序。例如，`bpftool perf`子命令允许您查看附加到与性能相关事件的程序，如下所示：

```
$ sudo bpftool perf show
pid 232272 fd 16: prog_id 392 kprobe func
__x64_sys_execve offset 0
pid 232272 fd 17: prog_id 394 kprobe func do_execve
offset 0
pid 232272 fd 19: prog_id 396 tracepoint
sys_enter_execve
pid 232272 fd 20: prog_id 397 raw_tracepoint
sched_process_exec
pid 232272 fd 21: prog_id 398 raw_tracepoint
sched_process_exec
```

The preceding output is what I see when running example code from the *hello.bpf.c* file in the *chapter7* directory, which attaches different programs to a variety of events that are all related to `execve()`. I'll discuss all of these types in this section, but as an overview, these programs are:

在运行位于chapter7目录中的hello.bpf.c文件的示例代码时，我看到的输出是前面的内容，它将不同的程序附加到与`execve()`相关的各种事件中。在本节中，我将讨论所有这些类型，但作为概述，这些程序是：

- A kprobe attached to the entry point to the `execve()` system call.
一个附加到`execve()`系统调用入口的kprobe。
- A kprobe attached to a kernel function, `do_execve()`.
一个附加到内核函数`do_execve()`的Kprobe。
- A tracepoint placed at the entry to the `execve()` syscall.
在`execve()`系统调用入口处放置的跟踪点。
- Two versions of a raw tracepoint called during the processing of `execve()`. One of these, as you'll see in this section, is a BTF-enabled version.

两个版本的原始跟踪点在处理`execve()`时被调用。你会在本节中看到其中一个版本是启用了BTF的。

You'll need `CAP_PERFMON` and `CAP_BPF` or `CAP_SYS_ADMIN` capabilities to use any of the tracing-related eBPF program types.

您将需要`CAP_PERFMON`和`CAP_BPF`或`CAP_SYS_ADMIN`权限才能使用任何与追踪相关的eBPF程序类型。

Kprobes and Kretprobes

Kprobes 和 Kretprobes

I discussed the concept of kprobes in [Chapter 1](#). You can attach kprobe programs almost anywhere in the kernel.¹ Commonly, they are attached using kprobes to the entry to a function and kretprobes to the exit of a function, but you can use kprobes to attach to an

instruction that is some specified offset after the entry to the function. If you choose to do this,² you'd need to be confident that the kernel version you're running on has the instruction you want to attach to where you think it is! Attaching to kernel function entry and exit points can be relatively stable, but arbitrary lines of code might easily be modified from one release to the next.

我在第1章中讨论了kprobes的概念。你几乎可以将kprobe程序附加到内核的任何位置。通常情况下，它们使用kprobes附加到函数的入口，并使用kretprobes附加到函数的出口，但你也可以使用kprobes附加到函数入口之后某个指定的偏移位置上的指令。如果你选择这样做，你需要确信你所运行的内核版本中具有你想要附加的指令所在的位置！附加到内核函数的入口和出口点可能相对稳定，但任意的代码行可能在下一个版本中轻易被修改。

Note

In the example output from `bpftool perf list`, you can see that there is an offset of 0 for both of the kprobes.

在bpftool perf list的示例输出中，您可以看到两个kprobe的偏移量都为0。

When the kernel is compiled, there's also the possibility that the compiler chooses to “inline” any given kernel function; that is, rather than jump from where the function is called, the compiler might emit the machine code to implement whatever the function does within the calling functions. If a function happens to get inlined, there won't be a kprobe entry point for your eBPF program to attach to.

当内核编译时，编译器还有可能选择对任何给定的内核函数进行“内联”；也就是说，编译器可能会在调用函数的位置，插入实现该函数功能的机器代码，而不是跳转到函数定义的位置。如果某个函数被内联了，那么你的 eBPF 程序就没有一个 kprobe 入口点可以附加到这个函数上。

Attaching kprobes to syscall entry points

将`Attaching kprobes to syscall entry points`翻译为简体中文: 将 kprobes 附加到系统调用入口点

The first example eBPF program for this chapter is called `kprobe_sys_execve`, and it is a kprobe attached to the `execve()` syscall. The function and its section definition look like this:

这一章的第一个eBPF程序示例被称为kprobe_sys_execve，它是一个附加到execve()系统调用的kprobe。该函数及其定义部分如下：

```
SEC("ksyscall/execve")
int BPF_KPROBE_SYSCALL(kprobe_sys_execve, char *pathname)
```

This is the same as what you saw in [Chapter 5](#).

这与你在第五章看到的是一样的。

One reason to attach to syscalls is that they are stable interfaces that won't change between kernel versions (the same is true of tracepoints, which we'll come to shortly). However, syscall kprobes shouldn't be relied on for security tooling, for reasons I'll cover in detail in [Chapter 9](#).

一种附加到系统调用的原因是它们是稳定的接口，在内核版本之间不会改变（同样适用于我们即将介绍的跟踪点）。然而，基于系统调用的探针不应该被视为安全工具，具体原因将在第9章详细讨论。

Attaching kprobes to other kernel functions

连接kprobes到其他内核函数

You can find lots of examples where eBPF-based tools use kprobes to attach to system calls, but, as mentioned earlier, kprobes can also be attached to any noninlined function in the kernel. I've provided an example in *hello.bpf.c* that attaches a kprobe to the function

`do_execve()`, and it's defined like this:

您可以找到很多例子，其中eBPF基于工具使用kprobes来连接到系统调用，但是正如之前提到的，kprobes也可以连接到内核中的任何非内联函数。我在hello.bpf.c中提供了一个示例，将一个kprobe连接到函数do_execve()，它的定义如下：

```
SEC("kprobe/do_execve")
int BPF_KPROBE(kprobe_do_execve, struct filename *filename
)
```

Because `do_execve()` isn't a system call, there are a few differences between this and the previous example:

因为do_execve()不是一个系统调用，所以与前面的示例有一些差异：

- The format of the SEC name is identical to the previous version attached to the syscall entry point, but there is no need to define

platform-specific variants because `do_execve()`, like most kernel functions, is common to all platforms.

SEC名称的格式与附加到syscall入口的先前版本相同，但无需定义特定于平台的变体，因为`do_execve()`和大多数内核函数一样，适用于所有平台。

- I used the `BPF_KPROBE` macro rather than `BPF_KPROBE_SYSCALL`. The intent is exactly the same, just that the latter handles syscall parameters.

我使用了BPF_KPROBE宏，而不是BPF_KPROBE_SYSCALL。意图完全相同，只是后者处理系统调用参数。

- There is another important difference: the `pathname` parameter to the syscall is a pointer to a string `(char *)`, but for this function the parameter is called `filename`, and it's a pointer to a `struct filename`, which is a data structure used within the kernel.

还有一个重要的区别：系统调用的`pathname`参数是一个指向字符串的指针（`char *`），但是对于这个函数，参数被称为`filename`，并且它是一个指向`struct filename`的指针，`struct filename`是内核中使用的数据结构。

You might well be wondering how I knew to use this type for this parameter. I'll show you. The `do_execve()` function in the kernel has the following signature:

你可能很好奇我是如何知道将这种类型用于这个参数的。我会向你展示的。内核中的`do_execve()`函数具有以下签名：

```
int do_execve(struct filename *filename,
              const char __user *const __user * __argv,
              const char __user *const __user * __envp)
```

I chose to ignore the `do_execve()` parameters `__argv` and `__envp`, and only declare the `filename` argument, using the type `struct filename *` to match the kernel function's definition. Given the way arguments are laid out sequentially in memory, it's OK to ignore the last n parameters, but you can't ignore an earlier argument in the list if you want to use a later one.

我选择忽略`do_execve()`中的参数`__argv`和`__envp`，只声明文件名参数，使用类型为`struct filename *`与内核函数的定义相匹配。考虑到参数在内存中是按顺序排列的，如果你想使用后面的参数，可以忽略最后 n 个参数，但不能忽略列表中较早的参数。

This `filename` structure is defined internal to the kernel, and it's an illustration of how eBPF programming is kernel programming: I had to look up the definition of `do_execve()` to find its arguments, and the definition of `struct filename`. The name of the executable that is about to be run is pointed to by `filename->name`. I'm retrieving this name in the example code with the following lines:

这个文件名结构在内核内部定义，并且它是eBPF编程是内核编程的一种示例：我不得不查找`do_execve()`的定义以找到它的参数，以及`struct filename`的定义。即将运行的可执行文件的名称由`filename->name`指向。我在示例代码中用以下几行代码获取了这个名称：

```
const char *name = BPF_CORE_READ(filename, name);
bpf_probe_read_kernel(&data.command, sizeof(data.command),
                      name);
```

So to recap: the context parameter to a syscall kprobe is a structure representing the values passed by user space into the syscall. The context parameter to a “regular” (nonsyscall) kprobe is a structure representing the parameters passed to the called function by whatever kernel code is calling it, so the structure depends on the function definition.

所以总结一下：syscall kprobe 的上下文参数是一个表示用户空间传递给系统调用的值的结构体。而“常规”（非系统调用）kprobe 的上下文参数是一个表示由调用它的内核代码传递给被调用函数的参数的结构体，因此结构体的内容取决于函数定义。

Kretprobes are very similar to kprobes, except that they are triggered when a function returns and can access the return value instead of the arguments.

Kretprobes与kprobes非常相似，不同之处在于它们会在函数返回时触发，并且可以访问返回值而不是参数。

Kprobes and kretprobes are a reasonable way to hook into kernel functions, but there’s a newer option you should consider if you’re running on recent kernels.

Kprobes和kretprobes是一种合理的方法，可以钩入内核函数，但如果你正在运行较新的内核，你应该考虑一个更新的选项。

Fentry/Fexit

进入/退出

A more efficient mechanism for tracing the entry to and exit from kernel functions was introduced along with the idea of *BPF trampoline* in kernel version 5.5 (on x86 processors; BPF trampoline support doesn't arrive for [ARM processors until Linux 6.0](#)). If you're using a recent enough kernel, fentry/fexit is now the preferred method for tracing the entry to or exit from a kernel function. You can write the same code inside a kprobe or fentry type program.

在内核版本5.5中引入了一种更高效的机制，用于跟踪进入和退出内核函数的路径（在x86处理器上引入了BPF跳板的概念；BPF跳板支持在Linux 6.0中才会到达ARM处理器）。如果您正在使用足够新的内核，fentry/fexit现在是跟踪进入或退出内核函数的首选方法。您可以在kprobe或fentry类型的程序中编写相同的代码。

There's an example fentry program called `fentry_execve()` in *chapter7/hello.bpf.c*. I declared the eBPF program for this kprobe using *libbpf's* macro `BPF_PROG`, which is another convenient wrapper giving access to typed parameters rather than the generic context pointer, but this version is used for fentry, fexit, and tracepoint program types. The definition looks like this:

在第七章的`hello.bpf.c`文件中有一个名为`fentry\_execve()`的示例fentry程序。我使用了libbpf的宏`BPF\_PROG`来声明此kprobe的eBPF程序。这个宏是另一个方便的封装，提供了对类型化参数而非通用上下文指针的访问，但这个版本适用于fentry、fexit和tracepoint程序类型。定义如下：

```
SEC("fentry/do_execve")
int BPF_PROG(fentry_execve, struct filename *filename)
```


The section name tells *libbpf* to attach to the fentry hook at the start of the `do_execve()` kernel function. Just as in the kprobe example, the context parameters reflect the parameters passed to the kernel function where you want to attach this eBPF program.

该节名称告诉libbpf在do_execve()内核函数的开始处附加fentry钩子。与kprobe示例类似，上下文参数反映了传递给要附加此eBPF程序的内核函数的参数。

Fentry and fexit attachment points were designed to be more efficient than kprobes, but there's another advantage when you want to generate an event at the end of a function: the fexit hook has access to the input parameters to the function, which kretprobe does not. You can see an example of this in [libbpf-bootstrap's examples](#). Both *kprobe.bpf.c* and *fentry.bpf.c* are equivalent examples that hook into the `do_unlinkat()` kernel function. The eBPF program attached to the kretprobe has the following signature:

Fentry和fexit附加点的设计比kprobe更高效，但是当你想在函数结束时生成事件时，还有一个优点：fexit钩子可以访问函数的输入参数，而kretprobe不能。你可以在libbpf-bootstrap的示例中看到一个例子。kprobe.bpf.c和fentry.bpf.c这两个示例是等价的，它们都钩入了do_unlinkat()内核函数。附加到kretprobe的eBPF程序具有以下签名：

```
SEC("kretprobe/do_unlinkat")
int BPF_KRETPROBE(do_unlinkat_exit, long ret)
```

The `BPF_KRETPROBE` macro expands to make this a kretprobe program on exit from `do_unlinkat()`. The only parameter the eBPF

program receives is `ret`, which holds the return value from `do_unlinkat()`. Compare this to the `fexit` version:

BPF_KRETPROBE宏展开后，将使之成为从do_unlinkat()退出时的一个kretprobe程序。eBPF程序接收的唯一参数是ret，它保存了从do_unlinkat()返回的值。与fexit版本进行比较：

```
SEC("fexit/do_unlinkat")
int BPF_PROG(do_unlinkat_exit, int dfd, struct filename *
name, long ret)
```

In this version the program gets access not just to the return value `ret`, but also to the input parameters to `do_unlinkat()`, which are `dfd` and `name`.

在这个版本中，程序不仅可以访问返回值ret，还可以访问do_unlinkat()的输入参数dfd和name。

Tracepoints

追踪点

[Tracepoints](#) are marked locations in the kernel code (we'll come to user space tracepoints later in this chapter). They're not by any means exclusive to eBPF and have long been used to generate kernel trace output and by tools like [SystemTap](#). Unlike attaching to arbitrary instructions using kprobes, tracepoints are stable between kernel releases (although an older kernel might not have the full set of tracepoints that have been added into a newer one).

追踪点是内核代码中标记的位置（我们稍后在本章中会谈用户空间追踪点）。它们并不专属于eBPF，并且长期以来已被用于生成内核跟踪输出以及像SystemTap这样的工具。与使用kprobes随意附加到指令不同，追踪点在内核版本之间是稳定的（尽管旧内核可能没有添加到新内核的所有追踪点）。

You can see the available set of tracing subsystems on your kernel by looking at `/sys/kernel/tracing/available_events`, as follows:

您可以通过查看`/sys/kernel/tracing/available_events`文件来查看内核上可用的跟踪子系统集合，如下所示：

```
$ cat /sys/kernel/tracing/available_events
tls:tls_device_offload_set
tls:tls_device_decrypted
...
syscalls:sys_exit_execveat
syscalls:sys_enter_execveat
syscalls:sys_exit_execve
syscalls:sys_enter_execve
...
```

My 5.15 version of the kernel has more than 1,400 tracepoints defined in this list. The section definition for a tracepoint eBPF program should match one of these items so that *libbpf* can automatically attach it to the tracepoint. The definition is in the form `SEC("tp/tracing subsystem/tracepoint name")`.

我5.15版本的内核在这个列表中定义了超过1,400个跟踪点。跟踪点eBPF程序的部分定义应与这些项目之一匹配，以便libbpf可以自动将其附加到跟踪点。定义的格式为`SEC("tp/跟踪子系统/跟踪点名称")`。

You'll find an example in the *chapter7/hello.bpf.c* files that matches the `syscalls:sys_enter_execve` tracepoint that gets hit when the kernel starts processing an `execve()` call. The section definition tells *libbpf* that this is a tracepoint program, and where it should be attached, like this:

您将在第7章/ *hello.bpf.c*文件中找到一个示例，该示例与syscalls:sys_enter_execve跟踪点匹配，当内核开始处理execve（）调用时会触发。节定义告诉libbpf这是一个跟踪点程序，并指示它应该被附加在哪里，就像这样：

```
SEC("tp/syscalls/sys_enter_execve")
```

What about the context parameter to a tracepoint? As I'll come to shortly, BTF can help us here, but first let's consider what is needed when BTF isn't available. Each tracepoint has a format describing the fields that get traced out from it. As an example, here's the format for the tracepoint at the entry to the `execve()` syscall:

上下文参数如何在跟踪点中使用？就像我马上要介绍的那样，BTF 可以在这里帮助我们，但首先让我们考虑当 BTF 不可用时需要什么。每个跟踪点都有一个描述从中跟踪出的字段的格式。举个例子，下面是进入 `execve()` 系统调用时的跟踪点的格式：

```
$ cat
/sys/kernel/tracing/events/syscalls/sys_enter_execve/format
name: sys_enter_execve
ID: 622
format:
    field:unsigned short common_type;           offset:0;
size:2; signed:0;
    field:unsigned char common_flags;           offset:2;
```

```

size:1; signed:0;
    field:unsigned char common_preempt_count; offset:3;
size:1; signed:0;
    field:int common_pid;                                offset:4;
size:4; signed:1;

    field:int __syscall_nr;                                offset:8;
size:4; signed:1;
    field:const char * filename;                          offset:16;
size:8; signed:0;
    field:const char *const * argv;                      offset:24;
size:8; signed:0;
    field:const char *const * envp;                      offset:32;
size:8; signed:0;

print fmt: "filename: 0x%08lx, argv: 0x%08lx, envp:
0x%08lx",
((unsigned long)(REC->filename)), ((unsigned long)(REC-
>argv)),
((unsigned long)(REC->envp))

```

I used this information to define a matching structure called

`my_syscalls_enter_execve` in *chapter7/hello.bpf.c*:

我用这些信息来定义一个叫做`my_syscalls_enter_execve`的匹配结构，在第7章 / `hello.bpf.c`中。

```

struct my_syscalls_enter_execve {
    unsigned short common_type;
    unsigned char common_flags;
    unsigned char common_preempt_count;
    int common_pid;

    long syscall_nr;
    long filename_ptr;
    long argv_ptr;
    long envp_ptr;
};

```

eBPF programs aren't allowed to access the first four of these fields.

If you try to access them, the program will fail verification with an

```
invalid bpf_context access error.
```

在eBPF程序中，不允许访问这些字段的前四个。如果尝试访问它们，程序将无法通过验证，出现无效的bpf_context访问错误。

My example eBPF program that attaches to this tracepoint can use a pointer to this type as its context parameter, like this:

我的示例eBPF程序可以依附到此追踪点，并将此类型的指针作为其上下文参数使用，就像这样：

```
int tp_sys_enter_execve(struct my_syscalls_enter_execve *
ctx) {
```

Then you can access the contents of this structure. For example, you can get the filename pointer as follows:

然后您可以访问此结构的内容。例如，您可以按如下方式获取文件名指针：

```
bpf_probe_read_user_str(&data.command, sizeof(data.command)
), ctx->filename_ptr);
```

When you use a tracepoint program type, the structure passed to the eBPF program has already been mapped from a set of raw arguments. For better performance, you can directly access these raw arguments with a raw tracepoint eBPF program type. The section definition should start with `raw_tp` (or `raw_tracepoint`) instead of `tp`. You'll need to convert the arguments from `__u64` to whatever type the tracepoint structure uses (when the tracepoint is the entry to a system call, these arguments are dependent on the chip architecture).

当您使用跟踪点程序类型时，传递给eBPF程序的结构已经从一组原始参数映射过来。为了更好的性能，您可以使用原始跟踪点eBPF程序类型直接访问这些原始参数。段定义应以`raw_tp`（或`raw_tracepoint`）开头，而不是`tp`。您需要将参数从`__u64`转换为跟踪点结构使用的任何类型（当跟踪点作为系统调用的入口时，这些参数取决于芯片架构）。

BTF-Enabled Tracepoints

BTF-启用跟踪点

In the previous example I wrote a structure called `my_syscalls_enter_execve` to define the context parameter for my eBPF program. But when you define a structure in your eBPF code or parse the raw arguments, there's a risk that your code might not match the kernel it's running on. The good news is that BTF, which you met in [Chapter 5](#), also solves this problem.

在之前的示例中，我编写了一个称为`my_syscalls_enter_execve`的结构来定义我的eBPF程序的上下文参数。但是，当在您的eBPF代码中定义结构或解析原始参数时，存在与运行它的内核不匹配的风险。好消息是，在第5章遇到的BTF也解决了这个问题。

With BTF support, there will be a structure defined in *vmlinux.h* that matches the context structure passed to a tracepoint eBPF program. Your eBPF program should use the section definition `SEC("tp_btf/tracepoint name")` where the tracepoint name is one of the available events listed in */sys/kernel/tracing/available_events*. The example program in *chapter7/hello.bpf.c* looks like this:

有了BTF的支持，在vmlinux.h中会定义一个与传递给tracepoint eBPF程序的上下文结构相匹配的结构。您的eBPF程序应该使用节定义SEC("tp_btf / tracepoint name")，其中tracepoint name是/sys/kernel/tracing/available_events中列出的可用事件之一。第7章/ hello.bpf.c中的示例程序如下所示：

```
SEC("tp_btf/sched_process_exec")
int handle_exec(struct trace_event_raw_sched_process_exec
*ctx)
```

As you can see, the structure name matches the tracepoint name, prefixed with `trace_event_raw_`.

正如你所看到的，结构名称与跟踪点名称相匹配，前缀为`trace_event_raw_`。

User Space Attachments

用户空间附件

So far I have shown examples of eBPF programs attaching to events defined within the kernel's source code. There are similar attachment points within user space code: uprobes and uretprobes for attaching to the entry and exit of user space functions, and user statically defined tracepoints (USDTs) for attaching to specified tracepoints within application code or user space libraries. These all use the `BPF_PROG_TYPE_KPROBE` program type.

到目前为止，我已经展示了一些示例，其中eBPF程序附加到内核源代码中定义的事件。在用户空间代码中也有类似的附加点：uprobes和uretprobes用于附加到用户空间函数的入口和出口，以及用户静态定

义的跟踪点（USDTs）用于附加到应用程序代码或用户空间库中指定的跟踪点。所有这些都使用BPF_PROG_TYPE_KPROBE程序类型。

Note

There are lots of public examples of programs attached to user space events. Here are a few from the BCC project:

有很多公共示例程序与用户空间事件相关联。以下是来自BCC项目中的一些示例：

- The [bashreadline](#) and [funclatency tools](#) attach to u(ret)probe.

bashreadline和funclatency工具附加到u（ret）probe。

- [USDT sample](#) in BCC.

USDT样本在BCC中。

If you're using *libbpf*, the `SEC()` macro lets you define the auto-attachment point for these user space probes. You'll find the format required for the section name in the [libbpf documentation](#). For example, to attach a uprobe to the start of the `SSL_write()` function in OpenSSL, you would define the section for the eBPF program with the following:

如果您正在使用libbpf，SEC（）宏可以让您为这些用户空间探针定义自动附加点。您可以在libbpf文档中找到所需的节名称格式。例如，要附加一个uprobe到OpenSSL中SSL_write（）函数的开头，您可以使用以下方式定义eBPF程序的节：

```
SEC(  
"uprobe/usr/lib/aarch64-linux-gnu/libssl.so.3/SSL_write")
```

There are a few gotchas to be aware of when instrumenting user space code:

在为用户空间代码进行仪器化操作时需要注意一些注意事项：

- Notice that the path to this shared library in this example is architecture specific, so you may need corresponding architecture-specific definitions.

请注意，此示例中共享库的路径是与特定体系结构相关的，因此您可能需要相应的体系结构特定定义。

- Unless you control the machine you're running the code on, you can't know what user space libraries and applications will be installed.

除非你控制代码运行的机器，否则你无法知道用户空间库和应用程序会安装什么。

- An application might be built as a standalone binary, so it won't hit any probes you might attach within shared libraries.

一个应用程序可以作为独立的二进制文件来构建，因此它不会触发您可能在共享库中连接的任何探测点。

- Containers typically run with their own copy of a filesystem, with their own set of dependencies installed in it. The path to a shared library used by a container won't be the same as the path to a shared library on the host machine.

容器通常会使用自己的文件系统运行，并安装了自己的依赖项。容器使用的共享库路径与主机上的共享库路径不同。

- Your eBPF program might need to be aware of the language in which an application was written. For example, in C the arguments to a function are generally passed using registers, but in Go they are passed using the stack,³ so the `pt_args` structure holding register information may be of less use.

您的eBPF程序可能需要了解应用程序使用的语言。例如，在C语言中，函数的参数通常是通过寄存器传递的，但在Go语言中，参数是通过栈传递的，因此包含寄存器信息的`pt_args`结构可能没有太大用处。

That said, there are lots of useful tools that instrument user space applications with eBPF. For example, you can hook into the SSL library to trace out decrypted versions of encrypted information—we'll explore this in more detail in the next chapter. Another example is continuous profiling of your applications, using tools such as [Parca](#).

有很多有用的工具可用于使用eBPF仪器化用户空间应用程序。例如，您可以钩入SSL库以跟踪解密后的加密信息的版本-我们将在下一章中详细探讨这一点。另一个例子是使用像Parca这样的工具对您的应用程序进行持续性分析。

LSM

主观地说，`LSM` 在简体中文中没有固定的意思。

`BPF_PROG_TYPE_LSM` programs are attached to the *Linux Security Module (LSM) API*, which is a stable interface within the kernel originally intended for kernel modules to use to enforce security policies. As you'll see in [Chapter 9](#), where I'll discuss this in more detail, eBPF security tooling can now use this interface too.

“BPF_PROG_TYPE_LSM程序附加到Linux安全模块（LSM）API上，该API是内核中的一个稳定接口，最初旨在供内核模块使用以强制执行安全策略。正如您将在第9章中看到的更详细讨论，eBPF安全工具现在也可以使用这个接口。”

`BPF_PROG_TYPE_LSM` programs are attached using `bpf(BPF_RAW_TRACEPOINT_OPEN)`, and in many ways they are treated like tracing programs. One interesting characteristic of `BPF_PROG_TYPE_LSM` programs is that the return value affects the way the kernel behaves. A nonzero return code indicates that the security check wasn't passed, so the kernel won't proceed with whatever operation it was asked to complete. This is a significant difference from perf-related program types where the return code is ignored.

BPF_PROG_TYPE_LSM程序是使用 `bpf(BPF_RAW_TRACEPOINT_OPEN)` 附加的，以多种方式处理它们就像跟踪程序一样。BPF_PROG_TYPE_LSM程序的一个有趣特点是其返回值影响内核的行为方式。非零返回代码表示安全检查未通过，因此内核不会继续完成请求的操作。这与与perf相关的程序类型明显不同，后者会忽略返回代码。

Note

The Linux kernel documentation covers [LSM BPF programs](#).

Linux内核文档涵盖了LSM BPF程序。

The LSM program type isn't the only one with a role to play in security. Many of the networking-related program types that you'll see in the next section can be used for network security to permit or deny networking traffic or networking-related operations. You'll also see more about eBPF being used for security purposes in [Chapter 9](#).

LSM程序类型并不是唯一扮演安全角色的类型。在下一节中，你会看到许多与网络相关的程序类型可以用于网络安全，以允许或拒绝网络流量或与网络相关的操作。在第9章中，你还会了解到更多关于eBPF在安全方面的应用。

So far in this chapter you have seen how a set of kernel and user space tracing program types enable visibility over the whole system. The next set of eBPF program types to consider are those that let us hook into the network stack, with the option not merely to observe but also to affect how it handles data being sent and received.

到目前为止，在本章中你已经看到了一组内核和用户空间跟踪程序类型如何使整个系统可见。下一组要考虑的eBPF程序类型是让我们能够钩入网络堆栈，不仅仅观察，还能影响它处理发送和接收数据的方式。

Networking

网络化

There are lots of different eBPF program types intended to process network messages as they pass through various points in the network stack. [Figure 7-1](#) shows where some of the commonly used program types attach. These program types all require `CAP_NET_ADMIN` and `CAP_BPF`, or `CAP_SYS_ADMIN`, capabilities to be permitted.

有许多不同类型的eBPF程序，旨在在网络堆栈的各个点上通过处理网络消息。图7-1显示了一些常用的程序类型连接的位置。所有这些程序类型都需要被允许具备CAP_NET_ADMIN和CAP_BPF或CAP_SYS_ADMIN的能力。

The context passed to these types of programs is the network message in question, although the type of structure depends on the data the kernel has at the relevant point in the network stack. At the bottom of the stack, data is held in the form of Layer 2 network packets, which are essentially a series of bytes that have been or are ready to be transmitted “on the wire.” At the top of the stack, applications use sockets, and the kernel creates socket buffers to handle data being sent and received from these sockets.

这些类型的程序接收到的上下文是网络消息，在网络堆栈的相关点，所附带的数据结构类型取决于内核所持有的数据。在堆栈底部，数据以第二层网络数据包的形式保存，它们实质上是一系列的字节，已经或准备传输到网络上。在堆栈顶部，应用程序使用套接字，内核会创建套接字缓冲区来处理从这些套接字发送和接收的数据。

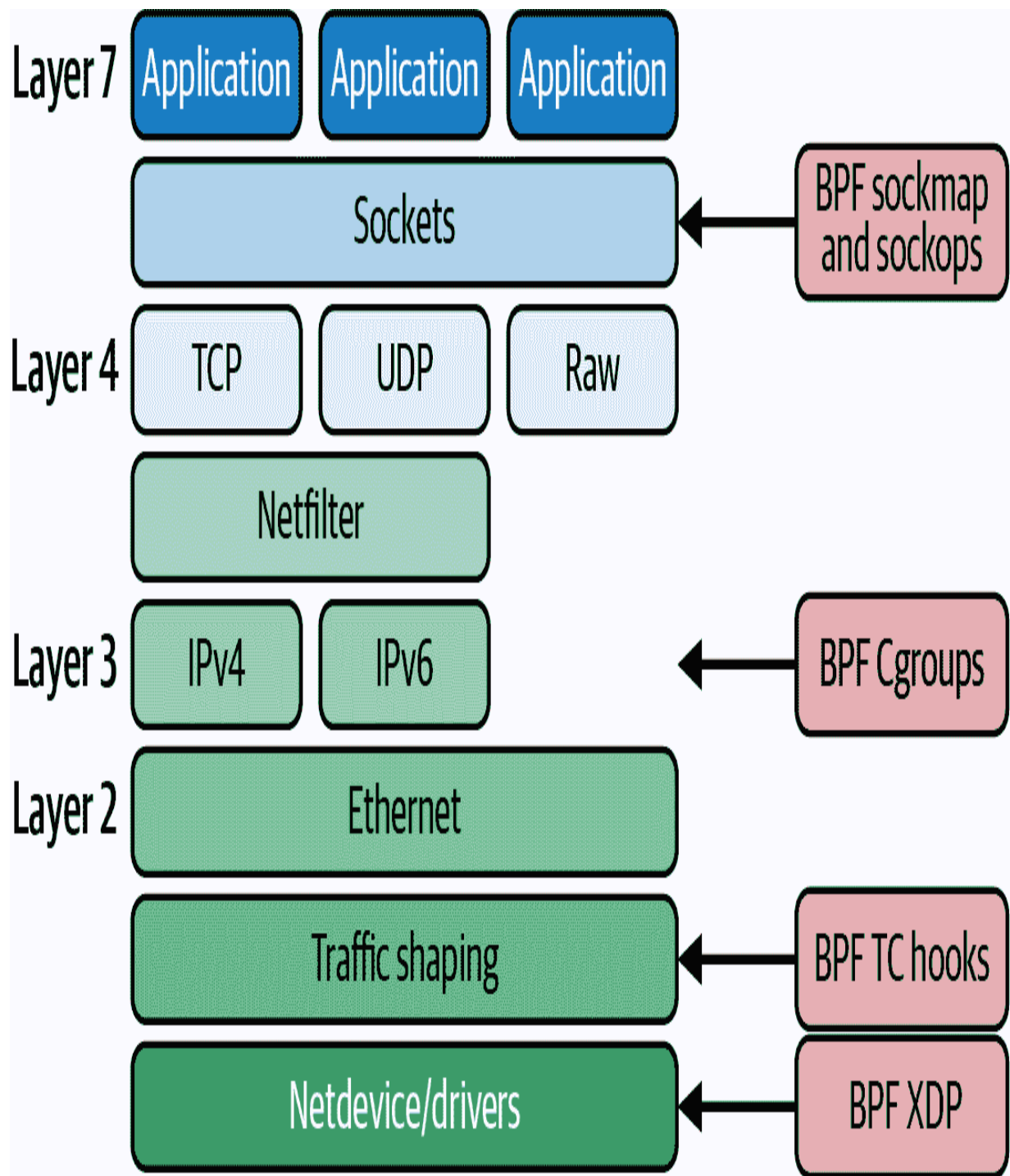


Figure 7-1. BPF program types hook into various points in the network stack

Note

The network layer model is beyond the scope of this book, but it's covered in many other books, posts, and training courses. I discussed it in [Chapter 10](#) of *Container Security* (O'Reilly). For the purposes of this book, it's sufficient to know that Layer 7 covers formats intended for applications to use, such as HTTP, DNS, or gRPC; TCP is at Layer 4; IP is at Layer 3; and Ethernet and WiFi are at Layer 2. One of the roles of the networking stack is to convert messages between these different formats.

网络层模型超出了本书的范围，但在许多其他书籍、帖子和培训课程中都有涵盖。我在《容器安全》（O'Reilly）的第10章中进行了讨论。对于本书来说，只需要知道第7层涵盖了应用程序所使用的格式，如HTTP、DNS或gRPC；TCP位于第4层；IP位于第3层；以太网和WiFi位于第2层即可。网络堆栈的一个作用是在这些不同的格式之间转换消息。

One big difference between the networking program types and the tracing-related types you saw earlier in this chapter is that they are generally intended to allow for the customization of networking behaviors. That involves two main characteristics:

网络程序类型和先前在本章中看到的与追踪相关的类型之间一个重要的区别是它们通常旨在允许定制网络行为。这涉及到两个主要特征：

1. Using a return code from the eBPF program to tell the kernel what to do with a network packet—which could involve processing it as usual, dropping it, or redirecting it to a different destination

使用eBPF程序的返回代码告诉内核如何处理网络数据包，包括常规处理、丢弃或将其重定向到不同的目标。

2. Allowing the eBPF program to modify network packets, socket configuration parameters, and so on

允许eBPF程序修改网络数据包、套接字配置参数等。

You'll see some examples of how these characteristics are used to build powerful networking capabilities in the next chapter, but for now, here's an overview of the eBPF program types.

在下一章中，您将看到一些示例，说明这些特性如何用于构建强大的网络功能。但现在，这里是eBPF程序类型的概述。

Sockets

套接字

At the top of the stack, a subset of these network-related program types relates to sockets and socket operations:

在堆栈的顶部，这些与网络相关的程序类型的子集与套接字和套接字操作有关。

- `BPF_PROG_TYPE_SOCKET_FILTER` was the first program type to be added to the kernel. You probably guessed from the name that this is used for socket filtering, but what's less obvious is that this doesn't mean filtering data being sent to or from an application. It's used to filter a *copy* of socket data that can be sent to an observability tool such as tcpdump.

BPF_PROG_TYPE_SOCKET_FILTER是第一个添加到内核中的程序类型。你可能从名称猜到它是用于套接字过滤，但不太明显的是，这并不意味着过滤发送到或从应用程序的数据。它用于过滤可以发送到可观察性工具（例如tcpdump）的套接字数据的副本。

- A socket is specific to a Layer 4 (TCP) connection.

`BPF_PROG_TYPE SOCK_OPS` allows eBPF programs to intercept various operations and actions that take place on a socket, and to set for that socket parameters such as TCP timeout values. Sockets only exist at the endpoints for a connection, and not on any middleboxes that they might pass through.

套接字是特定于第4层（TCP）连接的。

BPF_PROG_TYPE SOCK_OPS允许eBPF程序拦截在套接字上进行的各种操作和动作，并为该套接字设置参数，例如TCP超时值。套接字仅存在于连接的端点上，而不会存在于它们可能经过的任何中间设备上。

- `BPF_PROG_TYPE SK_SKB` programs are used in conjunction with a special map type that holds a set of references to sockets to provide what's known as [sockmap operations](#): redirecting traffic to different destinations at the socket layer.

• BPF_PROG_TYPE SK_SKB程序与一个特殊的地图类型一起使用，该地图类型持有一组对套接字的引用，提供所谓的sockmap操作：在套接字层面上将流量重定向到不同的目的地。

Traffic Control

交通管制

Further down the network stack comes “TC” or traffic control. There is a whole subsystem in the Linux kernel related to TC, and a glance at the [manpage for the `tc` command](#) will give you an idea of how complex it is and how important it is to computing in general to have deep levels of flexibility and configuration over the way network packets are handled.

网络堆栈的更低层次是“流量控制”或者称为TC。Linux内核中有一个完整的子系统与TC相关，查看tc命令的手册可以让你了解到它的复杂性，也可以明白在计算领域中拥有深层次的灵活性和配置能力对于处理网络数据包的方式来说是多么重要。

eBPF programs can be attached to provide custom filters and classifiers for network packets for both ingress and egress traffic. This is one of the building blocks of the Cilium project, and I’ll cover some examples in the next chapter. If you can’t wait until then, there are some good examples on [Quentin Monnet’s blog](#). This can be done programmatically, but you also have the option to use the `tc` command to manipulate these kinds of eBPF programs.

eBPF程序可以附加以提供自定义的入站和出站网络数据包过滤器和分类器。这是Cilium项目的一个基础模块，我将在下一章节中介绍一些示例。如果你不能等到那时，Quentin Monnet的博客上有一些很好的例子。这可以通过编程的方式来完成，但你也可以选择使用tc命令来操作这些类型的eBPF程序。

XDP

XDP translates to "XDP" in simplified Chinese. There is no translation available for this term.

You briefly met XDP (eXpress Data Path) eBPF programs in [Chapter 3](#). In that example I loaded the eBPF program and attached it to the `eth0` interface using the following commands:

在第三章中，你简要地接触到了XDP（eXpress Data Path）eBPF程序。在那个例子中，我使用以下命令加载了eBPF程序，并将其附加到eth0接口上。

```
bpftool prog load hello.bpf.o /sys/fs/bpf/hello
bpftool net attach xdp id 540 dev eth0
```

It's worth noting that XDP programs attach to a specific interface (or virtual interface), and you may very well have different XDP programs attached to different interfaces. In [Chapter 8](#) you'll learn more about how XDP programs can be offloaded to network cards or executed by network drivers.

值得注意的是XDP程序附加到特定的接口（或虚拟接口），你可能会在不同的XDP程序附加到不同的接口。在第8章中，你将了解更多关于如何将XDP程序卸载到网络卡或由网络驱动程序执行的内容。

XDP programs are another example of programs that can be managed using Linux network utilities—in this case, the `link` subcommand of [iproute2's ip](#). The roughly equivalent command for loading and attaching the program to `eth0` would be this:

XDP程序是可以使用Linux网络实用工具进行管理的程序的另一个示例，这里是使用iproute2的ip命令的链路子命令。加载并将程序附加到

eth0的大致等效命令如下：

```
$ ip link set dev eth0 xdp obj hello.bpf.o sec xdp
```

This command reads the eBPF program marked as section `xdp` from the `hello.bpf.o` object and attaches it to the `eth0` network interface. The `ip link show` command for this interface now includes some information about the XDP program that's attached to it:

这个命令从hello.bpf.o对象中读取标记为section xdp的eBPF程序，并将其附加到eth0网络接口上。现在，使用ip link show命令查看这个接口时，会包含一些关于附加到它的XDP程序的信息。

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
xdpgeneric qdisc fq_codel
state UP mode DEFAULT group default qlen 1000
    link/ether 52:55:55:3a:1b:a2 brd ff:ff:ff:ff:ff:ff
    prog/xdp id 1255 tag 9d0e949f89f1a82c jited
```

Removing the XDP program with `ip link` can be done like this:

用ip link移除XDP程序可以像这样完成：

```
$ ip link set dev eth0 xdp off
```

You'll see a lot more about XDP programs and their applications in the next chapter.

在下一章中，你会看到关于XDP程序及其应用的更多内容。

Flow Dissector

流分析器

A flow dissector is used at various points in the network stack to extract details from a packet's headers. eBPF programs of type `BPF_PROG_TYPE_FLOW_DISSECTOR` can implement custom packet dissection. There's a nice write-up in this LWN article on [writing network flow dissectors in BPF](#).

流解剖器在网络栈的各个点使用，以提取数据包头部的详细信息。`BPF_PROG_TYPE_FLOW_DISSECTOR` 类型的 eBPF 程序可以实现自定义数据包解剖。LWN 的一篇文章中有关于使用 BPF 编写网络流解剖器的详细介绍。

Lightweight Tunnels

轻量级隧道

The family of `BPF_PROG_TYPE_LWT_*` program types can be used to implement network encapsulation in eBPF programs. These program types can also be manipulated using the `ip` command, but this time it's the `route` subcommand that's involved. In practice, these are used infrequently.

`BPF_PROG_TYPE_LWT_*` 程序类型的家族可用于在 eBPF 程序中实现网络封装。这些程序类型也可以使用 `ip` 命令进行操作，但这次涉及的是 `route` 子命令。在实践中，这些很少使用。

Cgroups

Cgroups -> 控制组

eBPF programs can be attached to cgroups (short for “control groups”). *Cgroups* are a concept in the Linux kernel that restricts the

set of resources a given process or group of processes can have access to. Cgroups are one of the mechanisms that isolate one container (or one Kubernetes pod) from another. Attaching eBPF programs to a cgroup allows for custom behavior that only applies to that cgroup's processes. All processes are associated with a cgroup, including processes that are not running inside a container.

eBPF程序可以附加到cgroups（即“控制组”）。Cgroups是Linux内核中的一个概念，它限制了给定进程或一组进程可以访问的资源集合。Cgroups是隔离一个容器（或一个Kubernetes pod）与另一个容器的机制之一。将eBPF程序附加到一个cgroup允许自定义行为，仅适用于该cgroup的进程。所有进程都与一个cgroup相关联，包括不在容器内运行的进程。

There are several cgroup-related program types, and even more hooks where they can be attached. At least at the time of this writing, they are nearly all networking related, although there is also a `BPF_CGROUP_SYSCTL` program type that can be attached to `sysctl` commands affecting a particular cgroup.

有几种与cgroup相关的程序类型，以及更多可以附加它们的钩子。至少在撰写本文时，它们几乎都与网络相关，尽管还有一种名为 `BPF_CGROUP_SYSCTL` 的程序类型，可以附加到影响特定cgroup的 `sysctl` 命令上。

As an example, there are socket-related program types specific to cgroups `BPF_PROG_TYPE_CGROUP_SOCK` and `BPF_PROG_TYPE_CGROUP_SKB`. eBPF programs can determine whether a given cgroup is permitted to perform a requested socket operation

or data transmission. This is useful for network security policy enforcement (which I'll cover in the next chapter). Socket programs can also trick the calling process into thinking they are connecting to a particular destination address.

作为一个例子，有一些与控制组(cgroups)相关的程序类型，特别是与 `BPF_PROG_TYPE_CGROUP_SOCK` 和 `BPF_PROG_TYPE_CGROUP_SKB` 相关的类型。eBPF程序可以确定一个给定的控制组是否被允许执行所请求的套接字操作或数据传输。这在网络安全策略执行方面非常有用（我将在下一章中介绍）。套接字程序还可以欺骗调用进程，使其认为自己正在连接到特定的目标地址。

Infrared Controllers

红外控制器

Programs of type `BPF_PROG_TYPE_LIRC_MODE2` can be attached to the file descriptor for an infrared controller device to provide decoding for infrared protocols. At the time of this writing, this program type requires `CAP_NET_ADMIN`, but I think this illustrates that the division of program types into tracing related and networking related doesn't fully express the range of different applications that eBPF can address.

类型为 `BPF_PROG_TYPE_LIRC_MODE2` 的程序可以附加到红外控制器设备的文件描述符上，以提供红外协议的解码功能。在撰写本文时，此程序类型需要 `CAP_NET_ADMIN` 权限，但我认为这表明将程序

类型分为与跟踪相关和与网络相关并不能完全表达eBPF能够解决的不同应用领域的范围。

BPF Attachment Types

BPF附件类型

The attachment type offers more fine-grained control over where a program can be attached in the system. For some program types there is a one-to-one correlation to the type of hook that it can be attached to, so the attachment type is implicitly defined by the program type. For example, XDP programs are attached to XDP hooks in the network stack. For a few program types, an attachment type also has to be specified.

附件类型提供了对程序可以附加到系统的位置有更细粒度的控制。对于某些程序类型，与它可以附加到的钩子类型是一一对应的，因此附件类型是由程序类型隐式定义的。例如，XDP程序附加到网络堆栈中的XDP钩子上。对于少数程序类型，还必须指定附件类型。

The attachment type is involved in deciding which helper functions are valid, and it also restricts access to parts of the context information in some cases. There was an example of this earlier in this chapter where the verifier gives an `invalid bpf_context access` error.

附件类型参与决定哪些辅助函数是有效的，同时在某些情况下限制对上下文信息的访问。在本章的早些时候有一个例子，其中验证器给出了一个无效的 `bpf_context` 访问错误。

You can also see which program types need an attachment type to be specified, and which attachment types are valid, in the kernel function [bpf_prog_load_check_attach](#) (defined in [bpf/syscall.c](#)).

在内核函数**bpf_prog_load_check_attach**（在**bpf/syscall.c**中定义）中，您还可以看到需要指定附件类型的程序类型，以及哪些附件类型是有效的。

For example, here is the code that checks the attachment type for a program of type `CGROUP_SOCKET`:

例如，这里是检查类型为**CGROUP_SOCKET**的程序的附件类型的代码：

```
case BPF_PROG_TYPE_CGROUP_SOCKET:
    switch (expected_attach_type) {
        case BPF_CGROUP_INET_SOCKET_CREATE:
        case BPF_CGROUP_INET_SOCKET_RELEASE:
        case BPF_CGROUP_INET4_POST_BIND:
        case BPF_CGROUP_INET6_POST_BIND:
            return 0;
        default:
            return -EINVAL;
    }
```

This program type can be attached in multiple places: at socket creation, at socket release, or after a bind is completed in IPv4 or IPv6.

此程序类型可附加在多个地方：在套接字创建时，套接字释放时，或在IPv4或IPv6绑定完成后。

Another place to find a listing of the valid attachment types for programs is the [libbpf documentation](#), where you'll also find the

section names that *libbpf* understands for each program and attachment type.

在 *libbpf* 文档中可以找到程序的有效附件类型列表，您还可以找到 *libbpf* 对每个程序和附件类型理解的部分名称。

Summary

摘要

In this chapter you saw that various eBPF program types are used to attach into different hook points in the kernel. If you want to write code that responds to a particular event, you'll need to determine the program type(s) that are appropriate for hooking onto that event. The context passed into the program depends on the program type, and the kernel may also respond differently to the return code from your program, depending on its type.

在本章中，您看到了不同的eBPF程序类型被用于附加到内核中的不同钩子点上。如果您想编写代码来响应特定事件，您需要确定适合挂钩到该事件的程序类型。传递到程序中的上下文取决于程序类型，内核也可能根据程序的类型对您程序的返回代码做出不同的响应。

The example code for this chapter mostly focused on perf-related (tracing) events. In the next two chapters you'll see more details on different eBPF program types used for networking and security applications.

本章的示例代码主要关注于与性能相关的（追踪）事件。在接下来的两章中，您将看到有关用于网络和安全应用的不同 eBPF 程序类型的

更多细节。

Exercises

练习

The example code for this chapter includes kprobe, fentry, tracepoint, raw tracepoint, and BTF-enabled tracepoint programs that are all attached to the entry to the same system call. As you know, eBPF tracing programs can be attached to many other places besides syscalls.

本章的示例代码包括kprobe、fentry、tracepoint、原始tracepoint和启用了BTF的tracepoint程序，它们都附加到同一个系统调用的入口点上。如你所知，eBPF跟踪程序不仅可以附加到系统调用上，还可以附加到许多其他地方。

1. Run the example code using `strace` to capture the `bpf()` system calls, like this:

使用strace来捕获bpf()系统调用的示例代码，并按照以下方式运行：

```
strace -e bpf -o outfile ./hello
```

This will record information about each `bpf()` syscall into a file called *outfile*. Look for the `BPF_PROG_LOAD` instructions in that file, and see how the `prog_type` field varies for different programs. You can identify which program is which by the `prog_name` field in the trace, and match them to the source code in *chapter7/hello.bpf.c*.

这将会将每个bpf()系统调用的信息记录到一个名为outfile的文件中。在该文件中查找BPF_PROG_LOAD指令，并查看不同程序的prog_type文件如何变化。您可以通过追踪中的prog_name字段来确定每个程序，然后将其与第7章中的源代码hello.bpf.c匹配。

2. The example user space code in *hello.c* loads all the program objects defined in `hello.bpf.o`. As an exercise in writing *libbpf* user space code, modify the example code load and attach just one of the eBPF programs (pick whichever one you like), without removing those programs from *hello.bpf.c*.

在hello.c中，示例的用户空间代码加载了hello.bpf.o中定义的所有程序对象。作为编写libbpf用户空间代码的练习，修改示例代码，只加载并附加一个eBPF程序（选择您喜欢的任何一个），而不从hello.bpf.c中删除这些程序。

3. Write a kprobe and/or fentry program that is triggered when some other kernel function is called. You can find the available functions in your kernel version by looking at */proc/kallsyms*.

编写一个kprobe和/或fentry程序，在调用其他内核函数时触发。您可以通过查看/proc/kallsyms来查找您内核版本中可用的函数。

4. Write a regular, raw or BTF-enabled tracepoint program that attaches to some other kernel tracepoint. You can find the available tracepoints in `/sys/kernel/tracing/available_events`.

编写一个常规的、原始的或者启用了BTF的跟踪点程序，将其附加到其他内核跟踪点上。可以在/sys/kernel/tracing/available_events中找到可用的跟踪点。

5. Try to attach more than one XDP program to a given interface, and confirm that you can't! You should see an error that looks something like this:

试着将多个XDP程序附加到给定的接口上，并确认您无法这样做！您应该会看到类似如下的错误信息：

```
libbpf: Kernel error message: XDP program already
attached
Error: interface xdpgeneric attach failed: Device or
resource busy
```

- ¹ Except for a few parts of the kernel where kprobes aren't permitted for security reasons. These are listed in

`/sys/kernel/debug/kprobes/blacklist`.

除了出于安全原因在内核的一些部分不允许使用kprobes之外。这些在/sys/kernel/debug/kprobes/blacklist中列出。

- ² The only example I have seen so far is in the [cilium/ebpf test suite](#).

到目前为止，我见过的唯一一个例子是在纤毛/ebpf测试套件中。

- ³ Up to Go version 1.17, when a new register-based calling convention was introduced. Nevertheless, I think there will be Go executables built with older versions circulating for some time to come.

在Go的1.17版本之前，引入了一种新的基于寄存器的调用约定。尽管如此，我认为将来还会有使用旧版本构建的Go可执行文件在流通。

Chapter 8. eBPF for Networking

第8章。eBPF用于网络

As you saw in [Chapter 1](#), the dynamic nature of eBPF allows us to customize the behavior of the kernel. In the world of networking, there is a huge range of desirable behavior that depends on the application. For example, a telecommunications operator might have to interface with telco-specific protocols like SRv6; a Kubernetes environment might need to be integrated with legacy applications; dedicated hardware load balancers can be replaced with XDP programs running on commodity hardware. eBPF allows programmers to build networking features to meet specific needs, without having to force them on all upstream kernel users.

正如您在第1章中所看到的，eBPF的动态特性使我们能够定制内核的行为。在网络世界中，有许多不同的应用行为可供选择。例如，电信运营商可能需要与特定于电信的协议（如SRv6）进行接口交互；Kubernetes环境可能需要与传统应用程序集成；专用硬件负载均衡器可以被运行在通用硬件上的XDP程序取代。eBPF允许程序员根据具体需求构建网络功能，而无需强制应用于所有上游内核用户。正如您在第1章中所见的，eBPF的动态特性使我们能够定制内核的行为。在网络世界中，有许多不同的应用行为可供选择。例如，电信运营商可能需要与特定于电信的协议（如SRv6）进行接口交互；Kubernetes环境可能需要与传统应用程序集成；专用硬件负载均衡器可以被运行

在通用硬件上的XDP程序取代。eBPF允许程序员根据具体需求构建网络功能，而无需强制应用于所有上游内核用户。

Network tools based on eBPF are now widely used and have proven to be effective at prolific scale. The CNCF's [Cilium project](#), for example, uses eBPF as a platform for Kubernetes networking, standalone load balancing, and much more, and it's used by cloud native adopters in every conceivable industry vertical.¹ Meta has been using eBPF at a vast scale—every packet to and from Facebook since 2017 has been through an XDP program. Another public and hyper-scaled example is Cloudflare's use of eBPF for DDoS (distributed denial-of-service) protection.

基于eBPF的网络工具现在被广泛使用，并且已经被证明在大规模部署中非常有效。例如，CNCF的Cilium项目使用eBPF作为Kubernetes网络、独立负载均衡等方面的平台，被云原生行业的采用者在各个行业垂直领域中使用。自2017年以来，Meta一直在大规模使用eBPF——所有到达和离开Facebook的数据包都经过了XDP程序。另一个公开且超大规模的例子是Cloudflare使用eBPF进行分布式拒绝服务（DDoS）保护。

These are complex, production-ready solutions, and their details are far beyond the scope of this book, but by reading the examples in this chapter you can get a feel for how eBPF networking solutions like these are built.

这些是复杂的、可供生产使用的解决方案，它们的细节远超出了本书的范围，但通过阅读本章的示例，你可以感受到像这样的eBPF网络解决方案是如何构建的。

Note

The code examples for this chapter are in the *chapter8* directory of the repository at github.com/lizrice/learning-ebpf.

本章的代码示例位于github.com/lizrice/learning-ebpf仓库的chapter8目录中。

Packet Drops

数据包丢失

There are several network security features that involve dropping certain incoming packets and allowing others. These features include firewalling, DDoS protection, and mitigating packet-of-death vulnerabilities:

有几个网络安全功能涉及丢弃某些传入数据包并允许其他数据包。这些功能包括防火墙、DDoS保护和缓解致命数据包的漏洞。

- Firewalling involves deciding on a per-packet basis whether to allow a packet, based on the source and destination IP addresses and/or port numbers.

防火墙是基于源IP地址和目的IP地址和/或端口号对每个数据包进行决策，决定是否允许通过。

- DDoS protection adds some complexity, perhaps keeping track of the rate at which packets are arriving from a particular source and/or detecting certain characteristics of the packet contents to

determine that an attacker or set of attackers is trying to flood the interface with traffic.

DDoS防护增加了一些复杂性，可能要追踪来自特定源的数据包到达的速率并/或检测数据包内容的某些特征，以确定攻击者或一组攻击者试图用大量流量淹没接口。

- A packet-of-death vulnerability is a class of kernel vulnerability in which the kernel fails to safely process a packet crafted in a particular way. An attacker who sends packets with this particular format can exploit the vulnerability, which could potentially cause the kernel to crash. Traditionally, when a kernel vulnerability like this is found, it requires installing a new kernel with the fix, which in turn requires machine downtime. But an eBPF program that detects and drops these malicious packets can be installed dynamically, instantly protecting that host without affecting any applications running on the machine.

“死亡数据包漏洞是一类内核漏洞，指的是内核在处理以特定方式构建的数据包时未能安全地处理。攻击者通过发送这种特定格式的数据包可以利用此漏洞，潜在地导致内核崩溃。传统上，当发现这样的内核漏洞时，需要安装一个带有修复程序的新内核，这又需要机器停机时间。但是，可以动态安装一个检测并丢弃这些恶意数据包的 eBPF 程序，即时保护主机，而不影响机器上运行的任何应用程序。”

The decision-making algorithms for features like these are beyond the scope of this book, but let's explore how eBPF programs

attached to the XDP hook on a network interface drop certain packets, which is the basis for implementing these use cases.

这些特性的决策算法超出了本书的范围，但让我们探讨一下如何使用附加到网络接口的eBPF程序在XDP钩子上丢弃特定数据包，这是实现这些用例的基础。

XDP Program Return Codes

XDP程序返回码

An XDP program is triggered by the arrival of a network packet. The program examines the packet, and when it's done, the return code gives a *verdict* that indicates what to do next with that packet:

一个XDP程序在网络数据包到达时被触发。该程序检查数据包，并在完成后返回一个用于指示下一步操作的决策代码。

- `XDP_PASS` indicates that the packet should be sent to the network stack in the normal way (as it would have done if there were no XDP program).

`XDP_PASS`表示该数据包应以正常方式发送到网络栈（就像没有XDP程序一样）。

- `XDP_DROP` causes the packet to be discarded immediately.

`XDP_DROP`使得数据包立即被丢弃。

- `XDP_TX` sends the packet back out of the same interface it arrived on.

XDP_TX将数据包从它到达的相同接口上重新发送出去。

- XDP_REDIRECT is used to send it to a different network interface.

XDP_REDIRECT 用于将数据包发送到不同的网络接口。

- XDP_ABORTED results in the packet being dropped, but its use implies an error case or something unexpected, rather than a “normal” decision to discard a packet.

XDP_ABORTED导致数据包丢弃，但其使用意味着错误情况或意外情况，而不是一种“正常”决策来丢弃数据包。

For some use cases (like firewalling), the XDP program simply has to decide between passing the packet on or dropping it. An outline for an XDP program that decides whether to drop packets looks something like this:

对于某些用例（如防火墙），XDP程序只需要决定是传递数据包还是丢弃数据包。决定是否丢弃数据包的XDP程序大致如下：

```
SEC("xdp")
int hello(struct xdp_md *ctx) {
    bool drop;

    drop = <examine packet and decide whether to drop it>;

    if (drop)
        return XDP_DROP;
    else
        return XDP_PASS;
}
```

An XDP program can also manipulate the packet contents, but I'll come to that later in this chapter.

XDP程序还可以操作数据包的内容，但是我会在本章稍后讲述。

XDP programs get triggered whenever an inbound network packet arrives on the interface to which it is attached. The `ctx` parameter is a pointer to an `xdp_md` structure, which holds metadata about the incoming packet. Let's see how you can use this structure to examine the packet's contents in order to reach a verdict.

每当传入的网络数据包到达与其关联的接口时，XDP程序会被触发。`ctx`参数是指向`xdp_md`结构的指针，该结构包含有关传入数据包的元数据。让我们看看如何使用这个结构来检查数据包的内容以作出决定。

XDP Packet Parsing

XDP数据包解析

Here's the definition of the `xdp_md` structure:

下面是`xdp_md` 结构体的定义：

```
struct xdp_md {
    __u32 data;
    __u32 data_end;
    __u32 data_meta;
    /* Below access go through struct xdp_rxq_info */
    __u32 ingress_ifindex; /* rxq->dev->ifindex */
    __u32 rx_queue_index; /* rxq->queue_index */

    __u32 egress_ifindex; /* txq->dev->ifindex */
};
```

Don't be fooled by the `__u32` type for the first three fields, as they are really pointers. The `data` field indicates the location in memory

where the packet starts, and `data_end` shows where it ends. As you saw in [Chapter 6](#), to pass the eBPF verifier you will have to explicitly check that any reads or writes to the packet's contents are within the range `data` to `data_end`.

不要被前三个字段的__u32类型所迷惑，因为它们实际上是指针。数据字段指示数据包在内存中的起始位置，`data_end`则表示其结束位置。正如您在第6章中所看到的，为了通过eBPF验证器，您必须明确检查对数据包内容的任何读取或写入是否在`data`到`data_end`的范围内。

There is also an area in memory ahead of the packet, between `data_meta` and `data`, for storing metadata about this packet. This can be used for coordination between multiple eBPF programs that might process the same packet at various places on its journey through the stack.

在数据（`data`）和数据元信息（`data_meta`）之间，也有一个存储有关该数据包的元数据的内存区域。这可以用于在数据包通过堆栈的不同位置时，协调多个eBPF程序对同一个数据包的处理。

To illustrate the basics of parsing a network packet, there is an XDP program called `ping()` in the example code, which will simply generate a line of trace whenever it detects a ping (ICMP) packet. Here's the code for that program:

为了说明解析网络数据包的基础知识，在示例代码中有一个名为`ping()`的XDP程序，每当它检测到一个ping（ICMP）数据包时，它将简单地生成一行跟踪。以下是该程序的代码：

```
SEC("xdp")
int ping(struct xdp_md *ctx) {
    long protocol = lookup_protocol(ctx);
    if (protocol == 1) // ICMP
    {
        bpf_printk("Hello ping");
    }
    return XDP_PASS;
}
```

You can see this program in action by following these steps:

你可以按照以下步骤实时观看这个程序的运行情况：

1. Run `make` in the *chapter8* directory. This doesn't just build the code; it also attaches the XDP program to the loopback interface (called `lo`).

在第8章目录下运行make命令。这不仅会构建代码，还会将XDP程序附加到回环接口（称为lo）。

2. Run `ping localhost` in one terminal window.

在一个终端窗口中运行"ping localhost"。

3. In another terminal window, watch the output generated in the trace pipe by running `cat /sys/kernel/tracing/trace_pipe`.

在另一个终端窗口中，通过运行 `cat /sys/kernel/tracing/trace_pipe` 命令来观察在追踪管道中生成的输出。

You should see two lines of trace being generated approximately every second, and they should look like this:

你应该看到大约每秒钟生成两行跟踪信息，他们应该是这样的：

```
ping-26622      [000] d.s11 276880.862408: bpf_trace_printk:
Hello ping
ping-26622      [000] d.s11 276880.862459: bpf_trace_printk:
Hello ping
ping-26622      [000] d.s11 276881.889575: bpf_trace_printk:
Hello ping
ping-26622      [000] d.s11 276881.889676: bpf_trace_printk:
Hello ping
ping-26622      [000] d.s11 276882.910777: bpf_trace_printk:
Hello ping
ping-26622      [000] d.s11 276882.910930: bpf_trace_printk:
Hello ping
```

There are two lines of trace per second because the loopback interface is receiving both the ping requests and the ping responses.

每秒有两行跟踪信息，因为环回接口同时接收到ping请求和ping响应。

You can easily modify this code to drop ping packets by adding a line of code to return `XDP_DROP` when the protocol matches, like this:

您可以很容易地修改这段代码，通过添加一行代码，当协议匹配时返回`XDP_DROP`，就像这样：

```
if (protocol == 1) // ICMP
{
    bpf_printk("Hello ping");
    return XDP_DROP;
}
return XDP_PASS;
```

If you try this, you'll see that output resembling the following is only generated in the trace output once per second:

如果你尝试这样做，你会发现每秒只会生成类似以下追踪输出的输出内容。


```
ping-26639    [002] d.s11 277050.589356: bpf_trace_printk:
Hello ping
ping-26639    [002] d.s11 277051.615329: bpf_trace_printk:
Hello ping
ping-26639    [002] d.s11 277052.637708: bpf_trace_printk:
Hello ping
```

The loopback interface receives a ping request, and the XDP program drops it, so the request never gets far enough through the network stack to elicit a response.

环回接口接收到ping请求，并且XDP程序将其丢弃，因此请求从未足够通过网络堆栈到达从而引发响应。

Most of the work in this XDP program is being done in a function called `lookup_protocol()` that determines the Layer 4 protocol type. It's just an example, not a production-quality implementation of parsing a network packet! But it's sufficient to give you an idea of how parsing in eBPF works.

这个XDP程序中大部分的工作都在一个叫做`lookup_protocol()`的函数中完成，它确定了第四层协议的类型。这只是一个例子，不是一个用于生产环境的解析网络数据包的实现！但它足以让你了解eBPF中解析工作的原理。

The network packet that has been received consists of a string of bytes that are laid out as shown in [Figure 8-1](#).

接收到的网络数据包由按照图8-1所示排列的一串字节组成。

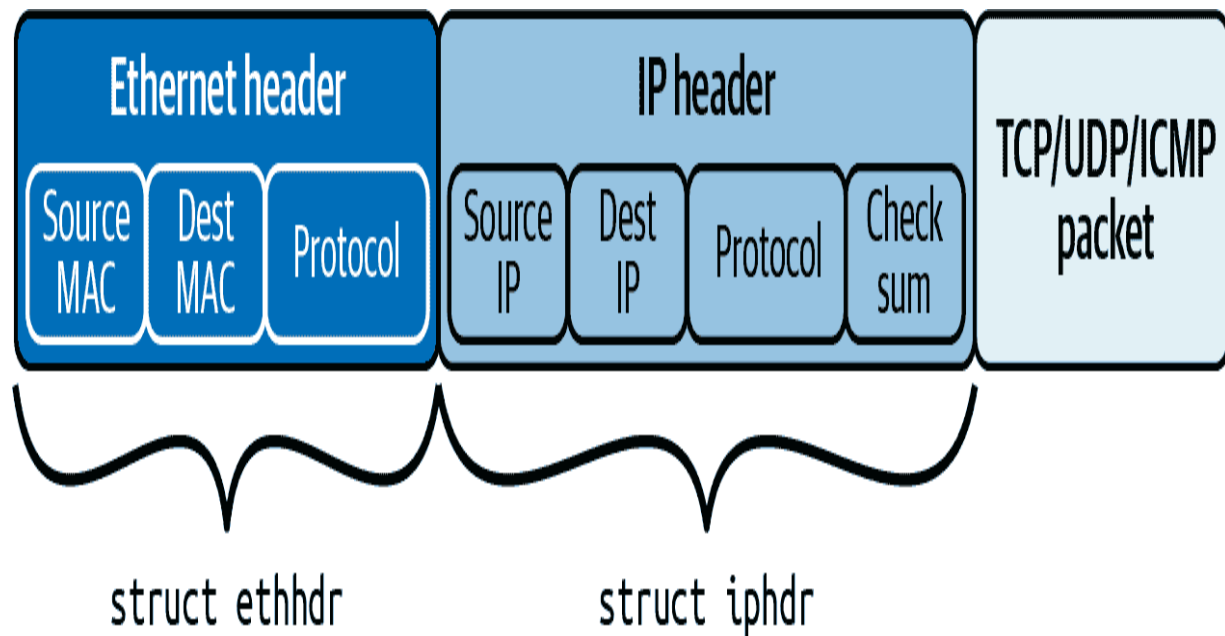


Figure 8-1. Layout of an IP network packet, starting with an Ethernet header, followed by an IP header, and then the Layer 4 data

The `lookup_protocol()` function takes the `ctx` structure that holds information about where this network packet is in memory and returns the protocol type that it finds in the IP header. The code is as follows:

`lookup_protocol()` 函数接受包含有关网络数据包在内存中位置信息的 `ctx` 结构，并返回它在 IP 头中找到的协议类型。代码如下所示：

```
unsigned char lookup_protocol(struct xdp_md *ctx)
{
    unsigned char protocol = 0;

    void *data = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;
    struct ethhdr *eth = data;
```

①

②

```

    if (data + sizeof(struct ethhdr) > data_end)
        return 0;

    // Check that it's an IP packet
    if (bpf_ntohs(eth-
>h_proto) == ETH_P_IP)
    {
        // Return the protocol of this packet
        // 1 = ICMP
        // 6 = TCP
        // 17 = UDP
        struct iphdr *iph = data + sizeof(struct ethhdr);

        if (data + sizeof(struct ethhdr) + sizeof(struct
iphdr) <= data_end)
            protocol = iph-
>protocol;
    }
    return protocol;
}

```

❶

The local variables `data` and `data_end` point to the start and end of the network packet.

❷

The network packet should start with an Ethernet header.

❸

But you can't simply assume this network packet is big enough to hold that Ethernet header! The verifier requires that you check this explicitly.

❹

The Ethernet header contains a 2-byte field that tells us the Layer 3 protocol.

5

If the protocol type indicates that it's an IP packet, the IP header immediately follows the Ethernet header.

6

You can't just assume there's enough room for that IP header in the network packet. Again the verifier requires that you check explicitly.

7

The IP header contains the protocol byte the function will return to its caller.

The `bpf_ntohs()` function used by this program ensures that the two bytes are in the order expected on this host. Network protocols are big-endian, but most processors are little-endian, meaning they hold multibyte values in a different order. This function converts (if necessary) from network ordering to host ordering. You should use this function whenever you extract a value from a field in a network packet that's more than one byte long.

这个程序中使用的**bpf_ntohs()**函数确保两个字节的顺序与主机上预期的顺序相同。网络协议使用大端序（Big-endian），但是大多数处理器是小端序（Little-endian），这意味着它们以不同的顺序保存多字节值。这个函数将（如果需要）从网络字节序转换为主机字节序。在从

网络数据包的字段中提取长度超过一个字节的值时，应该使用这个函数。

The simple example here shows how just a few lines of eBPF code can have a dramatic impact on networking functionality. It's not hard to imagine how more complex rules about which packets to pass and which packets to drop could result in the features I described at the start of this section: firewalling, DDoS protection, and packet-of-death vulnerability mitigation. Now let's consider how even more functionality can be provided given the power to modify network packets within eBPF programs.

这里的简单示例显示了几行eBPF代码如何对网络功能产生重大影响。很容易想象，关于通过哪些数据包和丢弃哪些数据包的更复杂规则可能会导致我在本节开头描述的功能：防火墙、DDoS保护和包死亡漏洞缓解。现在让我们考虑一下在eBPF程序中修改网络数据包的强大能力下，可以提供更多功能。

Load Balancing and Forwarding

负载均衡和转发

XDP programs aren't limited to inspecting the contents of a packet. They can also modify the packet's contents. Let's consider what's involved if you want to build a simple load balancer that takes packets sent to a given IP address and fans those requests to a number of backends that can fulfill the request.

XDP程序不仅仅局限于检查数据包的内容，还可以修改数据包的内容。如果你想构建一个简单的负载均衡器，将发送到特定IP地址的数据包分发给可以满足请求的多个后端服务器，那么我们来考虑一下所需的步骤。

There's an example of this in the GitHub repo.² The setup here is a set of containers that run on the same host. There's a client, a load balancer, and two backends, each running in their own container. As illustrated in [Figure 8-2](#), the load balancer receives traffic from the client and forwards it to one of the two backend containers.

在GitHub repo里有一个例子。这里的设置是一组在同一主机上运行的容器。有一个客户端，一个负载均衡器，以及两个后端，每个后端都在自己的容器中运行。正如图8-2所示，负载均衡器接收来自客户端的流量，并将其转发到两个后端容器中的一个。

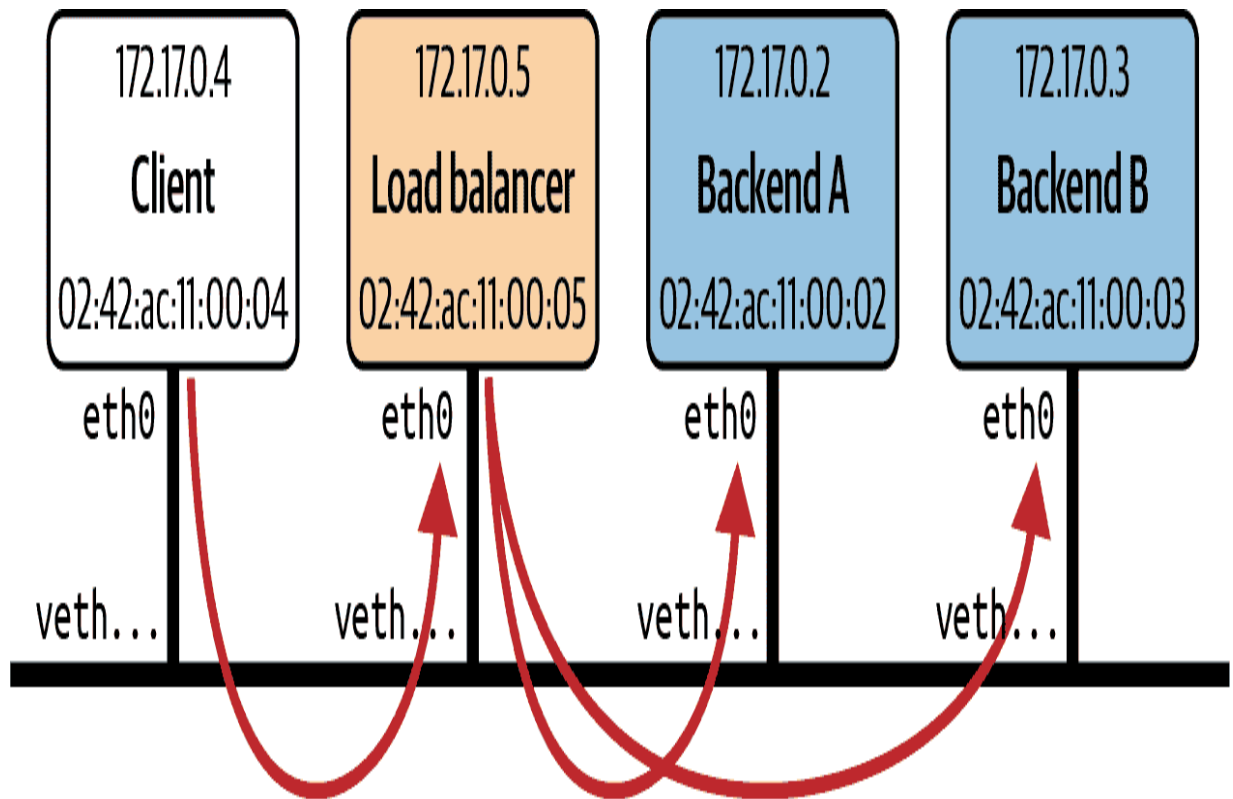


Figure 8-2. Example load balancer setup

The load balancing function is implemented as an XDP program attached to the load balancer's `eth0` network interface. The return code from this program is `XDP_TX`, indicating that the packet should be sent back out of the interface it came in on. But before that happens, the program has to update the address information in the packet headers.

负载均衡功能作为附加到负载均衡器的`eth0`网络接口的XDP程序实现的。这个程序的返回码是`XDP_TX`，表示应将数据包发送回原来进来的接口。但在这之前，程序必须更新数据包头部的地址信息。

Although I think it's useful as a learning exercise, this example code is very, very far from being production ready; for example, it uses

hard-coded addresses that assume the exact setup of IP addresses shown in [Figure 8-2](#). It assumes that the only TCP traffic it will ever receive is requests from the client or responses to the client. It also cheats by taking advantage of the way Docker sets up virtual MAC addresses, using each container's IP address as the last four bytes of the MAC address for the virtual Ethernet interface for each container. That virtual Ethernet interface is called eth0 from the perspective of the container.

尽管我认为这个示例代码作为学习练习很有用，但它离发布准备还相差很远；例如，它使用了硬编码的地址，假设了与图8-2中显示的IP地址设置完全相同。它假设它将只接收来自客户端的请求或回应。它还通过利用Docker设置虚拟MAC地址的方式进行了作弊，使用每个容器的IP地址作为每个容器的虚拟以太网接口的MAC地址的最后四个字节。从容器的角度来看，该虚拟以太网接口被称为eth0。

Here's the XDP program from the example load balancer code:

这是负载均衡代码示例中的XDP程序。

```
SEC("xdp_lb")
int xdp_load_balancer(struct xdp_md *ctx)
{
    void *data = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if (data + sizeof(struct ethhdr) > data_end)
        return XDP_ABORTED;

    if (bpf_ntohs(eth->h_proto) != ETH_P_IP)
        return XDP_PASS;

    struct iphdr *iph = data + sizeof(struct ethhdr);
    if (data + sizeof(struct ethhdr) + sizeof(struct iphdr)
```



```

) > data_end)
    return XDP_ABORTED;

    if (iph->protocol != IPPROTO_TCP)                ❷
        return XDP_PASS;

    if (iph->saddr == IP_ADDRESS(CLIENT))              ❸
    {
        char be = BACKEND_A;                          ❹
        if (bpf_get_prandom_u32() % 2)
            be = BACKEND_B;

        iph->daddr = IP_ADDRESS(be);                  ❺
        eth->h_dest[5] = be;
    }
    else
    {
        iph->daddr = IP_ADDRESS(CLIENT);              ❻
        eth->h_dest[5] = CLIENT;
    }
    iph->saddr = IP_ADDRESS(LB);                      ❼
    eth->h_source[5] = LB;

    iph->check = iph_csum(iph);                      ❽

    return XDP_TX;
}

```

❶

The first part of this function is practically the same as in the previous example: it locates the Ethernet header and then the IP header in the packet.

❷

This time it will process only TCP packets, passing anything else it receives on up the stack as if nothing had happened.

❸

Here the source IP address is checked. If this packet didn't come from the client, I will assume it is a response going to the client.

④

This code generates a pseudorandom choice between backends A and B.

⑤

The destination IP and MAC addresses are updated to match whichever backend was chosen...

⑥

...or if this is a response from a backend (which is the assumption here if it didn't come from a client), the destination IP and MAC addresses are updated to match the client.

⑦

Wherever this packet is going, the source addresses need to be updated so that it looks as though the packet originated from the load balancer.

⑧

The IP header includes a checksum calculated over its contents, and since the source and destination IP addresses have both been updated, the checksum also needs to be recalculated and replaced in this packet.

Note

Since this is a book on eBPF and not networking, I haven't delved into details such as why the IP and MAC addresses need to be updated or what happens if they aren't. If you're interested, I cover this some more in my [YouTube video of the eBPF Summit talk](#) where I originally wrote this example code.

由于这本书是关于eBPF而不是网络的内容，我没有深入探讨为什么需要更新IP和MAC地址，以及如果不更新会发生什么。如果你感兴趣，我在我写这个示例代码的eBPF峰会演讲的YouTube视频中对此进行了更详细的介绍。

Much like the previous example, the Makefile includes instructions to not only build the code but also use `bpftool` to load and attach the XDP program to the interface, like this:

与之前的例子类似，Makefile 中包含了一些指令，不仅用于构建代码，而且还使用 `bpftool` 来加载并将 XDP 程序附加到接口上，如下所示：

```
xdp: $(BPF_OBJ)
    bpftool net detach xdpgeneric dev eth0
    rm -f /sys/fs/bpf/$(TARGET)
    bpftool prog load $(BPF_OBJ) /sys/fs/bpf/$(TARGET)
    bpftool net attach xdpgeneric pinned
    /sys/fs/bpf/$(TARGET) dev eth0
```

This `make` instruction needs to be run *inside* the load balancer container so that `eth0` corresponds to its virtual Ethernet interface. This leads to an interesting point: an eBPF program is loaded into the kernel, of which there is only one; yet the attachment point may

be within a particular network namespace and visible only within that network namespace.³

该指令需要在负载均衡器容器内执行，以使eth0对应其虚拟以太网接口。这引出了一个有趣的问题：一个eBPF程序被加载到内核中，而内核只有一个；然而，附加点可能在特定的网络命名空间内，只能在该网络命名空间中可见。

XDP Offloading

XDP 卸载

The idea for XDP originated from a conversation speculating how useful it would be if you could run eBPF programs on a network card to make decisions about individual packets before they even reach the kernel's networking stack.⁴ There are some network interface cards that support this full *XDP offload* capability where they can indeed run eBPF programs on inbound packets on their own processor. This is illustrated in [Figure 8-3](#).

XDP的想法起源于一次谈话，人们猜测如果可以在网络适配卡上运行eBPF程序，在数据包到达内核的网络堆栈之前就能做出决策，那将是多么有用。有些网络接口卡支持完整的XDP卸载功能，在它们自己的处理器上确实可以运行eBPF程序来处理传入的数据包。如图8-3所示。

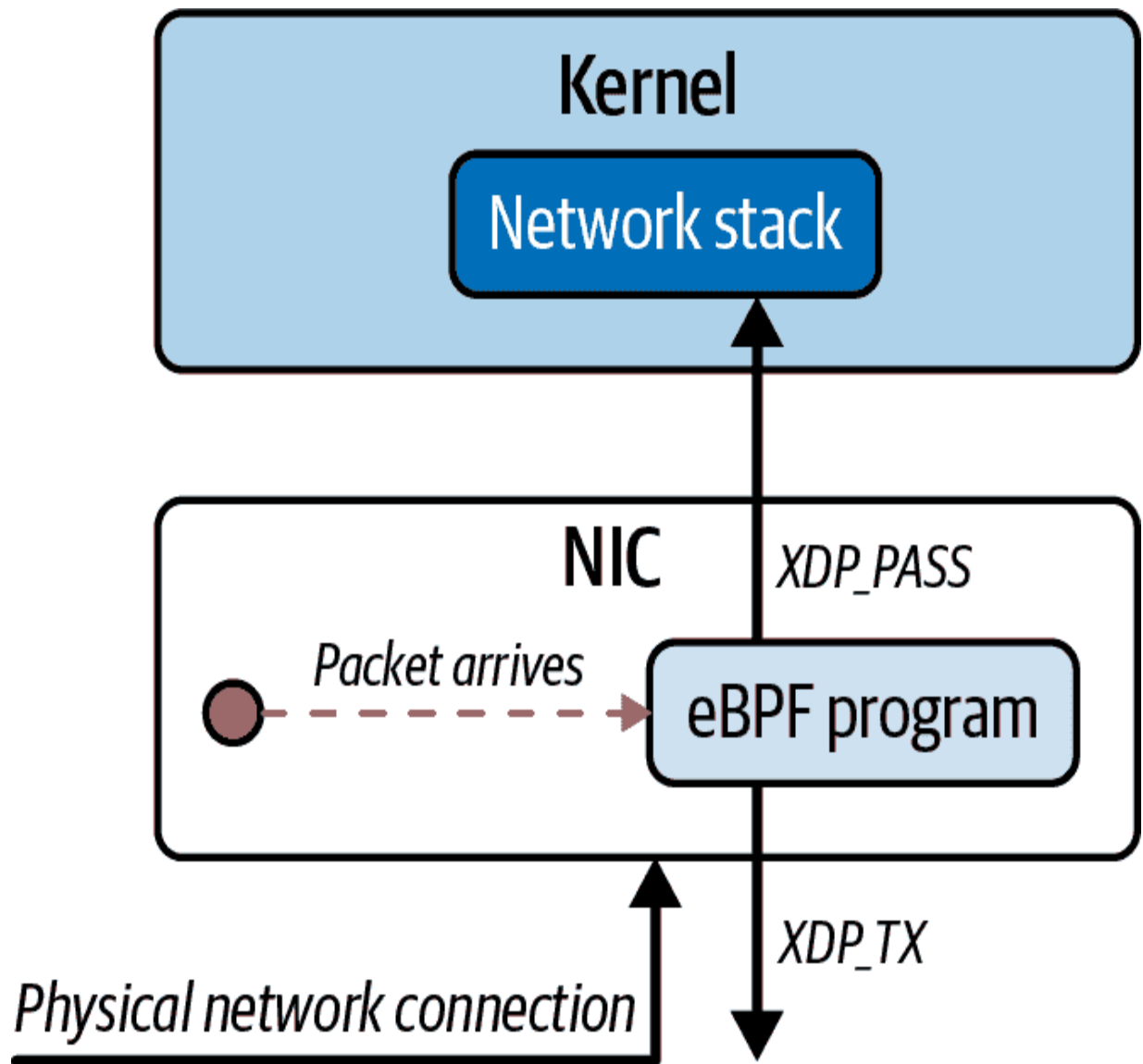


Figure 8-3. Network interface cards that support XDP offload can process, drop, and retransmit packets without any work required from the host CPU

This means a packet that gets dropped or redirected back out of the same physical interface—like the packet drop and load balancing examples earlier in this chapter—is never seen by the host’s kernel,

and no CPU cycles on the host machine are ever spent processing them, as all the work is done on the network card.

这意味着一个被丢弃或重新定向回同一个物理接口的数据包，就像本章前面的数据包丢弃和负载均衡示例一样，永远不会被主机内核看到，并且主机机器上的任何CPU周期都不会花费在处理它们上，因为所有的工作都在网络卡上完成。

Even if the physical network interface card doesn't support full XDP offload, many NIC drivers support XDP hooks, which minimizes the memory copying required for an eBPF program to process a packet.⁵

即使物理网络接口卡不支持完整的XDP卸载，许多网卡驱动程序支持XDP钩子，这样就最大程度地减少了eBPF程序处理数据包所需的内存复制。

This can result in significant performance benefits and allows functionality like load balancing to run very efficiently on commodity hardware.⁶

这可以带来显著的性能优势，并使负载均衡等功能在普通硬件上运行非常高效。

You've seen how XDP can be used to process inbound network packets, accessing them as soon as possible as they arrive on a machine. eBPF can also be used to process traffic at other points in the network stack, in whatever direction it is flowing. Let's move on and think about eBPF programs attached within the TC subsystem.

你已经看到了XDP如何被用于处理入站网络数据包，并在数据包到达机器时尽快对其进行访问。eBPF也可以用于处理网络栈中其他位置的流量，无论其流向如何。让我们继续思考在TC子系统中附加eBPF程序的情况。

Traffic Control (TC)

运输管制 (TC)

I mentioned traffic control in the previous chapter. By the time a network packet reaches this point it will be in kernel memory in the form of an `sk_buff`. This is a data structure that's used throughout the kernel's network stack. eBPF programs attached within the TC subsystem receive a pointer to the `sk_buff` structure as the context parameter.

我在前一章中提到了流量控制。当一个网络数据包到达这一点时，它将以一个`sk_buff`的形式存储在内核内存中。这是一个在内核的网络栈中广泛使用的数据结构。在TC子系统中附加的eBPF程序会接收一个指向`sk_buff`结构的指针作为上下文参数。

Note

You might be wondering why XDP programs don't also use this same structure for their context. The answer is that the XDP hook happens before the network data reaches the network stack and before the `sk_buff` structure has been set up.

你可能想知道为什么XDP程序不使用与上述结构相同的上下文结构。答案是XDP钩子发生在网络数据到达网络栈之前，并且在`sk_buff`结构

设置完毕之前。

The TC subsystem is intended to regulate how network traffic is scheduled. For example, you might want to limit the bandwidth available to each application so that they all get a fair chance. But when you're looking at scheduling individual packets, *bandwidth* isn't a terribly meaningful term, as it's used for the average amount of data being sent or received. A given application might be very bursty, or another application might be very sensitive to network latency, so TC gives much finer control over the way packets are handled and prioritized.⁷

TC子系统旨在调节网络流量的调度。例如，您可能希望限制每个应用程序可用的带宽，以便它们都能有公平的机会。但是当你看到调度单个数据包时，带宽并不是一个非常有意义的术语，因为它是用于发送或接收的平均数据量。一个特定的应用程序可能非常突发，或者另一个应用程序可能对网络延迟非常敏感，所以TC能够对数据包的处理和优先级提供更精细的控制。

eBPF programs were introduced here to give custom control over the algorithms used within TC. But with the power to manipulate, drop, or redirect packets, eBPF programs attached within TC can also be used as the building blocks for complex network behaviors.

在这里引入了eBPF程序，以便在TC中对使用的算法进行自定义控制。但是，由于可以操纵、丢弃或重定向数据包的能力，附在TC中的eBPF程序也可以用作复杂网络行为的基本构件。

A given piece of network data in the stack flows in one of two directions: *ingress* (inbound from the network interface) or *egress* (outbound toward the network interface). eBPF programs can be attached in either direction and will affect traffic only in that direction. Unlike XDP, it's possible to attach multiple eBPF programs that will be processed in sequence.

在堆栈中，一定的网络数据流动的方向有两个：入口（从网络接口进入）或出口（向网络接口出去）。eBPF 程序可以连接在任何一个方向上，并且只会影响该方向上的流量。与 XDP 不同的是，可以连接多个 eBPF 程序，并按顺序进行处理。

Traditional traffic control is split into *classifiers*, which classify packets based on some rule, and separate *actions*, which are taken based on the output from a classifier and determine what to do with a packet. There can be a series of classifiers, all defined as part of a *qdisc* or queuing discipline.

传统的流量控制被划分为分类器，根据某些规则对数据包进行分类，并且还有根据分类器的输出进行的分离动作，这些动作决定了如何处理数据包。在一个队列规则(qdisc)或排队规则中，可以存在一系列的分类器。

eBPF programs are attached as a classifier, but they can also determine what action to take within the same program. The action is indicated by the program's return code (whose values are defined in *linux/pkt_cls.h*):

eBPF程序作为分类器附加，但它们也可以在同一个程序中确定采取的操作。操作由程序的返回代码表示（其值在linux/pkt_cls.h中定义）：

- `TC_ACT_SHOT` tells the kernel to drop the packet.

`TC_ACT_SHOT`告诉内核丢弃该数据包。

- `TC_ACT_UNSPEC` behaves as if the eBPF program hadn't been run on this packet (so it would be passed to the next classifier in the sequence, if there is one).

`TC_ACT_UNSPEC` 表现得就像这个数据包没有进行 eBPF 程序的处理一样（所以它将被传递到下一个分类器，如果有的话）。

- `TC_ACT_OK` tells the kernel to pass the packet to the next layer in the stack.

`TC_ACT_OK`告诉内核将数据包传递给堆栈中的下一层。

- `TC_ACT_REDIRECT` sends the packet to the ingress or egress path of a different network device.

"`TC_ACT_REDIRECT`通过将数据包发送到不同网络设备的进入或出口路径来重定向数据包。"

Let's take a look at a few simple examples of programs that can be attached within TC. The first simply generates a line of trace and then tells the kernel to drop the packet:

让我们来看几个可以附加在TC中的简单程序示例。第一个程序只是生成一行跟踪，并告诉内核放弃该数据包。

```
int tc_drop(struct __sk_buff *skb) {
    bpf_trace_printk("[tc] dropping packet\n");
    return TC_ACT_SHOT;
}
```

Now let's consider how to drop only a subset of packets. This example drops ICMP (ping) request packets and is very similar to the XDP example you saw earlier in this chapter:

现在让我们考虑如何仅丢弃一个子集的数据包。这个例子丢弃 ICMP (ping) 请求包，非常类似于你在本章前面看到的 XDP 例子。

```
int tc(struct __sk_buff *skb) {
    void *data = (void *) (long) skb->data;
    void *data_end = (void *) (long) skb->data_end;

    if (is_icmp_ping_request(data, data_end)) {
        struct iphdr *iph = data + sizeof(struct ethhdr);
        struct icmp_hdr *icmp = data + sizeof(struct ethhdr) +
sizeof(struct iphdr);
        bpf_trace_printk("[tc] ICMP request for %x type %x\n",
iph->daddr,
                        icmp->type);
        return TC_ACT_SHOT;
    }
    return TC_ACT_OK;
}
```

The `sk_buff` structure has pointers to the start and end of the packet data, very much like the `xdp_md` structure, and packet parsing proceeds in very much the same way. Again, to pass verification you have to explicitly check that any access to data is within the range between `data` and `data_end`.

`sk_buff` 结构体有指向数据包起始和结束处的指针，就像 `xdp_md` 结构体一样，数据包解析也是以非常相似的方式进行。同样地，为了通过

验证，您必须显式地检查对数据的任何访问是否在data和data_end之间的范围内。

You might be wondering why you would want to implement something like this at the TC layer when you have already seen the same kind of functionality implemented with XDP. One good reason is that you can use TC programs for egress traffic, where XDP can only process ingress traffic. Another is that because XDP is triggered as soon as the packet arrives, there is no `sk_buff` kernel data structure related to the packet at that point. If the eBPF program is interested in or wants to manipulate the `sk_buff` the kernel creates for this packet, the TC attachment point is suitable.

你可能会想知道为什么在已经使用XDP实现了相同功能的情况下，你仍然想在TC层实施这样的功能。一个很好的理由是，你可以将TC程序用于出口流量，而XDP只能处理入口流量。另一个原因是，因为XDP是在数据包到达时立即触发的，所以在那个时刻与数据包相关的sk_buff内核数据结构不存在。如果eBPF程序对该数据包的sk_buff感兴趣或希望对其进行操作，那么TC附加点是适合的。

Note

To better understand the differences between XDP and TC eBPF programs, read the “Program Types” section in the [BPF and XDP Reference Guide](#) from the Cilium project.

为了更好地理解XDP和TC eBPF程序之间的差异，请阅读Cilium项目中的“程序类型”部分的BPF和XDP参考指南。

Now let's consider an example that doesn't just drop certain packets. This example identifies a ping request being received and responds with a ping response:

现在让我们考虑一个不仅仅丢弃某些数据包的例子。这个例子识别接收到的ping请求，并以ping响应进行回应。

```
int tc_pingpong(struct __sk_buff *skb) {
    void *data = (void *) (long) skb->data;
    void *data_end = (void *) (long) skb->data_end;

    if (!is_icmp_ping_request(data, data_end)) {           ❶
        return TC_ACT_OK;
    }

    struct iphdr *iph = data + sizeof(struct ethhdr);
    struct icmphdr *icmp = data + sizeof(struct ethhdr) +
sizeof(struct iphdr);

    swap_mac_addresses(skb);                               ❷
    swap_ip_addresses(skb);

    // Change the type of the ICMP packet to 0 (ICMP Echo Repl
    // (was 8 for ICMP Echo request)
    update_icmp_type(skb, 8, 0);                           ❸

    // Redirecting a clone of the modified skb back to the int
    // it arrived on
    bpf_clone_redirect(skb, skb->ifindex, 0);               ❹

    return TC_ACT_SHOT;                                    ❺
}
```

❶

The `is_icmp_ping_request()` function parses the packet and checks not only that it's an ICMP message, but also that it's an echo (ping) request.

②

Since this function is going to send a response to the sender, the source and destination addresses need to be swapped. (You can read the example code if you want to see the nitty-gritty details of this, which also includes updating the IP header checksum.)

③

This is converted to an echo response by changing the type field in the ICMP header.

④

This helper function sends a clone of the packet back through the interface (`skb->ifindex`) on which it was received.

⑤

Since the helper function cloned the packet before sending out the response, the original packet should be dropped.

In normal circumstances, a ping request would be handled later by the kernel's network stack, but this small example demonstrates how network functionality more generally can be replaced by an eBPF implementation.

在正常情况下，Ping请求将稍后由内核的网络堆栈处理，但此简单示例演示了如何通过eBPF实现来替代更一般的网络功能。

Lots of networking capabilities today are handled by user space services, but where they can be replaced by eBPF programs, it's likely to be great for performance. A packet that's processed within

the kernel doesn't have to complete its journey through the rest of the stack; there is no need for it to transition to user space for processing, and the response doesn't require a transition back into the kernel. What's more, the two could run in parallel—an eBPF program can return `TC_ACT_OK` for any packet that requires complex processing that it can't handle so that it gets passed up to the user space service as normal.

当今许多网络功能都由用户空间服务处理，但如果能用eBPF程序替代，性能将会大幅提升。在内核中处理的数据包无需完整地经过整个堆栈的流程；它不需要转入用户空间进行处理，也不需要再次回到内核。而且，这两者可以并行运行，对于那些复杂处理无法完成的数据包，eBPF程序可以返回`TC_ACT_OK`，使其像正常情况下一样传递给用户空间服务。

For me, this is an important aspect of implementing network functionality in eBPF. As the eBPF platform develops (e.g., more recent kernels allowing programs of one million instructions), it's possible to implement increasingly complex aspects of networking in the kernel. The parts that are not yet implemented in eBPF can still be handled either by the traditional stack within the kernel or in user space. Over time, more and more features can be moved from user space into the kernel, with the flexibility and dynamic nature of eBPF meaning you won't have to wait for them to be part of the kernel distribution itself. You can load eBPF implementations immediately, just as I discussed in [Chapter 1](#).

对我来说，这是在eBPF中实现网络功能的一个重要方面。随着eBPF平台的发展（例如，更近期的内核允许一百万条指令的程序），可以在内核中实现越来越复杂的网络方面。尚未在eBPF中实现的部分仍然可以通过内核中的传统堆栈或用户空间来处理。随着时间的推移，越来越多的功能可以从用户空间移动到内核中，由于eBPF的灵活性和动态特性，您无需等待它们成为内核分发的一部分。您可以立即加载eBPF实现，就像我在第1章中讨论的那样。

I'll return to the implementation of networking features in [“eBPF and Kubernetes Networking”](#). But first, let's consider another use case that eBPF enables: inspecting the decrypted contents of encrypted traffic.

我将回到“eBPF和Kubernetes网络”中网络功能的实现。但首先，让我们考虑eBPF可以实现的另一个用例：检查加密流量的解密内容。

Packet Encryption and Decryption

数据包加密与解密

If an application uses encryption to secure data it sends or receives, there will be a point before it's encrypted or after it's decrypted where the data is in the clear. Recall that eBPF can attach programs pretty much anywhere on a machine, so if you can hook into a point where data is being passed and isn't yet encrypted, or just after it has been decrypted, that would allow your eBPF program to observe that data in the clear. There's no need to supply any certificates to decrypt the traffic, as you would in a traditional SSL inspection tool.

如果一个应用程序使用加密来保护发送或接收的数据，在数据被加密之前或解密之后，会有一个明文数据的时刻。记住eBPF几乎可以在计算机的任何地方附加程序，所以如果你可以钩入数据传输时尚未加密或刚刚解密后的阶段，那么你的eBPF程序就可以观察到明文数据。无需提供任何证书来解密流量，就像传统SSL检查工具那样。

In many cases an application will encrypt data using a library like OpenSSL or BoringSSL that lives in user space. In this case the traffic will already be encrypted by the time it reaches the socket, which is the user space/kernel boundary for network traffic. If you want to trace out this data in its unencrypted form, you can use an eBPF program attached to the right place in the user space code.

在许多情况下，应用程序会使用类似OpenSSL或BoringSSL这样的库对数据进行加密，这些库存在于用户空间。在这种情况下，当流量到达套接字（也就是用户空间/内核之间的边界）时，它已经是加密的。如果你想跟踪这些数据的未加密形式，你可以在用户空间代码的适当位置附加一个eBPF程序。

User Space SSL Libraries

用户空间SSL库

One common way to trace out the decrypted content of encrypted packets is to hook into calls made to user space libraries like OpenSSL or BoringSSL. An application using OpenSSL sends data to be encrypted by making a call to a function called `SSL_write()` and retrieves cleartext data that was received over the network in encrypted form using `SSL_read()`. Hooking eBPF programs into

these functions with uprobes allows an application to observe the data *from any application that uses this shared library* in the clear, before it is encrypted or after it has been decrypted. And there is no need for any keys, because those are already being provided by the application.

追踪解密包的常见方法之一是钩入对用户空间库（如OpenSSL或BoringSSL）的调用。使用OpenSSL的应用程序通过调用名为SSL_write()的函数发送要加密的数据，并使用SSL_read()来检索以加密形式从网络接收到的明文数据。通过将eBPF程序钩入这些函数，应用程序可以在该共享库中使用的任何应用程序中观察到在加密之前或解密之后的明文数据。此外，由于应用程序已经提供了密钥，因此不需要任何密钥。

There is a fairly straightforward example called [openssl-tracer in the Pixie project](#),⁸ within which the eBPF programs are in a file called *openssl_tracer_bpf_funcs.c*. Here's the part of that code that sends data to user space, using a perf buffer (similar to examples you have seen earlier in this book):

有一个相当简单的例子叫做openssl-tracer，在Pixie项目中，其中eBPF程序存储在名为openssl_tracer_bpf_funcs.c的文件中。下面是该代码的一部分，它使用性能缓冲区将数据发送到用户空间（类似于您在本书中之前看到的示例）：

```
static int process_SSL_data(struct pt_regs* ctx, uint64_t
    id, enum
    ssl_data_event_type type, const char* buf) {
    ...
    bpf_probe_read(event->data, event->data_len, buf);
    tls_events.perf_submit(ctx, event, sizeof(struct
```

```
ssl_data_event_t));  
  
    return 0;  
}
```

You can see that data from `buf` gets read into an `event` structure using the helper function `bpf_probe_read()`, and then that `event` structure is submitted to a perf buffer.

您可以看到，使用辅助函数**bpf_probe_read()**将buf中的数据读入到事件结构中，然后将该事件结构提交到性能缓冲区。

If this data is being sent to user space, it's reasonable to assume this must be the data in unencrypted format. So where is this buffer of data obtained? You can work that out by seeing where the `process_SSL_data()` function is called. It's called in two places: one for data being read and one for data being written. [Figure 8-4](#) illustrates what is happening in the case of reading data that arrives on this machine in encrypted form.

如果这些数据被发送到用户空间，合理地假设这必须是未加密的数据。那么这个数据缓冲区是从哪里获取的呢？通过观察 `process_SSL_data()` 函数被调用的地方可以找到答案。它被调用的地方有两个：一个是用于读取数据，另一个是用于写入数据。图8-4说明了在读取以加密形式到达本机的数据的情况下正在发生的事情。

When you're reading data, you supply a pointer to a buffer to `SSL_read()`, and when the function returns, that buffer will contain the unencrypted data. Much like kprobes, the input parameters to a function—including that buffer pointer—are only available to a uprobe attached to the entry point, as the registers they're held in

might well get overwritten during the function's execution. But the data won't be available in the buffer until the function exits, when you can read it using a uretprobe.

当你读取数据时，你需要向SSL_read()提供一个指向缓冲区的指针，在函数返回时，该缓冲区将包含未加密的数据。与kprobes类似，函数的输入参数（包括缓冲区指针）只对附加到入口点的uprobe可用，因为它们所存储的寄存器在函数执行期间可能被覆盖。但是在函数退出时，数据将不会在缓冲区中可用，这时你可以使用uretprobe来读取它。

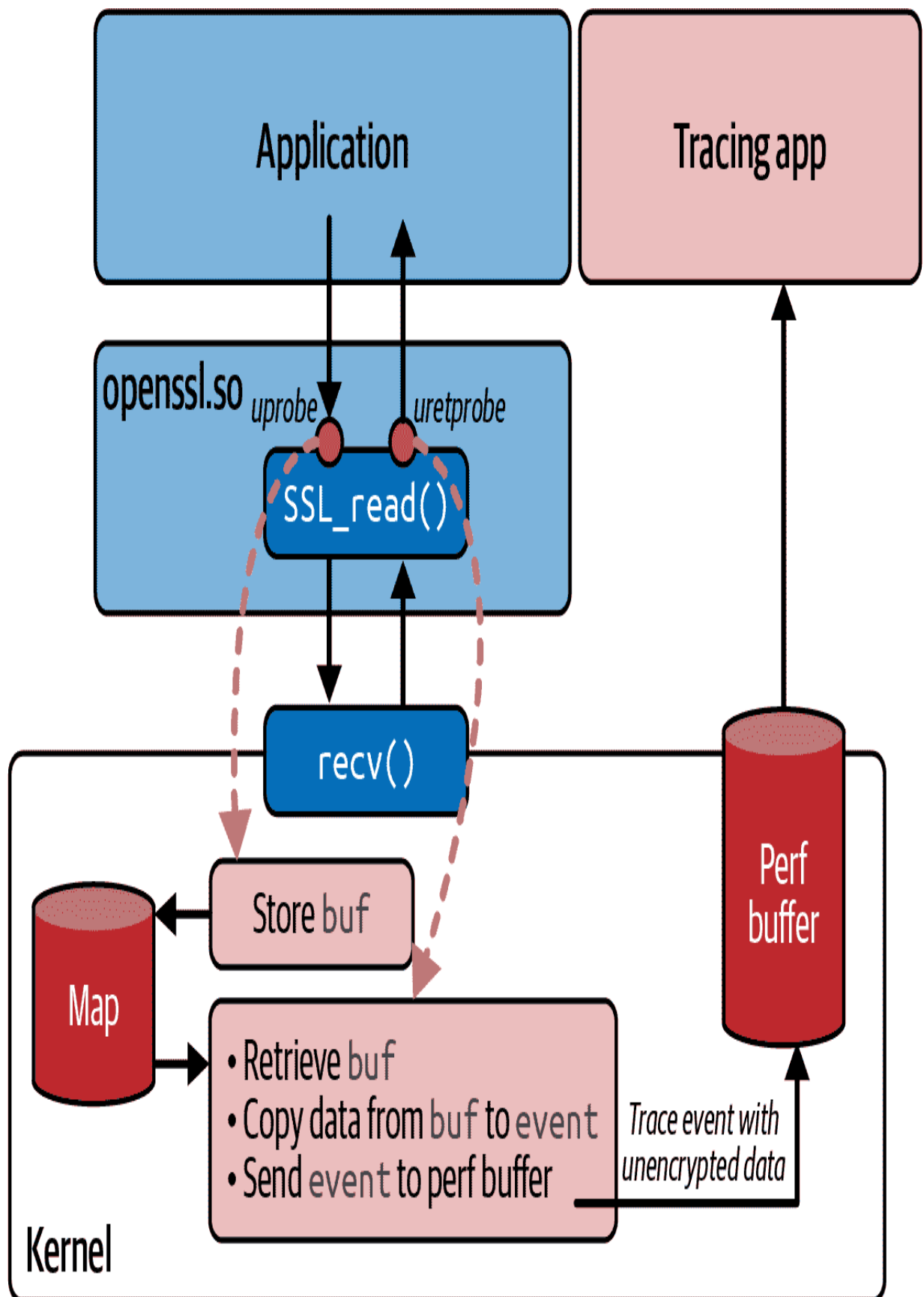


Figure 8-4. eBPF programs are hooked to uprobes at the entry to and exit from `SSL_read()` so that the unencrypted data can be read from the buffer pointer

So this example follows a common pattern for kprobes and uprobes, illustrated in [Figure 8-4](#), where the entry probe temporarily stores input parameters using a map, from which the exit probe can retrieve them. Let's look at the code that does this, starting with the eBPF program attached to the start of `SSL_read()`:

因此，此示例遵循kprobes和uprobes的常见模式，如图8-4所示，入口探针使用映射临时存储输入参数，而出口探针可以从中检索这些参数。让我们来看一下执行此操作的代码，从SSL_read()的开头附加的eBPF程序开始。

```
// Function signature being probed:
// int SSL_read(SSL *s, void *buf, int num)
int probe_entry_SSL_read(struct pt_regs* ctx) {
    uint64_t current_pid_tgid = bpf_get_current_pid_tgid();
    ...

    const char* buf = (const char*)PT_REGS_PARM2(ctx);
    ❶

    active_ssl_read_args_map.update(&current_pid_tgid, &buf)
    ; ❷
    return 0;
}
```

❶

As described in the comment for this function, the buffer pointer is the second parameter passed into the `SSL_read()` function to

which this probe will be attached. The `PT_REGS_PARM2` macro gets this parameter from the context.

②

The buffer pointer is stored in a hash map, for which the key is the current process and thread ID, obtained at the start of the function using the helper `bpf_get_current_pid_tgid()`.

Here's the corresponding program for the exit probe:

这是与退出探针对应的程序。

```
int probe_ret_SSL_read(struct pt_regs* ctx) {
    uint64_t current_pid_tgid = bpf_get_current_pid_tgid();

    ...
    const char** buf = active_ssl_read_args_map.lookup(&
current_pid_tgid); ❶
    if (buf != NULL) {
        process_SSL_data(ctx, current_pid_tgid, kSSLRead, *buf
); ❷
    }

    active_ssl_read_args_map.delete(&current_pid_tgid);
    ❸

    return 0;
}
```

❶

Having looked up the current process and thread ID, use this as the key to retrieve the buffer pointer from the hash map.

❷

If this isn't a null pointer, call `process_SSL_data()`, which is the function you saw earlier that sends the data from that buffer to

user space using the perf buffer.

③

Clean up the entry in the hash map, since every entry call should be paired with an exit.

This example shows how to trace out the cleartext version of encrypted data that gets sent and received by a user space application. The tracing itself is attached to a user space library, and there's no guarantee that every application will use a given SSL library. The BCC project includes a utility called [sslsniff](#) that also supports GnuTLS and NSS. But if someone's application uses some other encryption library (or even, heaven forbid, they chose to “roll their own crypto”), the uprobes simply won't have the right places to hook to and these tracing tools won't work.

这个例子展示了如何追踪用户空间应用程序发送和接收的加密数据的明文版本。追踪本身附加到一个用户空间库，不能保证每个应用程序都使用给定的SSL库。BCC项目包括一个名为sslsniff的实用工具，也支持GnuTLS和NSS。但是，如果某个应用程序使用其他加密库（甚至选择“自己编写加密算法”），那么uprobes就无法正确连接，这些追踪工具将无法工作。

There are even more common reasons why this uprobe-based approach might not be successful. Unlike the kernel (of which there is only one per [virtual] machine), there can be multiple copies of user space library code. If you're using containers, each one is likely to have its own set of all library dependencies. You can hook into

uprobes in these libraries, but you'd have to identify the right copy for the particular container you want to trace. Another possibility is that rather than using a shared, dynamically linked library, an application might be statically linked so that it's a single standalone executable.

这种基于uprobe的方法可能不成功的常见原因更多。与内核不同（每个[虚拟]机器只有一个内核），用户空间库代码可以有多个副本。如果您正在使用容器，那么每个容器很可能有自己的所有库依赖关系。您可以在这些库中接入uprobes，但必须识别出要跟踪的特定容器的正确副本。另一种可能性是，应用程序可能是静态链接的，而不是使用共享的动态链接库，使它成为单独的可执行文件。

eBPF and Kubernetes Networking

eBPF和Kubernetes网络

Although this book isn't about Kubernetes, eBPF is so widely used for Kubernetes networking that it's a great illustration of using the platform to customize the networking stack.

尽管这本书不是关于Kubernetes的，但是eBPF在Kubernetes网络中被广泛使用，因此它很好地说明了如何使用该平台来自定义网络堆栈。

In Kubernetes environments, applications are deployed in *pods*. Each pod is a group of one or more containers that share kernel namespaces and cgroups, isolating pods from each other and from the host machine they are running on.

在Kubernetes环境中，应用程序部署在Pod中。每个Pod是一组共享内核命名空间和cgroups的一个或多个容器，将Pod与彼此和宿主机隔离开来。

In particular (for the purposes of this chapter), a pod typically has its own network namespace and its own IP address.⁹ This means the kernel has a set of network stack structures for that namespace, separated from the host's and from other pods. As shown in [Figure 8-5](#), the pod is connected to the host by a virtual Ethernet connection, and it is allocated its own IP address.

具体来说（对于本章的目的），一个Pod通常有自己的网络命名空间和自己的IP地址。这意味着内核有一组针对该命名空间的网络栈结构，与宿主机和其他Pod分开。如图8-5所示，Pod通过虚拟以太网连接与宿主机相连，并分配了自己的IP地址。



Node

Pod

Process

Socket

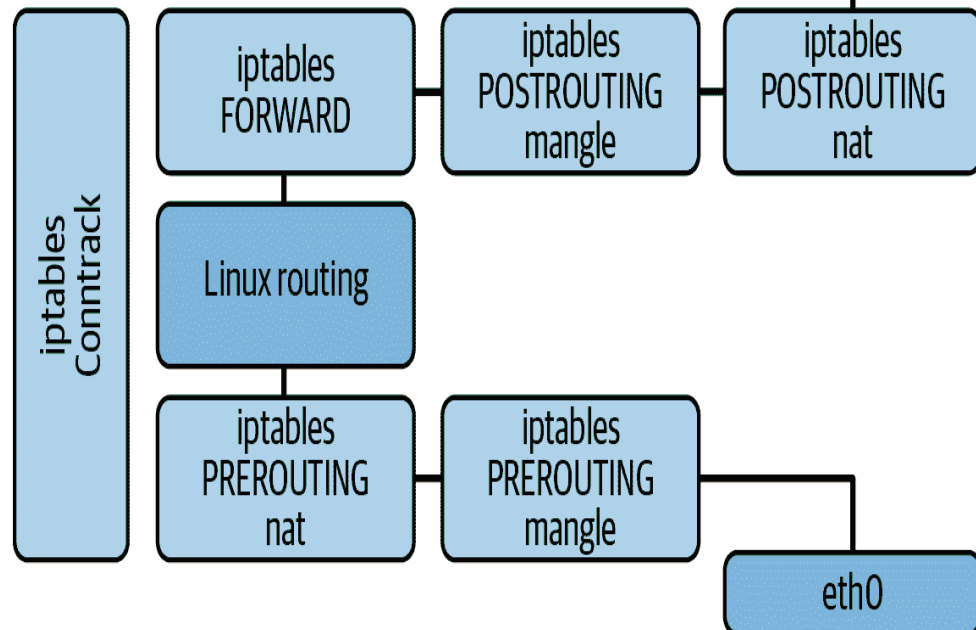
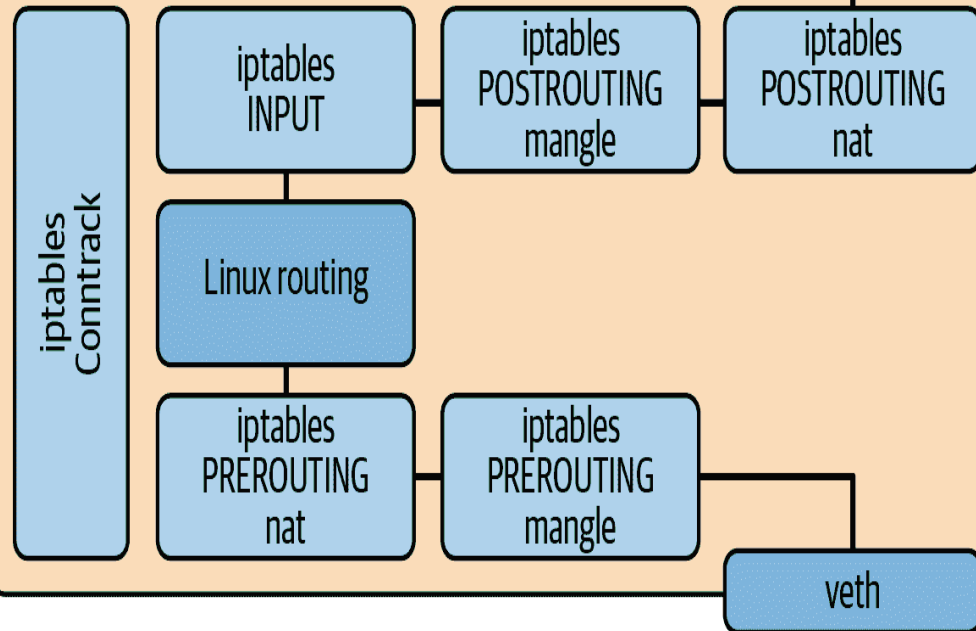


Figure 8-5. Network path in Kubernetes

You can see from [Figure 8-5](#) that a packet coming from outside the machine destined for an application pod has to travel through the network stack on the host, across the virtual Ethernet connection, and into the pod's network namespace, and then it has to traverse the network stack again to reach the application.

从图8-5中可以看出，从机器外部发出的一个数据包，目标是一个应用程序Pod，必须经过主机的网络堆栈、虚拟以太网连接、进入Pod的网络命名空间，然后再次穿过网络堆栈才能到达应用程序。

Those two network stacks are running in the same kernel, so the packet is really running through the same processing twice. The more code a network packet has to pass through, the higher the latency, so if it's possible to shorten the network path, that will likely bring about performance improvements.

这两个网络堆栈在同一个内核中运行，因此数据包实际上会经过相同的处理两次。网络数据包必须通过的代码越多，延迟就越高，所以如果能够缩短网络路径，很可能会带来性能改进。

An eBPF-based networking solution like Cilium can hook into the network stack to override the kernel's native networking behavior, as shown in [Figure 8-6](#).

一个基于eBPF的网络解决方案，例如Cilium，可以钩入网络堆栈以覆盖内核本身的网络行为，如图8-6所示。

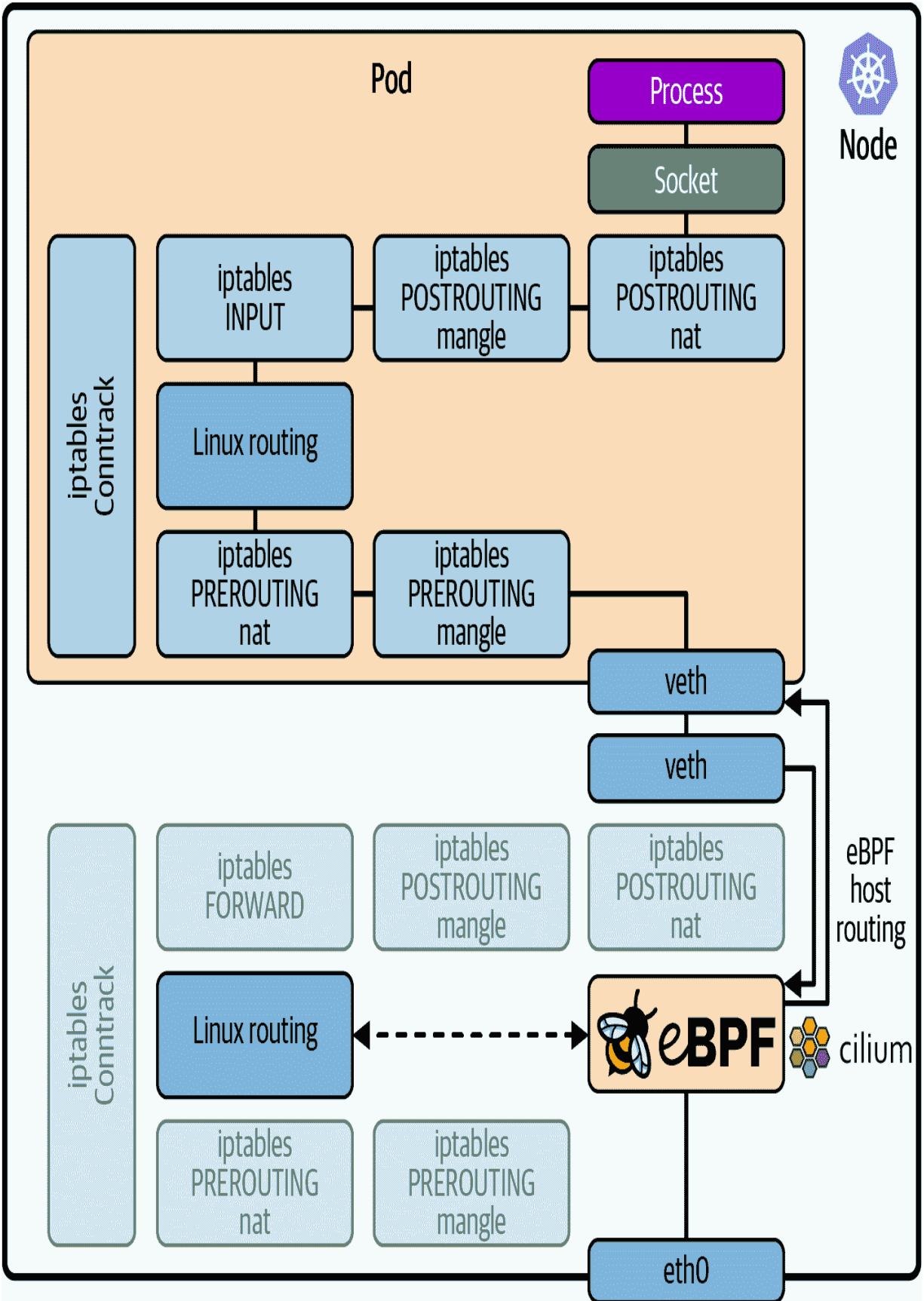


Figure 8-6. Bypassing iptables and conntrack processing with eBPF

In particular, eBPF enables replacing iptables and conntrack with a more efficient solution for managing network rules and connection tracking. Let's discuss why this results in a significant performance improvement in Kubernetes.

特别是，eBPF可以用更高效的解决方案替代iptables和conntrack，用于管理网络规则和连接跟踪。让我们讨论一下为什么这会在Kubernetes中带来显著的性能提升。

Avoiding iptables

避免使用 iptables

Kubernetes has a component called kube-proxy that implements load balancing behavior, allowing multiple pods to fulfill requests to a service. This has been implemented using iptables rules.

Kubernetes有一个称为kube-proxy的组件，它实现了负载均衡行为，允许多个pod来满足对服务的请求。这是通过使用iptables规则实现的。

Kubernetes offers users the choice of which networking solution to use through the use of the Container Network Interface (CNI). Some CNI plug-ins use iptables rules to implement L3/L4 network policy in Kubernetes; that is, the iptables rules indicate whether to drop a packet because it doesn't meet the network policy.

Kubernetes通过使用容器网络接口（CNI），为用户提供了选择网络解决方案的选项。一些CNI插件使用iptables规则在Kubernetes中实现L3/L4网络策略，即iptables规则指示是否丢弃一个数据包，因为它不符合网络策略。

Although iptables was effective for traditional (precontainer) networking, it has some weaknesses when it's used in Kubernetes. In this environment, pods—and their IP addresses—come and go dynamically, and each time a pod is added or removed, the iptables rules have to be rewritten in their entirety, and this impacts performance at scale. (A [talk](#) by Haibin Xie and Quinton Hoole at KubeCon in 2017 described how making a single rule update to iptables rules for 20,000 services could take five hours.)

尽管iptables在传统（非容器化）网络中非常有效，但在Kubernetes中使用时存在一些弱点。在这个环境中，Pod和它们的IP地址是动态变化的，每次添加或删除一个Pod时，iptables规则必须完全重新编写，这会影响到大规模的性能。（在2017年的KubeCon大会上，解海斌和Quinton Hoole的讲座描述了对20,000个服务进行单一规则更新可能需要花费五个小时的情况。）

Updates to iptables aren't the only performance issues: looking up a rule requires a linear search through the table, which is an $O(n)$ operation, growing linearly with the number of rules.

更新iptables不是唯一的性能问题：查找规则需要在表中进行线性搜索，这是一个 $O(n)$ 的操作，随着规则数量的增加而线性增长。

Cilium uses eBPF hash table maps to store network policy rules, connection tracking, and load balancer lookup tables, which can replace iptables for kube-proxy. Both looking up an entry in a hash table and inserting a new one are approximately $O(1)$ operations, which means they scale much, much better.

Cilium使用eBPF哈希表映射来存储网络策略规则、连接跟踪和负载均衡器查找表，可以替代kube-proxy中的iptables。在哈希表中查找或插入条目的时间复杂度都约为 $O(1)$ ，这意味着它们具有更好的扩展性。

You can read about the benchmarked performance improvements this achieves on the Cilium [blog](#). In the same post you'll see that Calico, another CNI that has an eBPF option, also achieves better performance when you pick its eBPF implementation over iptables. eBPF offers the most performant mechanisms for scalable, dynamic Kubernetes deployments.

您可以在Cilium博客上阅读有关此优化所实现的基准性能改进。在同一篇文章中，您还会看到Calico，另一个具有eBPF选项的CNI，在选择其eBPF实现而非iptables时也实现了更好的性能。eBPF为可扩展、动态的Kubernetes部署提供了最高性能的机制。

Coordinated Network Programs

协调网络计划

A complex networking implementation like Cilium can't be written as a single eBPF program. As shown in [Figure 8-7](#), it provides several different eBPF programs that are hooked into different parts of the kernel and its network stack.

Cilium这样的复杂网络实现无法编写为单个eBPF程序。如图8-7所示，它提供了几个不同的eBPF程序，这些程序被挂载到内核及其网络栈的不同部分。

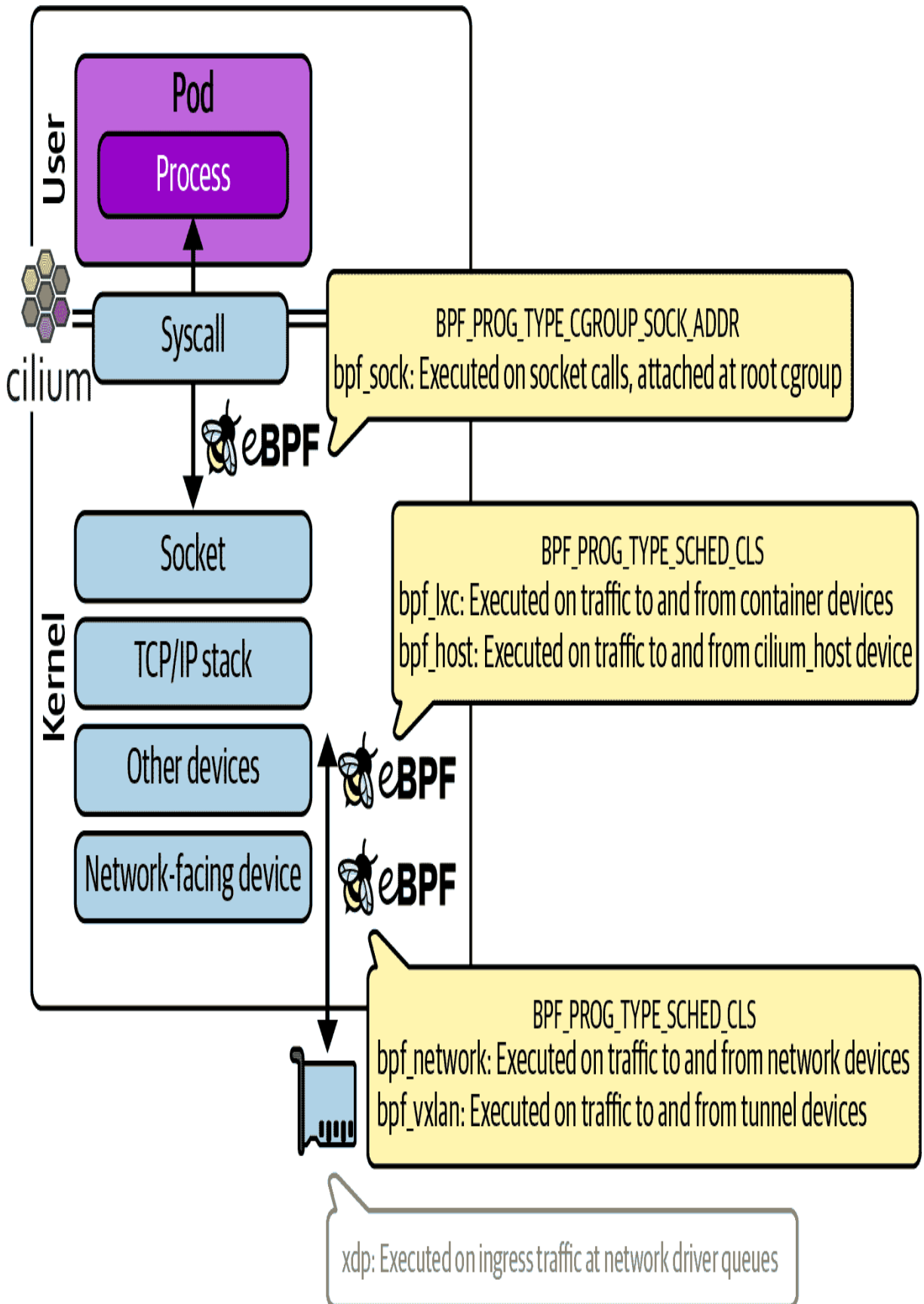


Figure 8-7. Cilium consists of multiple coordinated eBPF programs that hook into different points in the kernel

As a general principle, Cilium intercepts traffic as soon as it can in order to shorten the processing path for each packet. Messages flowing out from an application pod are intercepted at the socket layer, as close to the application as possible. Inbound packets from the external network are intercepted using XDP. But what about the additional attachment points?

作为一般原则，Cilium 在最早的时候拦截流量，以便缩短每个数据包的处理路径。从应用程序 Pod 流出的消息在套接字层被拦截，尽可能接近应用程序。来自外部网络的入站数据包使用 XDP 拦截。但是其他附加连接点呢？

Cilium supports different networking modes that suit different environments. A full description of this is beyond the scope of this book (you can find more information at [Cilium.io](https://cilium.io)), but I'll give a brief overview here so that you can see why there are so many different eBPF programs!

Cilium支持适用于不同环境的不同网络模式。本书的范围无法对其进行全面描述（您可以在Cilium.io找到更多信息），但我会在这里简要概述一下，让您能够了解为什么会有这么多不同的eBPF程序！

There is a simple, flat networking mode, in which Cilium allocates IP addresses for all the pods in a cluster from the same CIDR and directly routes traffic between them. There are also a couple of different tunneling modes, in which traffic intended for a pod on a

different node gets encapsulated in a message addressed to that destination node's IP address and decapsulated on that destination node for the final hop into the pod. Different eBPF programs get invoked to handle traffic depending on whether a packet is destined for a local container, the local host, another host on this network, or a tunnel.

Cilium有一个简单的、扁平的网络模式，它从同一CIDR中为集群中的所有Pod分配IP地址，并直接在它们之间进行流量路由。此外，还有几种不同的隧道模式，其中针对不同节点上的Pod的流量被封装在一个消息中，该消息被寻址给目标节点的IP地址，并在目标节点上进行解封装以进行最后的跳转到Pod上。根据数据包的目的地是一个本地容器、本地主机、本网络上的另一个主机还是一个隧道，不同的eBPF程序会被调用来处理流量。

In [Figure 8-7](#) you can see multiple TC programs that handle traffic to and from different devices. These devices represent the possible different real and virtual network interfaces where a packet might be flowing:

在图8-7中，您可以看到多个处理不同设备之间流量的TC程序。这些设备代表可能的不同真实和虚拟网络接口，数据包可能会在其中流动。

- The interface to a pod's network (one end of the virtual Ethernet connection between the pod and the host)

一个Pod网络的接口（Pod和宿主之间的虚拟以太网连接的一端）

- The interface to a network tunnel

网络隧道的接口

- The interface to a physical network device on the host

主机上物理网络设备的接口

- The host's own network interface

主机的网络接口

Note

If you're interested in learning more about how packets flow through Cilium, Arthur Chiao wrote this detailed and interesting blog post: [“Life of a Packet in Cilium: Discovering the Pod-to-Service Traffic Path and BPF Processing Logics”](#).

如果你想了解更多有关Cilium中数据包如何流动的信息，Arthur Chiao撰写了这篇详细而有趣的博客文章：“Cilium中数据包的生命周期：发现Pod到Service的流量路径和BPF处理逻辑”。

The different eBPF programs attached at these various points in the kernel communicate using eBPF maps and using the metadata that can be attached to network packets as they flow through the stack (which I mentioned when I discussed accessing network packets in the XDP example). These programs don't just route packets to their destination; they're also used to drop packets—just like you saw in earlier examples—based on network policies.

在内核中的各个不同的eBPF程序通过使用eBPF映射和附加到网络数据包的元数据进行通信（正如我在讨论XDP示例中访问网络数据包时

所提到的)。这些程序不仅仅将数据包路由到目标位置，还可以根据网络策略丢弃数据包，就像您在之前的示例中看到的一样。

Network Policy Enforcement

网络策略执行

You saw at the start of this chapter how eBPF programs can drop packets, and that means they simply won't reach their destination. This is the basis of network policy enforcement, and conceptually it's essentially the same whether we are thinking about "traditional" or cloud native firewalling. A policy determines whether a packet should be dropped or not, based on information about its source and/or destination.

在本章的开始，您看到了eBPF程序如何丢弃数据包。这意味着它们不会达到目的地。这是网络策略执行的基础，从概念上讲，无论是传统的防火墙还是云原生防火墙，它们本质上是相同的。策略根据有关数据包源和/或目的地的信息决定是否丢弃数据包。

In traditional environments, IP addresses are assigned to a particular server for a long period of time, but in Kubernetes, IP addresses come and go dynamically, and the address assigned today for a particular application pod might very well be reused for a completely different application tomorrow. This is why traditional firewalling isn't terribly effective in cloud native environments. It would be impractical to redefine firewall rules manually every time IP addresses change.

在传统环境中，IP地址长期分配给特定服务器，但在Kubernetes中，IP地址会动态分配，今天为特定应用程序容器分配的地址可能明天会

被完全不同的应用程序重新使用。这就是为什么传统的防火墙在云原生环境中效果不好。每次IP地址变化时手动重新定义防火墙规则是不切实际的。

Instead, Kubernetes supports the concept of a NetworkPolicy resource, which defines firewalling rules based on the labels applied to particular pods rather than based on their IP address. Although the resource type is native to Kubernetes, it's not implemented by Kubernetes itself. Instead, this functionality is delegated to whatever CNI plug-in you're using. If you choose a CNI that doesn't support NetworkPolicy resources, any rules you might configure are simply ignored. On the flip side, CNIs are free to configure custom resources that allow for more sophisticated network policy configurations than the native Kubernetes definition allows. For example, Cilium supports features like DNS-based network policy rules, so you can define whether traffic is or isn't allowed not based on an IP address but based on the DNS name (e.g., "*example.com*"). You can also define policies for various Layer 7 protocols, for example, allowing or denying traffic for HTTP GET calls but not for POST calls to a particular URL.

相反，Kubernetes支持NetworkPolicy资源的概念，并根据应用于特定pod的标签而不是基于其IP地址定义防火墙规则。尽管资源类型是Kubernetes原生支持的，但它并不是由Kubernetes本身实现的。实际上，这个功能由你使用的任何CNI插件来代理执行。如果选择的CNI不支持NetworkPolicy资源，那么您配置的任何规则都会被忽略。另一方面，CNIs可以自由配置自定义资源，允许比原生Kubernetes定义更复杂的网络策略配置。例如，Cilium支持基于DNS的网络策略规则，因

此您可以根据DNS名称（例如：“example.com”）来定义是否允许流量传输。还可以为各种第7层协议定义策略，例如允许或拒绝HTTP GET调用的流量，但不允许POST调用特定URL的流量。

Note

Isovalent’s free hands-on lab “[Getting Started with Cilium](#)” walks you through defining network policies at Layers 3/4 and Layer 7. Another very useful resource is the Network Policy Editor at networkpolicy.io, which visually presents the effects of a network policy.

Isovalent的免费实践实验室“入门Cilium”将引导您在第3/4层和第7层定义网络策略。另一个非常有用的资源是networkpolicy.io上的网络策略编辑器，它可直观呈现网络策略的影响。

As I discussed earlier in this chapter, it’s possible to use iptables rules to drop traffic, and that’s an approach some CNIs have taken to implement Kubernetes NetworkPolicy rules. Cilium uses eBPF programs to drop traffic that doesn’t match the set of rules currently in place. Having seen examples of dropping packets earlier in this chapter, I hope you have a rough mental model for how this would work.

正如我在本章前面所讨论的，可以使用iptables规则来丢弃流量，这是一些容器网络接口（CNI）采用的实现Kubernetes NetworkPolicy规则的方法。Cilium使用eBPF程序来丢弃不符合当前规则集的流量。在本章前面已经看到了丢弃数据包的示例，希望你对这个工作原理有一个大致的心理模型。

Cilium uses Kubernetes identities to determine whether a given network policy rule applies. In the same way labels define which pods are part of a service in Kubernetes, labels also define Cilium's security identity for the pod. eBPF hash tables, indexed by these service identities, make for very efficient rule lookups.

Cilium使用Kubernetes的身份来确定是否适用给定的网络策略规则。就像标签定义哪些pod是Kubernetes服务的一部分一样，标签也定义了Cilium对pod的安全身份。通过这些服务身份索引的eBPF哈希表使规则查找非常高效。

Encrypted Connections

加密连接

Many organizations have requirements to protect their deployments and their users' data by encrypting traffic between applications. This can be achieved by writing code in each application to ensure that it sets up secure connections, typically using mutual Traffic Layer Security (mTLS) underpinning an HTTP or gRPC connection. Setting up these connections requires first establishing the identities of the apps at either end of the connection (which is usually achieved by exchanging certificates) and then encrypting the data that flows between them.

许多组织需要通过加密应用程序之间的通信来保护其部署和用户数据。可以通过在每个应用程序中编写代码来实现此目的，确保它建立安全连接，通常使用互相认证的传输层安全（mTLS）作为HTTP或gRPC连接的基础。建立这些连接需要首先确定连接两端的应用程序

的身份（通常通过交换证书实现），然后加密在它们之间流动的数据。

In Kubernetes, it's possible to offload the requirement from the application, either to a service mesh layer or to the underlying network itself. A full discussion of service mesh is beyond the scope of this book, but you might be interested in a piece I wrote on the new stack: [“How eBPF Streamlines the Service Mesh”](#). Let's concentrate here on the network layer and how eBPF makes it possible to push the encryption requirement into the kernel.

在Kubernetes中，可以将应用程序的需求交给服务网格层或底层网络来处理。对服务网格的全面讨论超出了本书的范围，但你可能对我在The New Stack上写的一篇文章感兴趣：《如何通过eBPF简化服务网格》。在这里，我们将集中讨论网络层以及如何利用eBPF将加密需求推入内核中。

The simplest option to ensure that traffic is encrypted within a Kubernetes cluster is to use *transparent encryption*. It's called “transparent” because it takes place entirely at the network layer and it's extremely lightweight from an operational point of view. The applications themselves don't need to be aware of the encryption at all, and they don't need to set up HTTPS connections; nor does this approach require any additional infrastructure components running under Kubernetes.

确保在Kubernetes集群中的流量得到加密的最简单选择是使用透明加密。它被称为“透明”是因为它完全在网络层进行，并且从运维角度来看非常轻量级。应用程序本身不需要意识到加密的存在，也不需要建

立HTTPS连接；而且这种方法也不需要Kubernetes下运行任何额外的基础设施组件。

There are two in-kernel encryption protocols in common usage, IPsec and WireGuard^(R), and they're both supported in Kubernetes networking by Cilium and Calico CNIs. It's beyond the scope of this book to discuss the differences between these two protocols, but the key point is that they set up a secure tunnel between two machines. The CNI can choose to connect the eBPF endpoint for a pod via this secure tunnel.

在常见的使用中，有两种内核级加密协议，即IPsec和WireGuard，并且它们都由Cilium和Calico CNIs在Kubernetes网络中得到支持。本书的范围不包括讨论这两种协议之间的差异，但关键点是它们在两台机器之间建立了一个安全隧道。CNI可以选择通过这个安全隧道连接Pod的eBPF端点。

Note

There is a nice write-up on the [Cilium blog](#) of how Cilium uses WireGuard^(R) as well as IPsec to provide encrypted traffic between nodes. The post also gives a brief overview of the performance characteristics of both.

Cilium博客上有一篇很好的文章，介绍了Cilium如何使用WireGuard和IPsec来提供节点之间的加密流量。该文章还简要概述了它们的性能特点。

The secure tunnel is set up using the identities of the nodes at either end. These identities are managed by Kubernetes anyway, so the

administrative burden for an operator is minimal. For many purposes this is sufficient as it ensures that all network traffic in a cluster is encrypted. Transparent encryption can also be used unmodified with NetworkPolicy that uses Kubernetes identities to manage whether traffic can flow between different endpoints in the cluster.

安全通道通过两端节点的身份来建立。这些身份由Kubernetes管理，因此操作员的管理负担较小。对于许多用途而言，这已经足够，因为它确保了集群中的所有网络流量都是加密的。透明加密也可以与使用Kubernetes身份来管理集群中不同终端之间是否可以流动的NetworkPolicy一同使用。

Some organizations operate a multitenant environment where there's a need for strong multitenant boundaries and where it's essential to use certificates to identify every application endpoint. Handling this within every application is a significant burden, so it's something that more recently has been offloaded to a service mesh layer, but this requires a whole extra set of components to be deployed, causing additional resource consumption, latency, and operational complexity.

部分组织在多租户环境中运作，在这种环境中需要强大的多租户边界，并且必须使用证书来识别每个应用程序端点。在每个应用程序内处理这个问题是一项重大负担，因此最近这个问题已经被转移到服务网格层，但这需要部署一整套额外的组件，增加了资源消耗、延迟和操作复杂性的问题。

eBPF is now enabling a [new approach](#) that builds on transparent encryption but uses TLS for the initial certificate exchange and

endpoint authentication so that the identities can represent individual applications rather than the nodes they are running on, as depicted in [Figure 8-8](#).

eBPF现在提供了一种新的方法，它建立在透明加密之上，但使用TLS进行初始证书交换和端点身份验证，以便身份可以代表单个应用程序，而不是它们所运行的节点，如图8-8所示。

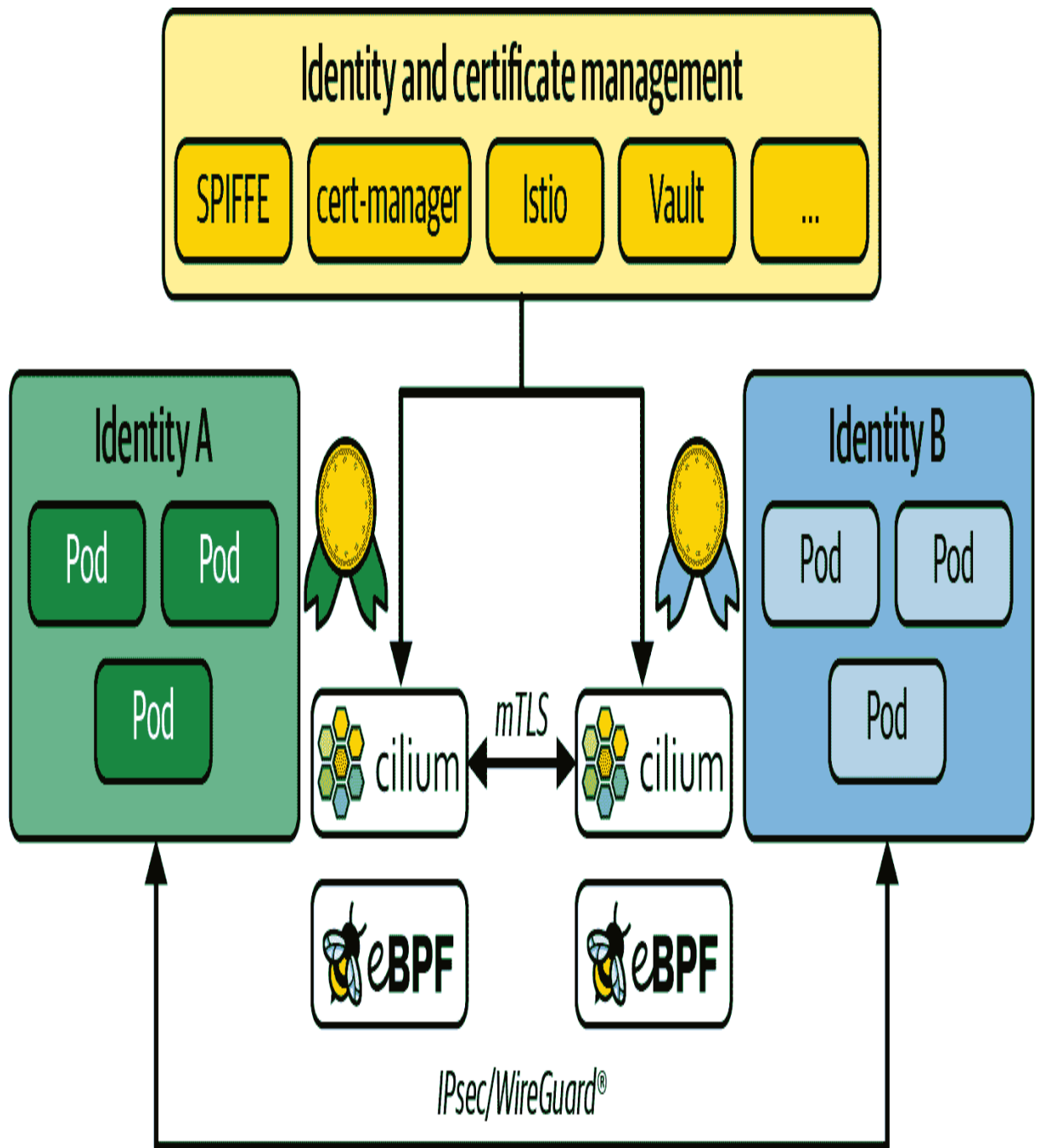


Figure 8-8. Transparent encryption between authenticated application identities

Once the authentication step has taken place, IPsec or WireGuard^(R) within the kernel is used to encrypt the traffic that flows between

those applications. This has a number of advantages. It allows third-party certificate and identity management tools like cert-manager or SPIFFE/SPIRE to handle the identity part, and the network takes care of encryption so that it's all entirely transparent to the application. Cilium supports NetworkPolicy definitions that specify endpoints by their SPIFFE ID rather than just by their Kubernetes labels. And perhaps most importantly, this approach can be used with any protocol that travels in IP packets. That's a big step up from mTLS, which works only for TCP-based connections.

一旦身份验证步骤完成，内核中的IPsec或WireGuard被用于加密这些应用程序之间的流量。这有很多优势。它允许第三方证书和身份管理工具，如cert-manager或SPIFFE/SPIRE来处理身份部分，网络负责加密，使应用程序完全透明。Cilium支持NetworkPolicy定义，通过其SPIFFE ID而不仅仅是Kubernetes标签来指定终端点。最重要的是，这种方法可以与以IP数据包传输的任何协议一起使用。这比仅适用于基于TCP的连接mTLS要进步得多。

There's not enough room in this book to dive deep into all the internals of Cilium, but I hope this section helped you understand how eBPF is a powerful platform for building complex networking functionality like a fully featured Kubernetes CNI.

这本书中没有足够的空间来深入探讨Cilium的所有内部机制，但我希望这一节对你理解eBPF是一个强大的平台来构建复杂的网络功能，比如一个完全功能的Kubernetes CNI，有所帮助。

Summary

摘要

In this chapter you saw eBPF programs attached at a variety of different points in the network stack. I showed examples of basic packet processing, and I hope these gave you an indication of how eBPF can create powerful networking features. You also saw some real-life examples of these networking features, including load balancing, firewalling, security mitigation, and Kubernetes networking.

在本章中，您看到了将eBPF程序附加到网络堆栈的各种不同点。我展示了一些基本的数据包处理示例，并希望这些示例给您提供了eBPF如何创建强大的网络功能的指示。您还看到了一些实际的网络功能示例，包括负载均衡、防火墙、安全性缓解和Kubernetes网络。

Exercises and Further Reading

练习和进一步阅读

Here are some ways to learn more about the range of networking use cases for eBPF:

以下是一些了解eBPF网络应用案例范围的方式：

1. Modify the example XDP program `ping()` so that it generates different trace messages for ping responses and ping requests. The ICMP header immediately follows the IP header in the network packet (just like the IP header follows the Ethernet header). You'll likely want to use `struct icmphdr` from

`linux/icmp.h` and look at whether the type field shows `ICMP_ECHO` or `ICMP_ECHOREPLY`.

修改例子XDP程序ping(), 使其为ping响应和ping请求生成不同的跟踪消息。ICMP头紧随网络数据包中的IP头(就像IP头紧随以太网头一样)。您可能想使用来自linux/icmp.h的struct icmp_hdr, 并查看类型字段是否显示为ICMP_ECHO或ICMP_ECHOREPLY。将函数ping()进行修改, 以生成不同的跟踪消息来区分ping响应和ping请求。网络数据包中的ICMP头跟随在IP头之后(与IP头跟随以太网头类似)。建议使用linux/icmp.h中的struct icmp_hdr, 并检查类型字段是否为ICMP_ECHO或ICMP_ECHOREPLY。

2. If you want to dive further into XDP programming, I recommend the xdp-project's [xdp-tutorial](#).

如果你想深入研究XDP编程, 我推荐xdp-project的xdp-tutorial。

3. Use [sslsniff](#) from the BCC project to view the contents of encrypted traffic.

使用BCC项目中的sslsniff工具查看加密流量的内容。

4. Explore Cilium by using tutorials and labs linked to from the [Cilium website](#).

通过从Cilium 网站链接的教程和实验室, 探索Cilium。

5. Use the editor at [networkpolicy.io](#) to visualize the effect of network policies in a Kubernetes deployment.

使用 networkpolicy.io 上的编辑器来可视化 Kubernetes 部署中网络策略的影响。

¹ At the time of this writing, around 100 organizations have publicly announced their use of Cilium in its [USERS.md file](#), though this number is growing quickly. Cilium has also been adopted by AWS, Google, and Microsoft.

截至撰写本文时，约有100家组织在其USERS.md文件中公开宣布使用Cilium，但这个数字正在快速增长。Cilium还被亚马逊、谷歌和微软采用。

² This example is based on a talk I gave at eBPF Summit 2021 called “[A Load Balancer from scratch](#)”. Build an eBPF load balancer in just over 15 minutes!

这个例子基于我在eBPF峰会2021上的演讲，题为“从头开始构建一个负载均衡器”。在短短15分钟内构建一个eBPF负载均衡器！

³ If you want to explore this, try [CTF Challenge 3 from eBPF Summit 2022](#). I won't give spoilers here in the book, but you can see the solution in [a walkthrough given by Duffie Cooley and me here](#).

如果你想探索这个问题，请尝试参加 2022 年的 eBPF 峰会的 CTF Challenge 3。我不会在这本书中透露任何剧透，但是你可以在 Duffie Cooley 和我这里的解决方案解说中看到答案。

⁴ See Daniel Borkmann's presentation “[Little Helper Minions for Scaling Microservices](#)” that includes a history of eBPF, where he tells this anecdote.

请帮我翻译成简体中文，“请看Daniel Borkmann的演讲“Little Helper Minions for Scaling Microservices”，其中包括eBPF的历史，并讲述

了这个轶事。”请返回仅翻译后的内容，不包括原文。

⁵ Cilium maintains a [list of drivers that support XDP](#) within the [BPF and XDP Reference Guide](#).

Cilium在《BPF和XDP参考指南》中维护着一个支持XDP的驱动程序列表。

⁶ Ceznam shared data about the performance boost its team saw when experimenting with an eBPF-based load balancer in [this blog post](#).

谢楠在这篇博客文章中分享了Ceznam团队在尝试使用基于eBPF的负载均衡器时所见到的性能提升数据。

⁷ For a more complete overview of TC and its concepts, I recommend Quentin Monnet’s post [“Understanding tc “direct action” mode for BPF”](#).

推荐阅读Quentin Monnet的文章“理解BPF的tc“直接操作”模式”，以获得TC和其概念的更全面的概述。

⁸ There is also a blog post that accompanies this example at <https://blog.px.dev/ebpf-openssl-tracing>.

还有一个博客文章与此示例配套，网址为<https://blog.px.dev/ebpf-openssl-tracing>。

⁹ It’s possible for pods to be run in the host’s network namespace so that they share the IP address of the host, but this isn’t usually done

unless there's a good reason for an application running in the pod to require it.

Pods 可以在主机的网络命名空间中运行，这样它们可以共享主机的 IP 地址，但通常情况下会避免这样做，除非 Pod 中的应用程序有必要使用它。

Chapter 9. eBPF for Security

第9章 安全性的eBPF

You've seen how eBPF can be used to observe events across a system and report information about those events to user space tools. In this chapter you'll consider how to build on the concept of event detection to create eBPF-based security tools that can detect, or even prevent, malicious activity. I'll start by helping you understand what makes security different from other types of observability.

你已经看到了eBPF如何用于观察系统中的事件并将有关这些事件的信息报告给用户空间工具。在本章中，您将考虑如何在事件检测的概念上建立，创建基于eBPF的安全工具，可以检测甚至防止恶意活动。我将从帮助您理解安全性与其他类型的可观察性有何不同开始。

Note

Example code for this chapter is in the [GitHub repo](#) in the *chapter9* directory.

本章的示例代码位于GitHub存储库的chapter9目录中。

Security Observability Requires Policy and Context

安全可观察性需要策略和上下文

The difference between a security tool and an observability tool that reports on events is that a security tool needs to be able to distinguish between events that are expected under normal circumstances and events that suggest malicious activity might be taking place. For example, suppose you have an application that writes data to a local file as part of its normal processing. Let's say the app is expected to write to */home/<username>/<filename>*, so this activity isn't something you're interested in from a security perspective. However, you would want to be notified if the app writes to one of the many sensitive file locations in Linux. For example, it is unlikely that it needs to modify the password information stored in */etc/passwd*.

安全工具和事件报告的可观测性工具之间的区别在于，安全工具需要能够区分在正常情况下预期发生的事件和暗示可能发生恶意活动的事件。例如，假设您有一个应用程序，作为其正常处理的一部分，将数据写入到本地文件中。假设该应用程序预计会写入*/home/<用户名>/<文件名>*，因此从安全角度来说，您对这种活动并不感兴趣。但是，如果该应用程序写入Linux中的许多敏感文件位置之一，您将希望收到通知。例如，它可能不需要修改存储在*/etc/passwd*中的密码信息。

Policies have to take into account not just normal behavior when systems are fully functional, but also the expected error path behavior. For example, if a physical disk gets full, the application might start sending network messages to alert about this situation. These network messages shouldn't be seen as security events—even though they are unusual, they're not suspicious. Taking error

paths into account can make it challenging to create effective policies, and we'll come back to that challenge later in this chapter.

策略必须考虑到系统正常运行时的行为，同时也要考虑预期的错误路径行为。例如，如果物理磁盘变满了，应用程序可能会开始发送网络消息来警告这种情况。这些网络消息不应被视为安全事件——虽然它们不寻常，但并不可疑。考虑错误路径可能会使创建有效策略变得有挑战性，我们将在本章后面再谈到这个挑战。

Defining what is and isn't expected behavior is the job of a policy. A security tool compares activity to a policy and takes some action when the activity is outside the policy, making it suspicious. That action would typically involve generating a security event log, which would usually get sent to a Security Information Event Management (SIEM) platform. It might also result in an alert to a human who will be called on to investigate what happened.

定义预期行为与非预期行为的责任落在政策的范畴之内。安全工具会将活动与政策进行对比，当活动超出政策范围时，会采取一些行动使其变得可疑。这些行动通常包括生成安全事件日志，该日志通常会被发送到安全信息事件管理（SIEM）平台。这也可能导致人员接收到警报，需要对所发生的事情进行调查。

The more contextual information that's available to the investigator, the more likely they will be able to find out the root cause of the event and determine whether it was an attack, which components were affected, how and when the attack took place, and who was responsible. As illustrated in [Figure 9-1](#), being able to answer

questions like this takes a tool from mere logging to deserving the nomenclature “security observability.”

调查者拥有更多的语境信息，就越有可能找出事件的根本原因，并确定它是否是一次攻击，受到了哪些组件的影响，攻击是如何发生的，以及谁负责。如图9-1所示，能够回答类似这样的问题将使工具从简单的日志记录提升到“安全可观察性”的称号。

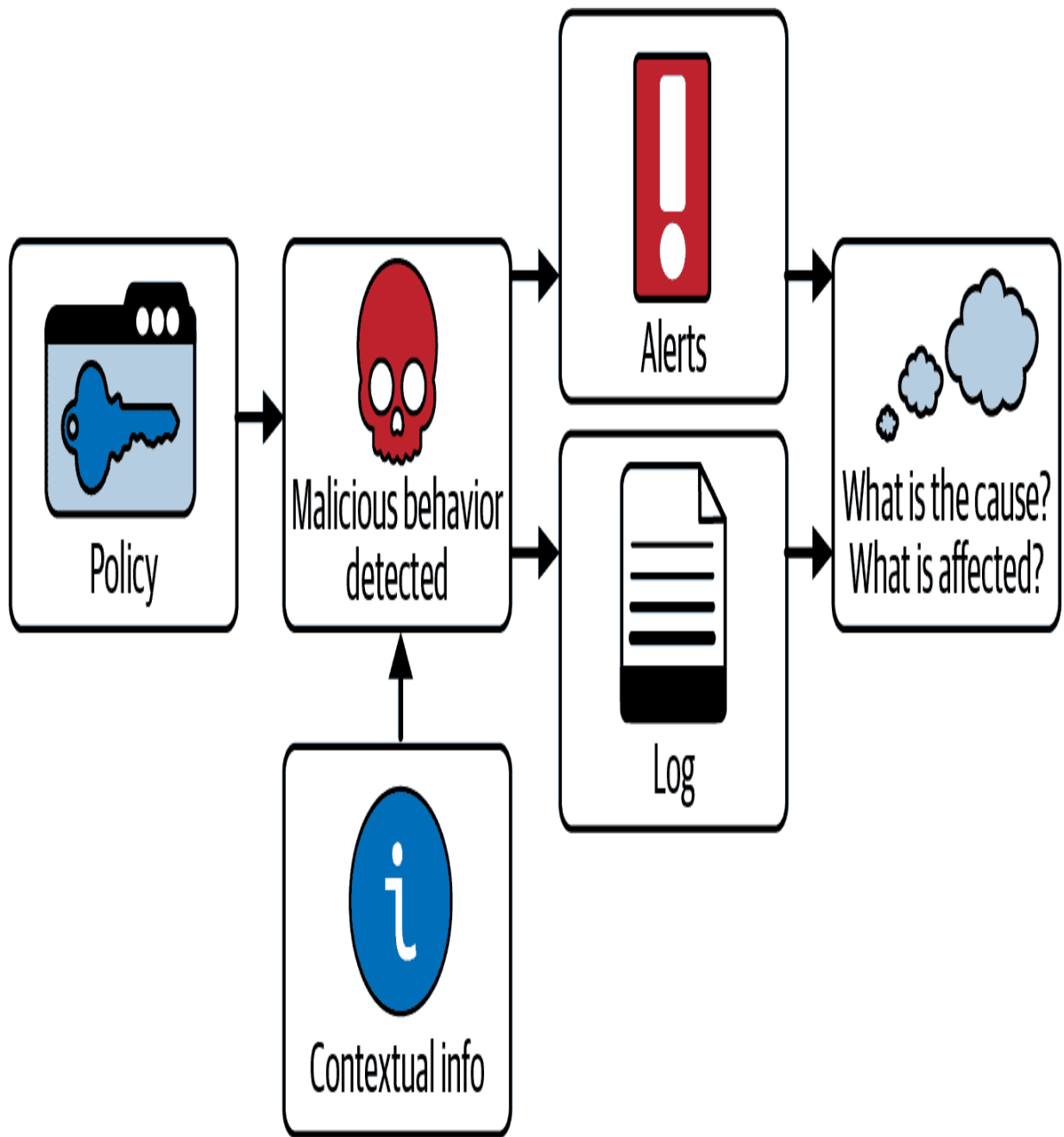


Figure 9-1. Contextual information is required alongside out-of-policy event detection for security observability

Let's explore some of the ways eBPF programs are being used to detect and enforce security events. As you know, eBPF programs can be attached to a variety of events, and one set of events that

have been commonly used for security for many years are syscalls. We'll start our discussion with syscalls, but as you'll see, syscalls might not be the most effective way to implement security tooling with eBPF. We'll see some newer and more sophisticated approaches later in this chapter.

让我们来探索一下如何使用eBPF程序来检测和执行安全事件。正如您所了解的，eBPF程序可以附加到各种事件，而长期以来一直被广泛用于安全性的一组事件是系统调用。我们将从系统调用开始讨论，但正如您将看到的，系统调用可能不是使用eBPF实施安全工具的最有效方式。在本章稍后我们将看到一些更新和更复杂的方法。

Using System Calls for Security Events

使用系统调用进行安全事件

System calls (or syscalls) are the interface between user space applications and the kernel. If you can restrict the set of syscalls an app can make, that will limit what it is able to do. For example, if you stop an app from making syscalls in the `open*()` family, it won't be able to open files. If you have an application that you never expect to open files, you might want to create this limitation so that even if the app gets compromised, it won't be able to open files maliciously. If you've been using Docker or Kubernetes over the past few years, there is a very good chance you have already come across a security tool that uses BPF to limit syscalls: seccomp.

系统调用（或称为syscalls）是用户空间应用程序与内核之间的接口。如果你可以限制应用程序可以进行的系统调用集合，那将限制其能够做的事情。举个例子，如果你阻止应用程序在`open*()`函数族中进行系统调用，它将无法打开文件。如果你有一个应用程序，你永远不希望它打开文件，那么你可能希望创建这个限制，即使应用程序被入侵，它也无法恶意打开文件。如果你在过去几年中一直在使用Docker或Kubernetes，那么你很有可能已经遇到过使用BPF限制系统调用的安全工具：seccomp。

Seccomp

系统调用过滤器

The name *seccomp* is a contraction of “SECure COMputing.” In its original, or “strict,” form, seccomp is used to limit the set of syscalls a process can use to a very small subset: `read()`, `write()`, `_exit()`, and `sigreturn()`. The intention of this strict mode was to allow users to run untrusted code (perhaps a program downloaded from the internet) without any possibility of that code doing malicious things.

seccomp的名字是“SECure COMputing”的缩写。在其原始或“严格”形式中，seccomp被用于将进程可以使用的系统调用集合限制为非常小的子集：`read()`、`write()`、`_exit()`和`sigreturn()`。这种严格模式的目的是允许用户运行不可信任的代码（比如从互联网下载的程序），而不会造成任何恶意行为的可能性。

Strict mode is very restrictive, and many applications need to use a much larger set of syscalls—but that doesn’t mean they need all 400 and more of them. It makes sense to allow a more flexible method

for restricting the set that any given application can use. This is the reasoning behind the flavor of seccomp that most of us from the container world have encountered, which is more properly known as seccomp-bpf. Instead of having a fixed subset of syscalls that it permits, this mode of seccomp uses BPF code to filter the syscalls that are and aren't allowed.

严格模式非常严格，许多应用程序需要使用更大的系统调用集合，但这并不意味着它们需要全部400多个系统调用。允许更灵活的方法来限制任何给定应用程序可以使用的集合是有道理的。这就是我们从容器世界中遇到的seccomp的推理，更准确地说是seccomp-bpf。与允许一组固定子系统调用的方式不同，这种seccomp模式使用BPF代码来过滤允许和不允许的系统调用。

In seccomp-bpf, a set of BPF instructions are loaded that act as a filter. Each time a syscall is called, the filter is triggered. The filter code has access to the arguments that are passed to the syscall so that it can make decisions based on both the syscall itself and the arguments that have been passed to it. The outcome is one of a set of possible actions that include:

在seccomp-bpf中，加载了一组充当过滤器的BPF指令。每次调用系统调用时，都会触发该过滤器。过滤器代码可以访问传递给系统调用的参数，以便它可以根据系统调用本身和传递给它的参数做出决策。结果是一组可能的操作，包括：

- Allowing the syscall to go ahead

允许系统调用继续执行

- Returning an error code to the user space application
返回错误代码给用户空间应用程序
- Killing the thread
结束线程
- Notifying a user space application (seccomp-unotify) (as of kernel version 5.0)
通知用户空间应用程序 (seccomp-unotify) (适用于内核版本 5.0)

Note

If you want to explore writing your own BPF filter code, Michael Kerrisk has some good examples at <https://oreil.ly/cJ6HL>.

如果你想探索编写自己的BPF过滤器代码，Michael Kerrisk在<https://oreil.ly/cJ6HL>有一些很好的例子。

Some of the arguments passed to syscalls are pointers, and the BPF code in seccomp-bpf is not able to dereference these pointers. This limits the flexibility of a seccomp profile, as it can only use value arguments in its decision-making process. Also, it has to be applied to the process when it starts—you can't modify the profile that is being applied to a given application process.

一些传递给系统调用的参数是指针，而seccomp-bpf中的BPF代码无法对这些指针进行解引用。这限制了seccomp配置文件的灵活性，因

为它只能在决策过程中使用值参数。此外，它必须在进程启动时应用，不能修改应用程序进程所使用的配置文件。

You may well have used seccomp-bpf without writing BPF code, as the code is often derived from a human-readable seccomp profile.

[Docker's default profile](#) is a good example. This is a general-purpose profile intended to be usable with pretty much any normal, containerized application. That inevitably means it allows most syscalls and disallows only a few that are unlikely to be appropriate in any application, `reboot()` being a great example.

您可能已经在不编写BPF代码的情况下使用了seccomp-bpf，因为代码通常是从可读的seccomp配置文件中派生的。Docker的默认配置文件是一个很好的例子。这是一个通用配置文件，可用于几乎任何正常的容器化应用程序。这必然意味着它允许大多数系统调用，只禁止了一些在任何应用程序中都不太可能适用的调用，比如reboot()。

[According to Aqua Security](#), most containerized apps use somewhere in the range of 40 to 70 syscalls. For better security, it would be preferable to use a more constrained profile that is targeted at each specific application and only allows the syscalls it actually uses.

根据Aqua Security的说法，大多数容器化应用程序使用的系统调用在40到70个之间。为了更好的安全性，最好使用一个更受限制的配置文件，针对每个特定应用程序并且只允许实际使用的系统调用。

Generating Seccomp Profiles

生成 Seccomp 话题

If you ask the average app developer to tell you what syscalls one of their programs makes, you're likely to get a blank look. That's not intended to be insulting. It's just that most developers write in programming languages that give them higher-level abstractions far removed from the details of syscalls. For example, they might know what files their application opens, but it's less likely that they could tell you whether they are opened using `open()` or `openat()`. This makes it unlikely that you'll get a positive response if you ask the developer to handcraft an appropriate seccomp profile along with their application code.

如果你询问一个普通的应用程序开发者，让他告诉你他们的程序使用了哪些系统调用，你可能会得到一脸茫然的表情。这并非是要冒犯他们，只是因为大多数开发者使用的编程语言给予了他们更高级的抽象，远离系统调用的细节。例如，他们可能知道他们的应用程序打开了哪些文件，但很少能告诉你它们是使用了`open()`还是`openat()`。这意味着如果你要求开发者在他们的应用程序代码中手动创建一个适当的seccomp配置文件，你可能得不到积极的反应。

Automation is the way forward: the idea is to use a tool to record the set of syscalls an application makes. In the early days, seccomp profiles were generally compiled using `strace` to gather the set of syscalls an application calls.¹ This isn't a wonderful solution in the cloud native age, as there's no easy way to point `strace` at a specific container or Kubernetes pod. It would also be more helpful to generate the profile not just as a list of syscalls, but in the JSON

format that Kubernetes and OCI-compatible container runtimes can take as input. There are a couple of tools that do this, using eBPF to gather information about all the syscalls being called:

自动化是前进的道路：这个想法是使用工具记录应用程序所调用的系统调用集合。在早期，通常使用strace编译seccomp配置文件来收集应用程序调用的系统调用集合。但在云原生时代，这并不是一个理想的解决方案，因为没有一种简单的方法将strace指向特定的容器或Kubernetes pod。生成配置文件不仅仅作为系统调用列表，而是采用Kubernetes和OCI兼容容器运行时可以作为输入的JSON格式将更加有帮助。有一些工具可以使用eBPF来收集所有被调用的系统调用的信息。

- [Inspektor Gadget](#) includes a seccomp profiler that allows you to generate a custom seccomp profile for the containers in a Kubernetes pod.²

Inspektor Gadget包含一个seccomp分析器，可以为Kubernetes pod中的容器生成自定义的seccomp配置文件。

- Red Hat created a seccomp profiler in the form of an [OCI runtime hook](#).

红帽公司创建了一个以OCI运行时钩子形式的seccomp分析器。

With these profilers, you need to run the application for some arbitrary amount of time to generate a profile that includes the full list of the syscalls it might legitimately call. As discussed earlier in this chapter, this list needs to include error paths. If your application can't behave correctly under error conditions because the syscalls it

needs to call are blocked, this might cause a bigger problem. And since seccomp profiles deal with a lower abstraction level than most developers are familiar with, it's hard to review them manually to see if they cover all the right cases.

使用这些分析器，需要运行应用程序一段任意的时间来生成一个包含其可能合法调用的系统调用列表的分析结果。如本章前面所讨论的，这个列表需要包括错误处理路径。如果你的应用程序在错误条件下无法正确运行，因为它需要调用的系统调用被阻塞了，这可能会引发更大的问题。由于seccomp分析器处理的是比大多数开发者熟悉的更低层次的抽象，手动审查它们以确保它们涵盖了所有正确的情况是困难的。

Taking the OCI runtime hook as an example, an eBPF program is [attached to the `syscall\_enter` raw tracepoint](#) and maintains an eBPF map that keeps track of [which syscalls have been seen](#). The user space parts of this tool are written in Go and use the [iovisor/gobpf library](#). (I'll discuss this and other Golang libraries for eBPF in [Chapter 10](#).)

以OCI运行时钩子为例，将一个eBPF程序附加到syscall_enter原始跟踪点，并维护一个eBPF映射以跟踪已经被触发的系统调用。此工具的用户空间部分使用Go编写，并使用iovisor/gobpf库。（我将在第10章讨论这个以及其他针对eBPF的Golang库。）

The following are the [lines of code](#) from the OCI runtime hook that load the eBPF program into the kernel and attach it to the tracepoint (a few lines have been omitted for brevity):

以下是OCI运行时钩子的代码行，用于将eBPF程序加载到内核并将其附加到跟踪点（为简洁起见，已省略几行）。

```
src := strings.Replace(source, "$PARENT_PID", strconv.Itoa
(pid), -1) ❶
m := bcc.NewModule(src, []string{})
defer m.Close()

...
enterTrace, err := m.LoadTracepoint("enter_trace") ❷
...
if err := m.AttachTracepoint("raw_syscalls:sys_enter",
enterTrace); err != nil ❸
{
    return fmt.Errorf("error attaching to tracepoint: %v",
err)
}
```

❶

This line does something quite interesting: it replaces a variable named `$PARENT_PID` in the eBPF source code with a numeric process ID. This is a common pattern, and it indicates that this tool will load individual eBPF programs for each process being instrumented.

❷

Here, an eBPF program called `enter_trace` gets loaded into the kernel.

❸

The `enter_trace` program gets attached to the tracepoint `raw_syscalls:sys_enter`. This is the tracepoint at the point of

entry to any syscall, which you've encountered in earlier examples. Whenever any user space code makes a syscall, this tracepoint will be hit.

These profilers use eBPF code attached to `sys_enter` to keep track of the set of syscalls that have been used, and they generate a seccomp profile to be used with seccomp, which does the actual job of enforcing the profile. The next class of eBPF tools we'll consider also attach to `sys_enter`, but they use syscalls to track the behavior of an application and compare it against security policies.

这些分析器使用与`sys_enter`关联的eBPF代码来跟踪已使用的系统调用集，并生成一个用于seccomp的seccomp配置文件，seccomp的实际工作是强制执行该配置文件。下一个类别的eBPF工具也连接到`sys_enter`，但它们使用系统调用来跟踪应用程序的行为并将其与安全策略进行比较。

Syscall-Tracking Security Tools

系统调用跟踪安全工具

The best-known tool that falls into this category of syscall-tracking security tools is the CNCF project [Falco](#), which provides security alerts. By default, Falco is installed as a kernel module, but there is also an eBPF version. Users can define [rules](#) to determine what events are security relevant, and Falco can generate alerts in a variety of formats when events happen that don't match the policies defined in these rules.

这个属于系统调用跟踪安全工具类别中最为知名的工具是CNCF项目 Falco，它提供安全警报。默认情况下，Falco被安装为内核模块，但也有一种eBPF版本。用户可以定义规则来确定哪些事件与安全相关，当事件不符合这些规则中定义的策略时，Falco可以生成各种格式的警报。

Both the kernel module driver and the eBPF-based driver attach to system calls. If you examine the [Falco eBPF programs on GitHub](#) you'll see lines like the following that attach probes to raw syscall entry and exit points (plus a few other events, such as page faults):

内核模块驱动程序和基于eBPF的驱动程序都会连接到系统调用。如果你在GitHub上查看Falco的eBPF程序，你会发现一些如下的行代码，它们会将探针连接到原始系统调用进入和退出点上（还有一些其他事件，如页面错误）。

```
BPF_PROBE("raw_syscalls/", sys_enter, sys_enter_args)

BPF_PROBE("raw_syscalls/", sys_exit, sys_exit_args)
```

Since eBPF programs can be loaded dynamically and can detect events triggered by preexisting processes, tools like Falco can apply policies to application workloads that are already running. Users can modify the set of rules being applied without having to modify the applications or their configuration. This is in contrast to seccomp profiles, which have to be applied to the application process when it is launched.

由于eBPF程序可以动态加载并且可以检测到现有进程触发的事件，像 Falco这样的工具可以将策略应用于已经运行的应用工作负载。用户可

以在不修改应用程序或其配置的情况下修改应用的规则集。这与在应用程序启动时应用seccomp配置文件的方式有所不同。

Unfortunately there is a problem with this approach of using syscall entry points for security tooling: there is a Time Of Check to Time Of Use (TOCTOU) issue.

很遗憾，使用系统调用入口点进行安全工具开发的方法存在一个问题：这里涉及到了检查时机与使用时机（TOCTOU）的问题。

When an eBPF program is triggered at the entry point to a system call, it can access the arguments that user space has passed to that system call. If those arguments are pointers, the kernel will need to copy the pointed-to data into its own data structures before acting on that data. As illustrated in [Figure 9-2](#), there is a window of opportunity for an attacker to modify this data, after it has been inspected by the eBPF program but before the kernel copies it. Thus, the data being acted on might not be the same as what was captured by the eBPF program.³

当eBPF程序在系统调用的入口点触发时，它可以访问用户空间传递给该系统调用的参数。如果这些参数是指针，内核需要在操作这些数据之前将指向的数据复制到自己的数据结构中。如图9-2所示，存在一个攻击者在eBPF程序检查数据后但内核复制之前修改此数据的机会窗口。因此，正在操作的数据可能与eBPF程序捕获的数据并不相同。当eBPF程序在系统调用的入口点被触发时，它可以访问用户空间向该系统调用传递的参数。如果这些参数是指针，内核将在操作这些数据之前需要将指针所指向的数据复制到自己的数据结构中。如图9-2所示，攻击者可以在eBPF程序检查完数据后但内核复制之前修改该数据

的窗口时机，因此，在实际操作时的数据可能与eBPF程序捕获到的数据不同。

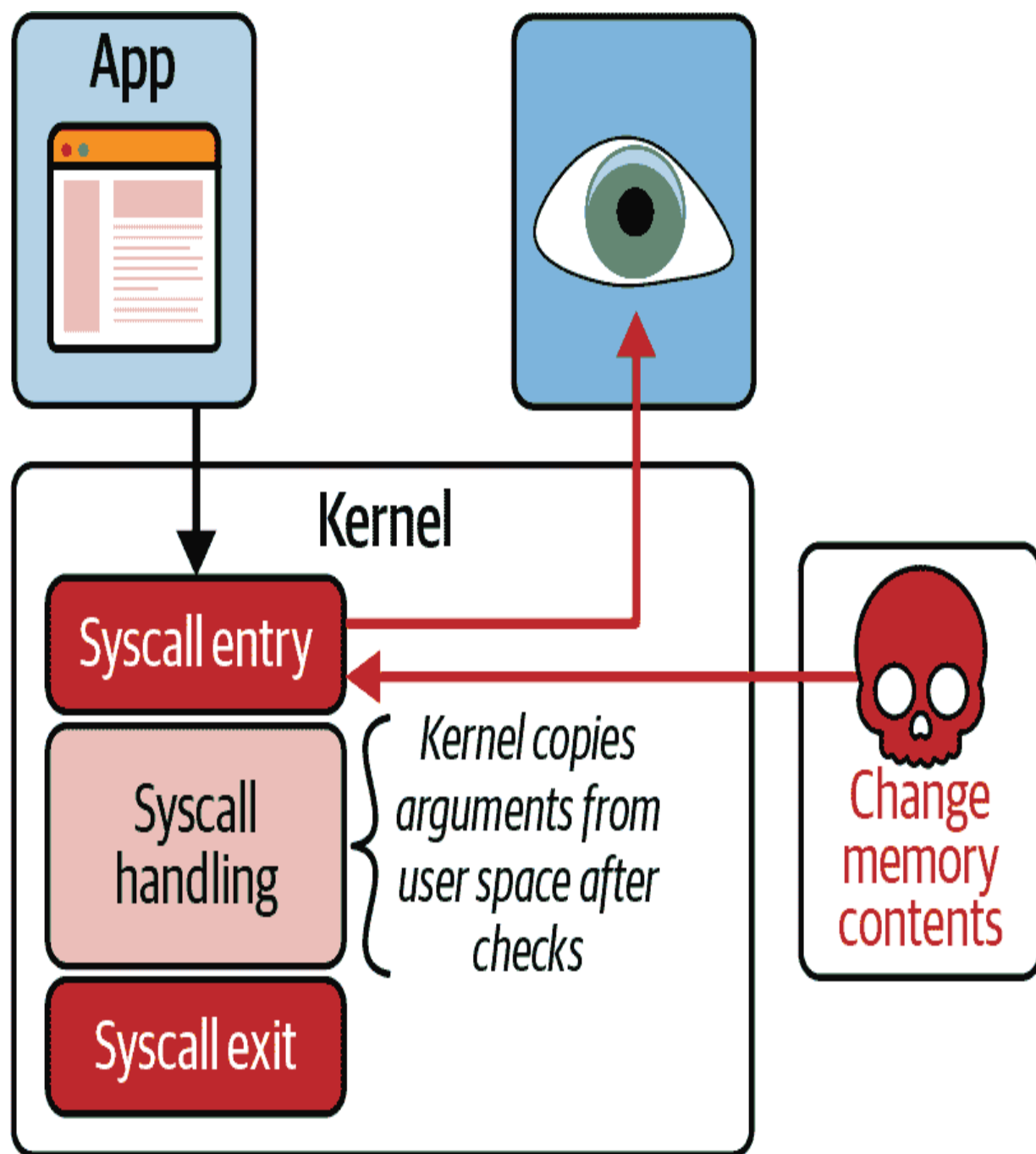


Figure 9-2. An attacker can change syscall arguments before they are accessed by the kernel

The same window would apply for seccomp-bpf, were it not for the fact that in seccomp-bpf the program is not allowed to dereference the user space pointer, so it's not possible to examine the data at all.

如果不考虑一个问题的话，对于seccomp-bpf来说应用相同的窗口效果是适用的。这个问题就是在seccomp-bpf中，程序是不被允许解引用用户空间指针的，因此无法检查数据。

The TOCTOU issue does apply for seccomp_unotify, a recently added mode of seccomp where a violation can be reported to user space. The [manpage for seccomp_unotify](#) explicitly notes that “It should thus be absolutely clear that the seccomp user-space notification mechanism *cannot* be used to implement a security policy!”

TOCTOU问题适用于seccomp_unotify，这是seccomp的最近添加的一种模式，可以将违规情况报告给用户空间。seccomp_unotify的man页明确指出：“因此，应当明确的是，seccomp用户空间通知机制不能用于实现安全策略！”

The syscall entry point might be very convenient for observability purposes, but for a serious security tool it's really not sufficient.

系统调用入口点对于可观察性目的可能非常方便，但对于严肃的安全工具来说并不足够。

The [Sysmon for Linux tool](#) addresses the TOCTOU window by attaching to both the entry and exit points for syscalls. Once the call has completed, it looks at the kernel's data structures to get an accurate view. For example, if the syscall returns a file descriptor, the

eBPF program attached to the exit can retrieve correct information about the object that the file descriptor represents by looking into the related process's file descriptor table. While this approach can result in an accurate record of security-related activity, it can't prevent an action from taking place, since the syscall has already completed by the time a check is made.

Sysmon for Linux 工具通过连接系统调用的进入点和退出点来解决 TOCTOU 窗口问题。一旦调用完成，它会查看内核的数据结构以获取准确的视图。例如，如果系统调用返回一个文件描述符，附加到退出点的 eBPF 程序可以通过查看相关进程的文件描述符表，来获取关于该文件描述符所代表对象的正确信息。尽管这种方法可以准确记录与安全相关的活动，但它不能阻止操作的发生，因为在进行检查时，系统调用已经完成。

To be certain that it is inspecting the same information the kernel will act on, the eBPF program should be attached to an event that occurs after the parameters have been copied into kernel memory. Unfortunately, there is no single common place in the kernel to do this, as the data is handled differently in syscall-specific code. However, there is a well-defined interface where eBPF programs can be safely attached: the Linux Security Module (LSM) API. This requires a relatively new eBPF feature: BPF LSM.

为了确保它在检查内核将要执行的相同信息，eBPF程序应该被附加到在参数被复制到内核内存之后发生的事件上。不幸的是，内核中没有单一的共同位置来完成这个操作，因为数据在特定于系统调用的代码中被处理方式不同。然而，在这里存在一个明确定义的接口，

eBPF程序可以安全地附加：Linux安全模块（LSM）API。这需要一个相对较新的eBPF功能：BPF LSM。

BPF LSM

BPF LSM will be translated to: BPF LSM 将被翻译为：

The LSM interface provides a set of hooks that each occur just before the kernel is about to act on a kernel data structure. The function called by a hook can make a decision about whether to allow the action to go ahead. This interface was originally provided to allow security tools to be implemented in the form of [kernel modules](#); [BPF LSM](#) extends this so that eBPF programs can be attached to the same hook points, as shown in [Figure 9-3](#).

LSM接口提供一组挂钩函数，在内核准备对内核数据结构进行操作之前触发。挂钩函数可以决定是否允许操作继续进行。这个接口最初是为了允许以内核模块形式实现安全工具而提供的；BPF LSM扩展了这个接口，使得eBPF程序可以附加到相同的挂钩点上，如图9-3所示。

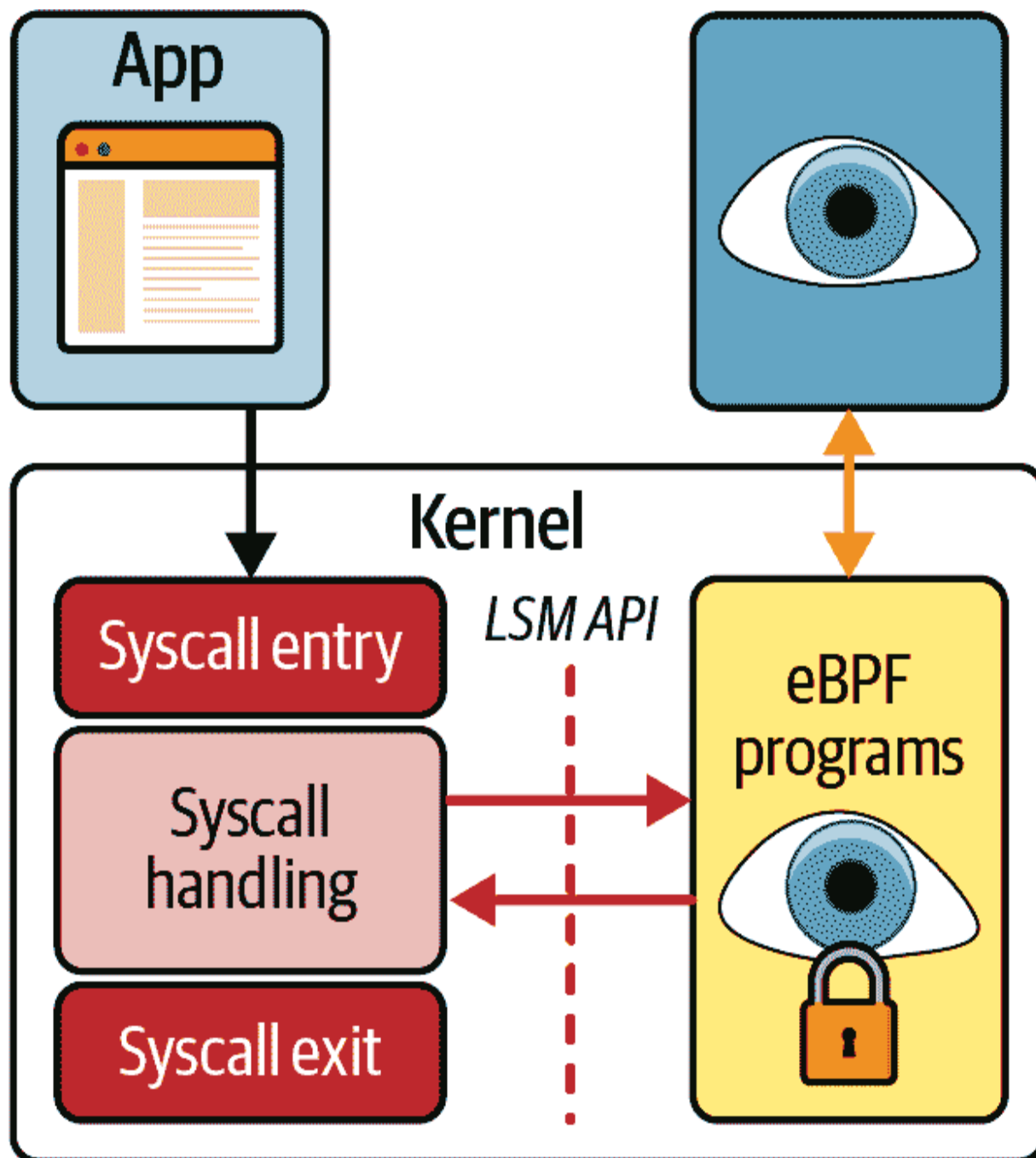


Figure 9-3. With LSM BPF, eBPF programs can be triggered by LSM hook events

There are hundreds of LSM hooks, and they're pretty nicely [documented in the kernel source code](#). To be clear, there isn't a one-

to-one mapping between syscalls and LSM hooks, but if a syscall has the potential to do something interesting from a security perspective, processing that syscall will trigger one or more of the hooks.

有数以百计的LSM钩子，并且在内核源代码中有很好的文档记录。请注意，系统调用与LSM钩子之间没有一对一的映射关系，但如果系统调用在安全角度上有可能执行一些有趣的操作，处理该系统调用将触发一个或多个钩子。

Here's a trivial example of an eBPF program attached to an LSM hook. This example is called during the processing of a `chmod` command ("chmod" stands for "change modes" and is mostly used to change the access permissions on a file):

这是一个eBPF程序附加到LSM钩子的一个简单例子。在处理chmod命令时调用此示例（“chmod”代表“更改模式”，主要用于更改文件的访问权限）：

```
SEC("lsm/path_chmod")
int BPF_PROG(path_chmod, const struct path *path, umode_t
mode)
{
    bpf_printk("Change mode of file name %s\n", path->
dentry->d_iname);
    return 0;
}
```

This example simply traces out the name of the file and always returns `0`, but you can imagine a real implementation that would make use of the arguments to decide whether to allow this change of mode. Returning a nonzero value would deny permission to make

this change, so the kernel wouldn't go ahead with it. It's worth noting that making policy checks entirely within the kernel like this is highly performant.

这个例子仅仅追踪文件名，并始终返回0，但你可以想象一个真实的实现，在决定是否允许这种模式改变时使用参数。返回非零值将拒绝允许这种改变，因此内核不会继续执行。值得注意的是，像这样完全在内核中进行策略检查的方式具有高性能。

The `path` argument to `BPF_PROG()` is the kernel data structure representing the file, and the `mode` argument is the desired new mode value. You can see the name of the file being accessed from the field `path->dentry->d_iname`.

`BPF_PROG()` 的路径参数是表示文件的内核数据结构，而模式参数是期望的新模式值。你可以从 `path->dentry->d_iname` 字段中看到正在访问的文件的名称。

LSM BPF was added in kernel version 5.7, which means that (at least at the time of this writing) it's not yet available on many supported Linux distributions, but I expect that over the next couple of years many vendors will develop security tooling that makes use of this interface. Before LSM BPF is made widely available, there is another possible approach, as used by the developers of Cilium Tetragon.

LSM BPF在内核版本5.7中添加了，这意味着（至少在撰写本文时）它尚未在许多支持的Linux发行版中提供，但我预计在接下来的几年里，许多供应商将开发利用这个接口的安全工具。在LSM BPF普及之

前，还有另一种可能的方法，正如Cilium Tetragon的开发所使用的一样。

Cilium Tetragon

四方眼鱼鳍

Tetragon is part of the Cilium project (also part of the CNCF). Rather than attaching to LSM API hooks, Tetragon's approach is to build a framework for attaching eBPF programs to arbitrary functions in the Linux kernel.

Tetragon是Cilium项目的一部分（也是CNCF的一部分）。与将其附加到LSM API挂钩不同，Tetragon的方法是构建一个框架，用于将eBPF程序附加到Linux内核的任意函数上。

Tetragon is designed for use in a Kubernetes environment, and the project defines a custom Kubernetes resource type called a *TracingPolicy*. This is used to define a set of events to which eBPF programs should be attached, conditions that need to be checked by eBPF code, and actions to take if the conditions are met. The following is an extract from a sample TracingPolicy:

Tetragon被设计用于Kubernetes环境中，并且该项目定义了一种名为TracingPolicy的自定义Kubernetes资源类型。这被用于定义一组应该附加eBPF程序的事件，需要由eBPF代码检查的条件，以及如果满足条件应采取的操作。以下是示例TracingPolicy的摘录： Tetragon被设计用于在Kubernetes环境中使用，该项目定义了一种名为TracingPolicy的自定义Kubernetes资源类型。它用于定义一组需要附

加eBPF程序的事件、eBPF代码需要检查的条件以及满足条件时所采取的操作。以下是示例TracingPolicy的摘录：

```
spec:
  kprobes:
    - call: "fd_install"
    ...
    matchArgs:
      - index: 1
        operator: "Prefix"
        values:
          - "/etc/"
    ...
```

This policy defines a set of kprobes to attach programs to, the first of which is the kernel function `fd_install`. This is an internal function in the kernel. Let's explore why you might choose to attach to a function like that.

该策略定义了一组kprobes，用于将程序附加到其中，其中第一个函数是内核函数`fd_install`。这是内核中的一个内部函数。让我们探讨一下为什么你会选择附加到这样的函数。

Attaching to Internal Kernel Functions

内核函数的附加

The system call interface and the LSM interface are defined as stable interfaces in the Linux kernel; that is to say, they won't change in a backward-incompatible way. If you write code today that uses the functions in those interfaces, they will continue to work in future versions of the kernel. These interfaces represent just a tiny fraction of the 30 million lines of code that make up the Linux kernel. Parts of

that codebase are de facto stable, even if they are not officially declared as such; they haven't changed for a long time and are unlikely to do so in the future.

系统调用接口和LSM接口在Linux内核中被定义为稳定接口；换句话说，它们不会以不兼容的方式进行更改。如果您今天编写的代码使用了这些接口中的函数，在将来的内核版本中它们将继续工作。这些接口仅代表了构成Linux内核的3000万行代码的一小部分。尽管这些代码的一部分尚未正式声明为稳定，但它们已经很长时间没有更改，并且将来也不太可能更改。

It's perfectly reasonable to write eBPF programs that attach to kernel functions that aren't officially stable, with the expectation that they are likely to work for some considerable time to come. Also, given that it typically takes several years for a new kernel version to be widely deployed, it's a safe bet that there will be plenty of time to address any incompatibility issues that might arise.

编写附加到未正式稳定的核心函数的eBPF程序，并期望它们在相当长的时间内能够正常工作是完全合理的。此外，考虑到新的内核版本通常需要几年时间才能被广泛部署，可以放心地认为会有足够的时间来解决可能出现的任何不兼容问题。

The Tetragon contributors include a number of kernel developers who have used their knowledge of kernel internals to identify some good, safe places where eBPF programs can be attached for useful security purposes. There are several [example TracingPolicy definitions](#) that make use of this knowledge. These examples monitor security events covering file operations, network activity,

program executions, and changes to privileges—all the kinds of things a malicious actor would do as part of an attack.

Tetragon的贡献者包括一些内核开发人员，他们利用他们对内核内部的了解，找到了一些适合附加eBPF程序以供安全用途的良好且安全的位置。有一些示例的TracingPolicy定义利用了这些知识。这些示例监控了涵盖文件操作、网络活动、程序执行以及特权更改的安全事件，所有这些都是恶意行为者在攻击中会执行的类型。

Let's come back to that example policy definition that attaches to `fd_install`. The “fd” stands for “file descriptor,” and the [comment in the source code for this function](#) tells us this function “Install[s] a file pointer in the fd array.” This happens when a file is opened, and it's called after the file's data structure has been populated in the kernel. It's a safe place to check the name of the file—and in the earlier TracingPolicy example, it's only of interest if the filename starts with “/etc”.

让我们回到附加在fd_install上的示例策略定义。“fd”代表“文件描述符”，源代码中对该函数的注释告诉我们这个函数“在fd数组中安装一个文件指针”。这发生在打开文件时，在文件的数据结构在内核中被填充之后被调用。这是一个安全的地方来检查文件名，而且在前面的TracingPolicy示例中，只有当文件名以“/etc/”开头时才感兴趣。

Just as in LSM BPF programs, Tetragon eBPF programs have access to contextual information to allow them to make security decisions entirely within the kernel. Rather than reporting all events of a given type to user space, security-relevant events can be filtered

within the kernel so that only the out-of-policy events get reported to user space.

就像在LSM BPF程序中一样，Tetragon eBPF程序可以访问上下文信息，使其能够完全在内核中做出安全决策。与将给定类型的所有事件报告给用户空间不同，安全相关事件可以在内核中进行过滤，只报告违反策略的事件给用户空间。

Preventative Security

防范性安全 (Fángfàn Xìng Ānquán)

Most eBPF-based security tools have used eBPF programs to detect malicious events, which notify a user space application that can then take action. As you can see in [Figure 9-4](#), any action the user space app takes happens asynchronously, by which time it might be too late—perhaps data could have been exfiltrated, or the attacker could have persisted malicious code onto disk.

大多数基于eBPF的安全工具使用eBPF程序来检测恶意事件，并通知用户空间应用程序采取行动。如图9-4所示，用户空间应用程序采取的任何行动都是异步进行的，到那时可能已经太晚了，数据可能已被外泄，或者攻击者可能已将恶意代码持久化到磁盘上。

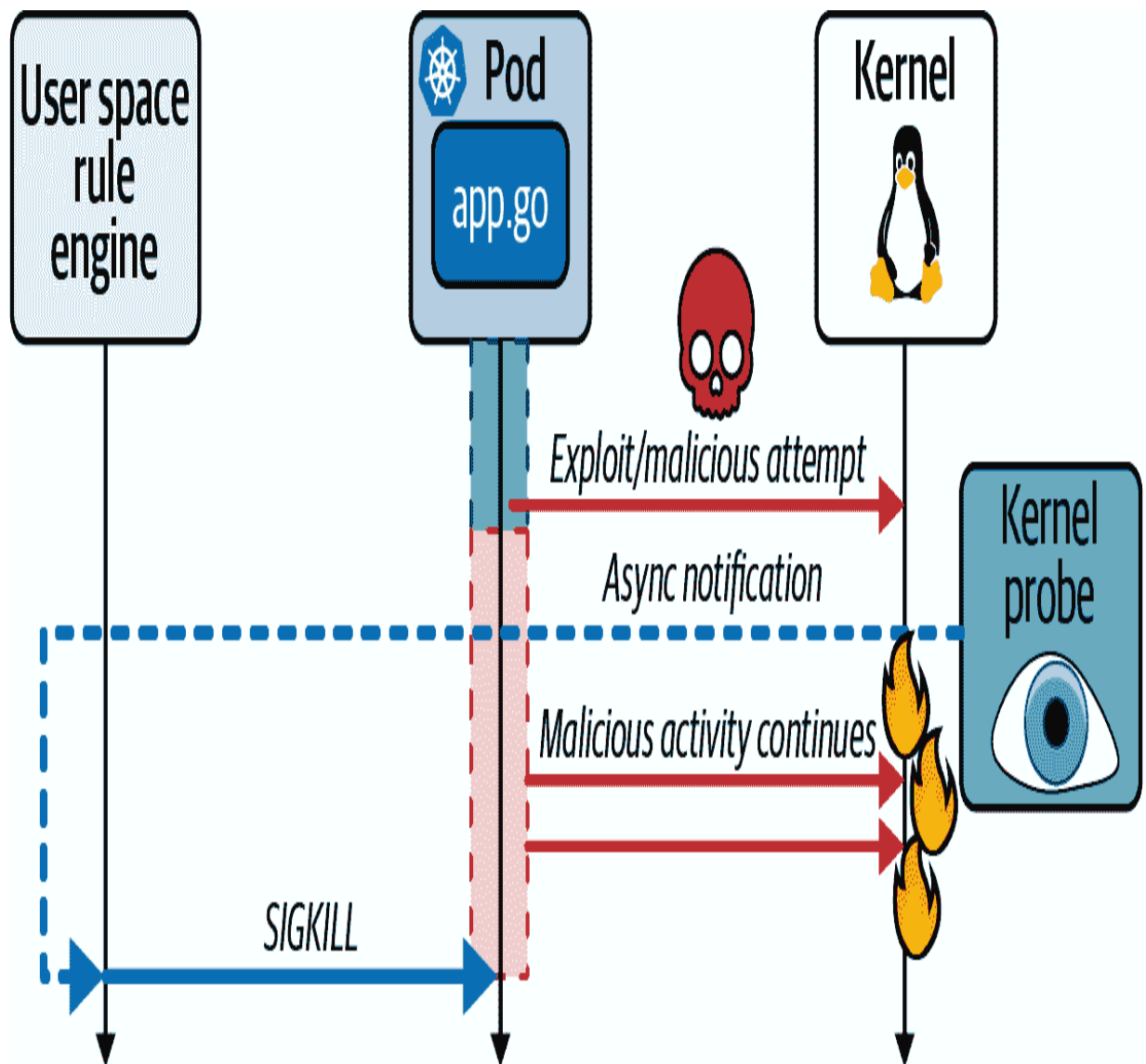


Figure 9-4. An asynchronous notification from kernel to user space allows some time for an attack to continue

In kernel versions 5.3 and up, there is a BPF helper function called `bpf_send_signal()`. Tetragon uses this function to implement preventative security. If a policy defines a Sigkill action, any matching events will cause Tetragon eBPF code to generate a SIGKILL signal that terminates the process that was attempting the

out-of-policy action. As shown in [Figure 9-5](#), this happens synchronously; that is, the activity the kernel was performing that the eBPF code determined to be out of policy is prevented from completing.

在内核版本5.3及以上，有一个称为**bpf_send_signal()**的BPF辅助函数。Tetragon使用此函数来实现预防性安全。如果策略定义了Sigkill动作，则任何匹配的事件将导致Tetragon的eBPF代码生成一个SIGKILL信号，终止试图超出策略的动作的进程。如图9-5所示，这发生在同步方式下；也就是说，内核正在执行的活动被eBPF代码确定为超出策略的活动将被阻止完成。

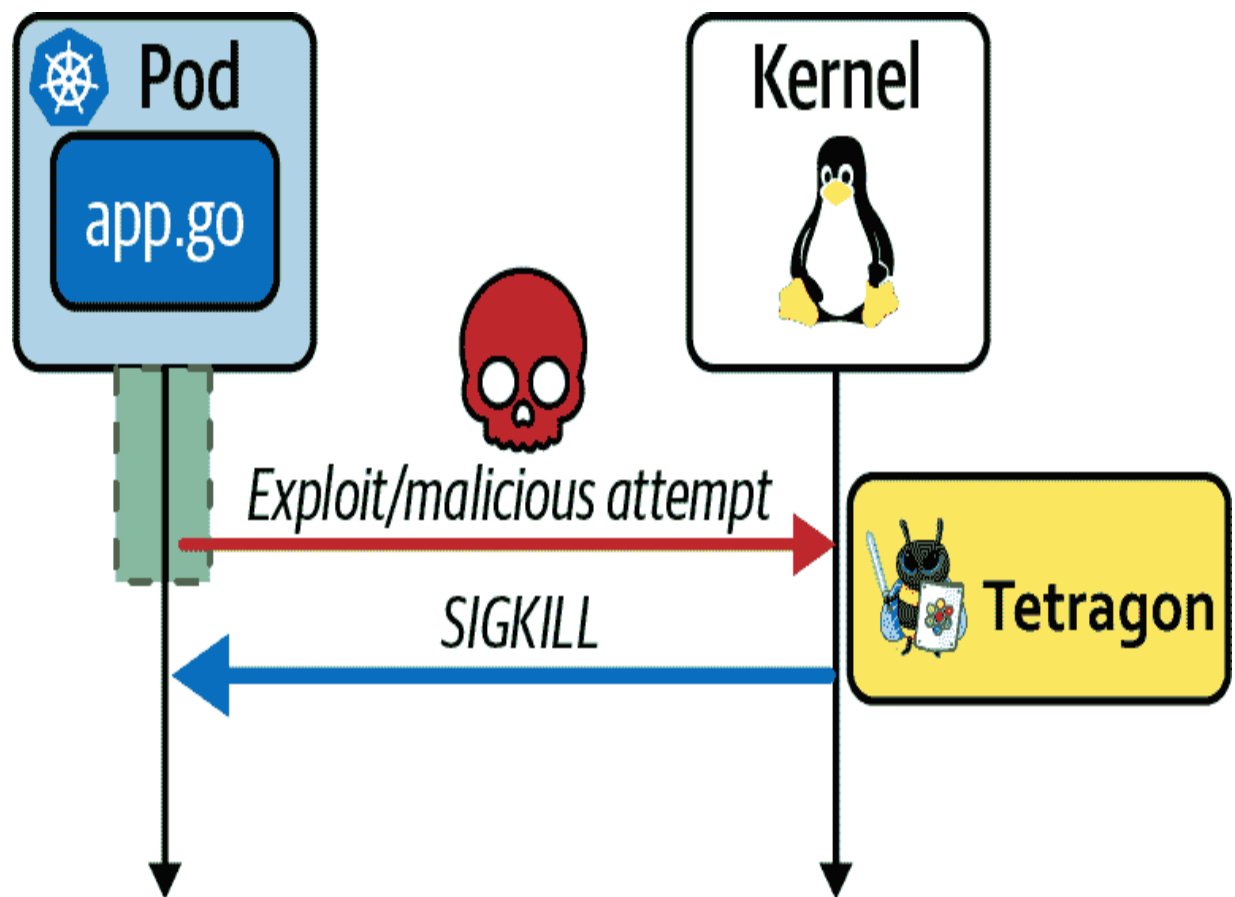


Figure 9-5. Tetragon kills malicious processes synchronously by sending a SIGKILL signal from the kernel

Sigkill policies need to be used with care, because an incorrectly configured policy could result in terminating applications unnecessarily, but it's an incredibly powerful use of eBPF for security purposes. You can start by running in an “audit” mode that generates security events but doesn't apply the SIGKILL enforcement, until you're confident that the policy won't break anything.

SIGKILL策略需要谨慎使用，因为配置不正确的策略可能会不必要地终止应用程序，但它是一种非常强大的用于安全目的的eBPF应用。您可以首先以“审计”模式运行，生成安全事件，但不应用SIGKILL强制执行，直到您确信策略不会导致任何问题。

If you're interested in learning more about using Cilium Tetragon for detecting security events, there is a report titled “[Security Observability with eBPF](#)” by Natália Réka Ivánkó and Jed Salazar that digs into much more detail.

如果您对了解如何使用Cilium Tetragon来检测安全事件感兴趣，有一篇名为“使用eBPF进行安全可观测性”的报告，作者是Natália Réka Ivánkó和Jed Salazar，其中对此进行了更详细的探讨。

Network Security

网络安全

[Chapter 8](#) discussed how eBPF can be used very effectively to implement network security mechanisms. To summarize:

第8章讨论了如何使用eBPF来实现网络安全机制。简而言之：

- Firewalling and DDoS protection are a natural fit for eBPF programs attached early in the ingress path for network packets. And with the possibility of XDP programs offloaded to hardware, malicious packets may never even reach the CPU!

防火墙和DDoS保护是早期连接网络数据包时与eBPF程序自然契合的。而且，通过将XDP程序卸载到硬件中，恶意数据包甚至可能从未到达CPU！

- For implementing more sophisticated network policies, such as Kubernetes policies determining which services are allowed to communicate with one another, eBPF programs that attach to points in the network stack can drop packets if they are determined to be out of policy.

为了实现更复杂的网络策略，比如Kubernetes策略确定允许哪些服务相互通信，可以使用附着到网络堆栈中的eBPF程序来丢弃违反策略的数据包。

Network security tools are very often used in a preventative mode, dropping packets rather than just auditing malicious activity. This is because it's so easy for bad actors to mount network-related attacks; if you give a device a public IP address exposed to the internet, it won't be long before you start seeing suspicious traffic, so organizations are forced to use preventative measures.

网络安全工具经常被用于预防模式，而不仅仅是检测恶意活动。这是因为坏人很容易发起网络攻击；如果你给一个设备一个对外暴露的公

共IP地址，不久后你就会开始看到可疑的流量，所以组织被迫采取预防措施。

In contrast, lots of organizations use intrusion detection tools in an audit mode, and they rely on forensics to determine whether a suspicious event was really malicious and what remedial action needs to be taken. If a given security tool is too blunt an instrument and is prone to detecting false-positives, it's not surprising that it needs to be run in audit mode rather than preventative mode. It's my belief that eBPF is enabling more sophisticated security tools with finer-grained, accurate controls. Just as we consider firewalls today to be sufficiently accurate to use in preventative mode, we'll see increased use of preventative tooling that acts on other, non-networking events. This could even include eBPF-based controls being packaged as part of an application product so that it can provide its own runtime security.

相反，许多组织在审核模式下使用入侵检测工具，并依赖取证分析来确定可疑事件是否真的恶意以及需要采取什么补救行动。如果某个安全工具过于粗糙并容易产生误报，那么需要在审核模式下运行也就不足为奇了。我认为eBPF能够让更复杂的安全工具具备更细粒度、准确的控制能力。就像今天我们认为防火墙已经足够准确可以在预防模式下使用一样，我们将会看到越来越多的预防性工具应用于其他非网络事件上。甚至可以将基于eBPF的控制功能打包作为应用产品的一部分，以便提供自身的运行时安全性。

Summary

总结

In this chapter you saw how eBPF's use in security has evolved from low-level checks on system calls to much more sophisticated use of eBPF programs for security policy checks, in-kernel event filtering, and runtime enforcement.

在本章中，你看到了eBPF在安全领域的应用是如何从对系统调用进行低级检查演变到对安全策略进行更加复杂的eBPF程序检查、内核事件过滤和运行时执行的。

There's still much active development in the area of using eBPF for security purposes. I believe we will see tools in this area evolving and becoming widely adopted over the coming years.

在使用eBPF进行安全目的方面仍然存在着许多活跃的开发。我相信在接下来的几年里，我们将会看到这个领域的工具逐渐演进并得到广泛应用。

¹ See, for example, this post from Jess Frazelle, who developed the default seccomp profile for Docker: [“How to Use the New Docker Seccomp Profiles”](#).

请参考Jess Frazelle的这篇帖子，她为Docker开发了默认的seccomp配置文件：“如何使用新的Docker Seccomp配置文件”。

² The documentation for Inspektor Gadget's seccomp profiler is quite dry, but [this video overview from Jose Blanquicet](#) is more accessible.

南方公园的完整文档相当冗长，但Jose Blanquicet的视频概述更容易理解。

³ Exploiting this window was discussed in a DEFCON 29 talk titled “[Phantom Attack: Evading System Call Monitoring](#)” by Rex Guo and Junyuan Zeng, and its impact on Falco was covered in more detail in the talk “[LSM BPF Change Everything](#)” by Leo Di Donato and KP Singh.

在DEFCON 29的名为“幻影攻击：规避系统调用监控”的讲座中，Rex Guo和Junyuan Zeng讨论了利用这个漏洞的问题。在Leo Di Donato和KP Singh的“LSM BPF改变一切”讲座中更详细地讨论了它对Falco的影响。

Chapter 10. eBPF Programming

第十章。eBPF编程

In this book so far, you've learned a lot about eBPF and seen many examples of how it's used for a variety of applications. But what if you want to implement your own ideas based on eBPF? This chapter discusses your options when it comes to writing your own eBPF code.

到目前为止，通过这本书，你已经学到了很多关于eBPF的知识，并且看到了许多关于它如何用于各种应用的例子。但是，如果你想根据自己的想法实现eBPF，你有几种选择。本章讨论了在编写自己的eBPF代码时的选择。

As you know from reading this book, eBPF programming consists of two parts:

从阅读本书你可以知道，eBPF编程包括两个部分：

- Writing eBPF programs that run in the kernel
在内核中编写运行的eBPF程序
- Writing the user space code that manages and interacts with eBPF programs
编写管理和与eBPF程序交互的用户空间代码

Most of the libraries and languages I'll discuss in this chapter require you as a programmer to handle both parts, with an awareness of what is being handled where. But `bpfttrace`, perhaps the simplest eBPF programming language, masks this distinction from the programmer.

大多数在本章讨论的库和编程语言都要求程序员同时处理这两个部分，并意识到正在处理的位置。但是 `bpfttrace`，可能是最简单的 eBPF 编程语言，将程序员与这种区别隔离开来。

Bpfttrace

Bpfttrace: 抽丝剥茧

As described on the project's *README* page, "`bpfttrace` is a high-level tracing language for Linux eBPF ... inspired by awk and C, and predecessor tracers such as DTrace and SystemTap."

如项目的README页面所述：“bpfttrace是Linux eBPF的高级跟踪语言……灵感来自awk和C，以及前身跟踪工具，如DTrace和SystemTap。”

The `bpfttrace` command-line tool converts programs written in this high-level language into eBPF kernel code and provides some output formatting for the results within the terminal. As a user, you don't really need to think about the kernel–user space split.

`bpfttrace`命令行工具将使用这种高级语言编写的程序转换为eBPF内核代码，并在终端中提供一些输出格式化的结果。作为用户，您不需要过多考虑内核和用户空间的分离。

You'll find several examples of useful one-liners in the project documentation, including a nice [tutorial](#) that takes you from writing a simple “Hello World” script up to writing more sophisticated scripts that can trace out data read from within kernel data structures.

在项目文档中，你会找到一些有用的一行代码的示例，包括一个很好的教程，它会引导你从编写简单的“Hello World”脚本，一直到编写更复杂的脚本，可以追踪内核数据结构中读取到的数据。

Note

Get a feel for the range of capabilities that `bpfttrace` provides from Brendan Gregg's [bpfttrace cheat sheet](#). Or, for in-depth coverage of both `bpfttrace` and BCC, see his book *BPF Performance Tools*.

从Brendan Gregg的bpfttrace备忘单上了解bpfttrace提供的各种功能。或者，了解有关bpfttrace和BCC的详细内容，请参阅他的书《BPF性能工具》。

As its name suggests, `bpfttrace` can attach to tracing (also known as perf-related) events, including kprobes, uprobes, and tracepoints. For example, you can list the available tracepoints and kprobes on a machine with the `-l` option, like this:

正如其名，bpfttrace可以连接到追踪（也称为perf相关）事件，包括kprobes、uprobes和tracepoints。例如，您可以使用-l选项在机器上列出可用的tracepoints和kprobes，就像这样：

```
$ bpfttrace -l "*execve*"
tracepoint:syscalls:sys_enter_execve
tracepoint:syscalls:sys_exit_execve
...
```

```
kprobe:do_execve_file
kprobe:do_execve
kprobe:__ia32_sys_execve
kprobe:__x64_sys_execve
...
```

This example finds all the available attachment points that contain “execve.” From this output you can see that it’s possible to attach to a kprobe called `do_execve`. Here’s a `bpfttrace` one-line script that attaches to that event:

此示例找到所有包含“execve”的可用附加点。从输出中可以看到，可以附加到名为`do_execve`的kprobe。下面是一个附加到该事件的bpfttrace一行脚本。

```
bpfttrace -e 'kprobe:do_execve { @[comm] = count(); }'
Attaching 1 probe...
^C

@[node]: 6
@[sh]: 6
@[cpuUsage.sh]: 18
```

The `{ @[comm] = count(); }` part is the script attached to that event. This example keeps track of the number of times the event was triggered by different executables.

`{ @[comm] = count(); }` 部分是附加到该事件的脚本。此示例跟踪了不同可执行文件触发该事件的次数。

Scripts for `bpfttrace` can coordinate multiple eBPF programs attached to different events. For example, consider the [opensnoop.bt script](#) that reports on files being opened. Here is an extract:

bpftrace脚本可以协调附加到不同事件的多个eBPF程序。例如，考虑opensnoop.bt脚本，它报告文件的打开情况。以下是摘要：

```
tracepoint:syscalls:sys_enter_open,
tracepoint:syscalls:sys_enter_openat
{
    @filename[tid] = args->filename;
}

tracepoint:syscalls:sys_exit_open,
tracepoint:syscalls:sys_exit_openat
/@filename[tid]/
{
    $ret = args->ret;
    $fd = $ret > 0 ? $ret : -1;
    $errno = $ret > 0 ? 0 : - $ret;

    printf("%-6d %-16s %4d %3d %s\n", pid, comm, $fd,
$errno,
        str(@filename[tid]));
    delete(@filename[tid]);
}
```

This script defines two different eBPF programs, each attached to two different kernel tracepoints, at the entry to and exit from the `open()` and `openat()` syscalls.¹ Both of these syscalls are used to open files and take a filename as an input argument. The program triggered by either flavor of syscall entry caches that filename, storing it in a map where the key is the current thread ID. When the exit tracepoint is hit, the cached filename is retrieved from this map by the `/@filename[tid]/` line in the script.

这个脚本定义了两个不同的eBPF程序，分别附加到两个不同的内核追踪点，即`open()`和`openat()`系统调用的进入和退出点。这两个系统调用用于打开文件，并将文件名作为输入参数。由任一类型的系统调用

进入触发的程序将缓存该文件名，将其存储在一个映射中，其中键是当前线程ID。当到达退出追踪点时，通过脚本中的/@filename[tid]/行从该映射中检索缓存的文件名。

Running this script generates output like this:

运行这个脚本会生成如下的输出：

```
./opensnoop.bt
Attaching 6 probes...
Tracing open syscalls... Hit Ctrl-C to end.
PID      COMM                      FD ERR PATH
297388 node                          30   0 /home/liz/.vscode-
server/data/User/
workspaceStorage/73ace3ed015
297360 node                          23   0 /proc/307224/cmdline
297360 node                          23   0 /proc/305897/cmdline
297360 node                          23   0 /proc/307224/cmdline
```

I've just told you there are four eBPF programs attached to tracepoints, so why does this output say there are six probes? The answer is that there are two “special probes” for the `BEGIN` and `END` clauses that the full version of this program includes to initialize and clean up the script (very similar to the awk language). I've omitted those clauses here for brevity, but you can find them in the [source code in GitHub](#).

我刚刚告诉你，有四个与跟踪点相关联的eBPF程序，那么为什么这个输出显示有六个探针？答案是此程序的完整版本包含用于初始化和清理脚本的BEGIN和END子句所需的两个“特殊探针”（类似于awk语言）。为了简洁起见，我在这里省略了这些子句，但你可以在GitHub的源代码中找到它们。

If you're using `bpftrace`, you don't need to know about the underlying programs and maps, but for those of you who have read earlier chapters of this book, those concepts should be familiar to you by now. If you're interested to see the programs and maps that are loaded in the kernel while a `bpftrace` program is running, you can easily do this with `bpftool` (just as you saw in [Chapter 3](#)).

Here's the output I got while running *opensnoop.bt*:

如果你正在使用**bpftrace**，你不需要了解底层的程序和映射，但对于那些已经阅读过本书前几章的人来说，这些概念现在应该很熟悉了。如果你有兴趣查看在运行**bpftrace**程序时加载到内核中的程序和映射，你可以轻松地通过**bpftool**来实现（就像你在第3章中看到的那样）。这是我在运行*opensnoop.bt*时得到的输出结果：

```
$ bpftool prog list
...
494: tracepoint name sys_enter_open tag 6f08c3c150c4ce6e
gpl
    loaded_at 2022-11-18T12:44:05+0000 uid 0
    xlated 128B jited 93B memlock 4096B map_ids 254
495: tracepoint name sys_enter_opena tag
26c093d1d907ce74 gpl
    loaded_at 2022-11-18T12:44:05+0000 uid 0
    xlated 128B jited 93B memlock 4096B map_ids 254
496: tracepoint name sys_exit_open tag 0484b911472301f7
gpl
    loaded_at 2022-11-18T12:44:05+0000 uid 0
    xlated 936B jited 565B memlock 4096B map_ids
254,255
497: tracepoint name sys_exit_openat tag
0484b911472301f7 gpl
    loaded_at 2022-11-18T12:44:05+0000 uid 0
    xlated 936B jited 565B memlock 4096B map_ids
254,255

$ bpftool map list
254: hash flags 0x0
```

```
key 8B  value 8B  max_entries 4096  memlock
331776B
255: perf_event_array  name printf  flags 0x0
key 4B  value 4B  max_entries 2  memlock 4096B
```

You can clearly see the four tracepoint programs, plus the hash map that's used for caching the filenames and the `perf_event_array` that is being used to pass output data from kernel to user space.

您可以清楚地看到四个跟踪点程序，以及用于缓存文件名的哈希映射和用于从内核传递输出数据到用户空间的`perf_event_array`。

Note

The `bpfftrace` utility is built on top of BCC, which you met elsewhere in this book and which I'll cover later in this chapter. `bpfftrace` scripts get converted into BCC programs, which are then compiled at runtime using the LLVM/Clang toolchain.

`bpfftrace` 实用工具是建立在 BCC 之上的，你在本书其他部分可能会遇到，并且我会在本章后面进行详细介绍。`bpfftrace` 脚本会被转化为 BCC 程序，然后在运行时使用 LLVM/Clang 工具链进行编译。

If you want command-line tools for eBPF-based performance measurement, you may well find that your needs are met using `bpfftrace`. But although `bpfftrace` can be a powerful tool for using eBPF for tracing, it doesn't open up the full range of possibilities that eBPF enables.

如果你需要用于基于eBPF的性能测量的命令行工具，你可能会发现`bpfftrace`可以满足你的需求。但是尽管`bpfftrace`是一个适用于使用

eBPF进行跟踪的强大工具，但它并未开启eBPF所提供的全部可能性。

To unlock the full potential of eBPF, you'll need to directly write eBPF programs yourself for the kernel and also handle the user space part. These two aspects can, and often are, written in entirely different languages. Let's start with the choices for eBPF code that runs in the kernel.

要发挥eBPF的全部潜力，您需要直接为内核编写eBPF程序，并处理用户空间部分。这两个方面可以使用完全不同的编程语言编写。让我们从内核中运行eBPF代码的选择开始。

Language Choices for eBPF in the Kernel

内核中eBPF的语言选择

eBPF programs can be written directly in eBPF bytecode,² but in practice, most are compiled to bytecode from either C or Rust. These languages have compilers that support eBPF bytecode as a target output.

eBPF程序可以直接编写成eBPF字节码，但实际上，大多数程序是通过C或Rust编译成字节码的。这些语言的编译器支持将eBPF字节码作为目标输出。

Note

eBPF bytecode isn't a suitable target for all compiled languages. If the language involves a runtime component (like Go, or Java's virtual machine), it's likely to be incompatible with eBPF's verifier. For example, it's hard to imagine how memory garbage collection could work hand in hand with the verifier's checks on safe use of memory. Similarly, eBPF programs are required to be single threaded, so any concurrency features in a language couldn't be used.

eBPF 字节码并不适合所有的编译语言。如果语言包含运行时组件（例如 Go 或 Java 的虚拟机），很可能与 eBPF 的验证程序不兼容。例如，很难想象内存垃圾回收如何与验证程序对内存安全使用的检查相结合工作。同样，eBPF 程序需要是单线程的，所以语言中的任何并发特性都不能使用。

Although not really eBPF, there is an interesting project called [XDPLua](#) that proposes writing XDP programs in Lua scripts that run directly within the kernel. However, the initial research in this project suggested that eBPF would likely be more performant, and as eBPF becomes more powerful with each kernel release (e.g., now being able to implement loops), it's not clear that there is much advantage other than the personal preference that some people might have to write code in Lua scripts.

虽然XDPLua这个项目并不是真正的eBPF，但它是一个有趣的项目，可以用Lua脚本编写直接在内核中运行的XDP程序。然而，该项目的初始研究表明，eBPF可能更具性能，而且随着每个内核发布中eBPF变得更加强大（例如，现在能够实现循环），除了一些人可能更喜欢用Lua脚本编写代码，目前还不清楚是否有太多优势。

I would hazard a guess that most people who choose to write eBPF kernel code in Rust would also opt for the same language for the user space code, since shared data structures wouldn't need to be rewritten. It's not obligatory, though—you can mix and match eBPF code with whatever user space language you choose.

我敢猜想，选择使用Rust编写eBPF内核代码的大多数人也会选择同一种语言来编写用户空间代码，因为共享数据结构不需要重新编写。但这并非强制要求，您可以混合和匹配eBPF代码和您选择的任何用户空间语言。

Those who choose to write the kernel-side code in C also have the option to write user space code in C (you've seen plenty of examples of that in this book already). But C is a pretty low-level language that requires programmers to handle lots of details for themselves, notably, memory management. While some people are comfortable doing this, many people would prefer to write the user space code in another, higher-level language. Whatever your preferred language, you'd like a library that provides eBPF support so that you don't have to write directly to the system call interface you saw in [Chapter 3](#). In the rest of this chapter we'll discuss some of the most popular options for eBPF libraries in a variety of languages.

那些选择在C语言中编写内核端代码的人也可以选择用C语言编写用户空间代码（你在本书中已经看到了大量的示例）。但是C语言是一种相当低级的语言，需要程序员自己处理许多细节，尤其是内存管理。虽然有些人习惯这样做，但很多人更喜欢用其他更高级的语言编写用户空间代码。无论你偏爱的语言是什么，你都希望有一个提供

eBPF支持的库，这样你就不需要直接编写第三章中介绍的系统调用接口。在本章的剩余部分，我们将讨论各种语言中最受欢迎的eBPF库选项。

BCC Python/Lua/C++

请帮我将`BCC Python/Lua/C++`翻译成简体中文。

Back in [Chapter 2](#), the first “Hello World” example I gave you was a Python program written using the BCC library. This project includes plenty of useful performance measurement tools implemented using this same library (as well as newer implementations based on *libbpf* that I’ll come to momentarily).

在第2章中，我给你的第一个“Hello World”示例是使用BCC库编写的Python程序。该项目包括许多有用的性能测量工具，这些工具使用了相同的库进行实现（以及基于libbpf的新实现，我稍后会讲到）。

In addition to the [documentation](#) that describes how to use the provided BCC tools to measure performance, BCC also includes a [reference guide](#) and a [Python programming tutorial](#) to help you develop your own eBPF tools in this framework.

除了描述如何使用提供的BCC工具来测量性能的文档之外，BCC还包括一个参考指南和一个Python编程教程，帮助您在这个框架中开发自己的eBPF工具。

Chapter 5 included a discussion of BCC's approach to portability, which is to compile the eBPF code at runtime to ensure that it's compatible with the target machine's kernel data structures. In BCC, you define the kernel-side eBPF program code as a string (or the contents of a file that BCC reads into a string). This string gets passed to Clang for compilation, but before that happens, BCC does some preprocessing on the string. This enables it to provide handy shortcuts for the programmer, some of which you've seen already in this book. For example, here are some relevant lines from the example code in *chapter2/hello_map.py*:

第5章介绍了BCC对可移植性的处理方式，即在运行时编译eBPF代码，以确保与目标机器的内核数据结构兼容。在BCC中，您将内核端的eBPF程序代码定义为一个字符串（或者BCC读入一个文件并将其内容转换为字符串）。在将该字符串传递给Clang进行编译之前，BCC对字符串进行了一些预处理。这使得它能够为程序员提供一些便利的快捷方式，其中一些您在本书中已经见过。例如，以下是第2章中的示例代码中的一些相关行：第5章讨论了BCC对可移植性的处理方式，该方式是在运行时编译eBPF代码，以确保其与目标机器的内核数据结构兼容。在BCC中，您将内核端的eBPF程序代码定义为字符串（或BCC读取的文件内容）。在传递给Clang进行编译之前，BCC对该字符串进行了一些预处理。这使其能够为程序员提供一些方便的快捷方式，其中一些您在本书中已经看到过。例如，以下是第2章中示例代码中的一些相关行：

```
#!/usr/bin/python3
from bcc import BPF

program = """
```

❶

❷

```

BPF_RINGBUF_OUTPUT(output, 1); ❸
...
int hello(void *ctx) {
    ...
    output.ringbuf_output(&data, sizeof(data), 0); ❹
    return 0;
}
"""

b = BPF(text=program) ❺
...

b["output"].open_ring_buffer(print_event) ❻
...

```

❶

This is a Python program, which will run in user space.

❷

The `program` string holds the eBPF program to be compiled and then loaded into the kernel.

❸

`BPF_RINGBUF_OUTPUT` is a BCC macro that defines a ring buffer called `output`. This is part of the `program` string, so it's natural to assume it's defining the buffer from the kernel's perspective. Hold that thought until we get to callout 6.

❹

This line looks like a `ringbuf_output()` method on an object called `object`. But wait a minute—methods on objects aren't even part of the C language! BCC is doing some heavy lifting

here, [expanding methods](#) like these into underlying BPF helper functions, `bpf_ringbuf_output()` in this case.

5

This is where the program string gets rewritten into the BPF C code that Clang can compile. This line also loads the resultant program into the kernel.

6

There is no other place in the code that defines the ring buffer called `output`, and yet it's accessible from the Python user space code here. BCC does double duty when it preprocesses the line in callout 3, as it defines the ring buffer for both the user space and kernel parts.

As this example shows, BCC essentially provides its own C-like language for BPF programming. It makes life easy for the programmer, handling things like shared structure definitions for both kernel and user space and providing convenient shortcuts to wrap around BPF helper functions. This means BCC is an accessible way to get into eBPF programming if you're new to the field, especially if you're already comfortable with Python.

正如本示例所示，BCC基本上为BPF编程提供了自己的类C语言。它使程序员的工作变得更轻松，处理内核和用户空间的共享结构定义，同时提供方便的快捷方式，用于封装BPF辅助函数。这意味着如果你对eBPF编程还不熟悉，尤其是如果你已经熟悉Python，BCC是一种简单易用的进入eBPF编程的方式。

Note

If you'd like to explore BCC programming, this [tutorial aimed at Python programmers](#) is a great way to walk through many more of the features and capabilities of BCC than there is space for in this book.

如果您想探索BCC编程，这个面向Python程序员的教程是一种非常好的方式，可以演示更多BCC的特性和能力，远超出本书篇幅所能容纳的。

The documentation doesn't make it terribly clear, but as well as supporting Python as the language for the user space part of eBPF tools, BCC enables writing tools in Lua and C++. There are *lua* and *cpp* directories within the supplied [examples](#) that you can base your own code on, should you be keen to try this approach.

文档并不是特别清楚，但是除了支持Python作为eBPF工具用户空间部分的语言外，BCC还支持使用Lua和C++编写工具。提供的示例中有lua和cpp目录，您可以在其基础上编写自己的代码，如果您有兴趣尝试这种方法。

BCC may be convenient for the programmer, but because of the inefficiencies of distributing a compiler toolchain alongside your utility (discussed in more depth in [Chapter 5](#)), if you're looking to write production-quality tools that you intend to distribute, I recommend considering some of the other libraries discussed in this chapter.

虽然BCC对于程序员来说可能很方便，但由于在你的实用工具中分发编译器工具链的低效率（在第5章中有更深入的讨论），如果你想要

编写并打算分发生产质量的工具，我建议考虑本章讨论的其他库。

C and Libbpf

C和Libbpf

You've seen lots of examples in this book of eBPF programs written in C, using the LLVM toolchain to compile into eBPF bytecode. You've also seen that extensions were added to support BTF and CO-RE. Many C programmers will also be familiar with the other major C compiler, GCC, and will be happy to hear that [GCC from version 10](#) also supports compiling to eBPF as a target; however, there are still some gaps compared with the functionality provided in LLVM.

你在这本书中看到了许多用C语言编写的eBPF程序示例，使用LLVM工具链编译成eBPF字节码。你还看到了为了支持BTF和CO-RE而添加的扩展功能。许多C语言程序员也熟悉另一个主要的C编译器GCC，并且会很高兴听到从10版本开始，GCC也支持将代码编译成eBPF目标；然而，与LLVM提供的功能相比，仍然存在一些差距。

As you saw in [Chapter 5](#), CO-RE and *libbpf* enabled an approach to portable eBPF programming that doesn't require shipping a compiler toolchain alongside every eBPF tool. The BCC project took advantage of this and, in addition to the original set of BCC performance tracing tools, it now has versions of these tools rewritten to take advantage of *libbpf*. The general consensus is that the versions of the BCC tools that have been rewritten based on

libbpf are the better option to use, since they have a significantly lower memory footprint³ and don't involve a start-up delay while the compilation step takes place.

正如您在第5章中所看到的，CO-RE和libbpf为可移植的eBPF编程提供了一种方法，无需在每个eBPF工具旁边都安装编译器工具链。BCC项目利用了这一点，并且除了原始的一套BCC性能跟踪工具，现在还有使用libbpf重写的这些工具的版本。普遍的共识是，基于libbpf重写的BCC工具版本是更好的选择，因为它们具有显著较低的内存占用，并且不会出现启动延迟，因为编译步骤正在进行中。

If you're comfortable with programming in C, using *libbpf* can make a lot of sense. You've seen lots of examples of this already in this book.

如果您对C语言编程感到自在，那么使用libbpf会是一个很有意义的选择。在本书中，您已经看到了很多关于此的示例。

Note

To write your own *libbpf* programs in C, the best place to start (now that you have read this book!) is [libbpf-bootstrap](#). Read Andrii Nakryiko's [blog post about it](#) as a good introduction to the motivations behind this project.

要在C语言中编写自己的libbpf程序，最好的入门方式（在您阅读了本书后！）是使用libbpf-bootstrap。阅读Andrii Nakryiko的博文来了解这个项目背后的动机是一个很好的介绍。

There's also a library called [libxdp](#) that builds on *libbpf* to allow for easier development and management of XDP programs. This is part of xdp-tools, which also holds one of my favorite learning resources for eBPF programming: the [XDP Tutorial](#).⁴

还有一个名为libxdp的库，它基于libbpf，可以更轻松地开发和管理XDP程序。这是xdp-tools的一部分，该部分还包含了我最喜欢的eBPF编程学习资源之一：XDP教程。

But C is quite a challenging, low-level language. C programmers have to take responsibility for things like memory management and buffer handling, and it's very easy to end up writing code with security vulnerabilities, not to mention crashes due to mishandling pointers. The eBPF verifier helps out on the kernel side, but there is no equivalent protection for your user space code.

但是C是一种相当具有挑战性的低级语言。C程序员必须对内存管理和缓冲区处理等事项负责，并且很容易编写出具有安全漏洞的代码，更不用说由于指针处理不当而导致的崩溃了。eBPF验证器可以在内核层面提供一些帮助，但是对于用户空间代码来说，没有等效的保护措施。

The good news is that there are libraries for other programming languages that interface to *libbpf*, or provide similar relocation functionality to allow for portable eBPF programs. Here are a few of the most popular ones.

好消息是，对于其他编程语言，有一些库可以与libbpf进行接口，或者提供类似的重定位功能，以便支持可移植的eBPF程序。以下是一些最

受欢迎的库。

Go

去

The Go language has been widely adopted for infrastructure and cloud native tooling, so it's natural that there should be options for writing eBPF code in it.

Go语言已被广泛应用于基础设施和云原生工具，因此自然而然地应该有用于编写eBPF代码的选项。

Note

[This article by Michael Kashin](#) provides another perspective comparing different eBPF libraries for Go.

在这篇由Michael Kashin撰写的文章中，提供了比较不同Go语言的eBPF库的另一个观点。

Gobpf

Gobpf (高级 Berkeley Packet Filter) 的翻译是高级伯克利包过滤器。

Possibly the first serious Golang implementation was the [gobpf](#) project that sits alongside BCC as part of lovisor. However, it hasn't been actively maintained for a while, and as I write this, there's some [discussion of deprecating it](#), so bear this in mind when making your library choice.

可能第一个认真的Golang实现是gobpf项目，它作为lovisor的一部分与BCC并存存在。然而，它已经有一段时间没有得到积极的维护了，而且在我写这篇文章时，有一些关于弃用它的讨论，因此在选择库时请记住这一点。

Ebpf-go

Ebpf-go的翻译为简体中文为"白名单引导程序-go"。

The [eBPF Go library included as part of the Cilium project](#) is widely used (I found around 10,000 references on GitHub, and the project has close to 4,000 stars). It provides convenient functions for managing and loading eBPF programs and maps, including CO-RE support, all implemented purely in Go.

Cilium项目中包含的eBPF Go库被广泛使用（我在GitHub上找到了大约10,000个相关引用，并且该项目获得了接近4,000颗星）。它提供了方便的函数来管理和加载eBPF程序和映射，包括纯Go语言实现的CO-RE支持。

With this library you have the option to compile your eBPF programs to bytecode and embed that bytecode into Go source code, using a supplied tool called [bpf2go](#). You need the LLVM/Clang compiler to do this generation as part of the build step. Once the Go code is compiled, you have a single Go binary that you can distribute that includes the eBPF bytecode and is portable to different kernels, without any dependencies other than the Linux kernel itself.

通过这个库，您可以选择将您的eBPF程序编译为字节码，并将该字节码嵌入到Go源代码中，使用一个名为bpf2go的工具。您需要使用

LLVM/Clang编译器在构建步骤中生成字节码。一旦Go代码编译完成，您将得到一个单一的Go二进制文件，可以分发包含eBPF字节码且可在不同内核之间移植，而不需要任何依赖，仅依赖于Linux内核本身。

The *cilium/ebpf* library also supports loading and managing eBPF programs built as standalone ELF files (like the **.bpf.o* examples you have seen in this book).

*cilium/ebpf*库还支持加载和管理作为独立ELF文件构建的eBPF程序（例如你在本书中看到的**.bpf.o*示例）。

At the time of this writing, the *cilium/ebpf* library supports the perf events for tracing, including the relatively recent fentry events, as well as an extensive set of network program types like XDP and cgroup socket attachments.

截至目前，*cilium/ebpf*库支持对性能事件进行跟踪，包括相对较新的fentry事件，以及一系列广泛的网络程序类型，如XDP和cgroup socket附件。

In this project's [examples directory under *cilium/ebpf*](#), you'll see that the C code for in-kernel programs sits in the same directories as the corresponding user space code in Go:

在此项目的示例目录下的*cilium/ebpf*文件夹中，你会看到内核程序的C代码与相应的Go用户空间代码位于同一目录下。

- The C files start with `// +build ignore`, which tells the Go compiler to ignore them. At the time of this writing there is an

[update in progress](#) to change to the newer `//go:build` style of build tag.

C文件以`// +build ignore`开头，告诉Go编译器忽略它们。在撰写本文时，正在进行更新，改用更新的`//go:build`构建标签。`

- The user space files include a line like the following, which tells the Go compiler to invoke the `bpf2go` tool on the C file(s):

用户空间文件包括以下一行，告诉Go编译器在C文件上调用**bpf2go**工具。

```
//go:generate go run github.com/cilium/ebpf/cmd/bpf2go
-cc $BPF_CLANG
                                -cflags $BPF_CFLAGS bpf <C
filename> -- -I../headers
```

Running `go:generate` on the package rebuilds the eBPF program and regenerates the skeleton in a single step.

在包上运行`go:generate`会在一个步骤中重新构建eBPF程序并重新生成框架。

Much like `bpftool gen skeleton`, which you saw in [Chapter 5](#), `bpf2go` generates skeleton code for manipulating the eBPF objects, minimizing the user space code you need to write yourself (except it's generating Go code rather than C). The output files also include the `.o` object files containing the bytecode.

与Chapter 5中所见的`bpftool gen skeleton`类似，`bpf2go` 生成用于操作eBPF对象的骨架代码，最大限度地减少您需要自己编写的用户空间

代码（只是生成Go代码而不是C代码）。输出文件还包括包含字节码的.o目标文件。

In fact, `bpf2go` generates two versions of the bytecode .o files, for big- and little-endian architectures. There are also two correspondingly generated .go files, and the correct versions for the target platform get used at compile time. As an example, the auto-generated files in the [kprobe example from cilium/ebpf](#) are:

实际上，bpf2go生成两个版本的字节码.o文件，用于大端和小端体系结构。还有两个相应生成的.go文件，编译时将使用适用于目标平台的正确版本。以cilium/ebpf中的kprobe示例为例，自动生成的文件如下：

- The *bpf_bpfeb.o* and *bpf_bpfel.o* ELF files containing eBPF bytecode

包含eBPF字节码的bpf_bpfeb.o和bpf_bpfel.o ELF文件

- The *bpf_bpfeb.go* and *bpf_bpfel.go* files, which define Go structures and functions that correspond to the maps, programs, and links defined in that bytecode

bpf_bpfeb.go和bpf_bpfel.go文件定义了与字节码中定义的映射、程序和链接相对应的Go结构和函数。

You can relate the objects defined in the auto-generated Go code to the C code from which it was generated. Here are the objects defined in the C code for that kprobe example:

在自动生成的Go代码中，您可以将其中定义的对象与生成它的C代码相关联。以下是该kprobe示例中C代码中定义的对象：

```
struct bpf_map_def SEC("maps") kprobe_map = {
    ...
};

SEC("kprobe/sys_execve")
int kprobe_execve() {
    ...
}
```

The auto-generated Go code includes structures representing all the maps and programs (in this case, there is only one of each):

自动生成的Go代码包含表示所有映射和程序的结构（在这种情况下，每种只有一个）。

```
type bpfMaps struct {
    KprobeMap *ebpf.Map `ebpf:"kprobe_map"`
}

type bpfPrograms struct {
    KprobeExecve *ebpf.Program `ebpf:"kprobe_execve"`
}
```

The names “KprobeMap” and “KprobeExecve” are derived from the map and program names used in the C code. These objects are grouped into a `bpfObjects` structure representing everything that’s being loaded into the kernel:

“KprobeMap”和“KprobeExecve”这两个名称是从C代码中使用的地图和程序名称派生而来的。这些对象被组合成一个结构，代表了加载到内核中的所有内容。

```
type bpfObjects struct {  
    bpfPrograms  
    bpfMaps  
}
```

You can then use these object definitions and related auto-generated functions in your user space Go code. To give you an idea of what this might involve, here's an extract based on the main function from the same [kprobe example](#) (omitting error handling for brevity):

您可以在您的用户空间Go代码中使用这些对象定义和相关的自动生成的函数。为了让您对可能涉及的内容有个简要了解，以下是基于相同kprobe示例的main函数的一个片段（省略了错误处理以简洁起见）：

```
objs := bpfObjects{}  
loadBpfObjects(&objs, nil)                                ❶  
defer objs.Close()  
  
kp, _ := link.Kprobe("sys_execve",  
                     objs.KprobeExecve, nil)              ❷  
defer kp.Close()  
  
ticker := time.NewTicker(1 * time.Second)                 ❸  
defer ticker.Stop()  
  
for range ticker.C {  
    var value uint64  
    objs.KprobeMap.Lookup(mapKey, &value)                 ❹  
    log.Printf("%s called %d times\n", fn, value)  
}
```

❶

Load all the BPF objects that were embedded in bytecode form, into the `bpfObjects` I just showed you defined by the auto-generated code.

②

Attach the program to the `sys_execve` kprobe.

③

Set up a ticker so that the code can poll the map once per second.

④

Read an item out of the map.

There are several other examples in the *cilium/ebpf* directory that you can use for reference and inspiration.

你可以在cilium/ebpf目录中找到其他几个示例，供你参考和启发。

Libbpfgo

Libbpfgo 的翻译是 "Libbpfgo"

The [libbpfgo project](#) by Aqua Security implements a Go wrapper around *libbpf*'s C code, providing utilities for loading and attaching programs and using Go-native features like channels for receiving events. Because it's built on *libbpf*, it supports CO-RE.

Aqua Security的libbpfgo项目实现了对libbpf的C代码的Go封装，提供用于加载和附加程序以及使用Go本地功能（如使用通道接收事件）的实用工具。由于它构建在libbpf上，因此支持CO-RE。

Here's an extract from the example from *libbpfgo*'s *README*, which gives a good high-level view of what to expect from this library:

这是libbpfgo的README中的一个示例片段，它提供了对这个库可以期望的高层次概览。

```
bpfModule := bpf.NewModuleFromFile(bpfObjectPath)
bpfModule.BPFLoadObject()

mymap, _ := bpfModule.GetMap("mymap")
mymap.Update(key, value)

rb, _ := bpfModule.InitRingBuffer("events", eventsChannel,
    buffSize)
rb.Start()
e := <-
eventsChannel
```

❶

Read eBPF bytecode from an object file.

❷

Load that bytecode into the kernel.

❸

Manipulate an entry in an eBPF map.

❹

Go programmers will appreciate receiving data from a ring or perf buffer on a channel, which is a language feature designed to handle asynchronous events.

This library was created for Aqua's [Tracee](#) security project, and it's also being used by other projects such as [Parca](#) from Polar Signals,

which provides eBPF-based CPU profiling. The only concern about this project's approach is the CGo boundary between the *libbpf* C code and Go, which can cause performance and other issues.⁵

这个库是为 Aqua 的 Tracee 安全项目创建的，它也被其他项目使用，例如 Polar Signals 的 Parca，该项目提供基于 eBPF 的 CPU 分析。对于这个项目的方法，唯一的关注点是 libbpf C 代码和 Go 之间的 CGo 边界，可能会导致性能和其他问题。

While Go has been the established language for lots of infrastructure coding for around a decade, there has more recently been a growing body of developers who prefer to use Rust.

虽然 Go 语言已经成为了十多年的基础架构编码的首选语言，但最近有越来越多的开发者更喜欢使用 Rust。

Rust

锈

Rust is increasingly being used for building infrastructure tools. It allows for the low-level access of C, but with the added benefit of memory safety. Indeed, Linus Torvalds [confirmed in 2022](#) that the Linux kernel itself will start to incorporate Rust code, and the recent [6.1 release has some initial Rust support](#).

Rust越来越多地被用于构建基础设施工具。它允许使用C的底层访问方式，但又增加了内存安全的好处。事实上，林纳斯·托瓦兹于2022年

确认，Linux内核本身将开始融入Rust代码，并且最近发布的6.1版本具有一些初始的Rust支持。

As I discussed earlier in this chapter, Rust can be compiled to eBPF bytecode, meaning that (with the right library support) it's possible to write both the user space and kernel code for eBPF utilities in Rust.

正如我在前面的章节中讨论的那样，Rust 可以编译成 eBPF 字节码，这意味着（在正确的库支持下）可以使用 Rust 编写 eBPF 实用程序的用户空间和内核代码。

There are a few options for Rust eBPF development: *libbpf-rs*, *Redbpf*, and Aya.

有几个选项可供Rust eBPF开发使用：libbpf-rs、Redbpf和Aya。

Libbpf-rs

Libbpf-rs --> Libbpf-rs

Libbpf-rs is part of the *libbpf* project, and provides a Rust wrapper around the *libbpf* C code so that you can write the user space parts of eBPF code in Rust. As you can see from the project's [examples](#), the eBPF programs themselves are written in C.

Libbpf-rs 是 libbpf 项目的一部分，它为 libbpf C 代码提供了一个 Rust 的包装器，因此您可以用 Rust 语言编写 eBPF 代码的用户空间部分。正如您从项目的示例中看到的，eBPF 程序本身是以 C 语言编写的。

Note

There are further examples in Rust in the [libbpf-bootstrap](#) project, designed to help you get off the ground if you want to try building your own code using this crate.

在libbpf-bootstrap项目中还有更多的例子，旨在帮助您开始尝试使用这个crate构建自己的代码。

This crate is helpful for incorporating eBPF programs into a Rust-based project, but it doesn't fulfill the desire that many people have to write the kernel-side code in Rust as well. Let's look at some other projects that enable that.

这个板条箱对于将eBPF程序整合到基于Rust的项目中非常有帮助，但它并不能满足许多人希望在内核端也用Rust编写代码的愿望。让我们来看看其他一些能够实现这一愿望的项目。

Redbpf

红色安全平台 (Hóngsè ānquán píngtái)

[Redbpf](#) is a set of Rust crates that interface with [libbpf](#), developed as part of [foniod](#), an eBPF-based security monitoring agent.

Redbpf是一套与libbpf接口的Rust crates，作为基于eBPF的安全监控代理程序foniod的一部分进行开发。

Redbpf predates Rust's ability to compile to eBPF bytecode, so it uses a [multistep compilation process](#) that involves compiling from Rust to LLVM bitcode and then using the LLVM toolchain to generate eBPF bytecode in ELF format. *Redbpf* supports a range of program

types including tracepoints, kprobes and uprobes, XDP, TC, and some socket events.

Redbpf在Rust能够编译为eBPF字节码之前就出现了，因此它使用了一个多步编译过程，包括从Rust编译为LLVM位码，然后使用LLVM工具链生成ELF格式的eBPF字节码。Redbpf支持一系列程序类型，包括跟踪点、kprobes和uprobes、XDP、TC以及一些套接字事件。

As the Rust compiler `rustc` gained the ability to generate eBPF bytecode directly, this was leveraged by a project called Aya. At the time of this writing, Aya is considered “emerging” according to the [community site at ebpf.io](#), while *Redbpf* is listed as a major project, but my personal perspective is that momentum seems to be moving toward Aya.

随着Rust编译器rustc直接生成eBPF字节码的能力，这为一个名为Aya的项目提供了支持。在撰写本文时，根据ebpf.io上的社区网站，Aya被认为是“新兴项目”，而Redbpf被列为重要项目，但个人观点是趋势似乎在向Aya发展。

Aya

阿雅

[Aya](#) is built in Rust directly to the syscall level, so it doesn't depend on *libbpf* (or indeed on BCC or the LLVM toolchain). But it does support the BTF format, the same relocations that *libbpf* does (as described in [Chapter 5](#)), so it's providing the same CO-RE abilities to compile once and run on other kernels. At the time of this writing, it supports a wider range of eBPF program types than *Redbpf*,

including tracing/perf-related events, XDP and TC, cgroups, and LSM attachments.

Aya是直接使用Rust编写的，在syscall级别上进行构建，因此它不依赖于libbpf（或者BCC或者LLVM工具链）。不过，它确实支持BTF格式，以及与libbpf相同的重定位（正如第5章所述），因此它提供了相同的CO-RE功能，可以一次编译，然后运行在其他内核上。在撰写本文时，Aya支持更广泛的eBPF程序类型，包括追踪/性能相关事件、XDP和TC、cgroups以及LSM附加等。

As I mentioned, the Rust compiler also supports [compiling to eBPF bytecode](#), so this language can be used for both kernel and user space eBPF programming.

正如我之前提到的那样，Rust编译器还支持编译为eBPF字节码，因此这种语言可用于内核和用户空间的eBPF编程。

Note

The ability to write both the kernel side and the user space side natively in Rust without the intermediate dependency on LLVM has attracted Rust programmers to this option. There's an interesting [discussion on GitHub](#) about why the developers of the [lockc project](#) (an eBPF-based project that enhances the security of container workloads using LSM hooks) decided to port their project from *libbpf-rs* to Aya.

使用Rust本地编写内核和用户空间，无需依赖LLVM中间件的能力，吸引了Rust程序员选择这个选项。关于为什么lockc项目的开发人员（一个基于eBPF的项目，使用LSM钩子增强容器负载的安全性）决

定将其项目从libbpf-rs迁移到Aya的讨论在GitHub上引起了人们的兴趣。

The project includes [aya-tool](#), a utility for generating Rust structure definitions that match kernel data structures so that you don't have to write them yourself.

该项目包括aya-tool，一个用于生成与内核数据结构相匹配的Rust结构定义的实用工具，这样您就不必自己编写它们了。

The Aya project strongly emphasizes developer experience and makes it easy for newcomers to get started. With that in mind, the “[Aya book](#)” is a very readable introduction with some good example code, annotated with helpful explanations.

The Aya项目非常强调开发者体验，使新手能够轻松入门。在此基础上，“Aya书籍”是一本非常易读的介绍性书籍，附有一些很好的示例代码，并配有有用的解释。

To give you a brief idea of what eBPF code looks like in Rust, here's an extract from Aya's basic XDP example that permits all traffic:

为了给您一个大致了解在Rust中eBPF代码的样子，这里是Aya的基本XDP示例的一部分，它允许所有的流量通过。

```
#[xdp(name="myapp")]  
  
pub fn myapp(ctx: XdpContext) -> u32 {  
    match unsafe { try_myapp(ctx) } {  
        Ok(ret) => ret,  
        Err(_) => xdp_action::XDP_ABORTED,  
    }  
}
```

```
unsafe fn try_myapp(ctx: XdpContext) -> Result<u32, u32> {  
    ❸ info!(&ctx, "received a packet");  
    Ok(xdp_action::XDP_PASS)  
}
```

❶

This line is what defines the section name, equivalent to

`SEC("xdp/myapp")` in C.

❷

The eBPF program called `myapp` calls the function `try_myapp` to process a network packet received at XDP.

❸

The `try_myapp` function logs the fact that a packet was received and always returns the `XDP_PASS` value that tells the kernel to carry on processing the packet as usual.

Just as we've seen in C-based examples throughout this book, the eBPF program gets compiled to an ELF object file. The difference is that Aya uses the Rust compiler instead of Clang to create that file.

就像我们在本书中看到的基于C的例子一样，eBPF程序被编译成了一个ELF目标文件。不同的是，Aya使用Rust编译器而不是Clang来创建该文件。

Aya also generates code for the user space activities of loading the eBPF program into the kernel and attaching it to an event. Here are a few key lines from the user space side of that same basic example:

Aya还会为用户空间活动生成代码，包括将eBPF程序加载到内核并将其附加到事件。以下是来自用户空间部分的一些关键行代码，属于同一基本示例： 请只返回翻译后的内容，不包括原文。

```
let mut bpf = Bpf::load(include_bytes_aligned!(
    "../../target/bpfel-unknown-none/release/myapp"
))?;

let program: &mut Xdp = bpf.program_mut("myapp").unwrap().
try_into()?;    ❷

program.load()?;    ❸

program.attach(&opt.iface, XdpFlags::default())    ❹
```

❶

Read the eBPF bytecode from the ELF object file produced by the compiler.

❷

Find the program called `myapp` in that bytecode.

❸

Load it into the kernel.

❹

Attach it to the XDP event on a specified network interface.

If you're a Rust programmer, I highly recommend you explore the [additional examples](#) in the “Aya book” in more detail. There's also a nice [blog post from Kong](#) that walks through writing an XDP load balancer using Aya.

如果您是Rust程序员，我强烈推荐您详细研究"Aya书"中的其他示例。还有一篇来自Kong的博客文章，详细介绍了如何使用Aya编写XDP负载均衡器。

Note

Aya maintainers Dave Tucker and Alessandro Decina joined me for [episode 25 of the “eBPF and Cilium Office Hours” livestream](#) where they demonstrated and gave an introduction to eBPF programming with Aya.

Aya维护者Dave Tucker和Alessandro Decina在第25期的“eBPF和Cilium办公时间”直播中与我一起参与，他们展示了以及向我们介绍了使用Aya进行eBPF编程。

Rust-bcc

Rust-bcc => Rust-bcc

[Rust-bcc](#) provides Rust bindings that mimic the BCC project's Python bindings, along with some Rust implementations of some of the BCC set of tracing [tools](#).

Rust-bcc提供了模仿BCC项目Python绑定的Rust绑定，以及一些BCC跟踪工具的Rust实现。

Testing BPF Programs

测试BPF程序

There's a `bpf()` command, `BPF_PROG_RUN`, that allows for running an eBPF program from user space for test purposes.

有一个**bpf()**命令，`BPF_PROG_RUN`，允许从用户空间运行eBPF程序进行测试目的。

`BPF_PROG_RUN` (currently) works only with a subset of BPF program types that are mostly networking related.

`BPF_PROG_RUN`（目前）仅适用于大多数与网络相关的BPF程序类型。

You can also get information about eBPF program performance with some built-in statistics information. Run the following command to enable it:

您还可以通过一些内置的统计信息来了解eBPF程序的性能情况。运行以下命令以启用此功能：

```
$ sysctl -w kernel.bpf_stats_enabled=1
```

This will show additional information in `bpftool`'s output about programs, like this:

这将在bpftool输出中显示有关程序的额外信息，就像这样：

```
$ bpftool prog list
...
2179: raw_tracepoint  name raw_tp_exec  tag
7f6d182e48b7ed38  gpl
      run_time_ns 316876 run_cnt 4
      loaded_at 2023-01-09T11:07:31+0000  uid 0
      xlated 216B  jited 264B  memlock 4096B  map_ids
780,777
```

```
btf_id 953  
pids hello(19173)
```

The additional statistics are shown in bold, and here they show that the program has run four times, taking about 300 microseconds in total.

附加统计数据以粗体显示，数据显示该程序已运行四次，总共耗时约300微秒。

Note

Learn more from Quentin Monnet’s FOSDEM 2020 talk titled [“Tools and mechanisms to debug BPF programs.”](#)

从Quentin Monnet在FOSDEM 2020的演讲中，`从他的名为“调试BPF程序的工具和机制”的演讲中获取更多信息。`

Multiple eBPF Programs

多个 eBPF 程序

An eBPF program is a function attached to an event in the kernel. Many applications need to track more than one event to achieve their goals. A simple example of this is opensnoop.⁶ I covered the `bpftrace` version of this early in this chapter, and you saw that it attaches BPF programs to four different syscall tracepoints:

eBPF程序是附加到内核事件的函数。许多应用需要追踪多个事件以实现自己的目标。一个简单的例子是opensnoop。我在本章早些时候介

绍了bpftrace版本的opensnoop，你看到它将BPF程序附加到了四个不同的syscall跟踪点。

- `syscall_enter_open`

`syscall\_enter\_open` in simplified Chinese is "系统调用输入打开".

- `syscall_exit_open`

退出系统调用打开

- `syscall_enter_openat`

`syscall\_enter\_openat` translates to `系统调用进入\_打开指定路径下文件` in simplified Chinese.

- `syscall_exit_openat`

`syscall_exit_openat` -> `系统调用退出\_openat`

These are the entry and exit points to the kernel's handling of the `open()` and `openat()` system calls. These two system calls can be used for opening files, and the opensnoop tool tracks both of them.

这些是内核处理open()和openat()系统调用的入口和出口点。这两个系统调用可用于打开文件，opensnoop工具可以跟踪这两个系统调用。

But why does it need to track both entry and exit for these system calls? The entry points are used because that's when the system call arguments are available, and these include the filename and any flags being passed to the `open[at]` syscall. But at that stage it's too

soon to know whether the file will be opened successfully or not. That explains why it's necessary to have eBPF programs attached to the exit points too.

为什么需要跟踪这些系统调用的进入和退出？进入点是因为这时可以获取到系统调用的参数，包括文件名和传递给打开系统调用的任何标志。但在这个阶段，我们还无法确定文件是否会成功打开。这就解释了为什么需要将 eBPF 程序附加到退出点的原因。

If you look at the [libbpf-tools version of opensnoop](#), you'll see there's just one user space program, and it loads all four eBPF programs into the kernel and attaches them to their events. The eBPF programs themselves are essentially independent, but they use eBPF maps to coordinate among themselves.

如果您查看opensnoop的libbpf-tools版本，您会发现只有一个用户空间程序，并将所有四个eBPF程序加载到内核中并将它们附加到各自的事件上。这些eBPF程序本身基本上是独立的，但它们使用eBPF映射来协调彼此之间的操作。

A complex application might even need to add and remove eBPF programs dynamically throughout a long period of time. There may not even be a fixed number of eBPF programs for any given application. For example, Cilium attaches eBPF programs to each virtual networking interface, and in a Kubernetes environment these interfaces come and go depending on how many pods are running.

一个复杂的应用可能需要在很长时间内动态地添加和移除eBPF程序。对于任何给定的应用程序，甚至可能没有固定数量的eBPF程序。例

如，在Kubernetes环境中，Cilium会将eBPF程序附加到每个虚拟网络接口上，而这些接口会根据正在运行的Pod数量的变化而来。

Most of the libraries in this chapter handle this multiplicity of eBPF programs automatically. For example, *libbpf* and *ebpf-go* generate skeleton code that will load *all* the programs and maps from the bytecode in an object file or buffer in one function call. They also generate finer-granularity functions so that you can manipulate programs and maps individually.

本章中的大多数库都可以自动处理多个eBPF程序。例如，libbpf和ebpf-go会生成一个骨架代码，通过一个函数调用从目标文件或缓冲区加载所有程序和映射的字节码。它们还会生成更细粒度的函数，以便您可以单独操作程序和映射。

Summary

摘要

The vast majority of people who use eBPF-based tooling won't need to write eBPF code themselves, but if you do find yourself wanting to implement something yourself, you have a lot of options. This is a changing field, so it's very possible that by the time you read this, new language libraries and frameworks might exist, or consensus may have gathered around some of the libraries I've highlighted in this chapter. You'll find an up-to-date list of the major language projects around eBPF on the [Infrastructure page of ebpf.io's list of significant projects](#).

大多数使用基于eBPF的工具的人不需要自己编写eBPF代码，但如果您确实想要自己实现一些功能，您有很多选择。由于这是一个不断变化的领域，所以在您阅读本章时，可能已经出现了新的语言库和框架，或者可能已经就本章中我强调的一些库达成共识。您可以在 ebpf.io 的基础设施页面上找到关于eBPF主要语言项目的最新列表。

For quickly collecting trace information, `bpftrace` can be a very valuable option.

快速收集追踪信息时，`bpftrace`可以是一个非常有价值的选择。

For more flexibility and control, BCC is a fast way to build an eBPF tool if you're comfortable with Python, provided that you don't care about the compilation step that takes place at runtime.

要获得更多的灵活性和控制，BCC 是一种快速构建 eBPF 工具的方法，如果你熟悉 Python，并且在运行时不介意发生编译步骤的话。

If you're writing eBPF code to be widely distributed and portable across different kernel versions, you'll probably want to take advantage of CO-RE. The user space frameworks that support CO-RE at time of this writing are *libbpf* for C, *cilium/ebpf* and *libbpfgo* for Go, and Aya for Rust.

如果您正在编写eBPF代码，以便在不同的内核版本间广泛分发和移植，您可能会想要利用CO-RE。目前支持CO-RE的用户空间框架包括C语言中的`libbpf`，Go语言中的`cilium/ebpf`和`libbpfgo`，以及Rust语言中的Aya。

For further advice, I highly recommend joining the [eBPF Slack](#) and discussing your questions there. You'll likely find the maintainers of many of these language libraries in that community.

进一步咨询，请强烈建议加入 eBPF Slack 并在那里讨论你的问题。你很可能会在那个社区找到许多这些语言库的维护者。

Exercises

练习

If you'd like to try one or more of the libraries discussed in this chapter, “Hello World” is always a good place to start:

如果你想尝试本章讨论的一个或多个库，“Hello World” 总是一个很好的起点。

1. Using one or more libraries of your choosing, write an example “Hello World” program that outputs a simple trace message.

使用一个或多个您选择的库，编写一个示例“Hello World”程序，输出一个简单的跟踪消息。

2. Use `llvm-objdump` to compare the bytecode produced with the “Hello World” example from [Chapter 3](#). You'll find lots of similarities!

使用`llvm-objdump`来比较与第3章“Hello World”示例生成的字节码。你会发现很多相似之处！

3. As you saw in [Chapter 4](#), you can use `strace -e bpf` to see when `bpf()` system calls are made. Try that on your “Hello World” program to see if it’s behaving as you expect.

正如您在第4章中所看到的那样，您可以使用`strace -e bpf`来查看`bpf()`系统调用的使用情况。在您的“Hello World”程序上试试看，看看它是否按照您的预期运行。

¹ Being attached to syscall entry points means this script has the same Time Of Check To Time Of Use (TOCTOU) vulnerability discussed in the previous chapter. That doesn’t stop it from being a useful tool; it’s just that you shouldn’t rely on it as your only line of defense for security purposes.

附加到系统调用入口点意味着该脚本具有前一章中讨论的时间检查到使用（TOCTOU）漏洞。这并不妨碍它作为一个有用的工具；只是你不应该仅仅依赖它作为你安全防御的唯一防线。

² For an example of this, check out Cloudflare’s blog post “[eBPF, Sockets, Hop Distance and manually writing eBPF assembly](#)”.

如需了解更多示例，请查看Cloudflare的博文“eBPF、Socket、跳跃距离和手动编写eBPF汇编”。

³ See, for example, Brendan Gregg’s [observation](#) that the *libbpf*-based version of opensnoop required around 9 MB compared with 80 MB for the Python-based version.

例如，看看Brendan Gregg的观察，使用基于libbpf的版本的opensnoop所需的大小约为9 MB，而基于Python的版本则需要80

MB。

⁴ Watch me working through some of the XDP Tutorial examples in [episode 13 of the eBPF and Cilium Office Hours](#) livestream.

请帮助我翻译：`在“eBPF和Cilium Office Hours”直播的第13集中，观看我在一些XDP教程示例中的工作过程。`

⁵ Dave Cheney’s 2016 post “[cgo is not Go](#)” remains a good overview of concerns related to the CGo boundary.

Dave Cheney在2016年的帖子“cgo不是Go”仍然是对与CGo边界相关的问题的良好概述。 请注意，此回答只包含翻译后的内容，不包括原文。

⁶ As well as the `bpftrace` version of this tool, there are equivalents in BCC and in *libbpf-tools*. They all do very much the same thing, generating a line of trace whenever a process opens a file. There’s a walkthrough of the eBPF code for BCC’s version of opensnoop in my report “[What Is eBPF?](#)”.

除了这个工具的bpftrace版本外，还有BCC和libbpf-tools中的等效工具。它们都完成了同样的功能，即在进程打开文件时生成一行追踪。我的报告《什么是eBPF？》中有对BCC版本opensnoop的eBPF代码的解释。

Chapter 11. The Future Evolution of eBPF

第11章。eBPF的未来演进

eBPF is not finished yet! Like most software, it's under continual development within the Linux kernel, and it's being added to the Windows operating system as well. In this chapter we'll look at some of the likely future paths for this technology.

eBPF还没有完成！就像大多数软件一样，它在Linux内核中持续开发，并且还正在被添加到Windows操作系统中。在本章中，我们将介绍这项技术的一些可能的未来发展路径。

Since it was introduced within the Linux kernel, BPF has evolved into its own subsystem with its own mailing list and maintainers.¹ As eBPF's popularity increased and interest broadened beyond the Linux kernel community, it made sense to create a neutral body that could coordinate among the different parties involved. That body is the eBPF Foundation.

自从引入Linux内核以来，BPF已经发展成为一个独立的子系统，拥有自己的邮件列表和维护者。随着eBPF的受欢迎程度增加，以及对Linux内核社区以外的广泛兴趣，创建一个中立的机构来协调各方工作就显得很有意义。这个机构就是eBPF基金会。

The eBPF Foundation

eBPF基金会

The [eBPF Foundation](#) was set up in 2021 by Google, Isovalent, Meta (then known as Facebook), Microsoft, and Netflix, under the auspices of the Linux Foundation. The foundation acts as a neutral body that can hold funds and intellectual property, such that various commercial companies can collaborate with one another.

eBPF 基金会成立于2021年，由谷歌、Isovalent、Meta（当时为Facebook）、微软和Netflix在Linux基金会的庇护下创立。该基金会作为一个中立的机构，可以持有资金和知识产权，使各个商业公司能够相互合作。

The intent was not to change anything about the way eBPF technology is developed by the Linux kernel community and contributors to the Linux BPF subsystem. The foundation's activities are directed by the BPF Steering Committee, which is entirely composed of the technical experts who build the technology, including the Linux kernel BPF maintainers and representatives from other core eBPF projects.

意图并不是改变Linux内核社区和Linux BPF子系统的开发方式。基金会的活动由BPF指导委员会负责，该委员会完全由构建该技术的技术专家组成，包括Linux内核BPF维护者和其他核心eBPF项目的代表。

The eBPF Foundation focuses on eBPF as a technology platform and the ecosystem of tools that enable eBPF development. Projects that build on top of eBPF looking for neutral ownership may find a better home in other foundations. For example, Cilium, Pixie, and

Falco are all part of the CNCF, which makes sense as they are all intended to be used in cloud native environments.

eBPF基金会专注于作为技术平台的eBPF以及支持eBPF开发的生态系统工具。希望在eBPF基础上构建项目并寻找中立的所有权的，可能会在其他基金会中找到更合适的归属地。例如，Cilium、Pixie和Falco都是CNCF的一部分，这是有道理的，因为它们都被设计用于云原生环境。

One of the key drivers of this collaboration beyond the existing Linux maintainers was the interest that Microsoft had in developing eBPF within the Windows operating system. This brings about a need to define a standard for eBPF,² such that programs written for one OS can be used on another. This work is being done under the auspices of the eBPF Foundation.

这种合作的一个主要推动因素是微软对在Windows操作系统中发展eBPF的兴趣，除了现有的Linux维护者。这带来了>eBPF的标准定义的需要，以便可以在不同操作系统上使用相同的程序。这项工作是在eBPF基金会的保护下进行的。

eBPF for Windows

eBPF为Windows

Work is well underway at Microsoft to support [eBPF for Windows](#). As I write this in the closing months of 2022, there are already [functional demos](#) that show Cilium Layer 4 load balancing and eBPF-based connection tracking running on Windows.

微软已经着手为Windows支持eBPF。当我在2022年结束的几个月内写下这段话时，已经有了可行的演示，展示了在Windows上运行的Cilium第四层负载均衡和基于eBPF的连接跟踪。

I've said before that eBPF programming is kernel programming, and at first glance it might seem unintuitive that a program written to run in the Linux kernel and that has access to Linux kernel data structures would in any way be able to operate in an entirely different operating system. But in practice, particularly when it comes to networking, all operating systems will have quite a lot in common. A network packet has the same structure whether it was created on a Windows or Linux machine, and the layers of the network stack have to be handled the same way.

我之前说过eBPF编程就是内核编程，乍一看，一个在Linux内核中运行并具有访问Linux内核数据结构能力的程序如何能在完全不同的操作系统中运行可能会让人感到不可思议。但实际上，在实践中，尤其是在网络方面，所有操作系统之间都有很多共同点。无论是在Windows还是Linux机器上创建的网络数据包都具有相同的结构，并且网络协议栈的处理方式也是相同的。

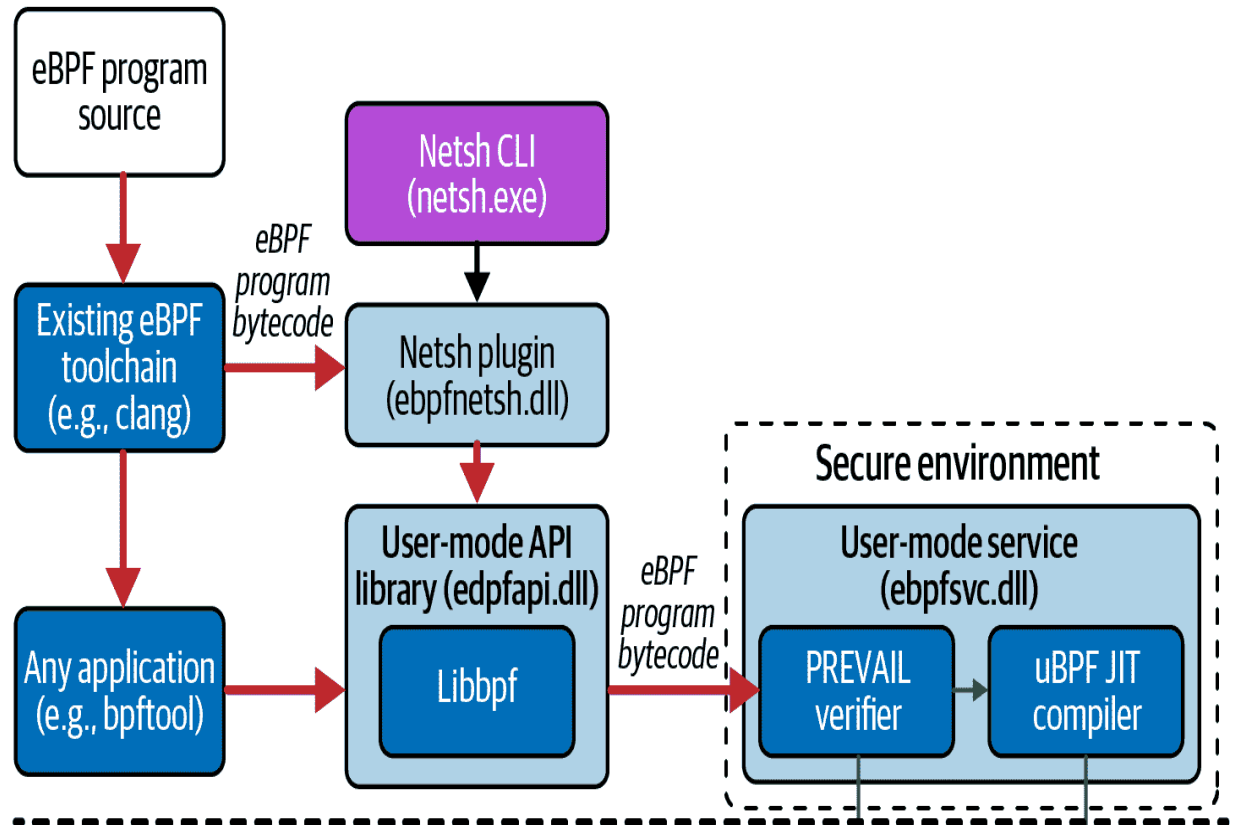
You'll also recall that eBPF programs consist of a set of bytecode instructions that are processed by a virtual machine (VM) implemented within the kernel. That VM can be implemented within Windows too!

你还记得eBPF程序由一组字节码指令组成，这些指令由内核中的虚拟机（VM）进行处理。那个虚拟机也可以在Windows中实现！

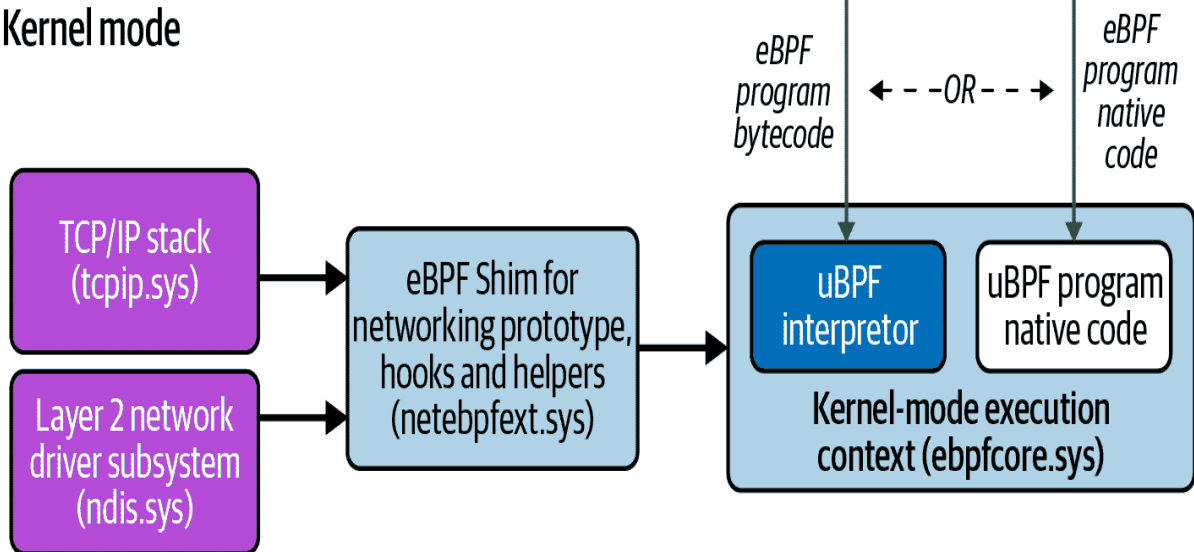
Figure 11-1 shows the eBPF for Windows architectural overview, taken from [the project's GitHub repo](#). As you can see from this diagram, eBPF for Windows reuses some open source components from the existing eBPF ecosystem, such as *libbpf*, and the support in Clang for producing eBPF bytecode. The Linux kernel is licensed under GPL, and Windows is proprietary, so the Windows project can't reuse any parts of the Linux kernel's implementation of the verifier.³ Instead, it uses the [PREVAIL verifier](#) and the [uBPF JIT compiler](#) (both of which are permissively licensed so that they can be used by a broader range of projects and organizations).

图11-1显示了来自该项目的GitHub仓库的eBPF for Windows架构概述。从该图可以看出，eBPF for Windows重用了一些现有eBPF生态系统中的开源组件，如libbpf和Clang中用于生成eBPF字节码的支持。Linux内核采用GPL许可，而Windows是专有的，因此Windows项目无法重用Linux内核实现的验证器的任何部分。相反，它使用PREVAIL验证器和uBPF JIT编译器（两者都具有宽松许可，可以被更广泛的项目和组织使用）。

User mode



Kernel mode



Key:

Unmodified Windows component

Open source: ebpf-for-windows

Open source: other projects

Figure 11-1. Architectural overview of eBPF for Windows,
adapted from <https://oreil.ly/HxKsu>

One interesting difference is that eBPF code is verified and JIT-compiled in a Windows Secure environment in user space rather than within the kernel (the uBPF interpreter shown in the kernel in [Figure 11-1](#) is used only in debug builds and not production environments).

一个有趣的区别是，eBPF代码在Windows安全环境的用户空间中进行验证和即时编译，而不是在内核中（在图11-1中显示的uBPF解释器只用于调试版本，而不用于生产环境）。

It would be unrealistic to expect that every single eBPF program written to run on Linux will work on Windows. But this isn't so different from the challenge of getting eBPF programs to run on different Linux kernel versions: even with CO-RE support, internal kernel data structures can be changed as well as added or removed between versions. It is the eBPF programmer's job to handle these possibilities gracefully.

不可能期望每个编写在Linux上运行的eBPF程序也能在Windows上运行，这是不现实的。但这与让eBPF程序在不同的Linux内核版本上运行的挑战并没有太大不同：即使有CO-RE的支持，内核数据结构也可能在不同版本间进行更改、添加或删除。eBPF程序员的工作就是优雅地处理这些可能性。

Speaking of changes to the Linux kernel, what changes can we expect to see in eBPF in the coming years?

谈到Linux内核的变化，我们在未来年份能够预期到eBPF会有哪些变化？

Linux eBPF Evolution

Linux eBPF进化

The capabilities of eBPF have evolved with practically every kernel release since 3.15. If you want to know what features are available in any given version, the BCC project maintains a [useful list](#). And I certainly expect more additions over the coming years.

自 Linux 内核 3.15 版本以来，eBPF 的功能不断发展。如果您想知道特定版本中有哪些功能可用，BCC 项目维护了一个有用的清单。我相信在未来几年还会有更多功能的添加。

The best way to predict what's coming is simply to listen to the people who are working on it. For example, at the 2022 Linux Plumbers Conference, eBPF maintainer Alexei Starovoitov gave a talk discussing how he expects to see the C language used by eBPF programs to evolve.⁴ We've already seen eBPF evolve from supporting a few thousand instructions to practically unlimited complexity, with the addition of support for loops and an ever-increasing set of BPF helper functions. As additional capabilities are added into the C that's supported, and with the support of the verifier, eBPF C could evolve to allow all the flexibility of developing kernel modules, but with the safety and dynamic loading characteristics of eBPF.

预测未来最好的方法就是听取那些正在从事相关工作的人的意见。例如，在2022年的Linux Plumbers Conference上，eBPF的维护者Alexei Starovoitov在一次演讲中讨论了他对eBPF程序中C语言的使用将如何发展的预期。我们已经看到eBPF从支持几千条指令发展为几乎无限复杂性，新增了对循环和越来越多的BPF辅助函数的支持。随着支持的C语言功能的不断增加，并且得到验证器的支持，eBPF C语言的发展可能会允许所有内核模块开发的灵活性，同时具备eBPF的安全性和动态加载特性。

Some of the other ideas being discussed and developed for new eBPF features and capabilities include:

一些正在讨论和开发的新eBPF功能和能力的其他想法包括：

Signed eBPF programs

Software supply chain security has been a hot topic for the past few years, and a key element is the ability to check that a program you're thinking of running comes from the expected source and has not been tampered with. One way to achieve this is, in general, to validate a cryptographic signature that accompanies a program. You might think this is something the kernel could do for eBPF programs, perhaps as part of the verification step, but unfortunately this is not straightforward! As you've seen in this book, user space loaders dynamically adjust programs with information about where maps are located, and for CO-RE purposes, which from a signing perspective is hard to distinguish from malicious modifications. This is a problem for which the eBPF community is [keen to find a solution](#).

Long-lived kernel pointers

An eBPF program can retrieve a pointer to a kernel object using a helper function or a kfunc, but a pointer is valid only during that execution of the program. The pointer cannot be stored in a map for later retrieval. The idea of [typed pointer support](#) will allow for more flexibility in this area.

Memory allocation

It's not safe for eBPF programs to simply call memory allocation functions like `kmalloc()`, but there is [a proposal that suggests](#) an eBPF-specific alternative.

When will you be able to take advantage of new eBPF features as they emerge? As an end user, the features you're able to take advantage of depend on the version of the kernel you're running in production, and as I discussed in [Chapter 1](#), it can take several years for kernel releases to make it to stable distributions of Linux. As an individual you might opt for a bleeding-edge kernel, but the vast majority of organizations running server deployments use a stable, supported version. eBPF programmers have to take into account that if they write code that takes advantage of the newest features added to the kernel, the features are unlikely to be usable in most production environments for some years to come. Some organizations will have sufficiently urgent needs that it's worth rolling out newer kernel versions more quickly in order to early-adopt new eBPF features.

你什么时候能够利用新的eBPF功能？作为最终用户，你能够利用的功能取决于你在生产环境中运行的内核版本，正如我在第一章中所讨论的，Linux内核的稳定版本通常需要几年的时间才会进入到稳定的Linux发布版本中。作为个人用户，你可能选择使用先进的内核版本，但是绝大多数运行服务器部署的组织会选择使用稳定且受支持的版本。eBPF程序员必须考虑到，如果他们编写的代码利用了内核中新增的最新功能，这些功能很可能在大多数生产环境中无法使用几年的时间。一些组织可能有足够迫切的需求，值得更快地推出新的核心版本，以便更早地采用新的eBPF功能。

For example, in another forward-looking talk on [building tomorrow's networking](#), Daniel Borkmann discussed a feature called *Big TCP*. This was added to Linux in version 5.19 to enable network speeds of 100 GBit/s (and faster) by batching up network packets to be processed in the kernel. Most Linux distributions won't support a kernel this recent for a few years, but for specialist organizations dealing with large amounts of network traffic, it might well be worth upgrading sooner. Adding Big TCP support into eBPF and Cilium today means it's available for those massive-scale users, even if it's not something that can be enabled by most of us for a while.

例如，在另一场展望未来的网络建设的演讲中，Daniel Borkmann讨论了一个名为Big TCP的功能。Linux 5.19版中引入了这个功能，通过批处理网络数据包在内核中处理，从而实现了100 GBit/s（甚至更快）的网络速度。大多数Linux发行版在未来几年内不会支持如此新的内核版本，但对于处理大量网络流量的专业组织来说，提前升级可能是值得的。今天将Big TCP支持集成到eBPF和Cilium中意味着对于那

些大规模用户而言，即使大多数人暂时无法启用该功能，他们仍然可以使用它。

Since eBPF allows kernel code to be adjusted dynamically, it's reasonable to expect it to be used to address problems “in the field.” In [Chapter 9](#) you read about using eBPF to mitigate kernel vulnerabilities; work is also underway to use eBPF to help support hardware devices such as [human interface devices](#) like mice, keyboards, and game controllers. This builds on existing support for decoding the protocols used by infrared controllers that I mentioned in [Chapter 7](#).

由于eBPF允许内核代码动态调整，因此可以合理地期望它被用于解决“实地”中的问题。在第9章中，您了解到使用eBPF来减轻内核漏洞的影响；目前还在进行使用eBPF来支持硬件设备的工作，例如人机接口设备如鼠标、键盘和游戏控制器。这是在第7章中提到的解码红外控制器使用的协议的基础上进一步发展的。

eBPF Is a Platform, Not a Feature

eBPF不是一个功能，而是一个平台

Coming up to a decade ago, the hot new technology was containers, and it seemed as though everyone was talking about what they were and what advantages they would bring. We're at a similar stage with eBPF today, with lots of conference talks and blog posts—several of which I've referred to in this book—extolling the benefits of eBPF. Today, containers are part of daily life for many developers, whether

they're running code locally using Docker or other container runtimes, or deploying code to Kubernetes environments. Will eBPF become part of everyone's regular toolkit too?

大约十年前，炙手可热的新技术是容器，似乎每个人都在谈论它们以及它们带来的优势。如今，我们对电子伯克利数据包过滤（eBPF）也处于类似的阶段，有很多会议演讲和博客文章，本书中有几篇我参考过的，都赞扬了eBPF的好处。今天，无论是开发人员在本地使用Docker或其他容器运行时运行代码，还是将代码部署到Kubernetes环境中，容器已经成为许多开发人员日常生活的一部分。eBPF是否也会成为每个人的常规工具包的一部分呢？

The answer, I believe, is no—or at least, not directly. Most users won't write eBPF programs directly or manipulate them manually with utilities like `bpftool`. But they'll interact regularly with tools built using eBPF, whether that's for performance measurement, debugging, networking, security, tracing, or a whole host of other capabilities yet to be implemented using eBPF. Users might not be aware that they're using eBPF, much as they might not know that when they use containers, they're using kernel features like namespaces and cgroups.

答案，我相信是不，至少不是直接的。大多数用户不会直接编写eBPF程序或使用类似bpftool的工具进行手动操作。但是，他们将定期使用使用eBPF构建的工具进行交互，无论是用于性能测量、调试、网络、安全、跟踪还是其他使用eBPF尚未实现的一系列功能。用户可能并不知道他们正在使用eBPF，就像他们使用容器时可能不知道他们正在使用像命名空间和cgroups之类的内核特性一样。

Today, projects and vendors with knowledge of eBPF highlight their use of it because it's so powerful and implies many advantages. As eBPF-based projects and products gain traction and market share, eBPF is becoming the de facto default technology platform for infrastructure tooling.

今天，具有eBPF知识的项目和供应商强调它的使用，因为它非常强大并具有许多优势。随着基于eBPF的项目和产品获得认可 and 市场份额，eBPF正在成为基础设施工具的事实上的默认技术平台。

Knowledge of eBPF programming is—and will continue to be—a sought-after but relatively rare skill, just as kernel development today is much less common than developing, say, business applications or games. If you enjoy diving into the lower levels of systems and want to build essential infrastructure tooling, eBPF skills will serve you well. I hope this book has been of some assistance on your eBPF journey!

eBPF编程的知识是并且将继续是一项稀缺但受追捧的技能，就像今天相比开发业务应用程序或游戏来说，内核开发要少得多。如果您喜欢深入系统的底层并且想要构建基础设施工具，eBPF技能将对您大有裨益。希望这本书在您学习eBPF的旅途中有所帮助！

Further Reading

Throughout this book I have provided references to specific articles and documentation pages. Here is a list of some additional resources to help you on your eBPF journey:

在本书中，我提供了一些具体文章和文档页面的参考内容。以下是一些额外资源的列表，可以帮助你在eBPF之旅中获得帮助：

- The eBPF community site ebpf.io

eBPF社区网站ebpf.io

- The BPF and XDP reference in the [Cilium documentation](#)

Cilium 文档中的 BPF 和 XDP 参考文档

- [Linux kernel documentation on BPF](#)

Linux内核文档中的BPF

- [Brendan Gregg's website](#) on using eBPF for performance and observability

Brendan Gregg的网站使用eBPF进行性能和观测的内容。

- [Andrii Nakryiko's website](#), particularly for more information on CO-RE and *libbpf*

安德里·纳克里科的网站，特别是有关 CO-RE 和 libbpf 的更多信息。

- [Lwn.net](#), a wonderful resource for updates to the Linux kernel, including the BPF subsystem

Lwn.net是一个更新Linux内核的绝佳资源，包括BPF子系统。

- [Elixir.bootlin.com](#), where you can browse the Linux source code

在Elixir.bootlin.com上，您可以浏览Linux源代码。

- [eCHO](#), a weekly livestream covering topics from across the eBPF and Cilium community (in which this author is a regular presenter)

eCHO，每周直播涵盖eBPF和Cilium社区的各个话题（本文作者是定期主讲人之一）。

Conclusion

结论

Congratulations on reaching the end of this book!

恭喜你完成了这本书！

I hope that reading *Learning eBPF* has given you insight into the power of eBPF. Perhaps it has inspired you to write eBPF code yourself or experiment with some of the tools I've discussed. If you've decided to do some eBPF programming, I hope the book has given you some confidence about how to get started. And if you have been completing the exercises as you worked through the book, bravo!

我希望阅读《学习 eBPF》让你深入了解了 eBPF 的强大之处。也许它激发了你自己编写 eBPF 代码或尝试一些我所讲述的工具的想法。如果你决定进行一些 eBPF 编程，我希望这本书能给你一些关于如何入门的信心。如果你一边阅读书籍一边完成练习，那真棒！

If you're excited about eBPF there are plenty of ways to get involved in the community. The best starting point is the website ebpf.io. This

will point you to the latest news, projects, events, and happenings, and also to the [eBPF Slack](#) channel where you're likely to find someone with the expertise to answer any questions you might have.

如果您对eBPF感到兴奋，有很多方法可以参与到社区中。最好的起点是网站ebpf.io。这将为您提供最新的新闻、项目、事件和动态，并带您进入eBPF Slack频道，在那里您很可能会找到有专业知识的人来回答您可能有的任何问题。

I welcome your feedback, comments, and any corrections to this text. You can provide your input through the GitHub repository that accompanies this book: github.com/lizrice/learning-ebpf. I'd also be happy to hear your comments directly. You can find me as @lizrice in many places across the internet.

欢迎您提供反馈、评论以及对本文的任何修正意见。您可以通过随附本书的 GitHub 存储库（github.com/lizrice/learning-ebpf）提供您的意见。我也非常愿意直接听取您的评论。您可以在互联网的许多地方找到我，我的用户名是 @lizrice。

¹ Shout-out to Alexei Starovoitov and Andrii Nakryiko from Meta, and Daniel Borkmann from Isovalent, who maintain the BPF subtree in the Linux kernel.

向 Meta 的 Alexei Starovoitov、Andrii Nakryiko 和 Isovalent 的 Daniel Borkmann 表示致敬，他们在 Linux 内核中维护 BPF 子树。

² [Dave Thaler presented on the state of this standardization work](#) at the Linux Plumbers Conference.

戴夫·萨勒在Linux管道会议上介绍了这项标准化工作的进展情况。

³ Well, it *could*, but doing so would require Microsoft to also release the Windows source code under the GPL license.

嗯，这样做是可能的，但需要微软将Windows源代码以GPL许可证发布出来。

⁴ Alexei Starovoitov discusses the journey of BPF from restricted C language to extended and safe C [in this video](#).

Alexei Starovoitov在这个视频中讨论了BPF从受限制的C语言到扩展和安全C的过程。

About the Author

关于作者

Liz Rice is the chief open source officer with eBPF specialists at Isovalent, creators of the Cilium cloud native networking, security, and observability project. She sits on the CNCF Governing Board and on the Board of OpenUK. She was chair of the CNCF's Technical Oversight Committee in 2019–2022 and cochair of KubeCon + CloudNativeCon in 2018. She is also the author of *Container Security* published by O'Reilly.

Liz Rice 是Isovalent的eBPF专家团队的首席开源官员，他们是Cilium云原生网络、安全和可观测性项目的创造者。她是CNCF（云原生计算基金会）的理事会成员，也是OpenUK的董事会成员。她曾担任CNCF技术监督委员会主席（2019年-2022年），并在2018年担任KubeCon + CloudNativeCon联合主席。她还是《容器安全》一书的作者（由O'Reilly出版）。

She has a wealth of software development, team, and product management experience from working on network protocols and distributed systems and in digital technology sectors such as VOD, music, and VoIP. When not writing code, or talking about it, Liz loves riding bikes in places with better weather than her native London, competing in virtual races on Zwift, and making music under the pseudonym Insider Nine.

她在网络协议和分布式系统的工作中积累了丰富的软件开发、团队和产品管理经验，以及在VOD、音乐和VoIP等数字技术领域的经验。当她不写代码或讨论代码时，利兹喜欢在比她的故乡伦敦天气更好的地方骑自行车，在Zwift上参加虚拟比赛，并以Insider Nine的化名创作音乐。

Colophon

版权页

The animal on the cover of *Learning eBPF* is an early bumblebee (*Bombus pratorum*), a bumblebee species found in most of Europe (especially the UK) and parts of Asia.

学习eBPF封面上的动物是早期大黄蜂（*Bombus pratorum*），它是一种在大部分欧洲（尤其是英国）和亚洲部分地区发现的大黄蜂物种。

Bombus pratorum builds nests above ground in fields, parks, and sparse forest, even repurposing abandoned bird or rodent nests. The early bumblebee indeed emerges early in the year, typically from March to July, but in southern England, worker bees appear as early as February, making it quite common for two colony cycles to occur in a single year.

熊蜂在田地、公园和稀疏森林地区筑巢，甚至会改建废弃的鸟类或啮齿动物的巢穴。早期熊蜂确实在一年的早期出现，通常从三月到七月，但在英国南部，工蜂早在二月就会出现，因此在同一年内出现两个群体循环非常普遍。

This species of bumblebee is quite smaller than others. While there are slight variations among queen, worker, and male drone types, the early bumblebee's appearance is generally black with a yellow collar, another yellow band on the abdominal segment, and a reddish or dull orange tail.

这种大黄蜂的体型要比其他蜜蜂小一些。虽然在女王、工蜂和雄性蜂之间有一些细微差异，但早期大黄蜂的外观通常是黑色的，有一道黄色的领圈，腹部有一道黄色的条纹，尾部呈红色或暗橙色。

Early bumblebees form colonies with queen and worker bees but, unusually, queens use aggressive behavior rather than pheromones to establish dominance, using her mandibles to head-butt the strongest worker bees to maintain control of the colony. Worker bees forages for nectar and pollen of flowering plants such as white clover, thistle, sage, lavender; drones are created late in the hive cycle and leave the nest in search of new queens.

早期的大黄蜂会以女王和工蜂的形式组成群体，但与寻常不同的是，女王们使用攻击性行为而不是信息素来确立统治地位，她们用下颚对抗最强壮的工蜂以保持对群体的控制。工蜂会采集白三叶草、蓟、鼠尾草和薰衣草等开花植物的花蜜和花粉；雄蜂在蜂巢周期的后期被产生，并离巢寻找新的皇后蜂。

Many of the animals on O'Reilly covers are endangered; all of them are important to the world. The cover illustration is by Karen Montgomery, based on an antique line engraving from *The Animal Kingdom Illustrated*. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.

奥莱利出版物封面上的许多动物濒临灭绝，它们对世界都非常重要。封面插图由卡伦·蒙哥马利根据《动物王国插图集》中的一幅古董线刻而来。封面字体为Gilroy Semibold和Guardian Sans。正文字体为

Adobe Minion Pro；标题字体为Adobe Myriad Condensed；代码字体为Dalton Maag的Ubuntu Mono。